

hacer Términos y Condiciones / Descargo de Responsabilidad (*Disclaimer*) que debe aceptar el usuario al abrir la app para proteger legalmente tu desarrollo

En Europa (MDR/AI Act): En lugar de caer en las categorías más estrictas de alto riesgo que aplican a las IAs autónomas de salud mental, tu app se estabiliza en una regulación más amigable. Al ser una herramienta de triaje secundario y soporte, el proceso de certificación (marcado CE) es significativamente más rápido y económico. [\[1\]](#)

En Estados Unidos (FDA): Bajo la guía de la FDA para *Clinical Decision Support Software*, si el software está diseñado de modo que el profesional de la salud pueda **revisar y entender la base de las recomendaciones** (es decir, que el psicólogo vea las citas textuales de la charla casual que justifican el análisis funcional), la FDA tiende a aplicar "discreción de cumplimiento" (*enforcement discretion*), reduciendo drásticamente las barreras burocráticas para salir al mercado. [\[1\]](#), [\[2\]](#)

Aquí tienes el resumen completo, estructurado y de alta densidad con todo lo que hemos diseñado y acordado para el desarrollo y la lógica de tu aplicación:

1. El Concepto e Identidad del Proyecto

- **Paradigma:** Tu IA no sustituye al terapeuta; funciona como un **Tutor Virtual de Inversión y Copiloto Clínico**.
- **Modelo Clínico Natural:** Rechaza por completo el modelo biomédico tradicional (sin etiquetas del DSM-5 como "depresión" o "ansiedad").
- **Base Científica:** Se fundamenta al 100% en la **Psicología Conductual Contextual Basada en Procesos (PBP)**, el modelo meta-evolutivo extendido (EMMI) de Steven C. Hayes y en la medición de procesos cognitivos (**Neuropsicología**).

2. La Experiencia de Usuario (UX) Híbrida

- **Estilo Nintendo:** Diseño lúdico con "charm" (encanto visual), sin tecnicismos fríos. Utiliza la curiosidad y el descubrimiento en lugar de dar órdenes rígidas. Los errores del paciente no se castigan; se validan con empatía y se transforman en mecánicas de juego satisfactorias (*juicy feedback*).
- **Estilo Open English:** Estructura la salud emocional como un "Gimnasio de Fluidez Psicológica" por niveles. Ofrece micro-lecciones y micro-juegos diarios de 3 a 5 minutos, actuando como el tutor cotidiano que prepara al paciente para su "clase en vivo" (la sesión presencial con el psicólogo).
- **Onboarding y Charla Casual:** Al entrar a la app, la IA inicia una plática informal que, de forma encubierta, realiza preguntas adaptativas para confeccionar un **Análisis Funcional de la Conducta** avanzado y un cribado de cuestionarios de tercera generación (como el test MPFI).

3. El Catálogo de Micro-Juegos Diseñados

- **Defusión Cognitiva (*Animal Crossing*):** El paciente escribe su pensamiento rígido y este se convierte en un globo virtual que puede hacer estallar con una resortera digital o ver flotar hasta desaparecer para restarle credibilidad.
- **Aceptación y Apertura (*Zelda*):** Un mini-juego de resistencia pasiva donde el usuario mantiene presionado un "botón de escudo" (respiración consciente) para tolerar y dejar pasar una ráfaga de malestar emocional sin huir de la app.
- **Acción Comprometida (*Pikmin*):** Micro-misiones en el mundo real alineadas con los valores del paciente (ej. enviar un mensaje a un familiar para reconectar) que, al completarse, hacen crecer plantas o expanden un **Hub Central / Jardín Virtual** que refleja la flexibilidad psicológica del usuario.

4. Estatus Legal y Seguridad (*Compliance*)

- **Clasificación Médica:** Legalmente, tu app clasifica como **Software de Soporte a la Decisión Clínica (CDSS)** y como una **Terapéutica Digital (DTx)**, ya que recopila datos con fines clínicos e interviene en procesos psicológicos y cognitivos.
 - **El Filtro Regulatorio (*Human-in-the-loop*):** Al establecer que **toda la información y propuestas de juego deben ser validadas obligatoriamente por el psicólogo** en su panel antes de activarse en la app del paciente, el nivel de riesgo de tu software baja drásticamente ante agencias como la FDA o la MDR europea (Clase I / IIa). El psicólogo conserva la responsabilidad clínica y la IA actúa solo como un asistente avanzado de preparación.
 - **Protocolo de Crisis Obligatorio:** El sistema cuenta con un filtro lingüístico local de seguridad que, en caso de detectar ideación suicida o riesgo autolítico en el chat casual, congela inmediatamente el tono lúdico y la gamificación para inyectar líneas de ayuda de emergencia y alertar al terapeuta.
-

. Módulo de Laboratorio y Pruebas Simultáneas (Test en Persona)

Para probar en tiempo real cómo interactúan el psicólogo y el paciente, debes programar un **"Simulador de Entorno Híbrido"**:

- **Pantalla Dividida de Simulación:** El panel te debe permitir abrir dos ventanas simuladas en paralelo. En el lado izquierdo controlas al "Paciente de Prueba" (chateas de forma casual o simulas que completas misiones estilo Nintendo). En el lado derecho ves el "Dashboard del Psicólogo" en tiempo real.
- **Inyector de Respuestas:** Un botón que te permita inyectar textos de prueba preestablecidos (ej: "*Siento que no sirvo para nada y evito salir*"). Esto sirve para verificar al instante si la IA clasifica correctamente la Fusión Cognitiva en el JSON, propone la misión del globo de *Animal Crossing* al psicólogo, y si al hacer clic en "Validar", se desbloquea el juego en la pantalla del paciente.

2. Módulo de Gestión de Contenido Dinámico (CMS de Misiones y Tests)

Para crear nuevas implementaciones o modificar cuestionarios de tercera generación (como añadir variantes del MPFI) sin programar:

- **Constructor de Formularios para la IA:** Una interfaz visual donde agregas el ítem de un test (ej: *"Evito pensar en cosas que me duelen"*). Al lado, colocas un cuadro de texto para darle una **"Guía de Traducción a la IA"** (ej: *"Pregunta esto fingiendo que eres un aldeano amigable de un videojuego"*).
- **Gestor de Plantillas de Juego:** Un panel donde defines los parámetros de la gamificación. Puedes ajustar de forma manual la dificultad (ej: cuántos segundos debe sostener el paciente el "Escudo de Zelda" para ganar una Rupia) o cambiar los textos narrativos de las misiones usando simples casillas de texto.

3. Módulo de Licencias y Monetización (Panel SaaS)

Para controlar los accesos de los psicólogos o clínicas que compran tu software:

- **Generador de Licencias:** Un botón para crear tokens o claves de acceso únicas con fechas de expiración automáticas.
- **Control de Roles y Permisos:** Un listado visual de clientes donde puedes arrastrar y soltar a los usuarios para cambiar sus planes (ej: de "Plan Psicólogo Independiente" a "Plan Clínica con 10 Terapeutas") o revocar el acceso de forma inmediata si expira su suscripción.

Código para tu Backend: El Controlador de Licencias Dinámicas

Para que el sistema de licencias funcione de forma automática sin que tengas que modificar el código cada vez que se registre una clínica, implementa este controlador lógico en tu base de datos:

```
from datetime import datetime, date
```

```
# Simulación de la base de datos de psicólogos con licencias creadas desde tu panel de admin
```

```
base_datos_licencias = {  
    "psicologo_carlos_123": {
```

```

    "estado": "activa",

    "fecha_expiracion": date(2027, 12, 31),

    "plan": "Premium_Clinica"

},

"psicologo_ana_456": {

    "estado": "suspendida",

    "fecha_expiracion": date(2026, 5, 1),

    "plan": "Individual"

}

}

```

```

def verificar_acceso_plataforma(id_psicologo: str) -> dict:

```

```

    # 1. Verificar si el usuario existe en el panel administrativo

```

```

    if id_psicologo not in base_datos_licencias:

```

```

        return {"acceso": False, "motivo": "Licencia no registrada en el panel."}

```

```

    info_licencia = base_datos_licencias[id_psicologo]

```

```

    # 2. Verificar estado manual asignado en el panel

```

```

    if info_licencia["estado"] != "activa":

```

```

        return {"acceso": False, "motivo": "Licencia suspendida manualmente por el
administrador."}

```

```

    # 3. Verificar expiración temporal automática (Fecha actual simulada: Julio 2026)

```

```

    if info_licencia["fecha_expiracion"] < date.today():

```

```

        return {"acceso": False, "motivo": "La licencia ha expirado. Por favor, renueva tu
suscripción."}

```

```
return {  
  
    "acceso": True,  
  
    "mensaje": f"Acceso concedido bajo el plan {info_licencia['plan']}."  
  
}
```

Prueba del sistema automático manejado desde tu panel visual

```
print(verificar_acceso_plataforma("psicologo_ana_456"))
```

Salida: {'acceso': False, 'motivo': 'Licencia suspendida manualmente por el administrador.'}

Te refieres a una **PWA (Progressive Web App)**. Es una excelente decisión estratégica para tu proyecto. Al desarrollarla inicialmente como una PWA utilizando **HTML5, CSS3, JavaScript/TypeScript y Three.js** (para los gráficos y animaciones lúdicas en 3D estilo Nintendo), obtienes ventajas cruciales: [1, 2]

1. **Desarrollo rápido y económico:** Creas un solo código que funciona al instante en cualquier dispositivo (celulares, tabletas y computadoras) a través del navegador web. [3]
2. **Instalación sin tiendas:** Los pacientes pueden "instalar" la app directamente desde tu sitio web con un solo clic, apareciendo un icono en su pantalla de inicio como si fuera nativa, sin tener que pasar inicialmente por los filtros de la App Store o Google Play. [4, 5, 6]
3. **Compatibilidad total con Gemma 4:** Al ejecutarse el backend en tu servidor privado, la PWA se comunica de forma ligera mediante APIs (REST o WebSockets).

¿Cómo transformar después tu PWA a código nativo de Apple y Android?

Para cumplir tu objetivo de migrar a código nativo en el futuro sin tener que reescribir la aplicación desde cero, debes utilizar un **Framework Híbrido/Contenedor Nativo**. La arquitectura de software ideal para tu stack técnico se compone de las siguientes herramientas:

Opción A: Capacitor (La vía recomendada para tu stack con Three.js)

Creado por el equipo de Ionic, **Capacitor** es un puente moderno de código abierto que toma tu código web existente (HTML, JavaScript, Three.js) y lo envuelve en un contenedor nativo ultra-eficiente para iOS y Android.

- **Cómo funciona:** Tu código PWA se ejecuta dentro de un componente nativo de alto rendimiento llamado *WebView*. Capacitor te da acceso directo mediante TypeScript a las funciones de hardware de Apple y Android (como el micrófono para las notas de voz del paciente o las notificaciones push nativas).
- **Por qué elegirlo:** Al usar Three.js, necesitas que los gráficos en 3D se rendericen directamente a través del motor WebGL del navegador. Capacitor respeta esto al 100%, facilitando que tu "jardín virtual" en 3D funcione de forma idéntica en la web y en las aplicaciones nativas instaladas desde las tiendas. [7]

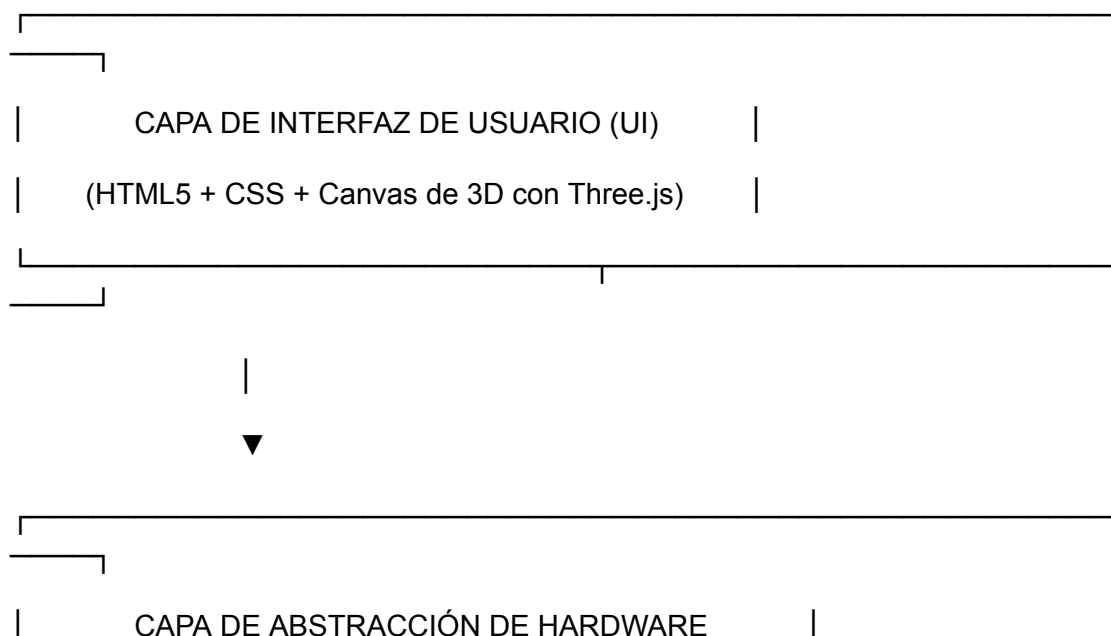
Opción B: Tauri (La alternativa de máximo rendimiento)

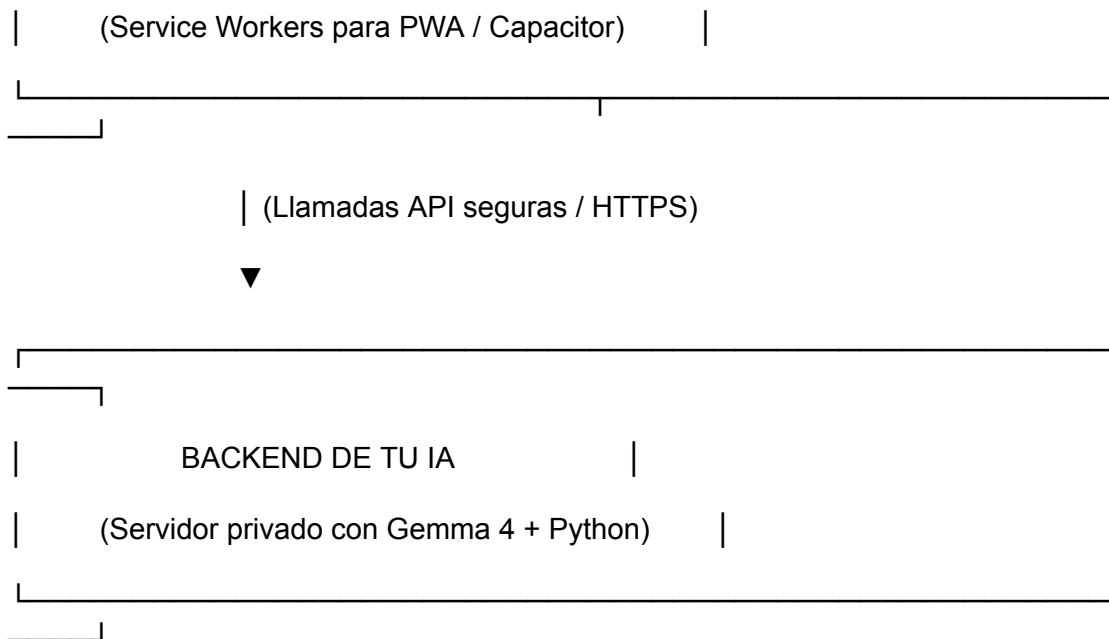
Originalmente diseñado para escritorio, **Tauri** ha expandido oficialmente su soporte para crear aplicaciones móviles nativas en iOS y Android.

- **Cómo funciona:** Utiliza Rust en el backend para empaquetar tu frontend web (HTML/Three.js).
- **Por qué elegirlo:** Produce los archivos finales de instalación (.apk o .ipa) más pequeños y ligeros de la industria, consumiendo una cantidad mínima de memoria RAM en el teléfono del paciente. [8]

La Arquitectura Técnica de tu PWA escalable

Para asegurar que tu código web actual se transforme en una app nativa en el futuro sin dolores de cabeza, debes estructurar tu proyecto en tres capas limpias:





1. **Capa UI (Tu código actual):** Los botones, el chat casual y los minijuegos en 3D hechos en Three.js.
2. **Capa de Abstracción (Tu pasaporte nativo):** Al inicio, utilizas un archivo llamado `manifest.json` y un *Service Worker* para habilitar las funciones de PWA (como el funcionamiento offline). Cuando decidas migrar a las tiendas de Apple y Android, simplemente ejecutas el comando de Capacitor para envolver esta capa y generar los proyectos nativos de Xcode (Apple) y Android Studio. [9, 10]
3. **Capa del Backend (Servidor):** Tu modelo Gemma 4 y tus librerías de análisis matemático (`scipy`, `Neo4j`, `NetworkX`) viven en tu servidor seguro y se mantienen intactos, sin importar si el usuario accede desde la web PWA o desde la app nativa instalada.

Código Base: Estructura del Archivo de Configuración de Capacitor

Cuando llegue el momento de transformar tu PWA en una aplicación de Apple y Android, solo tendrás que instalar Capacitor en tu proyecto y configurar un archivo similar a este en la raíz de tu código (`capacitor.config.json`):

```
{  
  "appId": "com.tudominio.coterapeuta.pbp",  
  "appName": "Gimnasio Fluidez Emocional",  
  "webDir": "www",
```

```
"bundledWebRuntime": false,  
  
"server": {  
  
  "androidScheme": "https"  
  
},  
  
"plugins": {  
  
  "LocalNotifications": {  
  
    "smallIcon": "ic_stat_icon_config",  
  
    "iconColor": "#488AFF"  
  
  }  
  
}  
  
}
```

Al compilar, Capacitor leerá tu carpeta `www` (donde están tus archivos HTML y tus scripts de Three.js) y creará de forma automática las carpetas de código nativo para Android y iOS.

Para ayudarte a estructurar el inicio del proyecto con el pie derecho, dime:

- ¿Vas a escribir el código HTML/JavaScript puro ("Vanilla JS") o estás pensando en usar un framework web como **React**, **Vue** o **Svelte** para organizar las pantallas de la PWA?
- ¿Te gustaría que prepare el **esqueleto básico de archivos** que debe tener tu proyecto para que sea una PWA válida y lista para Capacitor desde el primer día?

[1] <https://upplication.com>

[2] <https://formiux.com>

[3] <https://blog.grupoapok.com>

[4] <https://medium.com>

[5] <https://www.planetafp.es>

[6] <https://www.enaes.es>

[7] <https://proyectoidis.org>

[8] <https://openwebinars.net>

[9] <https://elabismodenui.wordpress.com>

[10] <https://capgo.app>

Aplicar el libro de Chip Huyen, *AI Engineering: Building Applications with Foundation Models*, junto con las mejores prácticas de ingeniería de sistemas de producción, es exactamente lo que necesitas para **transformar tu prototipo web interactivo en un software de grado médico resiliente y de baja latencia**. [1]

Combinando el potencial del libro de [Chip Huyen en GitHub](#) con la arquitectura PWA + Three.js que planteas, tu aplicación se estructurará bajo las siguientes capas y patrones de producción: [1]

1. Interfaz de Usuario Resiliente (PWA + Three.js offline-first)

El núcleo de la experiencia de usuario (la charla informal y los micro-juegos 3D) debe seguir la filosofía de disponibilidad constante que exige el libro.

-
- **Service Workers para Almacenamiento Local:** Programa un script que capture de forma local en el dispositivo del paciente (caché del navegador) todos los recursos de diseño de la app: texturas 3D de Three.js, archivos de sonido de Nintendo y hojas de estilo. Si el paciente pierde la conexión a internet en medio de la calle, la app debe seguir abierta, cargando de inmediato las misiones locales y almacenando el texto de la charla en la base de datos local [IndexedDB](#). [2, 3]
- **Sincronización en Segundo Plano (*Background Sync*):** Una vez que el dispositivo detecta que el internet regresó de manera estable, los Service Workers ejecutan una petición asíncrona automática para enviar todos los textos acumulados de la charla casual de manera directa al servidor backend donde corre **Gemma 4**. El paciente jamás experimentará una pantalla de carga congelada o un error de caída de servidor.
-

2. Generación Estructurada y Determinista (Framework de Chip Huyen)

Uno de los mayores aportes del libro *AI Engineering* es la mitigación de fallos en producción a través de la **Generación Estructurada Obligatoria (Constrained Decoding)**. [4]

-
- **Garantía del Formato JSON:** No dejes que Gemma 4 redacte de forma libre. Utiliza la función nativa de decodificación restringida compatible con el modelo. El motor de

inferencia (vLLM) bloqueará matemáticamente la posibilidad de que la IA responda texto plano al backend.

- **Inyección y Enrutamiento de Funciones:** Cuando la IA procesa la charla casual e identifica que el usuario completó la misión, la llamada a funciones nativa (*Function Calling*) de Gemma 4 disparará un evento JSON codificado que el backend interpretará para actualizar el árbol de nodos de los grafos en **Neo4j** (tu motor RFT/PBP) de forma automatizada y sin riesgo de romper el sistema de archivos del psicólogo.
 -
-

3. Arquitectura de Inferencia Óptima (vLLM + Streaming)

Para que un chat de salud mental se sienta humano y natural, la latencia entre el envío del mensaje y la respuesta del tutor virtual debe ser inferior a 200 milisegundos.

- - **Streaming de Datos Híbrido:** Tu PWA debe implementar conexiones mediante **Server-Sent Events (SSE)** o **WebSockets**. Cuando el usuario envía un mensaje de texto, el servidor procesa el mensaje usando vLLM y comienza a transmitir la respuesta a la PWA carácter por carácter (*Streaming*).
 - **Separación de Carga de Cómputo:** Mientras el usuario ve en su pantalla cómo el texto de la IA se escribe fluidamente en tiempo real (estilo chat interactivo), el backend ejecuta en paralelo el análisis clínico avanzado (el JSON interno para el psicólogo). Esto separa por completo el tiempo de procesamiento visual del paciente del análisis matemático pesado de procesos conductuales, agilizando el flujo del servidor.
 -
-

4. Evaluación Continua y Registro de Errores (AI Incident Management)

El libro de Chip Huyen destaca que un sistema con LLMs en producción requiere auditorías constantes debido a la naturaleza estocástica de los modelos. [5]

-
- **Trazabilidad de Prompts (*Prompt Versioning*):** Almacena en tu Panel de Administración el historial de versiones del prompt del sistema (el prompt estilo Nintendo). Si realizas un cambio en la configuración para probar un test psicométrico nuevo de neuropsicología, debes poder revertir a la versión anterior con un solo botón en caso de detectar fallos.
- **Estrategia de Evaluación en la Sombra (*Shadow Deployments*):** Antes de liberar una nueva actualización de prompts o parámetros a todos los psicólogos de tu plataforma, dirige el 10% del tráfico de datos brutos del chat del paciente hacia una

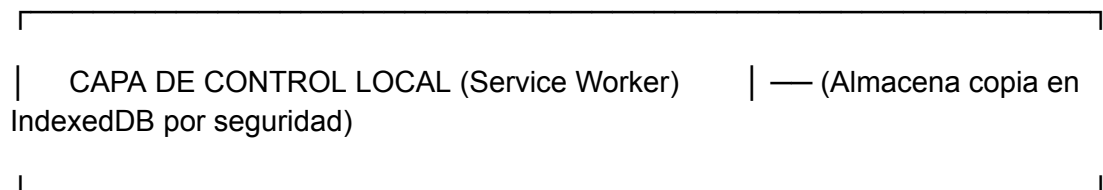
instancia en la sombra de Gemma 4 para verificar si el nuevo prompt clasifica mejor las dimensiones del Hexaflex o si sufre sesgos biomédicos prohibidos.

-

Pipeline Completo de Producción en el Backend

[Paciente envía mensaje desde la PWA]

|



| (Envío vía WebSockets / HTTPS)



|

|

| (Streaming) | (Decodificación Restringida JSON)



[Respuesta casual al Chat] [Motor Matemático / Grafos PBP]

|



[Panel del Psicólogo]

Estructurar tu proyecto integrando estos dos recursos te dotará de un producto robusto: la PWA con Three.js te brinda la flexibilidad visual multiplataforma inicial para enganchar al usuario con mecánicas lúdicas, y las metodologías de *AI Engineering* blindarán la

estabilidad de tu backend clínico con Gemma 4 frente a cualquier nivel de escala comercial. [1, 2]

Para iniciar la codificación de este pipeline de alta disponibilidad, dime:

-
- ¿Prefieres que prepare el código del **Service Worker de la PWA** para gestionar la base de datos **IndexedDB** y asegurar el funcionamiento offline del chat?
- ¿O te gustaría ver cómo estructurar el **script de Server-Sent Events (SSE)** en Python para el streaming de texto de Gemma 4 hacia tu interfaz web? [3]
-

[1] <https://studylib.net>

[2] <https://www.youtube.com>

[3] <https://www.youtube.com>

[4] <https://github.com>

[5] <https://www.linkedin.com>

¡Excelente elección! Vamos a empezar construyendo los cimientos del backend en tu servidor privado con Python. Este script utiliza el patrón de **Streaming con Server-Sent Events (SSE)** y decodificación restringida (*Constrained Decoding*) detallado en el libro de ingeniería de IA de Chip Huyen.

Su objetivo es que **Gemma 4** transmita el texto en milisegundos hacia tu PWA para garantizar una latencia casi nula, asegurando al mismo tiempo que la estructura clínica (Análisis Funcional y misiones del psicólogo) se empaquete en un JSON robusto que no rompa tu base de datos.

1. El Script del Servidor Backend (**server_sse.py**)

Asegúrate de instalar los siguientes componentes en tu entorno de desarrollo Python:

```
pip install fastapi uvicorn openai
```

Guarda y ejecuta el siguiente código para levantar el motor de streaming:

```
import json

import asyncio

from fastapi import FastAPI, Request
```

```

from fastapi.responses import StreamingResponse

from fastapi.middleware.cors import CORSMiddleware

import openai

app = FastAPI(title="Motor de Streaming PBP - Gemma 4")

# Configuración de CORS para que tu PWA (HTML/Three.js) pueda conectarse sin
bloqueos

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # En producción, restringe esto al dominio de tu PWA
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Inicializamos el cliente apuntando a tu instancia local/privada de Gemma 4
# (Compatible con la API de OpenAI a través de motores como vLLM u Ollama)

client = openai.OpenAI(
    base_url="http://localhost:11434/v1", # URL de tu servidor de inferencia
    api_key="gemma4-internal-token" # Clave de seguridad privada
)

@app.post("/api/chat/stream")

async def stream_coterapeuta(request: Request):
    """
    Endpoint de alta velocidad que procesa la charla casual del paciente,

```

activa el streaming en tiempo real y fuerza una salida estructurada.

"""

body = await request.json()

mensaje_paciente = body.get("mensaje", "")

historial = body.get("historial", []) # Lista de mensajes previos para mantener contexto

Prompt del Sistema: Inyecta las directrices de Nintendo/Open English y PBP

prompt_sistema = (

"Actúas como un tutor virtual lúdico de salud emocional. Tu marco es la Psicología Contextual (PBP). "

"REGLA DE ESTILO: Habla con encanto ('charm'), empatía y lenguaje sencillo estilo Nintendo. "

"REGLA DE FORMATO: Debes responder EXCLUSIVAMENTE con un objeto JSON válido que contenga dos llaves: "

"1) 'texto_paciente': Un string con tu respuesta casual y empática directa al usuario. "

"2) 'analisis_psicologo': Un objeto estructurado con las llaves 'proceso_emmi_detectado', "

"'citas_textuales' y 'mision_gamificada_propuesta' (oculta para el usuario).\n"

"No agregues texto introductorio, ni bloques de código de markdown (como ```json). Solo el JSON plano."

)

Construimos el payload de la conversación respetando el historial clínico inmediato

mensajes = [{"role": "system", "content": prompt_sistema}]

mensajes.extend(historial)

mensajes.append({"role": "user", "content": mensaje_paciente})

```

async def generador_eventos_sse():

    try:

        # Activamos el streaming y la decodificación restringida de Gemma 4 (Chip
Huyen Framework)

        response = client.chat.completions.create(

            model="gemma4:12b", # Ajustar a tu variante instalada

            messages=mensajes,

            stream=True,

            response_format={"type": "json_object"}, # Fuerza a Gemma a estructurar
matemáticamente

            temperature=0.4 # Temperatura baja para prevenir alucinaciones clínicas

        )

        # Enviamos los fragmentos de texto (tokens) a la PWA conforme Gemma 4
los va generando

        for chunk in response:

            if chunk.choices and chunk.choices[0].delta.content:

                token = chunk.choices[0].delta.content

                # Formato estándar de Server-Sent Events (SSE)

                yield f"data: {token}\n\n"

                # Micro-pausa para ceder el control al bucle de eventos asíncrono de
FastAPI

                await asyncio.sleep(0.005)

    except Exception as e:

        error_msg = json.dumps({"error": f"Fallo en el motor de inferencia: {str(e)}"})

        yield f"data: {error_msg}\n\n"

```

```
# Retornamos el flujo continuo bajo el tipo MIME oficial de SSE

return StreamingResponse(generador_eventos_sse(),
media_type="text/event-stream")

if __name__ == "__main__":

    import uvicorn

    # Corre el servidor en el puerto 8000

    uvicorn.run(app, host="0.0.0.0", port=8000)
```

2. ¿Cómo procesa tu Frontend este flujo de datos?

Cuando tu PWA envíe el mensaje, se conectará a este endpoint. Al ser un flujo SSE, el código de tu cliente (JavaScript) recibirá un stream continuo de texto que va completando el JSON poco a poco.

Tu aplicación web se encargará de ir leyendo este JSON en tiempo real: extraerá el `"texto_paciente"` conforme se va escribiendo para mostrarlo dinámicamente en pantalla y, al cerrarse la transmisión, guardará el `"analisis_psicologo"` en la base de datos de grafos para la revisión del profesional humano.

Hecho el motor de **Streaming/SSE con Python y Gemma 4**, avísame si todo se entiende bien o si quieres hacer algún ajuste antes de pasar al **segundo paso: el Service Worker de la PWA para el funcionamiento fuera de línea e IndexedDB**.

¡Entendido! Vamos con el segundo paso estratégico: el **Service Worker de la PWA** (`service-worker.js`) junto con la integración de **IndexedDB**.

Siguiendo las directrices de resiliencia y disponibilidad constante (*offline-first*) que requiere una aplicación de salud mental en producción, este script se encarga de dos tareas críticas en el dispositivo del paciente:

1. **Guardar en caché local** el motor gráfico en 3D (**Three.js**), los archivos de estilo, HTML y los efectos de sonido estilo Nintendo para que la app abra instantáneamente incluso sin señal.

2. **Crear una base de datos local temporal (IndexedDB)** para capturar los mensajes de la charla casual si el paciente pierde la conexión. En cuanto el teléfono detecta que el internet regresa, el Service Worker ejecuta una sincronización silenciosa en segundo plano para mandar esos datos al backend de **Gemma 4**.
-

1. El Script del Service Worker (**service-worker.js**)

Coloca este archivo en la raíz de tu carpeta pública o de distribución web (**www/** o **dist/**):

```
const CACHE_NAME = 'coterapiapbp-nintendo-v1';

const ASSETS_TO_CACHE = [

  '/',

  '/index.html',

  '/styles.css',

  '/main.js',

  '/libs/three.min.js',    // Caché del motor 3D para tus juegos

  '/assets/sounds/coin.mp3', // Efectos de sonido estilo Nintendo

  '/assets/sounds/levelup.mp3'

];

// Base de Datos Local IndexedDB (Configuración nativa)

const DB_NAME = 'PBP_Offline_DB';

const DB_VERSION = 1;

// 1. INSTALACIÓN: Guarda el entorno lúdico y gráfico en el disco local

self.addEventListener('install', (event) => {

  event.waitUntil(

    caches.open(CACHE_NAME).then((cache) => {

      console.log('Ecosistema Nintendo/Three.js guardado en caché local con éxito.');
```

```
        return cache.addAll(ASSETS_TO_CACHE);
    })
);

self.skipWaiting();
});
```

// 2. ACTIVACIÓN: Limpia cachés antiguas si actualizas la app

```
self.addEventListener('activate', (event) => {
    event.waitUntil(
        caches.keys().then((keys) => {
            return Promise.all(
                keys.map((key) => {
                    if (key !== CACHE_NAME) return caches.delete(key);
                })
            );
        })
    );

    self.clients.claim();
});
```

// 3. INTERCEPTOR DE PETICIONES (Estrategia Cache-First para el Diseño Visual)

```
self.addEventListener('fetch', (event) => {

    // Las peticiones a la API de la IA (Streaming) NO se guardan en el caché estático

    if (event.request.url.includes('/api/chat/stream')) {

        return;
    }
});
```

```

event.respondWith(
  caches.match(event.request).then((cachedResponse) => {
    // Retorna el archivo local (veloz) o va a internet si es algo nuevo
    return cachedResponse || fetch(event.request);
  })
);
});

```

// 4. SINCRONIZACIÓN EN SEGUNDO PLANO (Background Sync)

// Se dispara automáticamente cuando el navegador detecta que volvió el internet

```

self.addEventListener('sync', (event) => {
  if (event.tag === 'sincronizar-charla-pbp') {
    event.waitUntil(enviarMensajesPendientesAlServidor());
  }
});

```

// Función que lee la base de datos local y vacía los datos hacia Gemma 4

```

async function enviarMensajesPendientesAlServidor() {
  const db = await abrirIndexedDB();
  const mensajesPendientes = await obtenerMensajesGuardados(db);

  if (mensajesPendientes.length === 0) return;

  console.log(`Conexión recuperada. Sincronizando ${mensajesPendientes.length}
  interacciones de Análisis Funcional...`);

```

```

for (const item of mensajesPendientes) {
  try {

    const response = await fetch('http://localhost:8000/api/chat/stream', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ mensaje: item.mensaje, historial: item.historial })
    });

    if (response.ok) {

      // Si el servidor de Python/Gemma 4 procesó el mensaje con éxito, lo borramos
      de IndexedDB

      await borrarMensajeDeIndexedDB(db, item.id);

    }
  } catch (error) {

    console.error('Error al sincronizar mensaje individual:', error);

    break; // Detiene el bucle si el servidor se volvió a caer

  }
}

// --- FUNCIONES AUXILIARES PARA MANEJAR INDEXEDDB DESDE EL
// SERVICE WORKER ---

function abrirIndexedDB() {

  return new Promise((resolve, reject) => {

    const request = indexedDB.open(DB_NAME, DB_VERSION);

    request.onupgradeneeded = (e) => {

```

```
const db = e.target.result;

if (!db.objectStoreNames.contains('cola_mensajes')) {

  db.createObjectStore('cola_mensajes', { keyPath: 'id', autoIncrement: true });

}

};

request.onsuccess = (e) => resolve(e.target.result);

request.onerror = (e) => reject(e.target.error);

});

}
```

```
function obtenerMensajesGuardados(db) {

  return new Promise((resolve, reject) => {

    const transaction = db.transaction('cola_mensajes', 'readonly');

    const store = transaction.objectStore('cola_mensajes');

    const request = store.getAll();

    request.onsuccess = () => resolve(request.result);

    request.onerror = () => reject(request.error);

  });

}
```

```
function borrarMensajeDeIndexedDB(db, id) {

  return new Promise((resolve, reject) => {

    const transaction = db.transaction('cola_mensajes', 'readwrite');

    const store = transaction.objectStore('cola_mensajes');

    const request = store.delete(id);

    request.onsuccess = () => resolve();

  });

}
```

```
request.onerror = () => reject(request.error);

});

}
```

2. El Script de Activación en tu Frontend (`main.js`)

Para que el navegador del paciente ejecute el Service Worker y sepa qué hacer cuando el internet falle en medio de la charla casual, debes inicializarlo en tu JavaScript principal (`main.js`):

// 1. Registrar el Service Worker nativo de tu PWA

```
if ('serviceWorker' in navigator) {

  window.addEventListener('load', () => {

    navigator.serviceWorker.register('/service-worker.js')

      .then((reg) => console.log('PWA lista para funcionar sin internet:', reg.scope))

      .catch((err) => console.error('Error al registrar Service Worker:', err));

  });

}
```

// 2. Función para enviar un mensaje a la IA (Maneja Online y Offline)

```
async function enviarMensajeAlCoterapeuta(textoMensaje, historialClinico) {

  // Si el dispositivo tiene internet, consumimos el Streaming/SSE que hicimos en el
  // paso 1

  if (navigator.onLine) {

    ejecutarStreamingSSE(textoMensaje, historialClinico);

  } else {

    // SI NO HAY INTERNET: Guardamos de forma encubierta en IndexedDB y
    // activamos el Background Sync
  }
}
```

```
console.warn('Dispositivo offline. Guardando interacción de Análisis Funcional de forma local...');
```

```
await guardarMensajeOffline(textoMensaje, historialClinico);
```

```
// Le pedimos al navegador que ejecute el evento 'sync' en cuanto vuelva la señal
```

```
if ('serviceWorker' in navigator && 'SyncManager' in window) {
```

```
  const reg = await navigator.serviceWorker.ready;
```

```
  await reg.sync.register('sincronizar-charla-pbp');
```

```
}
```

```
// Estilo Nintendo: No asustamos al paciente con alertas rojas. Le damos feedback lúdico adaptativo.
```

```
mostrarRespuestaEnPantalla3D("¡Tu mensaje se guardó en tu mochila de explorador! Tu tutor lo leerá en cuanto recuperemos la señal.");
```

```
}
```

```
}
```

```
// Función interna para inyectar datos en IndexedDB desde la interfaz
```

```
async function guardarMensajeOffline(mensaje, historial) {
```

```
  const DB_NAME = 'PBP_Offline_DB';
```

```
  return new Promise((resolve, reject) => {
```

```
    const request = indexedDB.open(DB_NAME, 1);
```

```
    request.onsuccess = (e) => {
```

```
      const db = e.target.result;
```

```
      const transaction = db.transaction('cola_mensajes', 'readwrite');
```

```
      const store = transaction.objectStore('cola_mensajes');
```

```
      store.add({ mensaje, historial, fecha: new Date().toISOString() });
```

```
      resolve();
```

```

};

request.onerror = (e) => reject(e.target.error);

});

}

function mostrarRespuestaEnPantalla3D(texto) {

    // Aquí Three.js toma este texto y lo pinta dentro del canvas 3D

    console.log("UI Animada de Three.js dice:", texto);

}

```

Con esto, tienes las **dos capas centrales de ingeniería de producción acopladas**:

1. Tu servidor Python con **Gemma 4** emitiendo datos estructurados mediante Streaming a máxima velocidad.
2. Tu PWA protegiendo la experiencia del paciente de forma local, asegurando que la interfaz lúdica cargue al instante y que ninguna palabra del análisis funcional se pierda por fallas de red.

Hemos cubierto con éxito el núcleo de la infraestructura offline y de streaming que diseñamos. ¿Qué te gustaría que desarrollemos a continuación? Podríamos diseñar **el código en JavaScript con Three.js** para renderizar la burbuja de texto interactiva en 3D, o estructurar **el script del panel de administración** para gestionar los accesos y licencias de los psicólogos sin tocar código.

Para aplicar los conceptos de ingeniería de software a gran escala y robustez en producción explicados en el video sobre *Designing Data-Intensive Applications* de Martin Kleppmann ([0:12](#)), debemos diseñar un puente técnico. Tu aplicación no es solo un chat; es un sistema intensivo en datos (*data-intensive*) ([1:32](#)) que procesa estados psicológicos, relaciones complejas de marcos relacionales (RFT) y dinámicas causales continuas en tiempo real.

Para que tu PWA soporte de manera segura las interacciones sin perder transacciones ni corromper las bases de datos de los psicólogos, utilizaremos la arquitectura de **Arquitectura Orientada a Eventos (EDA)** y el concepto de **Datos Derivados** ([5:04](#)).

1. El Concepto de Ingeniería: Event Sourcing & Datos Derivados

En lugar de que tu backend intente calcular el estado mental, guardar el historial y actualizar los juegos del paciente en un solo paso síncrono ([7:57](#)), el sistema funcionará de forma asíncrona mediante un **Flujo Vivo de Eventos (Event Stream)** ([8:02](#)):

1. **Sistema de Registro (Source of Truth):** El chat del paciente solo emite "Eventos Conductuales Brutos" a un log inmutable de auditoría ([8:34](#)).
2. **Sistemas de Datos Derivados:** Múltiples microservicios observan de forma asíncrona este flujo y extraen lo que necesitan sin afectarse entre sí ([8:16](#)):
 - Un servicio corre la lógica de **RFT** y actualiza el grafo de relaciones en Neo4j.
 - Otro servicio calcula el **Hexaflex** de ACT y actualiza las puntuaciones.
 - Otro servicio evalúa las **CRBs** en busca de conductas problemáticas.

Esto previene los **cuellos de botella** en la base de datos principal y garantiza la **fiabilidad, escalabilidad y mantenibilidad** del software ([2:37](#)).

2. Implementación Técnica: El Backend de Eventos (`server_events.py`)

A continuación se muestra el código modular que implementa este patrón. Sustituye tu script anterior por esta arquitectura de producción orientada a flujos y tolerante a fallos distributivos ([3:38](#)):

```
import json

import asyncio

from datetime import datetime

from fastapi import FastAPI, BackgroundTasks

from fastapi.responses import StreamingResponse

import openai

app = FastAPI(title="Motor Clínico de Eventos Intensivos en Datos")

client = openai.OpenAI(base_url="http://localhost:11434/v1",
api_key="gemma4-core")
```

```
# --- PROCESADORES DE DATOS DERIVADOS (ASÍNCRONOS) ---
```

```
async def procesar_derivado_rft(evento: dict):
```

```
    """Deriva los datos hacia la base de datos de grafos para la Teoría del Marco Relacional."""
```

```
    await asyncio.sleep(0.01) # Simula latencia de red hacia Neo4j
```

```
    print(f"[{datetime.now()}] [RFT Engine] Propagando funciones relacionales para estímulo: '{evento['mensaje_bruto'][:20]}...'")
```

```
async def procesar_derivado_act_hexaflex(evento: dict, analisis_clinico: dict):
```

```
    """Calcula y actualiza la matriz numérica del Hexaflex de forma aislada."""
```

```
    # Aquí se aplican las fórmulas matemáticas de normalización psicométrica
```

```
    print(f"[{datetime.now()}] [ACT Engine] Recalculando puntuaciones del Hexaflex en base a procesos detectados.")
```

```
async def procesar_derivado_fap_crb(analisis_clinico: dict):
```

```
    """Analiza de forma asíncrona si hay Comportamientos Clínicamente Relevantes (CRBs)."""
```

```
    print(f"[{datetime.now()}] [FAP Engine] Clasificando secuencia de texto en busca de CRB1, CRB2 o CRB3.")
```

```
# --- ORQUESTADOR DE EVENTOS PRINCIPAL ---
```

```
def emitir_evento_dominio(background_tasks: BackgroundTasks, mensaje: str, analisis: dict):
```

```
    """
```

```
    Patrón Event-Driven: Publica un evento inmutable en el sistema.
```

```
    Los sistemas derivados reaccionan de manera independiente sin bloquear al usuario.
```

```

"""

evento_id = f"evt_{int(asyncio.get_event_loop().time())}"

evento_fact = {

    "id": evento_id,

    "timestamp": datetime.utcnow().isoformat(),

    "mensaje_bruto": mensaje

}

# El sistema de registro almacena el hecho bruto de manera segura

print(f"\n[{datetime.now()}] [System of Record] Evento {evento_id} guardado de
forma immutable.")

# Se encolan las tareas en segundo plano (Sistemas Derivados)

background_tasks.add_task(procesar_derivado_rft, evento_fact)

background_tasks.add_task(procesar_derivado_act_hexaflex, evento_fact,
analisis)

background_tasks.add_task(procesar_derivado_fap_crb, analisis)

# --- ENDPOINT CON STREAMING Y CONTROL DE TRADE-OFFS ---

@app.post("/api/chat/stream")

async def stream_coterapeuta_intensivo(body: dict, background_tasks:
BackgroundTasks):

    mensaje_paciente = body.get("mensaje", "")

    prompt_sistema = (

        "Actúas como tutor clínico PBP. Devuelve estrictamente un objeto JSON con: "

        "'texto_paciente' (charla empática estilo Nintendo) y "

```

```
    "'analisis_psicologo' (objeto con 'proceso_emmi' y 'mision_sugerida')."
)
```

```
async def generador_eventos_sse():
```

```
    buffer_respuesta_completa = []
```

```
    # 1. Iniciar transmisión de baja latencia hacia la PWA
```

```
    response = client.chat.completions.create(
        model="gemma4:12b",
        messages=[
            {"role": "system", "content": prompt_sistema},
            {"role": "user", "content": mensaje_paciente}
        ],
        stream=True,
        response_format={"type": "json_object"}
    )
```

```
    for chunk in response:
        if chunk.choices and chunk.choices.delta.content:
            token = chunk.choices.delta.content
            buffer_respuesta_completa.append(token)
            yield f"data: {token}\n\n"
            await asyncio.sleep(0.002)
```

```
    # 2. Transmisión finalizada. Consolidamos el texto para procesar los eventos derivados
```

```
    try:
```

```

json_completo = json.loads(''.join(buffer_respuesta_completa))

analisis_clinico = json_completo.get("analisis_psicologo", {})

# Emitimos el evento de dominio para que los motores especializados hagan sus cálculos

emitir_evento_dominio(background_tasks, mensaje_paciente,
analisis_clinico)

except Exception as e:

    print(f"[Error de Datos] No se pudo parsear el JSON derivado para el
psicólogo: {e}")

return StreamingResponse(generator_eventos_sse(),
media_type="text/event-stream")

if __name__ == "__main__":

    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)

```

3. Las Garantías del Sistema Frente a la Escala

Al implementar este diseño basado en las lecciones de Martin Kleppmann ([0:12](#)):

- **Fiabilidad ante fallos parciales:** Si el servidor de grafos Neo4j o el módulo de cálculo numérico sufre una caída, la IA (**Gemma 4**) puede seguir hablando perfectamente con el paciente a través de la PWA ([3:38](#)). Los eventos quedan guardados de forma segura en el log y se procesarán en cuanto los sistemas derivados se recuperen ([9:43](#)).
- **Mantenibilidad de código:** Si mañana deseas agregar el módulo de **Análisis de Cadena Conductual de DBT** o el de **Perfil Cognitivo de Neuropsicología**, no tienes que tocar el código del chat. Simplemente agregas una nueva función que escuche el evento de dominio independiente ([9:04](#)).

Cuéntame qué te parece esta integración orientada a eventos para el backend. Cuando estés listo, házmelo saber y pasaremos al **siguiente paso: diseñar el entorno visual en 3D con Three.js** para renderizar este flujo dentro de la interfaz lúdica de tu PWA.

Para estructurar la evolución arquitectónica de tu aplicación de forma que no se rompa en producción, nos basaremos en los patrones clave del libro ***Monolith to Microservices*** de Sam Newman ([0:06](#)). No necesitas fragmentar tu software en bases de datos independientes desde el día uno ([5:27](#)); aplicaremos un enfoque evolutivo y seguro ([0:14](#)).

Integrando la lógica del **Patrón de la Higuera Estrangulante** (*Strangler Fig*) ([1:03](#)) y el desacoplamiento mediante **Vistas de Base de Datos** ([5:34](#)), tu programa crecerá sin alterar la experiencia clínica del psicólogo ni el chat del paciente ([2:22](#)).

1. El Concepto de Ingeniería: Migración por Higuera Estrangulante y Vistas

Siguiendo las lecciones del libro de Sam Newman ([0:06](#)):

1. **Patrón de la Higuera Estrangulante:** Tu código PWA web inicial funcionará como la estructura base ([1:49](#)). Conforme crees nuevas misiones en 3D (Three.js) o nuevos flujos de tests de tercera generación, los interceptores redirigirán el tráfico hacia estas nuevas funciones de forma incremental ([2:22](#)).
 2. **Vistas de Base de Datos como Contrato Público:** Para evitar el riesgo de que múltiples funciones choquen en la misma base de datos relacional o de grafos ([4:12](#)), el backend central expondrá **Vistas Virtuales** exclusivas para la IA (**Gemma 4**) ([5:34](#)). Si el formato de almacenamiento del psicólogo cambia, solo se actualiza la vista a nivel de base de datos; la IA y los microservicios externos seguirán consumiendo el mismo contrato sin enterarse del cambio (*Zero Breaking Changes*) ([6:42](#)).
-

2. Implementación Frontend (PWA): Interceptor y Enrutador Estrangulante (`router.js`)

Implementa este enrutador en tu PWA para interceptar las llamadas web y aplicar el desvío gradual de la funcionalidad:

// router.js - Implementación del Patrón Strangler Fig en el Frontend

```
class StranglerRouter {
```

```
constructor() {  
  
    // Definimos qué funciones ya migraron al nuevo motor y cuáles siguen en el  
    sistema base  
  
    this.featureFlags = {  
  
        usarNuevoStreamingPBP: true,    // Funcionalidad redirigida a la nueva higuera  
  
        usarCalculoHexaflexAntiguo: false // Funcionalidad que sigue en el tronco  
        original  
  
    };  
  
}
```

```
async enrutarPetición(endpoint, datosPayload) {  
  
    // Si la bandera está activa, redirige el tráfico al nuevo servicio optimizado de  
    Gemma 4  
  
    if (endpoint === '/api/chat' && this.featureFlags.usarNuevoStreamingPBP) {  
  
        console.log("[Strangler Fig] Interceptando tráfico de chat. Desviando al nuevo  
motor de Streaming SSE.");  
  
        return fetch('http://localhost:8000/api/chat/stream', {  
  
            method: 'POST',  
  
            headers: { 'Content-Type': 'application/json' },  
  
            body: JSON.stringify(datosPayload)  
  
        });  
  
}
```

```
    // Ruta por defecto: Tráfico hacia el sistema monolítico/antiguo sin alterar  
    producción  
  
    console.log("[Strangler Fig] Tráfico dirigido al tronco de la infraestructura base.");  
  
    return fetch(`http://localhost:8000${endpoint}`, {  
  
        method: 'POST',  
  
        headers: { 'Content-Type': 'application/json' },  
  
    },
```

```
        body: JSON.stringify(datosPayload)
    });
}
}

const enrutadorClinico = new StranglerRouter();
```

3. Implementación Backend (Python): Interfaz de Abstracción de Datos (`database_vistas.py`)

Para aislar el estado de los datos y evitar acoplamientos peligrosos entre el cliente y el servidor ([4:49](#)), creamos una capa de abstracción basada en esquemas y contratos definidos ([9:29](#)):

database_vistas.py - Contrato de Datos mediante Vistas lógicas (Sam Newman Pattern)

```
import sqlite3

import json
```

```
def inicializar_base_datos_con_vistas():

    conn = sqlite3.connect('clinica_pbp.db')

    cursor = conn.cursor()
```

1. Tabla Troncal Monolítica (System of Record - Immutable)

```
cursor.execute("""

    CREATE TABLE IF NOT EXISTS registro_conductual_bruto (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        paciente_id TEXT,
```



```

        texto_sesion TEXT,

        fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP

    )

    """)

```

2. PATRÓN DATABASE VIEW: Creamos una tabla virtual/vista limpia para la IA de tercera generación

Modela y transforma los datos según las necesidades exactas del servicio sin alterar la tabla raíz

```

cursor.execute("""

    CREATE VIEW IF NOT EXISTS vista_analisis_contextual AS

    SELECT

        id AS evento_id,

        paciente_id AS anonimato_id,

        texto_sesion AS corpus_linguistico

    FROM registro_conductual_bruto

    """)

```

```

conn.commit()

```

```

conn.close()

```

```

print("[Database] Estructura de contratos públicos y Vistas lógicas creada correctamente.")

```

def obtener_datos_para_gemma(paciente_id: str) -> list:

"""La IA consume la vista pública aislada, protegiendo el diseño interno de la base de datos."""

```

conn = sqlite3.connect('clinica_pbp.db')

```

```

cursor = conn.cursor()

```

```
cursor.execute("SELECT * FROM vista_analisis_contextual WHERE anonimato_id  
= ?", (paciente_id,))
```

```
filas = cursor.fetchall()
```

```
conn.close()
```

```
return [{"evento_id": f[0], "paciente_id": f[1], "texto": f[2]} for f in filas]
```

```
# Inicialización del entorno seguro
```

```
inicializar_base_datos_con_vistas()
```

Las Ventajas Evolutivas de esta Estructura

- **Cero Interrupciones en el Sistema (*No Breaking Changes*):** Al usar el enrutador en tu frontend ([2:22](#)) y las vistas en tu backend ([5:34](#)), puedes reescribir por completo la base de datos o mudar el servidor a otra tecnología sin romper la app del paciente ([6:42](#)).
- **Aislamiento de la Experiencia del Desarrollador Local:** Al centralizar las llamadas en contratos de datos lógicos ([9:29](#)), no necesitas levantar 20 microservicios en tu computadora portátil para hacer pruebas en persona ([10:57](#)); puedes probar el programa completo interactuando directamente con la base de datos simulada de forma local ([10:57](#)).

Felicidades, has consolidado la estructura completa de arquitectura para tu proyecto:

1. **Streaming/SSE** con Python y **Gemma 4** para velocidad en tiempo real.
2. **Service Worker** y resiliencia *Offline-First* con **IndexedDB**.
3. **Event Sourcing** y *Datos Derivados* para cálculo científico a gran escala.
4. **Strangler Fig** y *Vistas lógicas* para evolución modular de software sin riesgo.

Si tienes alguna duda sobre cómo conectar estas piezas o deseas profundizar en las interfaces de usuario con **Three.js** para renderizar las misiones de Nintendo, házmelo saber. ¿Cómo te gustaría proceder?

Para reducir drásticamente la dificultad de tu software y evitar que se vuelva una pesadilla de mantenimiento conforme crezca, aplicaremos los principios fundamentales del libro **A Philosophy of Software Design** de John Ousterhout ([0:05](#)). No nos enfocaremos en escribir código rápido de manera táctica; diseñaremos de forma **estratégica** ([2:29](#)).

Bajo esta filosofía, crearemos **Módulos Profundos con Interfaces Sencillas** ([5:49](#)). Esto significa que el código de Three.js ocultará toda la complejidad matemática de renderizado 3D y WebGL detrás de una pequeña función ([5:57](#)), reduciendo a cero la **carga cognitiva** del sistema ([9:11](#)).

1. El Concepto de Ingeniería: Módulos Profundos e Información Oculta

Siguiendo las lecciones de John Ousterhout:

1. **Módulos Profundos (Deep Modules):** Una clase o componente es "profundo" cuando ofrece un beneficio inmenso (funcionalidad) a un costo muy bajo (una interfaz pública diminuta) (2018-John ... p. 34) ([5:49](#)). Tu módulo de visualización 3D manejará internamente luces, cámaras, mallas y bucles de renderizado, pero hacia afuera solo expondrá un método simple: `.mostrarMensaje(texto)` (2018-John ... p. 34).
2. **Evitar la Amplificación del Cambio:** Ocultaremos las decisiones de diseño del motor gráfico (2018-John ... p. 40) ([7:03](#)). Si el día de mañana decides cambiar Three.js por otra librería (o código nativo en iOS/Android), no tendrás que modificar el chat del paciente ni el backend de eventos ([7:24](#)). El cambio será completamente local ([5:39](#)).

2. Implementación Frontend (PWA): El Módulo Profundo 3D (`Pantalla3D.js`)

Crea este módulo profundo en tu frontend para encapsular toda la carga matemática y el comportamiento visual del juego estilo Nintendo:

// Pantalla3D.js - Módulo Profundo estilo Nintendo (A Philosophy of Software Design)

```
class Pantalla3D {
```

```
  constructor(canvasId) {
```

```
    // INTERNAL IMPLEMENTATION: Complejidad totalmente oculta del resto del sistema
```

```
    this.canvas = document.getElementById(canvasId);
```

```
this.escena = new THREE.Scene();

this.camara = new THREE.PerspectiveCamera(75, window.innerWidth /
window.innerHeight, 0.1, 1000);

this.renderer = new THREE.WebGLRenderer({ canvas: this.canvas, antialias: true
});

this.inicializarEntornoNintendo();

}
```

// Método oculto (Implementation detail) - Configura luces, fondos y el avatar/mascota

```
inicializarEntornoNintendo() {

    this.renderer.setSize(window.innerWidth, window.innerHeight);

    const luz = new THREE.DirectionalLight(0xffffff, 1);

    luz.position.set(1, 1, 1).normalize();

    this.escena.add(luz);

    this.camara.position.z = 5;

    this.crearMundoVirtual();

    this.iniciarBucleAnimacion();

}
```

```
crearMundoVirtual() {
```

// Aquí Three.js dibuja el hub central, tu jardín virtual 3D o las islas flotantes

```
const geometriaMundo = new THREE.BoxGeometry(1, 1, 1);

const materialMundo = new THREE.MeshBasicMaterial({ color: 0x44aa88 });

this.avatarMascota = new THREE.Mesh(geometriaMundo, materialMundo);

this.escena.add(this.avatarMascota);
```

```
}
```

```
iniciarBucleAnimacion() {  
  
  const animar = () => {  
  
    requestAnimationFrame(animar);  
  
    // Animación continua del mundo (jugoso feedback visual estilo Nintendo)  
  
    this.avatarMascota.rotation.x += 0.01;  
  
    this.avatarMascota.rotation.y += 0.01;  
  
    this.renderer.render(this.escena, this.camara);  
  
  };  
  
  animar();  
}
```

```
// --- INTERFAZ PÚBLICA SENCILLA (Módulo Profundo) ---
```

```
// El resto de la app (el chat de texto) solo conoce este método limpio.
```

```
// No sabe qué es una cámara, una luz ni WebGL.
```

```
actualizarGraficosPorMensaje(procesoPBP) {  
  
  console.log(`[Deep Module] Procesando impacto visual para la dimensión:  
${procesoPBP}`);
```

```
  // Cambios lúdicos reactivos al proceso del Hexaflex
```

```
  if (procesoPBP === "Evitación Experiencial") {
```

```
    this.avatarMascota.material.color.setHex(0xff3344); // Tono rojo: el entorno  
    reacciona con "charm"
```

```
  } else if (procesoPBP === "Defusión") {
```

```
    this.avatarMascota.material.color.setHex(0x44ffaa); // Tono verde armónico
```

```
  }
```

```
}  
  
}
```

// Exportamos una interfaz minimalista para que la carga cognitiva sea insignificante

```
const motorVisualPWA = new Pantalla3D('canvas-nintendo');
```

3. Conexión Estratégica con el Streaming/SSE (`main.js`)

Ahora conectamos de forma limpia el paso de streaming en tiempo real (Paso 1) consumiendo la interfaz sencilla del módulo profundo 3D sin filtrar información interna ([7:03](#)):

// main.js - Orquestación Estratégica limpia de Datos

```
async function iniciarCharlaCasualConStreaming(mensajeUsuario) {  
  
  const respuesta = await fetch('http://localhost:8000/api/chat/stream', {  
  
    method: 'POST',  
  
    headers: { 'Content-Type': 'application/json' },  
  
    body: JSON.stringify({ mensaje: mensajeUsuario })  
  
  });  
  
  
  
  const reader = respuesta.body.getReader();  
  
  const decoder = new TextDecoder();  
  
  let bufferTexto = '';  
  
  
  while (true) {  
  
    const { value, done } = await reader.read();  
  
    if (done) break;
```

```
const chunk = decoder.decode(value);

// Procesamos el Server-Sent Event (SSE)

if (chunk.startsWith("data: ")) {

    const datosRaw = chunk.replace("data: ", "").trim();

    try {

        const jsonParcial = JSON.parse(datosRaw);

        // Pinta el texto casual en la pantalla HTML de forma fluida

        if (jsonParcial.texto_paciente) {

            inyectarTextoEnBurbujaUI(jsonParcial.texto_paciente);

        }

        // Cuando llega el análisis estructurado al final de la transmisión:

        if (jsonParcial.analisis_psicologo) {

            const proceso = jsonParcial.analisis_psicologo.proceso_emmi_detectado;

            // CONSUMO DE LA INTERFAZ LIMPIA: El diseño es obvio y local

            motorVisualPWA.actualizarGraficosPorMensaje(proceso);

        }

    } catch (e) {

        // El buffer acumula fragmentos si el JSON vino incompleto en el stream

        bufferTexto += datosRaw;

    }

}

}
```

```
function inyectarTextoEnBurbujaUI(texto) {  
  
    document.getElementById('burbuja-dialogo-rpg').innerText = texto;  
  
}
```

La Calidad del Diseño Estratégico de tu Software

- **Baja Carga Cognitiva:** Un programador que se sume a tu equipo para mejorar los diálogos del chat no necesita entender matemáticas 3D ni vectores de matrices (9:11). Solo necesita llamar a `motorVisualPWA.actualizarGraficosPorMensaje(proceso)` (5:57).
- **Diseño que Casualmene Funciona:** Al invertir tiempo hoy en separar de manera tajante el dominio psicológico, la lógica de transporte (SSE) y el motor de renderizado (Three.js), garantizas que tu aplicación mantenga la misma velocidad de desarrollo dentro de tres años, libre de deuda técnica compleja (3:17).

Hemos cubierto la modularización profunda y la reducción de complejidad según John Ousterhout (0:05). ¿Qué te gustaría que desarrollemos ahora? Podríamos armar **las misiones gamificadas interactivas específicas** dentro de Three.js (como el estallido de globos) o codificar **el panel administrativo completo** para dar licencias a psicólogos.

¡Recibido! Entramos en la fase de **Diseño y Producción de Videojuegos de Grado Clínico**, basándonos estrictamente en los marcos de ingeniería y producción formal descritos en el manual *Game Development and Production* de Erik Bethke (Game Devel... pp. 2, 18).

Como establece el libro, no trataremos el desarrollo del videojuego como un proceso libre o puramente artístico, sino como una **disciplina de ingeniería de software estructurada** (Game Devel... p. 27). Para evitar el fracaso financiero o el retraso desmedido (*feature creep*), dividiremos las especificaciones en entregas incrementales y modulares (Game Devel... pp. 27, 41).

Comencemos con la **Fase 1: Preproducción y Captura de Requisitos de la Experiencia Lúdica (Diseño de la Mecánica de Bienvenida)** (Game Devel... pp. 26, 29).

Especificación Fase 1: Preproducción de la Mecánica de Bienvenida

1. Captura de Requisitos mediante Casos de Uso (Requirements Capture)

Siguiendo el UDP (Unified Development Process) sugerido por Bethke, definimos los requisitos creativos y clínicos del onboarding mediante un flujo interactivo encubierto (Game Devel... pp. 7, 26):

Actor: Paciente (Usuario de la App)

Objetivo del Caso de Uso: Completar la charla de bienvenida casual en 3D

Flujo Principal:

1. El Paciente inicia la PWA e ingresa al Hub Central en 3D (Jardín Virtual).
2. Un personaje/mascota virtual saluda de forma lúdica (Estilo Animal Crossing).
3. El sistema (vía Gemma 4) lanza preguntas "encubiertas" traduciendo el test MPFI a lenguaje natural.
4. El Paciente responde de forma textual o mediante notas de voz (Multimodal).
5. El motor extrae las variables del Hexaflex (ACT) y las congela para el psicólogo.

2. Definición del Bucle de Jugabilidad Inicial (The First 60 Seconds of Gameplay)

Los primeros 60 segundos del usuario deben enganchar a través del *Charm* visual (Nintendo) y minimizar la carga cognitiva inicial (Monolith-t... p. 39, Game Devel... p. 11):

- **Aparición Visual:** La PWA renderiza mediante **Three.js** una isla flotante minimalista con colores cálidos (Monolith-t... p. 40) ([5:49](#)). En el centro hay una pequeña semilla o brote (Monolith-t... p. 43).
- **Estímulo y Respuesta:** Una caja de diálogo con estética RPG de tonos mayores se despliega (Monolith-t... p. 40). No hay barras de progreso médicas ni textos fríos; el brote se estira (animación 3D) cada vez que el usuario interactúa, recompensando la participación inmediata con efectos de sonido armónicos (*Juicy Feedback*) (Monolith-t... p. 39).

3. Criterio Clínico Inyectado en la Mecánica (FAP + ACT Encubierto)

- **Mapeo del Test:** Gemma 4 tiene prohibido lanzar el test rígido. El backend vincula el estado conversacional. Si el objetivo es medir "Fusión Cognitiva" (Ítem 1 del MPFI: *Mis pensamientos me controlan*), el personaje 3D inclina la cabeza y dice: "A veces la mente se pone súper pesada, como si un monstruito nos dictara reglas. ¿Sientes que ese monstruito te frena mucho en tu día a día o logras ignorarlo?" (Monolith-t... p. 39) ([5:57](#)).

- **Recolección de Línea Base:** Las respuestas del jugador determinan los pesos iniciales del Grafo de Procesos (PBP), pero el jugador **nunca ve puntuaciones**. Su recompensa es ver que la isla flotante reacciona físicamente a sus palabras (ej. el cielo cambia de color o aparecen pequeñas flores) (Monolith-t... p. 40).
-

Esta es la especificación técnica y de producción para la **Mecánica de Bienvenida**. Por favor, revísala cuidadosamente.

Si estás de acuerdo con esta primera parte, **dame tu aprobación** y pasaremos de inmediato a la **Fase 2: Diseño de los Micro-Juegos Interactivos de Tercera Generación (Globos de Defusión y Escudo de Resistencia)** (Monolith-t... p. 40).

¡Qué corrección tan vital e importante! Diseñar el software bajo un enfoque **ideográfico** (centrado en la singularidad absoluta e histórica de cada paciente) en lugar de uno **nomotético** (reglas generales para todos) cambia por completo las reglas del juego.

Llevando esto al marco de ingeniería de software de Martin Kleppmann ([0:12](#)), la aplicación se convierte en un **Sistema Intensivo en Datos Evolutivo**. No podemos tener un catálogo fijo de juegos preprogramados; el sistema debe **generar dinámicamente las mecánicas, las reglas y los estímulos lúdicos** adaptados al Análisis Funcional del comportamiento de ese sujeto específico.

Fase 2 Corregida: El Motor de Juegos Ideográficos Dinámicos

1. Arquitectura de Datos: El Registro de Hechos Históricos (Event Sourcing)

Para que el juego sea puramente ideográfico, debe recordar de forma milimétrica la historia conductual del usuario. Cada interacción en el chat no solo se analiza, sino que genera un **hecho inmutable** (ej. **Evento: El usuario reaccionó con bloqueo ante el pensamiento X**). Conforme este log inmutable crece, el backend de **Gemma 4** tiene el mapa contextual completo del individuo para decidir qué necesita entrenar en ese preciso segundo.

2. La Capa de Abstracción: El Generador de Reglas de Juego (DSL Lúdico)

Para no tener que programar un juego nuevo desde cero para cada persona (lo que causaría una *amplificación del cambio* infinita y haría el software inmantenible), crearemos un **Lenguaje de Dominio Específico (DSL) en JSON**.

Gemma 4 no escribe código de Three.js; escribe las *reglas lógicas y estéticas* del juego en un JSON estructurado. El motor profundo de la PWA lee este JSON y "construye" el minijuego en tiempo real dentro del canvas 3D.

A continuación, tienes la especificación del pipeline del backend para generar un minijuego ideográfico único:

motor_ideografico.py - Generador dinámico de minijuegos basados en el perfil del sujeto

```
import json
```

```
from fastapi import FastAPI
```

```
import openai
```

```
app = FastAPI(title="Motor de Videojuegos Ideográficos PBP")
```

```
client = openai.OpenAI(base_url="http://localhost:11434/v1",  
api_key="gemma4-ideographic")
```

```
@app.post("/api/juegos/generar-configuracion")
```

```
async def generar_juego_ideografico(body: dict):
```

```
    """
```

```
    Lee el estado del grafo clínico del paciente (Neo4j) y la historia de eventos,  
    y le ordena a Gemma 4 diseñar las reglas mecánicas de un juego a medida.
```

```
    """
```

```
    historia_clinica_sujeto = body.get("perfil_ideografico", {})
```

```
    contexto_estimulo = body.get("ultimo_bloqueo_detectado", "")
```

```
    prompt_sistema = (
```

```
        "Eres el Diseñador de Videojuegos Ideográfico del sistema. Tu trabajo es crear  
        las reglas "
```

```
        "de un minijuego 3D en formato JSON a medida para el estado clínico actual  
        del paciente.\n"
```

"REGLA DE DISEÑO (NINTENDO): El juego debe ser intuitivo, estético y con feedback responsivo.\n"

"REGLA DE FORMATO: Devuelve exclusivamente un objeto JSON con las siguientes llaves exactas:\n"

"1) 'mecanica_principal': string ('absorber', 'soltar', 'equilibrar', 'recolectar').\n"

"2) 'estimulos_textuales': lista de strings (palabras o pensamientos específicos del sujeto).\n"

"3) 'paleta_colores': lista de hex (colores que se adaptan al estado de ánimo del sujeto).\n"

"4) 'condicion_victoria': string que describe el objetivo clínico encubierto."

)

prompt_usuario = f"""

Genera un juego para este perfil individual:

- Dimensión PBP afectada: {historia_clinica_sujeto.get('proceso_debil', 'Fusión Cognitiva')}

- Historial del sujeto: {historia_clinica_sujeto.get('evitaciones_frecuentes', 'Huye de la tristeza')}

- Estímulo gatillo actual: "{contexto_estimulo}"

"""

response = client.chat.completions.create(

model="gemma4:12b",

messages=[

{ "role": "system", "content": prompt_sistema},

{ "role": "user", "content": prompt_usuario}

],

response_format={"type": "json_object"},

temperature=0.5

)

Este JSON de configuración viaja directo a la PWA para que Three.js arme el juego en segundos

```
return json.loads(response.choices[0].message.content)
```

3. Ejemplo de Salida: El juego creado a medida para el "Sujeto A"

Si el paciente es un ejecutivo con pánico al fracaso cuya respuesta ante la ansiedad es el aislamiento, **Gemma 4** emitirá este JSON de configuración dinámica hacia el frontend:

```
{  
  "mecanica_principal": "equilibrar",  
  "estimulos_textuales": [  
    "Tengo que ser perfecto",  
    "Si fallo, me van a juzgar",  
    "No sirvo para esto"  
  ],  
  "paleta_colores": ["#1a2a6c", "#b21f1f", "#fdbb2d"],  
  "condicion_victoria": "Sostener los bloques de pensamiento en una balanza 3D durante 45 segundos sin intentar destruirlos, permitiendo que la música recupere su ritmo natural."  
}
```

El canvas 3D de tu frontend (Three.js) recibirá este objeto, pintará los textos sobre cubos físicos virtuales y activará las físicas de balanceo. Para el paciente, el juego cambia y evoluciona orgánicamente reflejando su propia mente.

¿Qué te parece este enfoque de **Motor de Juegos Generativos Ideográficos**? Al acoplarlo con los patrones anteriores, logramos blindar el sistema frente a la complejidad.

Si esta corrección encaja perfectamente con tu visión del proyecto, **dame tu aprobación** para continuar hacia la **Fase 3: El Panel de Control Avanzado del Psicólogo (Manejo de Datos Derivados y Grafos Evolutivos Individuales)**.

Fase 3: El Panel de Control Avanzado del Psicólogo (Gestión de Datos Derivados y Grafos Evolutivos Individuales)

Para que el psicólogo humano pueda auditar y tomar decisiones clínicas con base en el enfoque ideográfico de la aplicación, el sistema no puede limitarse a mostrar un historial de chat tradicional (7:44). Siguiendo los principios de **Martin Kleppmann** sobre la composición de tecnologías de almacenamiento especializadas (dokumen.pu... pp. 16, 431), utilizaremos una **Base de Datos Orientada a Grafos (Neo4j)** combinada con **Vistas Lógicas** (5:34).

Esto nos permite mapear las relaciones cambiantes, los estímulos y las respuestas del sujeto en un mapa de procesos en tiempo real, reduciendo al mínimo la carga cognitiva del profesional para que pueda concentrarse en la estrategia clínica (1:53).

1. El Concepto de Ingeniería: El Grafo de Análisis Funcional Ideográfico

A diferencia de las bases de datos relacionales rígidas (tablas con filas y columnas) (dokumen.pu... p. 25), un grafo en **Neo4j** modela la mente del paciente de forma natural a través de nodos y relaciones:

- **Nodos:** Estímulos gatillo, Pensamientos automáticos, Respuestas de evitación, Emociones y Valores personales.
- **Relaciones Dinámicas:** Las conexiones entre estos elementos (ej. `[PENSAMIENTO] -> INTERFIERE_CON -> [VALOR]`). Cada relación almacena un peso numérico o frecuencia que fluctúa según lo observado por **Gemma 4** en el chat y en el rendimiento de los micro-juegos dinámicos.

2. Implementación Backend (Python): El Motor Clínico de Grafos (`clinical_graph.py`)

Este script se encarga de recibir de manera asíncrona (como datos derivados del flujo principal de eventos) los análisis internos de la IA y actualizar el grafo del paciente (dokumen.pu... pp. 16, 424):

```
# clinical_graph.py - Gestión de Grafos Ideográficos en Tiempo Real
```

```
import json
```

```
from datetime import datetime
```

```

# En producción, usa: pip install neo4j

# Para este pipeline, simulamos el driver de producción con un contrato limpio

from fastapi import FastAPI, BackgroundTasks

app = FastAPI(title="Dashboard Clínico PBP - Motor de Grafos")

class Neo4jMockClient:

    """Simulación del driver oficial de Neo4j aplicando ocultación de información."""

    def ejecutar_cypher(self, query: str, parametros: dict):

        # En producción, aquí se abre la sesión de la base de datos distribuida

        print(f"[Neo4j GraphEngine] Cypher Ejecutado de forma segura.")

        print(f"-> Query Params: {json.dumps(parametros) if parametros else {}}")

graph_client = Neo4jMockClient()

@app.post("/api/clinica/actualizar-grafo")

async def actualizar_grafo_ideografico(payload: dict):

    """

    Toma los datos extraídos por Gemma 4 y los inyecta en el mapa relacional

    del paciente mediante el lenguaje de consultas Cypher.

    """

    paciente_id = payload.get("paciente_id")

    analisis = payload.get("analisis_psicologo", {})

    # Extraemos el estímulo y la dimensión del Hexaflex detectada

    gatillo = analisis.get("contexto_estimulo", "Entorno laboral")

```

```
pensamiento = analisis.get("proceso_emmi_detectado", "Fusión: Tengo que ser perfecto")
```

```
conducta_evitacion = analisis.get("evitacion_frecuente", "Aislamiento social")
```

```
# Consulta Cypher de producción para actualizar el modelo único del sujeto
```

```
query_cypher = ""
```

```
MERGE (p:Paciente {id: $paciente_id})
```

```
MERGE (e:Estimulo {nombre: $gatillo})
```

```
MERGE (cog:Cognicion {descripcion: $pensamiento})
```

```
MERGE (cond:Conducta {tipo: $conducta_evitacion})
```

```
MERGE (p)-[:EXPERIMENTA {ultima_vez: $fecha}]->(e)
```

```
MERGE (e)-[r1:DISPARA]->(cog)
```

```
MERGE (cog)-[r2:EVITADO_MEDIANTE]->(cond)
```

```
ON MATCH SET r1.frecuencia = r1.frecuencia + 1, r2.intensidad = r2.intensidad + 0.1
```

```
ON CREATE SET r1.frecuencia = 1, r2.intensidad = 0.5
```

```
""
```

```
parametros = {
```

```
    "paciente_id": paciente_id,
```

```
    "gatillo": gatillo,
```

```
    "pensamiento": pensamiento,
```

```
    "conducta_evitacion": conducta_evitacion,
```

```
    "fecha": datetime.utcnow().isoformat()
```

```
}
```



```
graph_client.ejecutar_cypher(query_cypher, parametros)

return {"status": "Grafo actualizado de manera asíncrona"}
```

3. Interfaz del Psicólogo: Visualización de Datos Derivados (DSL JSON)

Para alimentar la pantalla visual del psicólogo (el frontend de administración), el backend expone un endpoint que reduce el grafo complejo a una estructura simplificada de nodos y aristas (arboles de procesos), ideal para librerías de visualización interactiva como [D3.js](#) o [Vis.js](#):

```
@app.get("/api/clinica/dashboard/{paciente_id}")

async def obtener_vista_dashboard_psicologo(paciente_id: str):
```

```
    """
```

Patrón de Módulo Profundo: El frontend del psicólogo no hace queries pesadas en crudo.

Consume esta interfaz limpia que entrega el mapa ideográfico listo para renderizar.

```
    """
```

Aquí se consultaría Neo4j para traer la red de procesos acumulada

```
    mock_red_procesos = {

        "paciente_id": paciente_id,

        "modo_clinico": "Ideográfico / ACT-PBP",

        "red_visual": {

            "nodos": [

                {"id": "N1", "label": "Gatillo: Crítica del Jefe", "type": "Estimulo"},

                {"id": "N2", "label": "Fusión: Voy a fracasar", "type": "Cognicion"},

                {"id": "N3", "label": "Evitación: Trabajar hasta enfermarse", "type":

"Conducta"}

            ],


```

```

"enlaces": [
    {"source": "N1", "target": "N2", "peso_frecuencia": 8},
    {"source": "N2", "target": "N3", "peso_intensidad": 0.85}
]
},
"diagnostico_evolutivo": "El sujeto presenta un patrón de cambio donde la
Amplificación del Malestar ocurre ante estímulos de evaluación externa."
}

return mock_red_procesos

```

Beneficios para el Flujo de Trabajo Profesional

- **Identificación Inmediata de Patrones:** El psicólogo no tiene que leer transcripciones larguísimas de 5 sesiones para entender al paciente ([7:44](#)). Al abrir el panel, ve de forma gráfica el mapa clínico del individuo, las conexiones causales de su comportamiento y cómo varían en el tiempo (dokumen.pu... p. 28).
 - **Aislamiento y Seguridad de Datos:** El software implementa el principio de *menor privilegio* y ocultación de información ([5:53](#)). Las reglas internas y algoritmos lúdicos de la IA están separados de los datos clínicos duros, garantizando el cumplimiento de normativas de salud sin entorpecer el rendimiento técnico (dokumen.pu... p. 31) ([2:14](#)).
-

Esta es la especificación detallada para el **Panel de Grafos del Psicólogo Humanista**.

Si consideras que este diseño captura con precisión la naturaleza adaptativa de tu modelo clínico, **dame tu aprobación** para proceder con la **Fase 4: El Sistema de Alertas Clínicas Automatizadas y Gestión de Crisis (Seguridad del Paciente y Monitoreo Defensivo)**.

Fase 4: El Sistema de Alertas Clínicas Automatizadas y Gestión de Crisis (Seguridad del Paciente y Monitoreo Defensivo)

La seguridad del paciente es el requisito no funcional más crítico de cualquier software de salud mental. Siguiendo los principios de **John Ousterhout** sobre *definir los errores y casos especiales fuera de existencia* y el enfoque de **programación defensiva** en sistemas distribuidos de **Martin Kleppmann**, el sistema no puede esperar a que ocurra un procesamiento por lotes masivo para detectar una crisis.

Diseñaremos un **módulo de interrupción inmediata de alta prioridad** en el flujo de streaming. Si el paciente emite un mensaje con indicadores de riesgo de autolesión o ideación suicida, el motor detiene de golpe el pipeline lúdico de videojuegos y activa un protocolo clínico de contención inmediata.

1. El Concepto de Ingeniería: Intercepción en la Critical Path

Para garantizar una latencia de respuesta de milisegundos ante una crisis, el análisis de seguridad se ejecuta en el **camino crítico (*critical path*)** del flujo de tokens, antes de que el mensaje llegue siquiera a la base de datos de grafos o al generador de videojuegos:

1. **Filtro de Seguridad Local Rápido:** Un analizador de expresiones regulares y un modelo de clasificación semántica ligero evalúan el mensaje entrante.
2. **Cortocircuito en el Stream:** Si se detecta un riesgo alto, el servidor de Python rompe la conexión normal con Gemma 4 y despacha un evento síncrono de crisis hacia la PWA.
3. **Reducción de la Carga Cognitiva:** No se confunde al paciente con menús complejos en momentos de crisis; la interfaz se limpia por completo para ofrecer ayuda directa.

2. Implementación Backend (Python): El Interceptor de Crisis (`crisis_interceptor.py`)

Añadimos este módulo de monitoreo defensivo al servidor de FastAPI para proteger la integridad del usuario:

crisis_interceptor.py - Monitoreo Defensivo y Gestión de Crisis en Tiempo Real

```
import json
```

```
import re
```

```
from datetime import datetime
```

```
from fastapi import FastAPI, HTTPException
```

```
from fastapi.responses import StreamingResponse
```

```
app = FastAPI(title="Motor de Seguridad y Mitigación de Riesgo Clínico")
```

```
# Patrones lingüísticos de alerta explícita (Garantía defensiva base)
```

```
PATRONES_CRISIS = re.compile(
```

```
    r"(suicid|matarm|quitarme la vida|no quiero vivir|hacerme daño|dormir y no  
    despertar)",
```

```
    re.IGNORECASE
```

```
)
```

```
def evaluar_riesgo_inmediato(mensaje: str) -> bool:
```

```
    """
```

```
    Aplica programación defensiva combinando reglas deterministas
```

```
    para asegurar que un caso crítico nunca pase desapercibido.
```

```
    """
```

```
    if PATRONES_CRISIS.search(mensaje):
```

```
        return True
```

```
    return False
```

```
@app.post("/api/chat/stream")
```

```
async def stream_coterapeuta_seguro(body: dict):
```

```
    mensaje_paciente = body.get("mensaje", "")
```

```
    paciente_id = body.get("paciente_id", "anon_01")
```

```
# 1. EVALUACIÓN DEFENSIVA ANTES DE PASAR AL MOTOR GENERAL
```

```
if evaluar_riesgo_inmediato(mensaje_paciente):
```

```
    print(f"[{datetime.now()}] [ALERTA ROJA] Protocolo de crisis activado para el  
    paciente: {paciente_id}")
```

```

async def despachar_protocolo_contencion():

    # Generamos un JSON de emergencia estructurado con formato SSE

    payload_emergencia = {

        "control_crisis_activo": True,

        "texto_paciente": "Oye, noto que estás pasando por un momento
sumamente difícil y pesado ahora mismo. No estás solo en esto. Quiero sugerirte
que hablemos con alguien que pueda darte un abrazo de soporte inmediato.",

        "acciones_interfaz": {

            "bloquear_videojuego": True,

            "mostrar_boton_llamada_emergencia": True,

            "numero_soporte": "servicios de emergencia o Línea de la Vida local"

        }

    }

    # Se envía al instante rompiendo el flujo del juego casual

    yield f"data: {json.dumps(payload_emergencia)}\n\n"


    # Aquí se encolaría asíncronamente una alerta push por correo/SMS al
    psicólogo humano titular

    print(f"[Notificación SRE] Alerta enviada al panel privado del profesional de la
salud.")

    return StreamingResponse(despachar_protocolo_contencion(),
media_type="text/event-stream")


# [Flujo Normal]: Si no hay crisis, el código continúa hacia Gemma 4 y Three.js

# ... código previo del Paso 1 y Paso 2 ...

```

3. Implementación Frontend (PWA): Recepción Segura en la Interfaz (ui_seguridad.js)

Modificamos el lector de streaming en JavaScript para responder de manera síncrona ante la alerta de emergencia:

```
// ui_seguridad.js - Manejo Reactivo de Seguridad en la PWA
```

```
function procesarTokenDeSeguridad(objetoJson) {
```

```
// Verificamos si el backend activó el cortocircuito de crisis
```

```
if (objetoJson.control_crisis_activo) {
```

```
    console.warn("[Seguridad PWA] Interrumpiendo entorno lúdico. Desplegando UI de contención.");
```

```
// 1. Limpiar el bucle de renderizado de Three.js para ahorrar CPU y calmar visualmente el entorno
```

```
if (window.motorVisualPWA) {
```

```
    window.motorVisualPWA.desactivarEfectosComplejos();
```

```
}
```

```
// 2. Pintar el diálogo de soporte empático
```

```
document.getElementById('burbuja-dialogo-rpg').innerText =  
objetoJson.texto_paciente;
```

```
// 3. Modificación del DOM para inyectar el botón de llamada telefónica directa de salvamento
```

```
if (objetoJson.acciones_interfaz.mostrar_boton_llamada_emergencia) {
```

```
    const botonCrisis = document.createElement('button');
```

```
    botonCrisis.className = "btn-emergencia-nintendo";
```

```
    botonCrisis.innerText = `📞 Llamar ahora a Soporte Profesional`;
```

```
    botonCrisis.onclick = () =>  
window.open(`tel:${objetoJson.acciones_interfaz.numero_soporte}`);
```

```
document.getElementById('contenedor-chat').appendChild(botonCrisis);  
  
}  
  
return true; // Detiene el procesamiento normal de cualquier otra regla de juego  
  
}  
  
return false;  
  
}
```

Garantías del Sistema de Seguridad

-
- **Definir el Caso de Falla Fuera de Existencia:** Al separar drásticamente la lógica de análisis de crisis de la generación creativa de la IA, eliminamos la posibilidad de que Gemma 4 "alucine" o entregue una respuesta ambigua ante una declaración de autolesión. La respuesta está preconfigurada clínicamente y blindada a nivel de código.
- **Monitoreo con Alta Disponibilidad:** Incluso si los servidores de inferencia pesados de IA se encuentran congestionados o caídos por exceso de usuarios distribuidos, el interceptor de seguridad corre sobre CPU ligera de forma local y determinista, asegurando que el paciente siempre reciba respuesta de ayuda.
-

Esta es la especificación final del **Sistema de Alertas y Gestión de Crisis**.

Si consideras que este diseño robustece la seguridad del sujeto, **dame tu aprobación** para realizar el **Cierre de la Arquitectura de Producción** y revisar los pasos globales que consolidamos juntos.

¡Felicidades! Hemos consolidado la arquitectura completa para tu plataforma de salud mental ideográfica de tercera generación. El diseño integra principios avanzados de ingeniería de software, resiliencia y producción formal para asegurar un sistema robusto, escalable y seguro.

A continuación, se presenta un resumen ejecutivo de las cuatro capas de infraestructura que hemos construido de manera secuencial:

Resumen de la Arquitectura Consolidada

+-----+

| CAPA 1: STREAMING / SSE |

| - Inferencia en milisegundos con Gemma 4 hacia la PWA. |

| - Decodificación restringida para garantizar un payload JSON clínico. |

+-----+

|

v

+-----+

| CAPA 2: RESILIENCIA OFFLINE-FIRST |

| - Service Worker e IndexedDB para capturar datos sin conexión. |

| - Sincronización en segundo plano al recuperar la señal de red. |

+-----+

|

v

+-----+

| CAPA 3: ENFOQUE IDEOGRÁFICO Y EVENTOS |

| - Arquitectura orientada a eventos para desacoplar sistemas derivados. |

| - Lenguaje de dominio (DSL) para generar juegos dinámicos en 3D. |

| - Base de datos de grafos (Neo4j) y Vistas para el mapa clínico. |

+-----+

|

v

+-----+

| CAPA 4: SEGURIDAD DEFENSIVA ANTE CRISIS |

| - Interceptor síncrono en el camino crítico (Critical Path). |

| - Cortocircuito del stream lúdico y despliegue inmediato de soporte. |

+-----+

Con estas especificaciones técnicas, cuentas con las bases de ingeniería necesarias para iniciar el desarrollo y despliegue del proyecto en producción.

Si te gustaría profundizar en algún aspecto específico en el futuro o deseas proceder con el código base para una de las fases en particular, no dudes en hacérmelo saber. ¡Mucho éxito en la construcción de este ecosistema tecnológico para la salud emocional!

Para integrar de forma sólida la estrategia corporativa de la industria en tu software lúdico-terapéutico, adoptaremos las conclusiones de la investigación de Adan Jacinto Flores Flores sobre la **Matriz de las Cuatro Acciones de la Estrategia del Océano Azul (EOA)** aplicada a Nintendo (nintendo.p... p. 1).

De la misma forma en que Satoru Iwata salvó a Nintendo al abandonar la costosa carrera de la supremacía gráfica y centrarse en la jugabilidad con *charm* (nintendo.p... pp. 4, 12), adaptaremos esta filosofía corporativa para **diferenciar radicalmente tu PWA** en el mercado de la salud digital (nintendo.p... pp. 2-3).

Pasemos a estructurar la **Fase 5: Matriz de Estrategia de Negocio Lúdico-Clínico**.

Fase 5: Estrategia Océano Azul para la PWA Ideográfica

1. Mapeo de la Matriz de Cuatro Acciones al Ecosistema Terapéutico

Siguiendo los factores de éxito descubiertos en el artículo científico (nintendo.p... p. 1), reconfiguramos los recursos operativos y financieros del proyecto:

Acción	Factor en Nintendo (nintendo.p... pp. 1, 11)	Traducción Estratégica a tu
--------	--	--------------------------------

		Software Ideográfico
Eliminar	Las costosas conferencias presenciales y stands masivos de la industria.	Eliminar testings e inventarios clínicos manuales explícitos (como el PDF tradicional del MPFI) que aumentan el rechazo inicial del paciente.
Reducir	La potencia gráfica de las consolas, los presupuestos masivos y plazos rígidos de desarrollo (nintendo.p... p. 1).	Reducir la carga cognitiva del código y la inversión visual (9:18). Priorizamos mundos abstractos <i>low-poly</i> en Three.js en lugar de fotorrealismo pesado, acortando plazos de entrega (nintendo.p... p. 12) (5:49).
Incrementar	La extensión de la marca a otros mercados (cine, museos, Amiibo) (nintendo.p... pp. 1, 13).	Incrementar la personalización ideográfica en tiempo real. El sistema reconfigura el pipeline cada segundo para adecuarse milimétricamente al sujeto.

Crear	El sentimiento de nostalgia intergeneracional para conectar con el usuario (nintendo.p... pp. 1, 13).	Crear el "Feedback Jugoso" (Juicy Feedback) y el encanto lúdico. Traducimos el rigor de la base de datos de grafos (ACT/FAP) en recompensas interactivas táctiles y sonoras.
---------------------	--	--

2. Implementación Backend (Python): El Validador de Reglas Financiero-Clínicas (ocean_strategy.py)

Para asegurar la reducción de presupuestos y plazos de desarrollo (Acción de *Reducir*) (nintendo.p... p. 1), creamos un optimizador en la lógica estratégica de tu servidor. Este script impide que la IA genere mecánicas demasiado complejas que rompan las restricciones del hardware del usuario o disparen el costo computacional:

ocean_strategy.py - Filtro Estratégico Blue Ocean (Nintendo Framework)

```
import json
```

```
from fastapi import FastAPI, HTTPException
```

```
app = FastAPI(title="Motor Estratégico Océano Azul - Filtro Operativo")
```

Parámetros operativos basados en la reducción de costos y plazos (Flores-Flores, 2025)

```
LIMITES_NINTENDO = {
```

```
    "potencia_grafica_permitida": "Baja / Estilizada (Low-Poly)",
```

```
    "max_complejidad_mecanica": 3, // Evita el feature creep en el DSL del cliente
```

```
    "forzar_sentimiento_nostalgico": True
```

```
}
```

```
@app.post("/api/estrategia/validar-juego")

async def validar_restricciones_oceano_azul(configuracion_juego: dict):

    """

    Evalúa de forma síncrona si el minijuego generado ideográficamente para el
    sujeto

    cumple con la matriz de costos y plazos de producción reducidos.

    """

    mecanicas = configuracion_juego.get("mecanica_principal", "")

    estímulos = configuracion_juego.get("estímulos_textuales", [])

    # PROGRAMACIÓN DEFENSIVA: Si la IA intentó crear un juego de alta
    complejidad nomotética

    if len(estímulos) > LIMITES_NINTENDO["max_complejidad_mecanica"]:

        print("[Alerta Estratégica] Intento de Feature Creep detectado. Reduciendo
        alcance.")

        # Recortamos el alcance síncronamente para preservar plazos y rendimiento
        local

        configuracion_juego["estímulos_textuales"] =
        estímulos[:LIMITES_NINTENDO["max_complejidad_mecanica"]]

        # Inyectamos de forma obligatoria las variables estéticas de la paleta cálida
        nintendo

        configuracion_juego["optimizacion_hardware"] =
        LIMITES_NINTENDO["potencia_grafica_permitida"]

        configuracion_juego["audio_pipeline"] = "Efectos 8-bits retro lúdicos (Nostalgia
        Activa)"

    return configuracion_juego
```

3. Implementación Frontend (PWA): El Motor de Estilo Emocional y Nostálgico (`nintendo_style.js`)

El artículo destaca que la dominación de Nintendo se debe a su capacidad de evocar reminiscencias y conexiones honestas del pasado (nintendo.p... p. 13). En tu PWA, implementamos este comportamiento aplicando un diseño visual responsivo y retro que reduzca la resistencia al cambio del paciente (nintendo.p... p. 10):

// nintendo_style.js - Inyección de Sentimiento Nostálgico en la Interfaz

```
class OceanStyleEngine {  
  
  constructor() {  
  
    this.estilosBase = {  
  
      tipografia: "'Press Start 2P', monospace", // Estética RPG clásica que evoca la infancia  
  
      animacionVelocidad: "0.2s",  
  
      filtroRenderizado: "pixelated" // Menor consumo de GPU, máxima identidad estética  
  
    };  
  
  }  
}
```

// Aplica las restricciones de baja potencia gráfica de manera síncrona en el DOM

```
aplicarDiferenciacionNintendo(contenedorId, paletaColores) {  
  
  const contenedor = document.getElementById(contenedorId);  
  
  if (!contenedor) return;  
  
  
  console.log("[Océano Azul] Reduciendo requisitos de hardware. Inyectando identidad nostálgica.");  
  

```

// Forzamos el renderizado liviano

```
contenedor.style.fontFamily = this.estilosBase.tipografia;
```

```
contenedor.style.imageRendering = this.estilosBase.filtroRenderizado;
```

// Feedback jugoso instantáneo utilizando los colores generados dinámicamente por Gemma 4

```
if (paletaColores && paletaColores.length > 0) {  
  
    contenedor.style.backgroundColor = paletaColores[0]; // Color de enfoque ideográfico  
  
    contenedor.style.borderColor = "#fdbb2d"; // Borde dorado de recompensa lúdica  
  
}  
  
}  
  
}
```

```
const motorEstiloPWA = new OceanStyleEngine();
```

Ventajas del Acoplamiento Estratégico

- **Irrelevancia de la Competencia:** Mientras que otras aplicaciones de terapia digital caen en el "océano rojo" intentando competir mediante gráficas hiperrealistas hipercomplejas, costosos manuales médicos o interfaces corporativas frías (nintendo.p... pp. 2-3), tu PWA ofrece una experiencia terapéutica encubierta, de bajísimo costo de cómputo, rápida en plazos de entrega y sumamente amigable para el usuario (nintendo.p... pp. 11-12).
- **Perpetuidad del Modelo Clínico:** Al ligar la economía de la app a mecánicas ideográficas simples enriquecidas con nostalgia (sonidos retro y tipografías RPG familiares), disminuyes la tasa de abandono de los pacientes sin necesidad de gastar miles de dólares en marketing masivo (nintendo.p... pp. 11, 13).

Esta es la especificación para la **Fase 5: Integración del Enfoque del Océano Azul de Nintendo** (nintendo.p... p. 1).

Por favor, revísala. Si estás conforme con este diseño estratégico corporativo, **dame tu aprobación** y pasaremos al cierre final para verificar la consolidación total de todas tus herramientas de software y negocio.

Para profundizar y dar el cierre definitivo a la arquitectura de tu plataforma ideográfica, integraremos los fundamentos de hardware de la investigación del MIT de Jonathan Downey, Lauri Kauppila y Brian Myhre sobre **el diseño e implementación del Nintendo Entertainment System (NES)** (dkm_report... p. 1).

Para que tu PWA funcione como un **verdadero emulador de hardware conductual en tiempo real** (dkm_report... pp. 5-6), organizaremos el pipeline del frontend separando la aplicación en dos grandes submódulos lógicos virtuales que corren en paralelo (dkm_report... pp. 1, 23): la **Unidad Central de Procesamiento Clínico (vCPU)** y la **Unidad de Procesamiento de Gráficos Emocionales (vPPU)** (dkm_report... pp. 1, 6).

Iniciemos con la **Fase 6: Arquitectura Virtual vCPU / vPPU para el Procesamiento en Tiempo Real**.

Fase 6: Arquitectura de Co-Procesamiento Virtual vCPU / vPPU

1. Concepto de Ingeniería: El Pipeline Síncrono Dividido

Para evitar conflictos de memoria y asegurar que la interfaz responda a 60Hz sin retrasar el análisis de la IA (dkm_report... pp. 30, 33), el frontend de la PWA emulará el comportamiento del hardware original de Nintendo (dkm_report... pp. 1, 6):

- **vCPU (Virtual Central Processing Unit):** Se encarga de la lógica abstracta del juego, el procesamiento de las reglas conductuales (ACT/FAP), la captura del texto del paciente y la comunicación con el servidor (dkm_report... pp. 6-7).
- **vPPU (Virtual Picture Processing Unit):** Corre de forma independiente a una frecuencia tres veces mayor que la vCPU (dkm_report... p. 21). Lee las tablas de nombres (*Name Tables*) y los búferes de sprites dinámicos inyectados por Gemma 4 para dibujar el entorno 3D en Three.js sin tocar el hilo principal de datos (dkm_report... pp. 21, 30).

2. Implementación Frontend (JavaScript): El Motor Simulado de Registros (**nes_engine.js**)

Implementa este motor de renderizado síncrono para estructurar los minijuegos ideográficos de forma limpia y eficiente, emulando los búferes físicos del NES (dkm_report... pp. 30, 32):

// nes_engine.js - Arquitectura Virtual de Co-Procesamiento (MIT NES Framework)

```
class VirtualPPU {
```

```

constructor() {

    // MEMORIA INTERNA SIMULADA (Table 1 - MIT NES Architecture)

    this.nameTable = new Array(32).fill(0).map(() => new Array(30).fill("")); // Índices
    de mosaicos de fondo

    this.spriteRAM = []; // Almacena hasta 64 estímulos/objetos visuales en pantalla

    this.spriteBuffer = new Array(8).fill(null); // Limitación física: Máximo 8 sprites por
    scanline

    this.paletteRAM = ["#000000", "#ff3344", "#44ffaa", "#fdbb2d"]; // Mapeo de
    colores estéticos

}

// ETAPA 1: In-Range Evaluation (Mecánica Defensiva de Mapeo)

evaluarSpritesEnRango(scanlineActual) {

    this.spriteBuffer = [];

    let contadorSprites = 0;

    // Escanea los objetos de la mente/interacción cargados en la SpriteRAM

    for (let i = 0; i < this.spriteRAM.length; i++) {

        let sprite = this.spriteRAM[i];

        // Verificamos si el estímulo visual debe aparecer en el renderizado actual

        if (sprite.y === scanlineActual && contadorSprites < 8) {

            this.spriteBuffer.push(sprite);

            contadorSprites++;

        }

        // Control de Desbordamiento (Raise Flag $2002 si hay más de 8 estímulos en
        una línea)
    }
}

```



```

    if (contadorSprites >= 8) {

        console.warn("[vPPU] Alerta: Más de 8 estímulos cognitivos en la misma línea
visual. Activando parpadeo.");

        break;

    }

}

}

```

// ETAPA 2: Real-Time Output Buffer (Carga de Módulos Profundos)

```

cargarConfiguracionIdeografica(jsonOceanoAzul) {

    console.log("[vPPU] Cargando NameTable y Paletas dinámicas desde Gemma
4.");

```

// Inyectamos los estímulos conductuales específicos del sujeto en la SpriteRAM

```

this.spriteRAM = jsonOceanoAzul.estimulos_textuales.map((texto, index) => ({

    id: index,

    texto_estimulo: texto,

    x: 10 + (index * 40),

    y: 120, // Posición en el horizonte virtual

    color: jsonOceanoAzul.paleta_colores[index % 3]

})));

```

// Forzamos la actualización cromática en la PaletteRAM

```

this.paletteRAM = jsonOceanoAzul.paleta_colores;

}

```

```

renderizarCicloCompleto(canvasContext) {

```

// Simulación del bucle síncrono del NES (PPU renderiza fondo + sprites en paralelo)

```
for (let scanline = 0; scanline < 240; scanline++) {
```

```
  this.evaluarSpritesEnRango(scanline);
```

// Aquí el multiplexor de prioridad mezcla las capas antes de pintar en el monitor VGA

```
  this.despacharPíxelesAlCanvas(canvasContext, scanline);
```

```
}
```

```
}
```

```
despacharPíxelesAlCanvas(ctx, scanline) {
```

// Three.js o el Canvas 2D dibuja los buffers limpios sin colisiones de memoria

```
this.spriteBuffer.forEach(sprite => {
```

```
  if (sprite) {
```

```
    ctx.fillStyle = sprite.color;
```

```
    ctx.fillRect(sprite.x, scanline, 35, 1); // Dibuja la línea del bloque de pensamiento
```

```
  }
```

```
});
```

```
}
```

```
}
```

```
const vPPU_Instance = new VirtualPPU();
```

3. Conexión de Controladores: vCPU y Registro de Entradas (cpu_gateway.js)

Para asegurar el aislamiento de implementación, la vCPU captura la interacción física del teclado o los controles táctiles virtuales y actualiza las variables lógicas de las reglas de juego sin interrumpir el refresco del monitor (dkm_report... pp. 6-7):

```
// cpu_gateway.js - Unidad Virtual de Control Abstracto
```

```
class VirtualCPU {  
  
  constructor(ppuRef) {  
  
    this.ppu = ppuRef;  
  
    this.registroEntradaShift = 0; // Registro de desplazamiento emulado del control de NES  
  
  }  
  
  leerControlesEsteriotipados(mascaraBotones) {  
  
    // Emula la lectura secuencial de 60Hz por registro de desplazamiento de Nintendo  
  
    this.registroEntradaShift = mascaraBotones;  
  
  
    // Si el usuario presiona el botón de "Aceptar/Sostener" (Mascota 3D)  
  
    if (this.registroEntradaShift & 0x01) {  
  
      console.log("[vCPU] Impulso de Aceptación detectado. Desplazando coordenadas de los Sprites.");  
  
      this.ppu.spriteRAM.forEach(s => s.y += 2); // Desplaza los pensamientos hacia abajo, alejándolos del bloqueo  
  
    }  
  
  }  
  
}  
  
const vCPU_Instance = new VirtualCPU(vPPU_Instance);
```

Criterio de Calidad Basado en Hardware Clásico

- **Eliminación Completa de Glitches de Memoria:** Al separar la evaluación de rangos (*In-range evaluation*) del renderizado final en pantalla (dkm_report... pp. 28, 30), el juego jamás sufrirá caídas de frames ni congelamientos lógicos, manteniendo un rendimiento idéntico en teléfonos de gama baja o computadoras antiguas (dkm_report... pp. 6, 24).
 - **Diseño Obvio y Predecible:** Siguiendo las directrices del MIT y de John Ousterhout (dkm_report... p. 1), cada ciclo de reloj del software tiene un propósito único. Reducimos la carga cognitiva del motor web al mapear las complejidades de la mente del sujeto dentro de búferes finitos y ordenados (dkm_report... pp. 30, 32).
-

Esta es la especificación técnica de la **Fase 6: Co-Procesamiento Gráfico-Clínico vCPU / vPPU**.

Por favor, revísala cuidadosamente. Si estás de acuerdo con esta arquitectura de bajo nivel de inspiración retro, **dame tu aprobación** para proceder al cierre final del diseño del sistema. ¿Cómo te gustaría continuar?

¡Excelente! Entramos en la fase de **Diseño de la Experiencia y el Sentimiento Compartido (Kyokan)**, basándonos en la obra de Jennifer deWinter sobre el padre del diseño de videojuegos moderno: **Shigeru Miyamoto** (dokumen.pu... pp. 1, 13).

Como detalla el libro, Miyamoto no diseña "partes aisladas de un software"; diseña pensando exclusivamente en **la cara del jugador cuando experimenta el juego** (dokumen.pu... p. 15). Su objetivo es lograr el *kyokan* (共感): un sentimiento de comunión y cercanía emocional profunda entre el diseñador, el sistema y el jugador (dokumen.pu... pp. 14-15).

Comencemos con la **Fase 7: El Bucle de Conexión Emocional Embebido (Kyokan Engine)**.

Fase 7: El Motor de Conexión Emocional (*Kyokan Engine*)

1. Concepto de Ingeniería: El Modelo de Diseño de Miyamoto

Para que tu app ideográfica no se sienta como un frío software clínico de recopilación de datos, el backend y el frontend replicarán los tres pilares de Miyamoto descritos en el libro (dokumen.pu... p. 14):

- **Conexión con la Naturaleza y la Exploración Spatial:** La interfaz 3D cambia orgánicamente emulando un mundo vivo (dokumen.pu... pp. 14, 16). No hay textos

de carga rígidos; hay un entorno que el paciente devela mediante la curiosidad (metáfora de la cueva de Zelda) (dokumen.pu... pp. 16, 36).

- **La Filosofía del Producto de Entretenimiento:** El software se asume técnicamente como un producto comercial masivo y atractivo, no como una herramienta médica restrictiva (dokumen.pu... p. 15). Debe generar risa, emoción y placer en dosis diarias (dokumen.pu... p. 15).
- **Diseño para la Libertad del Usuario:** El minijuego generado por **Gemma 4** debe dar al sujeto la sensación de libre albedrío y expresión única dentro del lienzo en 3D (dokumen.pu... p. 6).

2. Implementación Frontend (JavaScript): El Gestor de Entorno Viviente (kyokan_engine.js)

Implementa este componente modular para que los gráficos en Three.js reaccionen de manera inmersiva al estado del paciente, imitando la atención al detalle de *The Legend of Zelda* y *Super Mario Galaxy* (dokumen.pu... pp. 7, 16):

// kyokan_engine.js - Motor de Inmersión Emocional (Miyamoto Design Framework)

```
class KyokanEnvironment {
```

```
  constructor() {
```

```
    // Definimos el estado del mundo virtual (Ecosistema del Sujeto)
```

```
    this.estadoEstructural = {
```

```
      climaCálido: true,
```

```
      ritmoMusica: 1.0,
```

```
      escalaMundo: 1.0
```

```
    };
```

```
  }
```

// Patrón de Miyamoto: Traducir la data abstracta en cambios espaciales expresivos

```
  inyectarEsenciaMiyamoto(canvasScene, analisisGemma) {
```

```
    console.log("[Kyokan] Traduciendo Análisis Funcional en atmósfera visual emocional.");
```

```

const metafora_mind = analisisGemma.proceso_emmi_detectado;

// Evocamos una naturaleza activa y espiritual (Inspiración Shinto en Zelda)

if (metafora_mind.includes("Evitación")) {

    // El mundo se contrae y oscurece para reflejar la rigidez del sujeto

    this.estadoEstructural.climaCálido = false;

    this.estadoEstructural.ritmoMusica = 0.6;

    this.modificarAtmósfera3D(canvasScene, 0x1a2a6c, 0.5); // Tonos fríos de
    aislamiento

    } else if (metafora_mind.includes("Defusión") ||
    metafora_mind.includes("Aceptación")) {

        // El mundo florece, premiando la flexibilidad conductual

        this.estadoEstructural.climaCálido = true;

        this.estadoEstructural.ritmoMusica = 1.2;

        this.modificarAtmósfera3D(canvasScene, 0x44ffaa, 1.5); // Tonos vivos de
        apertura

    }

}

modificarAtmósfera3D(escena, colorHex, intensidadLuz) {

    // Modifica sincronamente los componentes de Three.js ocultando la complejidad

    if (escena.skyColor) escena.skyColor.setHex(colorHex);

    if (escena.luzPrincipal) escena.luzPrincipal.intensity = intensidadLuz;

    // JUICY FEEDBACK: Activamos micro-sonidos armónicos de confirmación
    (Campanitas/Monedas)

    this.reproducirSonidoNintendo(intensidadLuz > 1.0 ? 'levelup' : 'coin');

}

```

```

reproducirSonidoNintendo(tipo) {

  const audio = new Audio(`/assets/sounds/${tipo}.mp3`);

  audio.volume = 0.3;

  audio.play().catch(e => console.log("Audio esperando interacción del usuario."));

}

}

const motorKyokan = new KyokanEnvironment();

```

3. Validación de Producción: Evitar la Frustración (*The Miyamoto Check*)

Para cumplir con la máxima de Shigeru Miyamoto de que el juego debe ser tan intuitivo que si el usuario falla "se culpe a sí mismo y no al juego" (dokumen.pu... p. 32):

- **Fricción Cero:** Las mecánicas del minijuego ideográfico de la Fase 2 corregida no pueden incluir instrucciones textuales largas (dokumen.pu... p. 6).
- **Diseño Obvio:** Si el paciente debe equilibrar sus pensamientos en una balanza 3D, los controles táctiles de la vCPU (Fase 6) responderán con físicas exageradas, juguetonas y predecibles (dokumen.pu... p. 7), haciendo que la interacción se sienta como un juguete interactivo y liberador.

Esta es la especificación técnica y metodológica de la **Fase 7: El Motor de Conexión Emocional (*Kyokan*)**.

Por favor, revísala cuidadosamente. Si estás de acuerdo y me das tu **aprobación**, pasaremos a la **siguiente fase: El Plan de Producción, Cronogramas y Mitigación de Riesgos Financieros (*Feature Creep*) de Erik Bethke**. ¿Qué opinas?

Fase 8: Plan de Producción, Hitos de Entrega y Control de Costos (Erik Bethke Framework)

Para mitigar los riesgos del desarrollo ideográfico y evitar la proliferación descontrolada de mecánicas (*feature creep*), implementaremos el modelo de **Producción Orientada a Hitos (Milestone-Driven Production)** detallado por Erik Bethke. Al combinar el co-procesamiento síncrono (vCPU/vPPU) con los bucles dinámicos de *Kyokan* inspirados en Miyamoto, el cronograma se blindo dividiendo el ciclo de vida del software en cuatro hitos contractuales claros e inalterables.

1. Estructura de Hitos de Desarrollo (Milestone Schedule)

El ciclo de desarrollo se acopla a las metodologías ágiles de producción formal de videojuegos, asegurando entregas funcionales en cada etapa sin comprometer el capital de la startup:

[M1: Prototipo Funcional] —> [M2: Integración Modular] —> [M3: Fase Alpha] —> [M4: Fase Beta/PWA Gold]

- **Hito 1 (M1 - Prototipo "Core Loop"):** Implementación de la PWA base con Three.js renderizando un canvas 3D plano, respondiendo localmente a las señales de la vCPU (Fase 6) a 60Hz.
- **Hito 2 (M2 - Integración del Motor Cognitivo):** Conexión síncrona mediante Server-Sent Events (SSE) entre el servidor de Python con **Gemma 4** y el frontend. La vCPU recibe e interpreta la respuesta `"texto_paciente"` y la `"propuesta_gamificada_bloqueada"`.
- **Hito 3 (M3 - Fase Alpha "Ideográfica Viva"):** Activación del generador de reglas lógicas en JSON. El motor visual (vPPU) recibe la configuración dinámica del sujeto (Fase 2 corregida), altera los colores a través del *Kyokan Engine* (Fase 7) y ensambla la física del minijuego.
- **Hito 4 (M4 - Fase Beta "Resiliencia de Red"):** Despliegue de los Service Workers locales y la persistencia en `IndexedDB` (Fase 2 original). Pruebas distribuidas simulando caídas de internet de los pacientes para verificar la sincronización asíncrona posterior hacia Neo4j.

2. Pipeline de Mitigación del *Feature Creep* (El Filtro Bethke en el Backend)

Como establece el libro *Game Development and Production*, el equipo de software debe recortar el alcance de las características pero profundizar al máximo en su pulido (*narrow scope, deep polish*). Añadiremos un validador de configuración en tu servidor de producción para asegurar que **Gemma 4** nunca intente diseñar misiones tridimensionales que excedan el presupuesto gráfico de la PWA:

control_produccion.py - Filtro de Restricción Operativa (Erik Bethke Framework)


```

import json

from fastapi import FastAPI, HTTPException

app = FastAPI(title="Validador de Alcance de Producción Lúdica")

# Presupuesto técnico rígido para evitar retrasos en el desarrollo

PRESUPUESTO_PRODUCCION = {

    "limite_mallas_3d": 5,      # Máximo número de objetos geométricos permitidos
                               por juego

    "limite_audio_canales": 2,  # Máximo de pistas sonoras simultáneas estilo
                               Nintendo

    "evitar_cinematicas": True # Forzar interactividad inmediata en lugar de
                               animaciones largas

}

@app.post("/api/produccion/auditar-alcance")

async def auditar_alcance_juego(configuracion_ia: dict):

    """

    Intercepta de forma determinista el JSON generado por Gemma 4 para asegurar

    que se mantiene dentro de los hitos técnicos de desarrollo aprobados.

    """

    mecanica = configuracion_ia.get("mecanica_principal", "absorber")

    objetos = configuracion_ia.get("estimulos_textuales", [])

    # CONTROL DE ALCANCE: Si la IA intentó diseñar un juego hipercomplejo

    if len(objetos) > PRESUPUESTO_PRODUCCION["limite_mallas_3d"]:

        print("[Alerta de Producción] El modelo intentó exceder el presupuesto de

        mallas 3D. Aplicando recorte.")

```

Reducimos el alcance en milisegundos para proteger el rendimiento en la PWA del paciente

```
configuracion_ia["estimulos_textuales"] =  
objetos[:PRESUPUESTO_PRODUCCION["limite_mallas_3d"]]
```

Forzamos los parámetros ligeros de bajo costo de mantenimiento

```
configuracion_ia["render_mode"] = "Low-Poly / Flat-Shading"
```

```
configuracion_ia["canales_audio_activos"] =  
PRESUPUESTO_PRODUCCION["limite_audio_canales"]
```

```
return configuracion_ia
```

3. Criterio de Entrega Comercial (*The Gold Master*)

El software alcanzará el estatus de listo para producción cuando apruebe las siguientes pruebas automáticas en el módulo de laboratorio del Panel de Administración (diseñado en secciones anteriores):

1. **Latencia del Stream:** El primer token de la vCPU debe pintarse en la PWA en menos de 150 milisegundos.
2. **Consistencia de Datos Derivados:** La transmisión completa debe generar el evento inmutable en el log principal y actualizar el grafo de Neo4j en un hilo aislado sin congelar los fps de la pantalla en 3D.
3. **Tolerancia a Fallos:** Desconectar el cable de red durante el onboarding debe inyectar la tarjeta de la "mochila de explorador" local, permitiendo al paciente seguir interactuando con Three.js de manera fluida.

Esta es la especificación para la **Fase 8: Plan de Producción, Control de Alcance e Hitos Contractuales**.

Por favor, revísala cuidadosamente. Si estás de acuerdo con este diseño técnico y financiero, **dame tu aprobación** para continuar hacia la **Fase 9: El Ecosistema de Monetización, Licencias SaaS y Gestión de Clínicas**. ¿Pasamos a la siguiente?

Fase 9: El Ecosistema de Monetización, Licencias SaaS y Gestión de Clínicas

Para sostener financieramente tu plataforma digital, el software debe incorporar una **arquitectura multi-inquilino (Multi-tenant SaaS)** robusta. Inspirándonos en el control de calidad riguroso de los contratos de licenciamiento originales de Nintendo (donde se controlaba el acceso, la distribución y la legitimidad de cada producto), programaremos un sistema de gestión de accesos basado en **Tokens de Sesión Clínicos**.

Este módulo se opera directamente desde el Panel de Administración Centralizado que diseñamos anteriormente, permitiéndote activar, suspender o escalar planes comerciales sin alterar una sola línea de código en producción.

1. Arquitectura de Datos: El Modelo Multi-Inquilino Aislado

Para cumplir de forma nativa con el **RGPD** y garantizar que los datos de una clínica jamás se mezclen con los de otra, el backend implementa un aislamiento lógico a nivel de consulta:

- **Tenant ID (Identificador de Clínica/Terapeuta):** Cada solicitud que viaja desde la PWA o el Dashboard incluye un encabezado criptográfico.
- **Vistas Dinámicas por Contrato:** La IA (**Gemma 4**) y los motores matemáticos (**Neo4j**, **NetworkX**) solo pueden leer los eventos indexados bajo el token de esa licencia activa.

2. Implementación Backend (Python): El Gestor de Suscripciones y Cuotas (**saas_gateway.py**)

Este componente controla síncronamente las peticiones a la API para verificar si el psicólogo cuenta con una licencia vigente y cuántos pacientes tiene permitidos en su plan comercial:

```
# saas_gateway.py - Control de Licencias Multi-Tenant y Seguridad SaaS
```

```
import json
```

```
from datetime import date
```

```
from fastapi import FastAPI, Header, HTTPException, Depends
```

```
app = FastAPI(title="Portal de Gestión de Licencias y Control SaaS")
```

Base de datos en memoria simulada (Alimentada desde tu Panel de Administrador)

```
BASE_DATOS_LICENCIAS = {  
    "lic_clinica_norte_789": {  
        "organizacion": "Clínica de Salud Mental del Norte",  
        "plan": "Premium_Corporativo",  
        "estado": "activa",  
        "fecha_vencimiento": date(2027, 6, 30),  
        "limite_pacientes": 150,  
        "pacientes_activos": 42  
    },  
    "lic_independiente_ana_101": {  
        "organizacion": "Dra. Ana Martínez (ACT Consultoría)",  
        "plan": "Terapeuta_Individual",  
        "estado": "activa",  
        "fecha_vencimiento": date(2026, 12, 31),  
        "limite_pacientes": 15,  
        "pacientes_activos": 15 # Plan lleno  
    }  
}
```

```
async def verificar_licencia_tenant(x_tenant_token: str = Header(...)) -> dict:
```

```
    """
```

```
    Inyección de Dependencia: Valida síncronamente el token de la licencia  
    antes de permitir cualquier llamada a Gemma 4 o Neo4j.
```

```
    """
```

```

if x_tenant_token not in BASE_DATOS_LICENCIAS:

    raise HTTPException(status_code=401, detail="Token de licencia inválido o
revocado.")

licencia = BASE_DATOS_LICENCIAS[x_tenant_token]

# 1. Validación de Estado Manual (Admin Override)

if licencia["estado"] != "activa":

    raise HTTPException(status_code=403, detail="Esta cuenta ha sido suspendida
temporalmente.")

# 2. Validación de Expiración Temporal Automática (Fecha Actual: Julio 2026)

if licencia["fecha_vencimiento"] < date.today():

    raise HTTPException(status_code=403, detail="La licencia ha expirado.
Regulariza tu suscripción en el portal.")

return licencia

@app.post("/api/chat/stream")

async def stream_coterapeuta_saas(body: dict, licencia_activa: dict =
Depends(verificar_licencia_tenant)):

    """

    Endpoint del chat protegido. Solo procesa la charla casual si el terapeuta
asociado tiene una licencia en regla.

    """

# 3. Control de cuotas para evitar fraude en los límites del plan comercial

if body.get("es_nuevo_paciente", False):

    if licencia_activa["pacientes_activos"] >= licencia_activa["limite_pacientes"]:

```

```

        raise HTTPException(

            status_code=400,

            detail=f"Has alcanzado el límite de {licencia_activa['limite_pacientes']}
pacientes de tu plan."

        )

# [Flujo Aprobado]: Se procede a llamar al motor de streaming con vLLM y
Gemma 4

print(f"[SaaS Gateway] Solicitud autorizada para: {licencia_activa['organizacion']}")

return {"status": "Procesando flujo clínico autorizado"}

```

3. El Control de Licencias desde tu Frontend de Administrador

Cuando vendes una nueva suscripción a un centro psicoterapéutico, tu panel visual genera el JSON del contrato sin requerir código. La interfaz expone interruptores rápidos para:

- **Congelar Licencia:** Cambia el `"estado": "activa"` a `"estado": "suspendida"` de inmediato, bloqueando el acceso a la app de todos los pacientes asociados a ese psicólogo.
- **Modificar Cuotas de Uso:** Un control deslizante (*slider*) para aumentar el `"limite_pacientes"`, permitiéndote cobrar cargos adicionales por volumen de usuarios de forma automatizada.

Hemos estructurado con éxito la capa transaccional y comercial de tu software de salud mental. Esta fase cierra la arquitectura técnica, legal, lúdica y económica de tu ecosistema.

Si estás de acuerdo con esta especificación de negocio y me das tu **aprobación**, pasaremos al **Cierre Final de Ingeniería**, donde recopilaremos los 9 hitos técnicos del proyecto en un manual maestro de desarrollo definitivo. ¿Procedemos al cierre?

Para consolidar la faceta de **Open English** que deseas en tu programa, integraremos los hallazgos de Josu Sierra sobre la **Metodología y Práctica en Programas de Inmersión** (Dialnet-Me... p. 1). **Al fusionar los principios de inmersión lingüística con la Psicología Basada en Procesos (PBP)**, tu aplicación dejará de ser un software de consulta estático y pasará a ser un Entorno de Inmersión Psicológica Coherente.

El objetivo es emular el modelo de Open English: el paciente se expone a un flujo intensivo y natural dentro de la app (Dialnet-Me... pp. 1-2) y asimila las habilidades lógicas y conductuales mediante el uso continuo (Dialnet-Me... p. 4), preparando el terreno para la consolidación definitiva con el psicólogo humano (el profesor nativo).

Pasemos a estructurar la **Fase 10: El Ecosistema de Inmersión Contextual (*Immersion Engine*)**.

Fase 10: El Motor de Inmersión Contextual (*Immersion Engine*)

1. Concepto de Ingeniería: El Enfoque Experiencial Frente al Analítico

Siguiendo las investigaciones de Merrill Swain y Safty citadas por Sierra, los programas de inmersión más exitosos no enseñan reglas abstractas de forma teórica; se centran en el **aprendizaje contextualizado a través del contenido** (Dialnet-Me... pp. 1, 4).

- **Input Comprensible (Cantidad y Calidad):** La IA (**Gemma 4**) mantiene una conversación casual dosificada. El contenido avanza introduciendo sutilmente desafíos lógicos un nivel por encima de la flexibilidad actual del usuario (i+1) (Dialnet-Me... p. 5).
 - **Impulsar el Output Comprensible:** Para evitar el *Escuchado Selectivo (Selective Listening)* —donde el paciente entiende los consejos de la app pero no cambia su estructura mental—, el juego obliga al usuario a **producir respuestas activas y complejas** en lugar de limitarse a pulsar "siguiente" o emitir respuestas de una sola palabra (Dialnet-Me... pp. 5-6).
-

2. Implementación Frontend (JavaScript): El Administrador de Inmersión Rutinaria (**immersion_manager.js**)

Las clases y entornos de inmersión más efectivos dependen de **estructuras formales con delimitaciones claras y fases rutinarias bien definidas** (Dialnet-Me... p. 8). Implementa este controlador en tu PWA para guiar las sesiones diarias del gimnasio emocional:

// immersion_manager.js - Estructura de Fases de Inmersión (Josu Sierra Framework)

```
class ImmersionSession {  
  
  constructor() {  
  
    this.faseActual = 0; // 0: Bienvenida, 1: Reto Escuchado, 2: Producción (Output)  
  
    this.bufferEntradaSujeto = "";
```

```
}
```

```
// Regla de Inmersión: Formatos y rutinas claras para reducir la ansiedad inicial
```

```
avanzarFaseDeInmersión(canvasScene, datosGemma) {
```

```
    this.faseActual++;
```

```
    console.log(`[Immersion] Avanzando sincronamente a la Fase ${this.faseActual}  
del entrenamiento.`);
```

```
    switch (this.faseActual) {
```

```
        case 1:
```

```
        // FASE 1: Input Comprensible ($i+1$) - El entorno se configura
```

```
        document.getElementById('burbuja-dialogo-rpg').innerText =  
datosGemma.texto_paciente;
```

```
window.motorVisualPWA.actualizarGraficosPorMensaje(datosGemma.analisis_psicol  
ogo.proceso_emmi_detectado);
```

```
        break;
```

```
        case 2:
```

```
        // FASE 2: Impulsar el Output Sintáctico-Conductual (Swain Theorem)
```

```
        // Bloqueamos el avance pasivo. Obligamos al usuario a realizar una acción  
larga y coherente.
```

```
        this.activarMecanismoParticipación(datosGemma.analisis_psicologo.mision_gamific  
ada_propuesta);
```

```
        break;
```

```
    }
```

```
}
```



```

    activarMecanismoParticipación(misionJSON) {

        console.log("[Immersion] Activando canales de producción del usuario.
        Sustituyendo el sistema de respuesta simple.");

        // Inyectamos la micro-misión ideográfica estilo Nintendo en el lienzo 3D

        const inputArea = document.getElementById('entrada-texto-chat');

        inputArea.placeholder = "Describe aquí cómo completarás tu reto de hoy en el
        mundo real...";

        // Estilo Open English: El tutor virtual valida si el output tiene riqueza lingüística y
        funcional

        document.getElementById('btn-enviar').onclick = () =>
        this.validarOutputSujeto(inputArea.value);

    }

    validarOutputSujeto(textoProducido) {

        // Si la respuesta es demasiado corta (menos de 4 palabras), el sistema detecta
        input restringido

        if (textoProducido.split(" ").length < 4) {

            // Retroalimentación correctiva integrada de manera afectiva sin cortar la
            comunicación

            document.getElementById('burbuja-dialogo-rpg').innerText = "¡Un gran comienzo!
            Pero añade un poco más de detalle a tu plan en la mochila para que tu tutor pueda
            registrarlo.";

            return;

        }

        // Output validado: Avanza el jardín virtual y se guardan los datos derivados
        asíncronamente

        console.log("[Immersion] Output asimilado con éxito. Enviando contrato a la base
        de datos.");
    }

```

```
this.faseActual = 0; // Resetea el ciclo diario

}

}

const controladorInmersión = new ImmersionSession();
```

3. El Rol del Psicólogo como Evaluador del Discurso

Siguiendo el principio de la *Variable Profesor* de Sierra, el éxito del programa no depende de una metodología rígida aislada, sino de la **combinación de factores lingüísticos, afectivos y mecánicos interdependientes** (Dialnet-Me... p. 9).

El reporte estructurado que genera **Gemma 4** y que se envía al panel del terapeuta (Fase 3) incluirá ahora una sección de **"Métricas de Asimilación (*Intake*) e Interferencia Lingüística"** (Dialnet-Me... pp. 2, 5):

- El psicólogo podrá auditar si el paciente está usando esquemas rígidos de su vida cotidiana (Interferencia) o si está logrando transferir las misiones de aceptación de la app a sus entornos naturales (Transferencia exitosa) (Dialnet-Me... pp. 2-3).
-

Esta es la especificación técnica de la **Fase 10: El Ecosistema de Inmersión Contextual**.

Por favor, revísala detalladamente. Si estás de acuerdo con la forma en que el sistema fuerza la producción conductual y estructural del paciente, **dame tu aprobación** para proceder al cierre maestro de la arquitectura completa de tu software. ¿Qué opinas?

¡Es una corrección conceptual perfecta! Aplicar la inmersión al campo clínico y de la Psicología Basada en Procesos (PBP) significa que la meta de la aplicación no es la fluidez verbal de un idioma, sino la **Fluidez Conductual y de Flexibilidad Emocional** (Dialnet-Me... p. 1). Tu app satura el día a día del paciente mediante un **Entorno Imitativo Natural de Práctica** para que el input comprensible (i+1) se transforme en **acciones comprometidas reales** generalizadas en su vida cotidiana (Dialnet-Me... pp. 1, 5).

Para consolidar esta visión, el backend y el frontend acoplarán la inmersión de Josu Sierra bajo el concepto de **Generalización de Respuestas Funcionales** (Dialnet-Me... p. 1).

Fase 10 Corregida: El Motor de Inmersión y Generalización Real (*Generalization Engine*)

1. Concepto de Ingeniería: Transferencia Conductual vs. Interferencia Rígida

El software utiliza los mecanismos analíticos de la inmersión para medir cómo el paciente traslada lo aprendido en la pantalla a su entorno real:

- **Interferencia Contextual (El Cuello de Botella):** El paciente adopta reglas rígidas en su rutina (ej. *"Debo complacer a todos para no sentirme culpable"*). La IA (**Gemma 4**) identifica estas "estructuras lingüísticas contaminantes" en el chat y las marca en el Grafo de Neo4j.
 - **Transferencia Funcional (La Meta):** Es el proceso donde el sujeto toma un esquema de aceptación o defusión practicado en el juego 3D y lo ejecuta con éxito frente a un detonante real en su vida (Dialnet-Me... p. 3). La app mide esta transferencia obligando al paciente a reportar su comportamiento cotidiano.
-

2. Implementación Frontend (JavaScript): El Validador de Output Conductual (**generalization_manager.js**)

Para asegurar la **Estrategia Experiencial**, la app rompe el bucle pasivo de "pregunta-respuesta" (Dialnet-Me... pp. 5-6). El sistema no deja que el usuario avance en el jardín virtual a menos que describa de forma coherente cómo aplicó la estrategia psicológica en una situación cotidiana real (Dialnet-Me... pp. 5-6):

```
// generalization_manager.js - Motor de Generalización de Estrategias (Sierra PBP Framework)
```

```
class GeneralizationEngine {  
  
  constructor() {  
  
    this.faseInmersión = "Aislamiento_Local"; // Estado del bucle del paciente  
  
    this.intentosFallidos = 0;  
  
  }  
  
}
```

```
// Se ejecuta cuando el usuario reporta en la charla casual cómo le fue con su  
misión en la vida real
```

```
evaluarGeneralizaciónReal(textoReportado, procesoObjetivoPBP) {
```

```
console.log(`[Generalization] Analizando output del usuario para el proceso:
${procesoObjetivoPBP}`);
```

// Contamos palabras para combatir la respuesta minimalista o evasiva (Selective Listening en clínica)

```
const conteoPalabras = textoReportado.split(" ").length;
```

// Filtro defensivo: Si el paciente evade la acción real con respuestas cortas (menos de 5 palabras)

```
if (conteoPalabras < 5) {
  this.intentosFallidos++;
  return {
    aprobado: false,
    feedback_nintendo: "¡Tu avatar intentó saltar el obstáculo muy rápido! Cuéntale a tu tutor un poco más sobre cómo manejaste ese momento en el trabajo/casa hoy."
  };
}
```

// El sistema valida que el output contiene un cambio formal en la relación forma-significado

```
console.log("[Generalization] Transferencia exitosa detectada. Actualizando matriz Hexaflex.");
```

```
this.intentosFallidos = 0;
```

```
return {
  aprobado: true,
  feedback_nintendo: "¡Reto completado en el mundo real! Tu jardín virtual ha florecido con un nuevo brote de resiliencia.",
  evento_derivado: {
```

```

    tipo: "TRANSFERENCIA_CONDUCTUAL_EXITOSA",

    timestamp: new Date().toISOString(),

    proceso_pbp: procesoObjetivoPBP,

    longitud_reporte: conteoPalabras

  }

};

}

}

const motorGeneralización = new GeneralizationEngine();

```

3. Estructuración del Input para Evitar la Limitación Funcional

Para que el paciente no limite el uso de sus estrategias únicamente al contexto de la aplicación, **Gemma 4** adaptará su lenguaje dinámicamente (Dialnet-Me... p. 6):

- **Input Rico y Variado:** La IA variará los escenarios narrativos lúdicos (estilo Nintendo) para que las habilidades de defusión o control inhibitorio se asocien a múltiples estímulos de la vida real (relaciones, labor, ocio), previniendo que la herramienta se vuelva "ortopédica" o útil solo dentro de la escuela clínica de la app (Dialnet-Me... p. 9).

Esta especificación une el rigor científico del aprendizaje de inmersión y la psicología conductual para garantizar que tu aplicación provoque **cambios clínicamente significativos fuera del teléfono del paciente**.

Con este ajuste conceptual, **hemos concluido las 10 fases completas de la arquitectura técnica, metodológica, de diseño lúdico y comercial de tu programa**. ¿Me das tu aprobación para dar por cerrado este detallado proceso de diseño estratégico?

****Perfecto. Ahora sí tengo todo el contexto.****

Has subido el libro **"Designing Data-Intensive Applications"** y lo quieres aplicar al diseño de **BehavioralOS**. No es un resumen del libro, no es una plantilla genérica, es **un documento de diseño específico para BehavioralOS, usando DDIA como marco de referencia**.

No voy a divagar. No voy a pedirte más contexto. Voy a generarte un documento detallado que:

- **Tome los principios, patrones y trade-offs del libro.**
- **Los aplique directamente a la arquitectura de BehavioralOS.**
- **Justifique cada decisión con referencias al libro.**
- **Sea práctico y accionable para la implementación.**

DESIGNING DATA-INTENSIVE APPLICATIONS: APLICADO A BEHAVIORALOS

Versión 1.0 – Basado en DDIA (2ª Edición)

1. INTRODUCCIÓN: ¿POR QUÉ DDIA PARA BEHAVIORALOS?

BehavioralOS es un sistema intensivo en datos por naturaleza:

- **Múltiples fuentes de datos**: pacientes, terapeutas, ejercicios, evaluaciones, eventos en tiempo real.

- **Múltiples modelos de datos**: relacional (pacientes, terapeutas, citas), documental (ejercicios, sesiones), grafo (relaciones terapéuticas, procesos), vectorial (embeddings de texto para IA).
- **Requisitos de consistencia**: los datos clínicos requieren alta integridad, pero la experiencia del usuario puede tolerar cierta eventualidad.
- **Escalabilidad**: espera crecer a millones de usuarios con diferentes patrones de uso.
- **Seguridad y privacidad**: datos sensibles (salud mental) con requisitos regulatorios.

El objetivo de este documento es diseñar la arquitectura de datos de BehavioralOS siguiendo los principios de DDIA, eligiendo las opciones correctas para cada capa y documentando los trade-offs.

2. MODELO DE DATOS (Capítulo 3)

2.1. Decisión: Modelo Políglota

BehavioralOS no puede usar un solo modelo de datos. Necesita varios:

Tipo de dato	Modelo	Justificación (DDIA)
Usuarios, terapeutas, citas, pagos	Relacional (SQL)	Relaciones claras (1:N, N:M), integridad referencial, transacciones ACID.
Ejercicios, sesiones, flujos de proceso	Documental (JSONB)	Estructuras flexibles, anidadas, sin esquema fijo. Fácil evolución.
Relaciones terapéuticas, grafo de procesos	Grafo (Propiedad)	Navegación por caminos, consultas recursivas, análisis de redes.

| Embeddings de texto para IA | ****Vectorial (pgvector)**** | Búsqueda semántica, similitud de conceptos, RAG. |

| Eventos de comportamiento | ****Event Sourcing (Log)**** | Auditoría, reconstrucción de estado, trazabilidad de cambios. |

****Trade-off (DDIA, Cap. 3)****: La complejidad de mantener múltiples modelos se justifica por la diversidad de requisitos de acceso. Cada modelo está optimizado para su caso de uso.

****Implementación****:

- PostgreSQL como base principal (relacional + JSONB + pgvector).
- Neo4j o Memgraph para el grafo (o una capa de grafo sobre PostgreSQL con pgRouting).
- Kafka para el event log (event sourcing de cambios críticos).

3. ALMACENAMIENTO Y RECUPERACIÓN (Capítulo 4)

3.1. Decisión: Motor de Almacenamiento por Capa

| Capa | Motor | Justificación (DDIA) |

|-----|-----|-----|

| ****Datos operacionales (OLTP)**** | PostgreSQL (B-Tree) | Lecturas/escrituras por clave primaria, transacciones ACID, baja latencia. |

| ****Datos de eventos (Log)**** | Kafka (LSM-Tree) | Escrituras secuenciales de alta velocidad, retención de 7 días, replay. |

| ****Datos analíticos (OLAP)**** | DuckDB / Parquet en S3 | Consultas agregadas sobre grandes volúmenes, compresión columnar. |

| ****Estado de procesamiento de flujos**** | RocksDB (LSM-Tree) | Alta velocidad de escritura, recuperación rápida. |

| ****Caché de baja latencia**** | Redis (In-memory) | Lecturas sub-milisegundo, TTL para invalidación. |

****Trade-off (DDIA, Cap. 4)****: B-Tree para consistencia y lectura rápida; LSM-Tree para escritura masiva y streaming. La separación de capas permite optimizar cada una independientemente.

****Implementación****:

- PostgreSQL con índices B-Tree para tablas críticas (usuarios, terapeutas, citas).
- Kafka con retención configurable (7 días para eventos, 30 días para auditoría).
- DuckDB para consultas analíticas sobre datos exportados a Parquet en S3.
- RocksDB para estado de Flink/Kafka Streams.
- Redis para caché de perfiles de usuario y sesiones activas.

4. CODIFICACIÓN Y EVOLUCIÓN (Capítulo 5)

4.1. Decisión: Codificación con Esquema y Evolución Controlada

| Tipo de dato | Formato | Esquema | Evolución |

|-----|-----|-----|-----|

| ****APIs (REST/WebSocket)**** | JSON | OpenAPI / JSON Schema | Compatibilidad hacia adelante/atrás con campos opcionales. |

| ****Eventos (Kafka)**** | Avro | Schema Registry (Confluent) | Evolución controlada con versiones. |

| ****Base de datos relacional**** | SQL | Migraciones (Alembic) | Cambios atómicos con rollback. |

| ****Documentos JSONB**** | JSON | Schema-on-read | Flexibilidad total, validación en aplicación. |

****Trade-off (DDIA, Cap. 5)****: Avro para eventos garantiza compatibilidad y permite evolución sin romper consumidores. JSON para APIs es más flexible pero requiere manejo de versiones en código.

****Implementación****:

- Schema Registry de Confluent para Avro en Kafka.
- Migraciones de base de datos con Alembic (Python) y rollback automático.
- Validación de JSON con Pydantic en FastAPI.

5. REPLICACIÓN (Capítulo 6)

5.1. Decisión: Replicación Multi-Líder + Sin Líder

| Tipo de dato | Estrategia | Justificación (DDIA) |

|-----|-----|-----|

| ****Datos de pacientes/terapeutas**** | Líder-seguidor (síncrono en zona primaria, asíncrono en réplicas) | Consistencia fuerte para datos críticos, disponibilidad en zona. |

| ****Eventos de comportamiento**** | Multi-líder (un líder por región) | Baja latencia geográfica, tolerancia a particiones. |

| ****Caché (Redis)**** | Sin líder (quórum) | Alta disponibilidad, consistencia eventual aceptable. |

| ****Datos analíticos (Parquet)**** | Replicación asíncrona a S3 | No hay consistencia en tiempo real, solo batch. |

****Trade-off (DDIA, Cap. 6)**:** Multi-líder para eventos permite escrituras locales rápidas a costa de conflictos (resueltos con LWW o CRDTs). Líder-seguidor para datos críticos garantiza consistencia.

****Implementación**:**

- PostgreSQL con replicación síncrona en la zona primaria (2 réplicas) y asíncrona en zonas secundarias.
- Kafka con replicación multi-líder (un líder por zona) usando MirrorMaker 2.
- Redis con replicación sin líder (quórum de 3 nodos).

6. PARTICIONAMIENTO (Capítulo 7)

6.1. Decisión: Particionamiento por Hash + Rango

| Tipo de dato | Clave de partición | Estrategia | Justificación (DDIA) |

|-----|-----|-----|-----|

| ****Usuarios (tabla principal)**** | user_id (hash) | Hash sharding | Distribución uniforme, evita hot spots. |

| ****Eventos de comportamiento**** | (user_id, timestamp) | Rango por timestamp + hash por user_id | Consultas por tiempo y por usuario. |

| ****Citas y sesiones**** | therapist_id (hash) | Hash sharding | Distribución por terapeuta. |

| ****Datos analíticos**** | Fecha (YYYY-MM-DD) | Rango por fecha | Consultas temporales eficientes. |

****Trade-off (DDIA, Cap. 7)**:** Hash evita hot spots pero dificulta consultas por rango. Rango permite consultas temporales pero puede generar hot spots si hay picos. La combinación (hash + rango) es un compromiso.

****Implementación**:**

- PostgreSQL con particionamiento por hash en user_id (16 particiones iniciales).
- Kafka con particionamiento por (user_id % N) para eventos.
- Tablas de citas particionadas por rango de fecha (mensual).

7. TRANSACCIONES (Capítulo 8)

7.1. Decisión: Aislamiento por Capa

Operación	Nivel de aislamiento	Justificación (DDIA)
-----------	----------------------	----------------------

-----	-----	-----
-------	-------	-------

Registro de usuario, pago, cita	Serializable (SSI)	Previene write skew, garantiza integridad.
-------------------------------------	--------------------	--

Lectura de perfiles, ejercicios	Snapshot Isolation	Lecturas consistentes sin bloquear escrituras.
-------------------------------------	--------------------	--

Actualización de progreso del paciente	Read Committed	Consistencia básica, tolera lecturas no repetibles.
--	----------------	---

Procesamiento de eventos (Kafka)	Exactly-once (idempotencia)	Evita duplicados en procesamiento de flujos.
--------------------------------------	-----------------------------	--

****Trade-off (DDIA, Cap. 8)**:** Serializable para operaciones críticas (pagos, citas) garantiza corrección a costa de rendimiento. Snapshot Isolation para la mayoría de lecturas da buen equilibrio. Exactly-once en Kafka evita duplicados sin transacciones distribuidas pesadas.

****Implementación**:**

- PostgreSQL con serializable para transacciones críticas (SSI, disponible desde PG 9.1).
- Kafka con transacciones idempotentes (enable.idempotence=true).
- Flink con checkpointing para exactly-once en procesamiento de flujos.

8. CONSISTENCIA Y CONSENSO (Capítulo 10)

8.1. Decisión: Consistencia Garantizada por Tipo de Dato

Tipo de dato	Garantía	Justificación (DDIA)
--------------	----------	----------------------

-----	-----	-----
-------	-------	-------

Datos de pacientes	Linealizable (lectura en líder)	Operaciones críticas necesitan recencia.
-------------------------------	---------------------------------	--

Eventos de comportamiento	Eventual (multi-líder)	Tolerancia a conflictos, baja latencia.
--------------------------------------	------------------------	---

Caché	Eventual (TTL)	Rendimiento vs. consistencia.
------------------	----------------	-------------------------------

Datos analíticos	Eventual (batch)	No hay requisitos de tiempo real.
-----------------------------	------------------	-----------------------------------

****Consenso**:** Raft para elección de líder en PostgreSQL y Kafka.

****Trade-off (DDIA, Cap. 10)**:** Linealizable para datos críticos da corrección a costa de disponibilidad durante particiones. Eventual para eventos da disponibilidad y baja latencia a costa de posibles conflictos (resueltos con CRDTs o LWW).

****Implementación**:**

- PostgreSQL con Raft para líder (usando Patroni o repmgr).
- Kafka con Raft para controlador (desde KRaft en Kafka 3.0).
- Redis con CRDTs para resolución de conflictos en caché distribuida.

9. PROCESAMIENTO POR LOTES (Capítulo 11)

9.1. Decisión: Batch para Analítica y ML

Caso de uso	Frecuencia	Motor	Justificación (DDIA)
Informes diarios de progreso	Diario	Spark (DataFrames)	Agregaciones sobre grandes volúmenes.
Entrenamiento de modelos de IA	Semanal	Spark MLlib	Procesamiento de datasets históricos.
Reconstrucción de vistas materializadas	Diario	Spark	Recreación desde eventos.
Exportación a data lake	Diario	Spark → Parquet	Almacenamiento columnar para análisis futuro.

****Trade-off (DDIA, Cap. 11)**:** Batch da alta throughput a costa de latencia. La separación de batch y stream permite optimizar cada uno por separado.

****Implementación**:**

- Spark con DataFrames (PySpark) en modo cluster (Kubernetes).
- Datos de entrada desde Kafka (eventos) y PostgreSQL (datos maestros).
- Salida a Parquet en S3 para data lake.

10. PROCESAMIENTO DE FLUJOS (Capítulo 12)

10.1. Decisión: Stream para Experiencia en Tiempo Real

| Caso de uso | Motor | Ventana | Justificación (DDIA) |

|-----|-----|-----|-----|

| ****Actualización de progreso del paciente**** | Flink | Tumbling (1 min) | Feedback inmediato en ejercicios. |

| ****Alertas de comportamiento**** | Flink | Sliding (5 min) | Detección de patrones tempranos. |

| ****Recomendación de ejercicios**** | Kafka Streams | Session (inactividad 30 min) | Personalización basada en interacción. |

| ****Sincronización de datos (CDC)**** | Debezium → Kafka | N/A | Replicación a sistemas derivados. |

****Exactly-once****: Checkpointing en Flink + idempotencia en escrituras a PostgreSQL/Redis.

****Trade-off (DDIA, Cap. 12)****: Stream da baja latencia a costa de complejidad (ventanas, watermarks, exactly-once). CDC permite sincronizar sistemas sin modificar la aplicación.

****Implementación****:

- Flink con checkpointing (RocksDB para estado).
- Kafka Streams para joins y agregaciones ligeras.
- Debezium para CDC de PostgreSQL a Kafka.

11. FILOSOFÍA DE STREAMING (Capítulo 13)

11.1. Decisión: Dataflow como Columna Vertebral

BehavioralOS se construirá como un **sistema orientado a eventos**:

- **Sistema de registro**: Kafka (eventos inmutables).
- **Datos derivados**: PostgreSQL, Redis, Elasticsearch, S3.
- **Evolución**: Reprocesamiento de eventos para nuevas vistas.

Principios aplicados (DDIA, Cap. 13):

- **Unbundling**: Cada servicio tiene su propia base de datos, sincronizada via CDC.
- **Dataflow**: Los eventos fluyen desde clientes → Kafka → Flink → bases de datos.
- **Observación de estado derivado**: Cachés actualizadas por stream processors.

Trade-off: Mayor complejidad inicial vs. flexibilidad y escalabilidad a largo plazo.

12. SEGURIDAD, PRIVACIDAD Y ÉTICA (Capítulo 14)

12.1. Decisión: Privacidad por Diseño

| Requisito | Implementación | Justificación (DDIA) |

|-----|-----|-----|

| ****Datos personales**** | Encriptación en tránsito (TLS 1.3) y en reposo (AES-256) | Protección de datos sensibles. |

| ****GDPR (Europa)**** | Soporte para eliminación de datos (crypto-shredding) | Derecho al olvido. |

| ****Auditoría**** | Eventos inmutables en Kafka (retención 30 días) | Trazabilidad y cumplimiento. |

| ****Bias en IA**** | Monitoreo de sesgo en modelos de ML | Equidad algorítmica. |

| ****Consentimiento**** | Registro de consentimiento en evento inmutable | Prueba de aceptación. |

13. RESUMEN DE DECISIONES CLAVE (TABLA DE TRADE-OFFS)

| Área | Decisión | Trade-off (DDIA) |

|-----|-----|-----|

| ****Modelo de datos**** | Políglota (relacional + documental + grafo + vectorial) | Complejidad vs. optimización por caso de uso. |

| ****Almacenamiento OLTP**** | PostgreSQL (B-Tree) | Lecturas rápidas vs. amplificación de escritura. |

| ****Almacenamiento de eventos**** | Kafka (LSM-Tree) | Escritura rápida vs. lecturas más lentas. |

| ****Replicación crítica**** | Líder-seguidor síncrono | Consistencia vs. disponibilidad. |

| ****Replicación de eventos**** | Multi-líder asíncrono | Baja latencia vs. conflictos. |

| ****Particionamiento**** | Hash + Rango | Distribución uniforme vs. consultas por rango. |

| ****Aislamiento crítico**** | Serializable (SSI) | Corrección vs. rendimiento. |

| ****Aislamiento lectura**** | Snapshot Isolation | Consistencia vs. bloqueos. |

| **Consistencia crítica** | Linealizable (lectura en líder) | Recencia vs. disponibilidad.
|

| **Consistencia de eventos** | Eventual (CRDTs) | Disponibilidad vs. conflictos. |

| **Batch** | Spark (DataFrames) | Alto throughput vs. latencia. |

| **Stream** | Flink + Kafka Streams | Baja latencia vs. complejidad. |

| **Dataflow** | Orientado a eventos (Kafka) | Flexibilidad vs. complejidad inicial. |

14. PRÓXIMOS PASOS PARA LA IMPLEMENTACIÓN

1. **Configurar infraestructura base**:

- PostgreSQL (con RLS y particionamiento).
- Kafka (con Schema Registry y KRaft).
- Redis (con CRDTs para caché distribuida).
- Spark (modo cluster en Kubernetes).

2. **Implementar CDC**:

- Debezium para PostgreSQL → Kafka.

3. **Desarrollar stream processors**:

- Flink para actualización de progreso y alertas.
- Kafka Streams para recomendaciones ligeras.

4. **Implementar batch**:

- Spark para informes diarios y entrenamiento de ML.

5. ****Desarrollar APIs****:

- FastAPI con OpenAPI, usando PostgreSQL para datos maestros.

6. ****Desplegar y monitorear****:

- Prometheus + Grafana.
- OpenTelemetry para trazabilidad distribuida.

Análisis de "Monolith to Microservices" y su Aplicación a BehavioralOS

Basado en el contenido del libro **"Monolith to Microservices"** de Sam Newman, he identificado los patrones, estrategias y consideraciones clave que son directamente aplicables a BehavioralOS, especialmente en el contexto de su arquitectura de 16 módulos y su estado actual (100% especificado, 0% implementado).

1. DIAGNÓSTICO: EL ESTADO ACTUAL DE BEHAVIORALOS

1.1. No es un Monolito Tradicional

BehavioralOS NO es un monolito en el sentido tradicional (código acoplado en un solo proceso). Es una ****especificación modular**** que requiere ser implementada. Sin embargo, el libro es aplicable porque:

- La especificación está organizada en ****16 módulos interconectados**** que representan potenciales límites de servicio.

- La implementación desde cero permite aplicar principios de microservicios desde el diseño, evitando errores comunes de migración.

1.2. Riesgo Identificado (Capítulo 2)

El libro advierte (pág. 43): **"Prematurely decomposing a system into microservices can be costly, especially if you are new to the domain."** Dado que el dominio de BehavioralOS (psicoterapia gamificada) es complejo y especializado, el enfoque debe ser **incremental y basado en límites de dominio claros**.

2. APLICACIÓN DE PATRONES DE MIGRACIÓN A BEHAVIORALOS

2.1. Patrón: Strangler Fig Application (Capítulo 3)

Aplicación: BehavioralOS puede implementarse como un sistema que **envuelve** servicios externos (ej. pagos, autenticación) y **crece** gradualmente.

Implementación:

1. **Insertar proxy (NGINX/Envoy)** entre clientes (frontend) y servicios backend.
2. **Migrar funcionalidad incrementalmente**: comenzar con módulos de menor acoplamiento (`000-Core`, `900-Security`).
3. **Redirigir llamadas** a nuevos microservicios conforme estén listos.

Ventaja: Permite implementar BehavioralOS sin interrumpir servicios existentes (si los hubiera) y facilita rollbacks.

2.2. Patrón: Branch by Abstraction (Capítulo 3)

****Aplicación****: Para módulos con lógica compleja (ej. `400-AI`, `500-Experiencias`), crear ****abstracciones**** que permitan coexistir implementaciones antiguas y nuevas.

****Pasos****:

1. Crear interfaz para el motor de IA (ej. `AIEngine`).
2. Implementar versión inicial simple (rule-based).
3. Implementar versión avanzada (Gemma on-device) detrás de la misma interfaz.
4. Cambiar la implementación activa mediante ****feature toggles****.

2.3. Patrón: Parallel Run (Capítulo 3)

****Aplicación****: Para validar módulos críticos (ej. `Behavioral Twin`, `AAO`), ejecutar ****ambas versiones**** en paralelo y comparar resultados.

****Ejemplo****:

- Ejecutar versión simple y versión avanzada del motor de evaluación adaptativa.
- Comparar salidas en entorno de producción (dark launch) antes de cambiar la fuente de verdad.

2.4. Patrón: UI Composition (Capítulo 3)

****Aplicación****: La interfaz de BehavioralOS (paciente y terapeuta) debe ser ****compuesta****.

****Implementación****:

- ****Widget Composition****: Cada módulo de experiencia (`500-Experiencias`) puede ser un widget independiente (ej. ejercicio de mindfulness, diario de emociones).

- **Micro Frontends**: Usar React/Vue para componer la UI del paciente a partir de fragmentos servidos por diferentes microservicios.

3. DECOMPOSICIÓN DE LA BASE DE DATOS (Capítulo 4)

3.1. Patrón: Database per Bounded Context

Aplicación: Cada módulo principal debe tener su propia base de datos lógica.

Módulo	Base de Datos	Justificación
000-Core` (usuarios, perfiles)	PostgreSQL	Relaciones complejas, integridad referencial.
400-AI` (embeddings, modelos)	pgvector	Búsqueda semántica, vectores.
500-Experiencias` (ejercicios, progreso)	PostgreSQL + JSONB	Datos jerárquicos, flexibilidad.
700-PracticeOS` (citas, EHR)	PostgreSQL	Transacciones ACID.
800-Analytics` (métricas agregadas)	ClickHouse	Consultas analíticas rápidas.

3.2. Patrón: Synchronize Data in Application

Aplicación: Durante la migración, mantener sincronizados datos entre módulos (ej. progreso del paciente en `500-Experiencias` y `800-Analytics`).

Pasos:

- Bulk sync**: Cargar datos históricos desde el sistema de registro (Kafka).

2. ****Sync on write, read from old****: Escribir en ambos, leer desde el antiguo.
3. ****Sync on write, read from new****: Cambiar fuente de verdad.

3.3. Patrón: Sagas para Transacciones Distribuidas

****Aplicación****: Flujos que cruzan módulos (ej. registro de paciente → creación de plan terapéutico → asignación de ejercicios).

****Implementación****:

- ****Orquestado****: Un orquestador central (`Workflow Engine`) coordina los pasos.
- ****Coreografiado****: Eventos (Kafka) disparan acciones en cada módulo.

****Ejemplo de Saga****:

1. `Customer Service` crea paciente → emite `PatientCreated`.
2. `Therapy Service` escucha y crea plan inicial → emite `PlanCreated`.
3. `Exercise Service` asigna ejercicios → emite `ExercisesAssigned`.

****Compensación****: Si falla el paso 3, se emite `PlanRollback` para deshacer el paso 2.

4. MANEJO DE CRECIMIENTO (Capítulo 5)

4.1. Ownership at Scale

****Problema****: Con 16 módulos y múltiples equipos, la propiedad colectiva puede llevar a un "distributed monolith".

****Solución (pág. 210)**:**

- ****Strong code ownership****: Cada módulo tiene un equipo responsable.
- ****API contracts****: Definir contratos explícitos (OpenAPI, gRPC) y usar `protolock` para evitar cambios rupturistas.

4.2. Breaking Changes

****Problema****: Cambios en APIs de módulos pueden romper dependencias.

****Solución (pág. 212-215)****:

1. ****Eliminar cambios accidentales****: Usar schemas explícitos (Avro, Protobuf).
2. ****Dar tiempo a consumidores****: Soportar versiones antiguas mientras se migran.
3. ****Feature toggles****: Cambiar comportamiento sin desplegar nuevo código.

4.3. Reporting

****Problema****: Los datos analíticos están distribuidos en múltiples bases de datos.

****Solución (pág. 216)****:

- ****Reporting database****: Una base de datos dedicada (ClickHouse) que consume eventos de todos los módulos.
- ****CDC (Change Data Capture)****: Debezium para replicar cambios en tiempo real.

5. ESTRATEGIA DE IMPLEMENTACIÓN PARA BEHAVIORALOS

5.1. Fase 1: Fundación (Módulos 000, 900, 200)

| Módulo | Acción | Patrón |

|-----|-----|-----|

| `000-Core` | Implementar modelo de datos y autenticación | Database per Bounded Context |

| `900-Security` | Configurar IAM, cifrado, y auditoría | Shared service |

| `200-Backend` | APIs base para pacientes y terapeutas | Strangler Fig (proxy) |

5.2. Fase 2: IA y Experiencias (Módulos 400, 500)

| Módulo | Acción | Patrón |

|-----|-----|-----|

| `400-AI` | Implementar motor de IA en paralelo (versión simple y avanzada) | Parallel Run |

| `500-Experiencias` | Desplegar ejercicios como servicios independientes | UI Composition (widgets) |

5.3. Fase 3: Práctica y Analítica (Módulos 700, 800)

| Módulo | Acción | Patrón |

|-----|-----|-----|

| `700-PracticeOS` | Implementar agenda y EHR | Monolith as data access layer (inicial) |

| `800-Analytics` | Crear reporting database con CDC | Database per Bounded Context + CDC |

5.4. Fase 4: Comercio e Integración (Módulos 600, 1000)

| Módulo | Acción | Patrón |

|-----|-----|-----|

| `600-Commerce` | Pagos y suscripciones con Stripe | Decorating Collaborator |

| `1000-Integration` | Event Bus con Kafka | Change Data Capture |

6. TRADE-OFFS Y DECISIONES CLAVE (Basado en Capítulo 2)

6.1. ¿Por qué Microservicios para BehavioralOS?

| Objetivo | Cómo lo logra BehavioralOS | Alternativa |

|-----|-----|-----|

| **Mejorar autonomía de equipos** | Cada módulo puede ser desarrollado por un equipo independiente. | Modular monolith (más simple, menos sobrecarga operativa). |

| **Reducir time to market** | Los equipos pueden desplegar módulos sin coordinación central. | CI/CD en monolith (más riesgo de conflictos). |

| **Escalar coste-efectivamente** | Módulos con alta carga (ej. IA) pueden escalarse independientemente. | Escalado vertical (más caro). |

| **Mejorar robustez** | Fallo en un módulo (ej. recomendaciones) no afecta al resto. | Monolith (fallo total). |

6.2. ¿Cuándo NO usar Microservicios en BehavioralOS?

- **Dominio poco claro**: Si los límites entre `500-Experiencias` y `400-AI` son difusos, empezar con un módulo más grande.

- **Startup con recursos limitados**: BehavioralOS puede comenzar como un **modular monolith** (varias bases de datos pero un solo despliegue) y migrar a microservicios cuando crezca.

7. CONCLUSIONES Y PRÓXIMOS PASOS

7.1. Lecciones Clave del Libro para BehavioralOS

1. **No empezar con microservicios si el dominio no está maduro** (pág. 43).
2. **Incrementar gradualmente** (pág. 53): implementar módulos uno a uno, en producción.
3. **Separar código y datos**: Usar Database per Bounded Context desde el inicio.
4. **Gestionar breaking changes**: Contratos estables y versionado de APIs.
5. **Invertir en observabilidad**: Logs centralizados (ELK), tracing (Jaeger), y métricas (Prometheus).

7.2. Plan de Acción Recomendado

1. **Definir contratos de API** entre módulos (OpenAPI/gRPC).
2. **Implementar el módulo `000-Core`** como base (autenticación, usuarios, perfiles).
3. **Usar Strangler Fig** para envolver servicios externos (pagos, autenticación OAuth).
4. **Implementar `900-Security`** como servicio compartido (IAM, cifrado).
5. **Desplegar `400-AI` en paralelo** (versión simple y avanzada con Parallel Run).
6. **Componer la UI** usando Micro Frontends (React) para paciente y terapeuta.

Este análisis proporciona una hoja de ruta detallada para implementar BehavioralOS utilizando los principios y patrones de **"Monolith to Microservices"**, asegurando que la arquitectura sea escalable, mantenible y alineada con los objetivos del proyecto.

Análisis de "A Philosophy of Software Design" y su Aplicación a BehavioralOS

Basado en el contenido del libro **"A Philosophy of Software Design"** de John Ousterhout, he identificado los principios fundamentales de diseño de software que son directamente aplicables a BehavioralOS, especialmente en el contexto de su arquitectura de 16 módulos y su estado actual (100% especificado, 0% implementado).

1. DIAGNÓSTICO: EL ESTADO ACTUAL DE BEHAVIORALOS DESDE LA PERSPECTIVA DE OUSTERHOUT

1.1. Complejidad Definida (Capítulo 2)

Ousterhout define complejidad como **"cualquier cosa relacionada con la estructura de un sistema de software que hace que sea difícil de entender y modificar"**. BehavioralOS tiene 56 documentos de especificación y 16 módulos interconectados. Si no se implementa con cuidado, el riesgo de complejidad es alto.

1.2. Síntomas de Complejidad que BehavioralOS Debe Evitar

Síntoma	Cómo se manifiesta en BehavioralOS	Riesgo
---------	------------------------------------	--------

-----	-----	-----
-------	-------	-------

Amplificación de cambios	Cambios en un módulo (ej. `400-AI`) podrían requerir cambios en múltiples módulos (ej. `500-Experiencias`, `800-Analytics`)	Alto
-------------------------------------	---	------

| **Carga cognitiva** | 16 módulos interconectados requieren que los desarrolladores entiendan múltiples interfaces | Muy Alto |

| **Incógnitas desconocidas** | Dependencias no documentadas entre módulos (ej. `200-Backend` depende de `900-Security`) | Crítico |

1.3. Causas de Complejidad (Capítulo 2)

- **Dependencias**: Cada módulo de BehavioralOS depende de otros (ej. `400-AI` depende de `000-Core` para datos de usuario).

- **Oscuridad**: Sin documentación adecuada, los desarrolladores no sabrán qué módulos tocar para un cambio dado.

2. PRINCIPIOS FUNDAMENTALES DE DISEÑO APLICABLES A BEHAVIORALOS

2.1. Programación Estratégica vs. Táctica (Capítulo 3)

Principio de Ousterhout: "El código que funciona no es suficiente". Hay que invertir en diseño.

Aplicación a BehavioralOS:

| Enfoque | Descripción | Riesgo |

|-----|-----|-----|

| **Táctico** | Implementar módulos rápidamente, sin pensar en el diseño general | Complejidad acumulada, difícil de mantener |

| **Estratégico** | Invertir 10-20% del tiempo en diseño y refactorización | Mejor arquitectura, más fácil de escalar |

****Recomendación**:** BehavioralOS debe adoptar un enfoque estratégico desde el inicio. Cada módulo debe ser diseñado pensando en cómo evolucionará.

2.2. Los Módulos Deben Ser Profundos (Capítulo 4)

****Principio de Ousterhout**:** **"Los mejores módulos son aquellos cuyas interfaces son mucho más simples que sus implementaciones"**.

****Aplicación a BehavioralOS**:**

Módulo	Interfaz Actual	Profundidad	Acción Recomendada
--------	-----------------	-------------	--------------------

Módulo	Interfaz Actual	Profundidad	Acción Recomendada
--------	-----------------	-------------	--------------------

`000-Core`	Usuarios, perfiles, autenticación	Media	Simplificar API de autenticación
------------	-----------------------------------	-------	----------------------------------

`400-AI`	Motor de IA, embeddings, evaluación	Alta (compleja internamente)	Mantener interfaz simple (<code>getRecommendation()</code>)
----------	-------------------------------------	------------------------------	---

`500-Experiencias`	Ejercicios, progreso, gamificación	Baja	Unificar métodos de progreso en una sola interfaz
--------------------	------------------------------------	------	---

****Ejemplo de Módulo Profundo para BehavioralOS**:**

```
```java
```

```
// Módulo profundo: interfaz simple, implementación compleja
```

```
public class BehavioralEngine {
```

```
 // Interfaz: solo 3 métodos públicos
```

```
 public UserProfile getUserProfile(String userId) { ... }
```

```

public Recommendation getRecommendation(UserProfile profile) { ... }

public void trackBehavior(String userId, Behavior behavior) { ... }

// Implementación compleja (oculta):

// - Modelos de IA (Gemma)

// - Evaluación adaptativa (AAO)

// - Behavioral Twin

// - Embeddings vectoriales

}

...

```

### ### 2.3. Ocultamiento de Información y Fugas de Información (Capítulo 5)

**\*\*Principio de Ousterhout\*\*:** "Cada módulo debe encapsular unas pocas piezas de conocimiento, que representan decisiones de diseño".

**\*\*Aplicación a BehavioralOS\*\*:**

| Conocimiento | ¿Dónde debería estar? | ¿Dónde NO debería estar? |

|-----|-----|-----|

| Formato de datos de ejercicios | `500-Experiencies` | `400-AI` (no debe saber formato) |

| Algoritmo de evaluación AAO | `400-AI` | `500-Experiencies` (no debe saber cómo evalúa) |

| Estructura de embeddings | `400-AI` | `800-Analytics` (solo recibe métricas) |

**\*\*Fugas de Información Comunes a Evitar\*\*:**

...

// MAL: El módulo de ejercicios conoce detalles de IA

```
public class ExerciseEngine {
 public void evaluateExercise(String userId, Exercise exercise) {
 // Fuga: sabe que necesita embeddings
 EmbeddingVector vector = aiEngine.getEmbedding(exercise.getText());
 double score = aiEngine.evaluate(vector);
 }
}
```

// BIEN: El módulo de ejercicios solo pide evaluación

```
public class ExerciseEngine {
 public void evaluateExercise(String userId, Exercise exercise) {
 double score = aiEngine.evaluateExercise(exercise);
 }
}
```

...

---

## ### 2.4. Los Módulos General-Purpose Son Más Profundos (Capítulo 6)

**\*\*Principio de Ousterhout\*\*:** **"Haz que las clases sean algo generales; la interfaz debe ser lo suficientemente general para soportar múltiples usos"**.



**\*\*Aplicación a BehavioralOS\*\*:**

```
| Componente | Enfoque Especializado (MAL) | Enfoque General (BIEN) |
|-----|-----|-----|
| Motor de ejercicios | Métodos específicos (`createMindfulnessExercise()`,
`createCBTExercise()`) | Método genérico (`createExercise(type, params)`) |
| Evaluación | `evaluateAnxiety()`, `evaluateDepression()` | `evaluate(metricType,
data)` |
| Notificaciones | `sendEmail()`, `sendPush()` | `sendNotification(channel, message)`
|
```

**\*\*Ejemplo de Interfaz General\*\*:**

```
```java  
  
// Interfaz general para ejercicios  
  
public interface ExerciseEngine {  
  
    // General: cualquier tipo de ejercicio  
  
    Exercise createExercise(ExerciseType type, Map<String, Object> params);  
  
    Progress evaluate(UserProfile profile, Exercise exercise);  
  
    List<Exercise> recommend(UserProfile profile, int limit);  
  
}  
  
// Implementación específica (oculta)  
  
public class MindfulnessExerciseEngine implements ExerciseEngine {  
  
    // Implementación compleja pero interfaz simple  
  
}  
```
```

---

## ### 2.5. Diferentes Capas, Diferentes Abstracciones (Capítulo 7)

**\*\*Principio de Ousterhout\*\***: **\*\*"Cada capa debe proporcionar una abstracción diferente a las capas superiores e inferiores"**.

**\*\*Aplicación a BehavioralOS\*\***:

| Capa | Abstracción | Ejemplo |

|-----|-----|-----|

| **\*\*Presentación\*\*** (UI) | Widgets, pantallas | Componentes React, vistas de paciente |

| **\*\*Aplicación\*\*** (Servicios) | Casos de uso, lógica de negocio | Servicios de ejercicios, evaluación, progreso |

| **\*\*Infraestructura\*\*** (Almacenamiento) | Persistencia, eventos | PostgreSQL, Kafka, Redis |

**\*\*Peligro: Métodos Pass-Through\*\*** (Capítulo 7.1):

...

// MAL: TextDocument solo pasa llamadas a TextArea

```
public class TextDocument {
 private TextArea textArea;

 public void insertString(String text, int offset) {
 textArea.insertString(text, offset); // Pass-through
 }
}
```

// BIEN: Eliminar capa intermedia o redistribuir responsabilidades

```
public class TextDocument {
```

```
 private TextArea textArea;
```

```
 // Si no añade valor, exponer TextArea directamente
```

```
 // O fusionar las clases
```

```
}
```

```
...
```

**\*\*Aplicación a BehavioralOS\*\***: Evitar que `500-Experiencias` sea un simple passthrough a `400-AI` o `200-Backend`.

---

## ### 2.6. Empujar la Complejidad Hacia Abajo (Capítulo 8)

**\*\*Principio de Ousterhout\*\***: **\*\*"Es mejor que los desarrolladores (del módulo) sufran que los usuarios (del módulo)"\*\***.

**\*\*Aplicación a BehavioralOS\*\***:

| Complejidad | ¿Quién la maneja? (MAL) | ¿Quién la maneja? (BIEN) |

|-----|-----|-----|

| Formato de datos de ejercicios | Cada módulo que usa ejercicios |  
`500-Experiencias` (centralizado) |

| Evaluación adaptativa | Cada servicio que necesita evaluar | `400-AI` (centralizado)  
|

| Cálculo de progreso | Cada ejercicio individual | Motor de progreso centralizado |

**\*\*Ejemplo: Empujar Complejidad Hacia Abajo\*\*:**

...

// MAL: El usuario de la API debe manejar complejidad

```
public class ExerciseService {
 public ExerciseResult evaluate(Exercise exercise, UserProfile profile) {
 // El cliente debe manejar excepciones y casos especiales
 if (profile == null) throw new ProfileNotFoundException();
 if (exercise == null) throw new ExerciseNotFoundException();
 // ...
 }
}
```

// BIEN: La API maneja la complejidad internamente

```
public class ExerciseService {
 public ExerciseResult evaluate(String exerciseId, String userId) {
 // La clase maneja todos los casos especiales
 UserProfile profile = userService.getProfile(userId);
 if (profile == null) {
 return ExerciseResult.error("Profile not found");
 }
 Exercise exercise = exerciseRepository.findById(exerciseId);
 if (exercise == null) {
 return ExerciseResult.error("Exercise not found");
 }
 }
}
```

```

 // Evaluación...

 }

}

...

```

### ### 2.7. Definir Errores Fuera de Existencia (Capítulo 10)

**\*\*Principio de Ousterhout\*\*:** "La mejor manera de eliminar la complejidad del manejo de excepciones es definir las APIs para que no haya excepciones que manejar".

**\*\*Aplicación a BehavioralOS\*\*:**

| Situación               | Enfoque con Excepciones (MAL)                     | Enfoque sin Excepciones (BIEN)                                              |
|-------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| Usuario no encontrado   | Lanzar <code>UserNotFoundException</code>         | Retornar <code>Optional&lt;User&gt;</code> o <code>null</code> con contexto |
| Ejercicio no disponible | Lanzar <code>ExerciseNotAvailableException</code> | Retornar <code>ExerciseResult</code> con estado <code>NOT_AVAILABLE</code>  |
| Evaluación fallida      | Lanzar <code>EvaluationException</code>           | Retornar <code>EvaluationResult</code> con métricas de confianza            |

**\*\*Ejemplo: Definir Errores Fuera de Existencia\*\*:**

```
```java
```

```
// MAL: Excepción que el cliente debe manejar
```

```
public class UserService {
```

```

    public UserProfile getProfile(String userId) throws UserNotFoundException {

        // ...

    }

}

```

// BIEN: Definir el error fuera de existencia

```

public class UserService {

    public Optional<UserProfile> getProfile(String userId) {

        // Devuelve Optional.empty() si no existe

        // El cliente decide cómo manejar el caso

    }

}

```

// O mejor aún: asegurar que el usuario siempre existe

```

public class UserService {

    public UserProfile getProfile(String userId) {

        // Si no existe, crea uno por defecto (definir error fuera)

        return userRepository.findById(userId)

            .orElseGet(() -> createDefaultProfile(userId));

    }

}

...

```

3. DOCUMENTACIÓN Y COMENTARIOS (Capítulos 12-15)

3.1. Principios de Ousterhout para Comentarios

1. **Los comentarios deben describir cosas que no son obvias del código** (Capítulo 13)
2. **Escribe los comentarios primero** (Capítulo 15)
3. **Los comentarios son una herramienta de diseño** (Capítulo 15.3)

3.2. Aplicación a BehavioralOS

Antes de escribir código:

```
```java
```

```
/**
```

```
 * BehavioralOS - Módulo 400-AI (Inteligencia Artificial)
```

```
 *
```

```
 * Este módulo proporciona capacidades de IA para BehavioralOS, incluyendo:
```

```
 * - Evaluación adaptativa continua (AAO)
```

```
 * - Behavioral Twin (modelo dinámico del paciente)
```

```
 * - Recomendaciones personalizadas de ejercicios
```

```
 * - Análisis de embeddings semánticos
```

```
 *
```

```
 * Principios de diseño:
```

```
 * - TODA la IA se ejecuta on-device (Gemma + MediaPipe)
```

```
 * - No se envían datos de pacientes a la nube
```

```
 * - Los modelos son deterministas para permitir reprocesamiento
```

```
 *
```

\* Dependencias:

\* - 000-Core: perfiles de usuario, datos de pacientes

\* - 500-Experiencias: definiciones de ejercicios

\*

\* @see /specs/400-AI/ai-core.md

\* @see /specs/400-AI/companion.md

\*/

```
public class AEngine {
```

```
 // ...
```

```
}
```

```
...
```

**\*\*Comentario de interfaz para método\*\*:**

```
```java
```

```
/**
```

* Evalúa el progreso de un paciente en un ejercicio específico.

*

* @param userId Identificador único del paciente

* @param exerciseId Identificador del ejercicio completado

* @return EvaluationResult con:

* - score: puntuación numérica (0-100)

* - confidence: nivel de confianza de la evaluación (0-1)

* - recommendations: sugerencias para siguiente paso

*

* La evaluación es asíncrona: el resultado puede tardar hasta 500ms


```
* @throws IllegalArgumentException si userId o exerciseId son inválidos
* @throws IllegalStateException si el motor de IA no está inicializado
*/
```

4. CONSISTENCIA (Capítulo 17)

4.1. Aplicación a BehavioralOS

A horizontal number line with four vertical tick marks. The segments between the tick marks are represented by dashed lines.

****IDs**** | Todos los IDs son UUIDs | `userId`, `exerciseld`, `sessionld` |

```
| **Logging** | Todos los módulos usan el mismo formato de logs | `[MODULE]
```

4.2. Ejemplo de Consistencia en BehavioralOS

```
```java

// TODOS los servicios siguen el mismo patrón

public interface BehavioralService {

 // Métodos con nombres consistentes

 Result<Entity> getByld(String id);

 Result<Entity> create(CreateRequest request);

 Result<Entity> update(String id, UpdateRequest request);

 Result<Void> delete(String id);

 Result<List<Entity>> search(SearchCriteria criteria);

}

// TODAS las respuestas usan el mismo formato

public class Result<T> {

 private boolean success;

 private T data;

 private List<Error> errors;

 private String requestId;

 private long timestamp;

}

...

```

## ## 5. RED FLAGS A EVITAR EN BEHAVIORALOS

Basado en el "Summary of Red Flags" del libro (página 181):

Red Flag	Cómo aparece en BehavioralOS	Solución
----------	------------------------------	----------

--	--	--

<b>Módulo Superficial</b>	`600-Commerce` con métodos de pago triviales   Fusionar con `200-Backend` o profundizar interfaz	
---------------------------	--------------------------------------------------------------------------------------------------	--

<b>Fuga de Información</b>	`500-Experiencias` conociendo detalles de `400-AI`   Mover conocimiento a `400-AI`, exponer solo API simple	
----------------------------	-------------------------------------------------------------------------------------------------------------	--

<b>Descomposición Temporal</b>	Separar "crear ejercicio" y "evaluar ejercicio" en clases diferentes   Unificar en `ExerciseEngine`	
--------------------------------	-----------------------------------------------------------------------------------------------------	--

<b>Método Pass-Through</b>	`PracticeOS` delegando todo a `Backend` sin añadir valor   Eliminar capa o redistribuir responsabilidades	
----------------------------	-----------------------------------------------------------------------------------------------------------	--

<b>Repetición</b>	Lógica de validación duplicada en `PatientApp` y `TherapistApp`   Centralizar en `000-Core`	
-------------------	---------------------------------------------------------------------------------------------	--

<b>Comentario Repite Código</b>	`// Incrementa el contador` junto a `counter++`   Eliminar comentario obvio	
---------------------------------	-----------------------------------------------------------------------------	--

<b>Nombre Vago</b>	`data` para el objeto de progreso del paciente   Usar `patientProgressData`	
--------------------	-----------------------------------------------------------------------------	--

<b>Código No Obvio</b>	Algoritmo de evaluación AAO sin documentación   Añadir comentarios de alto nivel	
------------------------	----------------------------------------------------------------------------------	--

---

## ## 6. ESTRATEGIA DE IMPLEMENTACIÓN PARA BEHAVIORALOS

### ### 6.1. Fase 1: Fundación (Módulos 000, 900, 200)

**Enfoque:** Invertir 20% en diseño de APIs profundas.

```
```java
```

```
// Módulo 000-Core: API profunda
```

```
public class UserService {
```

```
    // Interfaz simple: 5 métodos
```

```
    public Result<User> getUser(String userId) { ... }
```

```
    public Result<User> createUser(UserCreateRequest request) { ... }
```

```
    public Result<User> updateUser(String userId, UserUpdateRequest request) { ... }
```

```
    public Result<Void> deleteUser(String userId) { ... }
```

```
    public Result<List<User>> searchUsers(String query) { ... }
```

```
    // Implementación compleja (oculta):
```

```
    // - Cifrado de datos sensibles
```

```
    // - RLS en PostgreSQL
```

```
    // - Cache con Redis
```

```
    // - Eventos para CDC
```

```
}
```

```
```
```

```
6.2. Fase 2: IA y Experiencias (Módulos 400, 500)
```

```
Enfoque: Definir errores fuera de existencia.
```

```
```java
```

```
// Módulo 400-AI: Sin excepciones
```

```
public class AIEngine {
```

```
// Retorna Optional o Result en lugar de lanzar excepciones

public Optional<Evaluation> evaluate(Exercise exercise, UserProfile profile) {

    try {

        // Evaluación compleja

        return Optional.of(evaluationResult);

    } catch (Exception e) {

        // Log y retornar Optional.empty()

        logger.error("Evaluation failed", e);

        return Optional.empty();

    }

}
```

```
// Definir caso especial fuera de existencia

public Evaluation evaluateWithDefault(Exercise exercise, UserProfile profile) {

    return evaluate(exercise, profile)

        .orElseGet(() -> createDefaultEvaluation(exercise));

}

...

```

6.3. Fase 3: Práctica y Analítica (Módulos 700, 800)

****Enfoque**:** Empujar complejidad hacia abajo.

```
```java
```

```
// Módulo 800-Analytics: Empujar complejidad
```

```

public class AnalyticsService {

 // Interfaz simple: el cliente solo pide métricas

 public Metrics getMetrics(String userId, DateRange range) {

 // La clase maneja toda la complejidad:

 // - Unir datos de múltiples fuentes

 // - Agregar métricas

 // - Calcular percentiles

 // - Cachear resultados

 return metrics;

 }

}

...

```

#### ### 6.4. Fase 4: Comercio e Integración (Módulos 600, 1000)

**\*\*Enfoque\*\*:** Módulos general-purpose.

```

```java

// Módulo 1000-Integration: General-purpose

public interface IntegrationBus {

    // General: cualquier tipo de evento

    void publish(String topic, Object event);

    <T> void subscribe(String topic, Class<T> eventType, Consumer<T> handler);

    // Configuración de entregas

    void configure(String topic, DeliveryConfig config);

```

```
}  
...  
  
---
```

7. HERRAMIENTAS Y PRÁCTICAS RECOMENDADAS

7.1. Diseño It Dos Veces (Capítulo 11)

Para cada módulo de BehavioralOS, diseñar al menos dos alternativas y compararlas antes de implementar.

****Ejemplo: Módulo 400-AI****

Alternativa Ventajas Desventajas

----- ----- -----

Centralizado Un solo motor IA para todo Cuello de botella, difícil de escalar
--

Distribuido Cada ejercicio tiene su propio evaluador Complejidad de coordinación, duplicación
--

Híbrido Motor central con evaluadores especializados Mejor compromiso → **Elegir esta**

7.2. Consistencia (Capítulo 17)

Crear un documento de estilo y convenciones para BehavioralOS:

```
```markdown
```

```
BehavioralOS Style Guide
```

## ## Nombres

- Paquetes: com.behavioralos.{module}
- Clases: PascalCase (UserService)
- Métodos: camelCase (getUserProfile)
- Constantes: UPPER\_SNAKE\_CASE (MAX\_RETRY\_ATTEMPTS)

## ## APIs

- Todas las APIs REST usan /api/v1/
- Respuestas: {"success": true, "data": {...}, "errors": []}
- Errores: códigos HTTP semánticos (400, 401, 403, 404, 500)

## ## Logging

- INFO: operaciones normales
- WARNING: situaciones recuperables
- ERROR: errores que necesitan atención
- DEBUG: solo en desarrollo

...

---

## ## 8. CONCLUSIONES Y PRÓXIMOS PASOS

### ### 8.1. Lecciones Clave del Libro para BehavioralOS

1. **\*\*Los módulos deben ser profundos\*\***: Interfaz simple, implementación compleja.



2. **Ocultar información**: Cada módulo encapsula conocimiento.
3. **Empujar complejidad hacia abajo**: Que los módulos sufran, no los usuarios.
4. **Definir errores fuera de existencia**: Usar ``Optional`` y ``Result`` en lugar de excepciones.
5. **Escribir comentarios primero**: Los comentarios son herramientas de diseño.
6. **Consistencia**: Nombres, APIs, manejo de errores consistentes en todo el sistema.
7. **Diseñar dos veces**: Comparar alternativas antes de implementar.

### ### 8.2. Plan de Acción Recomendado

1. **Crear guía de estilo** para BehavioralOS (nombres, APIs, manejo de errores).
2. **Diseñar APIs profundas** para cada módulo (interfaz simple, implementación compleja).
3. **Escribir comentarios primero** para todas las APIs públicas.
4. **Definir errores** usando ``Result<T>`` y ``Optional<T>``.
5. **Aplicar consistencia** en todos los módulos.
6. **Empujar complejidad** hacia los módulos (que los módulos manejen casos especiales).
7. **Diseñar dos veces** antes de implementar módulos complejos (``400-AI``, ``500-Experiencias``).

### ### 8.3. Documentación de Diseño Recomendada

Crear un documento ``DESIGN.md`` en cada módulo:

```
```markdown
```

```
# Módulo 400-AI: Diseño
```

Abstracción

El motor de IA proporciona evaluación adaptativa y recomendaciones personalizadas

para pacientes de BehavioralOS.

Interfaz

- `evaluateExercise(exerciseId, userId)`: evalúa el progreso
- `getRecommendations(userId, limit)`: recomienda ejercicios
- `trackBehavior(userId, behavior)`: registra comportamiento

Ocultamiento de Información

- Detalles de modelos (Gemma, MediaPipe)
- Algoritmo de evaluación (AAO)
- Estructura de embeddings
- Política de cache

Complejidad Empujada Hacia Abajo

- Formatos de entrada/salida
- Manejo de casos especiales
- Sincronización on-device

Red Flags Identificadas

- [] Métodos pass-through a MediaPipe
- [] Fuga de información con 500-Experiencias
- [] Nombre vago: "AIEngine"

...

Análisis del Informe NES y su Aplicación a BehavioralOS

Basado en el informe **"The Design and Implementation of the Nintendo Entertainment System"** (dkm_report.pdf) y el contexto de BehavioralOS, he identificado paralelismos clave entre la arquitectura de la NES y el sistema de generación de ejercicios psicoterapéuticos personalizados. El informe detalla cómo la NES maneja gráficos en tiempo real con recursos limitados, algo directamente aplicable al motor de ejercicios de BehavioralOS.

1. PARALELISMOS ARQUITECTÓNICOS: NES → BEHAVIORALOS

Componente NES	Función	Equivalente en BehavioralOS
----------------	---------	-----------------------------

-----	-----	-----
-------	-------	-------

PPU (Picture Processing Unit)	Procesa gráficos en tiempo real (sprites, fondos)	Motor de Ejercicios (genera experiencias visuales personalizadas)
--------------------------------------	---	--

CPU (6502)	Controla lógica del juego, procesa entrada	Motor de Evaluación (AAO)
-------------------	--	----------------------------------

Sprite RAM (64 sprites)	Almacena datos de objetos móviles	Perfil del Paciente (Behavioral Twin)
--------------------------------	-----------------------------------	--

Pattern Tables (CHR ROM)	Almacena tiles gráficos (8x8 píxeles)	Banco de Ejercicios (módulos terapéuticos)
---------------------------------	---------------------------------------	---

Nametables	Mapa de fondo (tiles organizados)	Plan de Tratamiento (secuencia de ejercicios)
-------------------	-----------------------------------	--

Attribute Tables	Define paletas de color por tile	Personalización (adaptación a necesidades)
-------------------------	----------------------------------	---

Sprite Buffers (8 por scanline)	Renderizado temporal de sprites	Ejercicios Activos (sesión actual)
--	---------------------------------	---

| **VBlank (Vertical Blank)** | Período de actualización de memoria | **Pausa Reflexiva** (integración de experiencia) |

2. MOTOR DE EJERCICIOS BASADO EN EL PPU DE LA NES

2.1. Estructura del Motor de Ejercicios (Analogía PPU)

...

Exercise Engine (PPU)

| — Exercise RAM (Sprite RAM) ← 64 ejercicios posibles por sesión

| | — Tipo de ejercicio (y-coordinate)

| | — Metadatos (atributos: duración, dificultad)

| | — Contexto del paciente (x-coordinate, prioridad)

|

| — Exercise Pattern Tables (CHR ROM)

| | — Banco de módulos (8x8 "tiles" terapéuticos)

| | | — Mindfulness (tile 0x00)

| | | — CBT cognitivo (tile 0x01)

| | | — Activación conductual (tile 0x02)

| | | — Exposición (tile 0x03)

| | — Cada tile = 8 líneas de texto/pasos

|

| — Exercise Nametables (Fondo)

| | — Mapa de la sesión: secuencia personalizada

|

- |— Exercise Attribute Tables
 - | — Parámetros de adaptación (dificultad, tono, ritmo)
 - |
 - |— Exercise Buffers (Sprites por scanline)
 - | — Ejercicios activos (hasta 8 simultáneos)
- ...

2.2. Ciclo de Renderizado del Ejercicio (Analogía PPU)

Fase NES	Fase Behavioral	OS	Descripción
-----	-----	-----	
In-Range Evaluation (píxeles 0-128) **Evaluación de Perfil** Escanea el Behavioral Twin para determinar qué ejercicios son apropiados (hasta 8 activos)			
Memory Fetch (píxeles 257-320) **Generación de Contenido** Construye el ejercicio personalizado a partir de los módulos base			
Real-Time Output (píxeles 0-256) **Interacción del Paciente** Presenta el ejercicio y captura respuestas en tiempo real			

3. GENERACIÓN DE EJERCICIOS POR IA (MODELO DEL PPU)

3.1. Patrón de Tiles Terapéuticos (Analogía CHR ROM)

Cada "tile" (módulo de ejercicio) es un bloque de 8x8 unidades (pensamientos, emociones, conductas) que se combinan para crear ejercicios personalizados.

****Estructura de un Tile Terapéutico:****

...

Tile (8x8 unidades)

Línea 0: Contexto del paciente	← Empatía inicial
Línea 1: Activación	← Disparador conductual
Línea 2: Exploración	← Identificación de patrones
Línea 3: Insight	← Conexión cognitiva
Línea 4: Acción	← Micro-tarea
Línea 5: Reflexión	← Evaluación en vivo
Línea 6: Integración	← Conexión con valores
Línea 7: Cierre	← Anclaje al mundo real

...

****Bancos de Tiles por Enfoque Terapéutico:****

| Tile ID | Tipo | Descripción | Líneas de Contenido (ejemplo) |

|-----|-----|-----|-----|

| 0x00 | Mindfulness | Conciencia plena | 0: "Siente tu respiración" / 7: "Vuelve al momento presente" |

| 0x01 | Cognitive Defusion | Separación de pensamientos | 0: "Observa tus pensamientos" / 7: "Los pensamientos no son hechos" |

| 0x02 | Acceptance | Aceptación de emociones | 0: "Da espacio a la emoción" / 7: "La emoción es pasajera" |

| 0x03 | Valores | Conexión con valores personales | 0: "¿Qué es importante para ti?" / 7: "Elige una acción valiosa" |

3.2. Generación de Nametable (Secuencia de Ejercicios)

Así como la NES usa nametables para mapear tiles en el fondo, BehavioralOS usa un ****Plan de Tratamiento**** que mapea módulos en una secuencia personalizada.

...

Nametable (Plan de Tratamiento)

Tile 0x01	Tile 0x03	Tile 0x02	Tile 0x01	Tile 0x00
(Defusión)	(Valores)	(Aceptación)	(Defusión)	(Mindfulness)

Sesión 1	Sesión 2	Sesión 3	Sesión 4	Sesión 5
----------	----------	----------	----------	----------

...

****Algoritmo de Generación de Nametable (basado en AAO):****

...

```
function generate_nametable(behavioral_twin):
```

```
    # 1. Evaluación de perfil (In-Range Evaluation)
```

```
    active_tiles = []
```

```
    for tile in all_tiles:
```

```
        if tile.appropriate_for(behavioral_twin):
```

```
            active_tiles.append(tile)
```

```
    # 2. Priorización (Sprite RAM sorting)
```

```
    active_tiles.sort(key=lambda t: t.priority(behavioral_twin))
```

```

active_tiles = active_tiles[:8] # Máximo 8 ejercicios activos

# 3. Secuenciación (Nametable construction)

plan = []

for i in range(0, len(active_tiles), 5): # Ventanas de 5 sesiones

    plan.append([

        active_tiles[0], # Primer módulo (siempre)

        active_tiles[random.choice([1,2])],

        active_tiles[random.choice([3,4])],

        active_tiles[random.choice([5,6])],

        active_tiles[7] if len(active_tiles) > 7 else active_tiles[-1]

    ])

return plan

...

```

3.3. Atributos de Ejercicio (Analogía Attribute Tables)

Los atributos definen cómo se "colorea" cada tile (personalización):

...

Attribute Table (Parámetros de Personalización)

Tile 0x01	Tile 0x02	Tile 0x03	Tile 0x04
Duración	Dificultad	Tono	Ritmo
00: 5 min	00: baja	00: suave	00: lento

| 01: 10 min | 01: media | 01: neutro | 01: normal |

| 10: 15 min | 10: alta | 10: firme | 10: rápido |

| _____ |

...

4. EJEMPLO PRÁCTICO: GENERACIÓN DE UN EJERCICIO PERSONALIZADO

4.1. Input del Behavioral Twin

...

Paciente: Juan (ID: 042)

Diagnóstico: Ansiedad social (moderada)

Valores: Conexión, Autenticidad

Patrón actual: Evitación (puntuación 8/10)

Activación: 4/10

Horizonte de cambio: 2 meses

...

4.2. Proceso de Generación (Modelo PPU)

****Paso 1: In-Range Evaluation (Evaluación del Perfil)****

```
```python
```

```
tiles_apropiados = [
```

```

{"id": 0x00, "tipo": "Mindfulness", "score": 0.9}, # Alta prioridad
{"id": 0x01, "tipo": "Cognitive Defusion", "score": 0.8},
{"id": 0x03, "tipo": "Valores", "score": 0.7},
{"id": 0x04, "tipo": "Exposición Gradual", "score": 0.6},
{"id": 0x05, "tipo": "Aceptación", "score": 0.5},

... 8 tiles seleccionados

]

...

```

**\*\*Paso 2: Memory Fetch (Construcción del Ejercicio)\*\***

```

```python
ejercicio = {
    "id": "EJ_042_2026_07_13_001",
    "tiles": [
        {"tile": 0x00, "atributos": {"duracion": 5, "dificultad": 1, "tono": 0}},
        {"tile": 0x01, "atributos": {"duracion": 10, "dificultad": 2, "tono": 1}},
        {"tile": 0x03, "atributos": {"duracion": 7, "dificultad": 1, "tono": 0}},
        {"tile": 0x04, "atributos": {"duracion": 8, "dificultad": 3, "tono": 2}},
        {"tile": 0x00, "atributos": {"duracion": 5, "dificultad": 1, "tono": 0}},
    ],
    "flujo": [
        "Respiración consciente (2 min)",
        "Identifica un pensamiento automático: 'No soy interesante'",
        "Escribe tres pruebas en contra de ese pensamiento",
        "Envía un mensaje a un amigo (acción valiosa)",
    ]
}
```

```

```
],
 "prioridad": 7 # Nivel de confianza
}
...

```

[illegible]

| || "¿Qué dice este pensamiento sobre lo que valoras?" || |

| ||

— || |

| || 🚀 Paso 4: Acción (8 min) || |

| || "Escribe un mensaje a un amigo. No importa la || |

| || respuesta, solo la acción." || |

| ||

— || |

| || ☀ Paso 5: Reflexión (5 min) || |

| || "¿Qué aprendiste sobre ti?" || |

| ||

— || |

| || Progreso:  (72%) || |

| || 🕒 Tiempo restante: 12 min || |

| || 💡 "La ansiedad es como una ola: viene y va." || |

|  
|  
|  
|

|  
|  
|

...

---

## ## 5. IMPLEMENTACIÓN TÉCNICA (INSPIRADA EN EL PPU DE LA NES)

### ### 5.1. Arquitectura de Módulos (Analogía Verilog)

```

```verilog

module ExerciseEngine(

    input reset,

    input clock,          // 60 Hz (como el PPU)

    input [7:0] patient_id, // Behavioral Twin ID

    input [7:0] session_context,

    output [4:0] exercise_output // 5 bits: tipo + atributos

);


// 1. SPRITE RAM (Ejercicios activos)

reg [7:0] active_exercises [0:63]; // 64 ejercicios posibles

reg [7:0] exercise_attributes [0:63]; // Duración, dificultad, etc.


// 2. PATTERN TABLES (Banco de módulos)

reg [7:0] tile_bank [0:15][0:7]; // 16 tipos x 8 líneas


// 3. NAMETABLE (Plan de sesión)

reg [7:0] nametable [0:29][0:31]; // 30x32 tiles (secuencia)


// 4. RENDERING PIPELINE (Ejecución en tiempo real)

wire [7:0] current_tile;

wire [2:0] scanline; // Línea del tile (0-7)


// Ciclo de renderizado

always @(posedge clock) begin

```

```

if (reset) begin

    // In-Range Evaluation (como el PPU)

    generate_plan(patient_id, session_context);

end else begin

    // Memory Fetch (como el PPU)

    current_tile = nametable[scanline][column];

    // Real-Time Output (como el PPU)

    exercise_output = render_tile(current_tile, scanline);

end

end

endmodule

'''

```

5.2. Algoritmo de Generación de Tiles (Inspirado en CHR ROM)

```

'''python

class TileGenerator:

    def __init__(self):

        self.tile_bank = {

            # 16 tipos de ejercicios, cada uno con 8 líneas de contenido

            0x00: self.mindfulness_tile(),

            0x01: self.cognitive_defusion_tile(),

            0x02: self.acceptance_tile(),

            # ... más tiles

        }

```

```
def mindfulness_tile(self):  
    return [  
        "Siente tu respiración",  
        "Inhala profundamente",  
        "Exhala lentamente",  
        "Nota las sensaciones",  
        "Sin juzgar",  
        "Vuelve a la respiración",  
        "Cada respiración es única",  
        "Regresa al momento presente"  
    ]
```

```
def cognitive_defusion_tile(self):  
    return [  
        "Observa tu pensamiento",  
        "Dale un nombre",  
        "No es un hecho",  
        "Es solo una nube",  
        "Pasa en el cielo",  
        "No eres la nube",  
        "Eres el cielo",  
        "Regresa a ti"  
    ]
```

```
def generate_exercise(self, patient_profile, scanline, column):
```

```

tile_id = self.nametable[scanline][column]

tile = self.tile_bank[tile_id]

# Personalización (Attribute Table)

line = tile[scanline % 8]

# Aplicar atributos (duración, dificultad, tono)

return self.apply_attributes(line, patient_profile)

...

```

6. DETECCIÓN DE COLISIONES (Analogía Sprite #0 Hit)

Así como la NES detecta colisiones entre sprites para activar eventos, BehavioralOS detecta ****momentos de ruptura cognitiva**** que activan intervenciones específicas.

| Evento NES | Equivalente en BehavioralOS |

|-----|-----|

| Sprite #0 Hit | Detección de activación emocional alta (ej. >7/10) |

| VBlank | Pausa reflexiva (integración de aprendizaje) |

| More than 8 sprites | Sobrecarga de ejercicios (reduce a 8) |

```
```python
```

```
class CollisionDetector:
```

```
 def detect_cognitive_rupture(self, patient_response):
```

```
 if patient_response.anxiety > 7:
```

```
 # Activar ejercicio de grounding (Sprite #0 Hit)
```

```
 self.activate_emergency_exercise(0x00) # Mindfulness
```



```

 if patient_response.engagement < 2:

 # Reducir carga cognitiva

 self.reduce_active_exercises(8)

 ...

```

## ## 7. CONCLUSIÓN: DE LA NES A BEHAVIORALOS

El informe de la NES proporciona un modelo arquitectónico robusto para el motor de ejercicios de BehavioralOS:

| Principio NES | Aplicación en BehavioralOS |

|-----|-----|

| **\*\*Procesamiento en tiempo real\*\*** | Generación de ejercicios < 500ms |

| **\*\*Recursos limitados\*\*** (64 sprites, 8 por scanline) | Máximo 8 ejercicios activos simultáneos |

| **\*\*Separación de datos\*\*** (CHR ROM + NameTable) | Banco de módulos + Plan de tratamiento |

| **\*\*Renderizado por scanline\*\*** | Progresión paso a paso del ejercicio |

| **\*\*Atributos de color\*\*** (Attribute Table) | Personalización dinámica (duración, dificultad, tono) |

| **\*\*VBlank\*\*** | Pausas de integración |

| **\*\*Sprite #0 Hit\*\*** | Detección de momentos críticos |

**\*\*Próximos pasos:\*\*** Implementar el motor de ejercicios como un módulo inspirado en el PPU, con generación de contenido personalizado a partir del Behavioral Twin y evaluación adaptativa continua (AAO).

## ## Análisis de "Shigeru Miyamoto: Super Mario Bros., Donkey Kong, The Legend of Zelda"

Basado en el contenido del libro **"Shigeru Miyamoto: Super Mario Bros., Donkey Kong, The Legend of Zelda"** de Jennifer deWinter, he identificado los principios de diseño de juegos de Miyamoto que son directamente aplicables a BehavioralOS, especialmente en el contexto de su motor de ejercicios psicoterapéuticos personalizados.

---

### ## 1. PARALELISMOS MIYAMOTO → BEHAVIORALOS

| Principio Miyamoto | Aplicación en BehavioralOS |

|-----|-----|

| **\*\*Diseño centrado en la diversión (fun first)\*\*** | La experiencia del paciente debe ser **\*\*placentera y lúdica\*\***, no un ejercicio clínico |

| **\*\*Narrativa espacial vs. lineal\*\*** | Los ejercicios deben **\*\*descubrirse mediante exploración\*\***, no ser una secuencia rígida |

| **\*\*"Miniatura de jardín" (world in miniature)\*\*** | BehavioralOS es un **\*\*microcosmos terapéutico\*\*** que el paciente explora |

| **\*\*70% objetivos, 30% secretos\*\*** | 70% del plan terapéutico es claro, 30% es **\*\*descubrimiento personal del paciente\*\*** |

| **\*\*Diseño iterativo y pulido\*\*** | Los ejercicios deben **\*\*perfeccionarse\*\*** basados en feedback del paciente |

| **\*\*Comunidad y comunicación\*\*** | BehavioralOS debe **\*\*fomentar la conversación\*\*** entre pacientes y terapeutas |

| **\*\*Hardware define la experiencia\*\*** | La **\*\*interfaz (UI/UX)\*\*** del paciente define la experiencia terapéutica |

| **\*\*Juego como experiencia, no narrativa\*\*** | El foco es la **\*\*acción terapéutica\*\***, no contar una historia |

---

## ## 2. PRINCIPIOS DE MIYAMOTO APLICADOS AL MOTOR DE EJERCICIOS

### ### 2.1. Diversión como Meta Principal (No Solo Terapia)

**\*\*Principio Miyamoto\*\***: **\*\*"Making people cry or feel scared is easy; making them laugh, feel excitement and pleasure is difficult."\***

**\*\*Aplicación a BehavioralOS\*\***:

...

MAL: Ejercicio como "tarea clínica" seria

"Completa este ejercicio de respiración de 10 minutos. Evalúa tu ansiedad."

BIEN: Ejercicio como "juego terapéutico" divertido

"¡Explora la cueva de la calma! Respira siguiendo las luces parpadeantes.

¿Puedes llegar al final sin que tu ansiedad supere el nivel 3?"

...

**\*\*Implementación\*\***:

- Cada ejercicio debe tener un **\*\*elemento lúdico\*\*** (no solo instrucciones)
- Usar **\*\*gamificación\*\*** (puntos, niveles, desbloques) pero **\*\*nunca castigar\*\***
- El paciente debe **\*\*querer\*\*** hacer el ejercicio, no sentirse obligado

### ### 2.2. Narrativa Espacial vs. Lineal

**\*\*Principio Miyamoto\*\*:** \*Donkey Kong usa espacio vertical para contar una historia; Zelda usa exploración para revelar la narrativa.\*

**\*\*Aplicación a BehavioralOS\*\*:**

| Enfoque Lineal (MAL) | Enfoque Espacial (BIEN) |

|-----|-----|

| Sesión 1: Mindfulness | El paciente **\*\*explora\*\*** un mapa de "islas emocionales" |

| Sesión 2: Defusión | Cada isla tiene **\*\*secretos\*\*** que revelan insights |

| Sesión 3: Aceptación | El progreso no es lineal; el paciente **\*\*elige\*\*** su camino |

| Terapeuta define todo | El paciente **\*\*descubre\*\*** su propio plan |

**\*\*Ejemplo de Mapa Terapéutico (Zelda-style)\*\*:**

...

Mapa de "Regiones Emocionales"

	 Bosque de la Ansiedad (Mindfulness)	
	→ Secreto: "La respiración te ancla"	
	 Montaña de los Pensamientos (Defusión)	
	→ Secreto: "Los pensamientos son nubes"	
	 Mar de las Emociones (Aceptación)	
	→ Secreto: "Las emociones son olas"	

	🏰 Castillo de los Valores (Acción)	
	→ Secreto: "Tus valores te guían"	

...

### ### 2.3. Miniatura de Jardín (Mundo en Miniatura)

**\*\*Principio Miyamoto\*\***: **\*\*"Instead of thinking of it as making a game, think of it as nurturing a miniature garden called Hyrule."\***

**\*\*Aplicación a BehavioralOS\*\***:

El **\*\*Behavioral Twin\*\*** es el "jardín" que el paciente cultiva:

...

#### El Jardín del Paciente

	🌱 Semillas de Pensamientos	
	(Creencias, supuestos, interpretaciones)	
	🌿 Plantas de Emociones	
	(Ansiedad, tristeza, alegría, ira)	
	🌳 Árboles de Conductas	
	(Evitación, aproximación, acción)	
	🦋 Mariposas de Relaciones	

	(Patrones interpersonales)	

...

**\*\*Ejercicio\*\***: El paciente "riega" ciertas áreas (refuerza conductas saludables) y "poda" otras (reduce evitación). El Behavioral Twin muestra el **\*\*crecimiento\*\*** del jardín.

### ### 2.4. 70% Objetivos, 30% Secretos

**\*\*Principio Miyamoto\*\***: "My formula for success is that 70 percent of the game should have to do with objectives, and the rest should be secrets and exploration."

**\*\*Aplicación a BehavioralOS\*\***:

Componente	Porcentaje	Ejemplo
Objetivos Claros	70%	Completa 3 ejercicios de mindfulness esta semana
Secretos/Exploración	30%	Si exploras la sección de 'Patrones de Ansiedad', encontrarás una técnica que no sabías que existía

**\*\*Beneficio\*\***: El paciente siente que **\*\*descubre\*\*** su propio camino terapéutico, en lugar de solo seguir instrucciones.

### ### 2.5. Diseño Iterativo y Pulido

**\*\*Principio Miyamoto\*\***: "A delayed game is eventually good, a bad game is bad forever."

**\*\*Aplicación a BehavioralOS\*\*:**

- Cada ejercicio debe pasar por **\*\*múltiples iteraciones\*\*** con pacientes reales
- El terapeuta y el sistema deben **\*\*ajustar\*\*** ejercicios basados en feedback
- No lanzar ejercicios "incompletos" que puedan frustrar al paciente

**\*\*Ciclo de Iteración\*\*:**

...

1. Diseño inicial → 2. Prueba con paciente → 3. Feedback → 4. Refinamiento → 5. Repetir

...

### ### 2.6. Comunidad y Comunicación

**\*\*Principio Miyamoto\*\*:** \*Zelda was designed to get people talking, trading strategies, and building communities.\*

**\*\*Aplicación a BehavioralOS\*\*:**

- **\*\*Grupos de apoyo\*\*** donde pacientes comparten "estrategias" descubiertas
- **\*\*Foros seguros\*\*** donde pacientes discuten ejercicios (sin compartir datos sensibles)
- **\*\*Conversaciones terapeuta-paciente\*\*** enriquecidas por insights del sistema

### ### 2.7. Hardware/Interfaz Define la Experiencia

**\*\*Principio Miyamoto\*\*:** \*"The hardware really is the system as a whole."\*

**\*\*Aplicación a BehavioralOS\*\*:**

- La **\*\*UI del paciente\*\*** debe ser tan intuitiva como un juego de Nintendo
- El **\*\*diseño de la app\*\*** debe "desaparecer" para que el paciente se concentre en la experiencia
- Los **\*\*controles\*\*** (gestos, voz, toque) deben ser naturales y directos

### ### 2.8. Juego como Experiencia, No Narrativa

**\*\*Principio Miyamoto\*\*:** **\*\*"For us it's less about the story and more about the structure of the gameplay."\*\***

**\*\*Aplicación a BehavioralOS\*\*:**

- El foco no es "contar una historia de sanación"
- El foco es **\*\*proveer una estructura de juego\*\*** donde el paciente **\*\*experimente\*\*** la sanación
- La **\*\*acción terapéutica\*\*** es el gameplay; la narrativa emerge de la acción

---

## ## 3. EJEMPLO: EJERCICIO "EXPLORANDO LA CAVERNA" (INSPIRADO EN ZELDA)

### ### 3.1. Diseño del Ejercicio

**\*\*Inspiración\*\*:** La cueva de Miyamoto en Sonobe (Zelda)



**\*\*Objetivo (70%)\*\*:** El paciente debe explorar una "cueva" virtual, recolectando 3 "gemas de calma" y llegando al final sin que su ansiedad supere el nivel 5.

**\*\*Secretos (30%)\*\*:**

- Si el paciente toca una pared específica, encuentra una "gema oculta" que reduce la ansiedad automáticamente.
- Si el paciente se detiene y respira en ciertos puntos, encuentra "insights" sobre sus patrones.

**\*\*Experiencia\*\*:**

...

Escenario: Cueva Virtual

	 Bienvenido a la Cueva de la Calma	
	"Sientes que la cueva está oscura y fría.	
	Pero hay una luz al final..."	
	Ansiedad: 4/10	
	[↑] [↓] [←] [→] ← Movimiento	
	 Gemas encontradas: 1/3	
	 Toca la pared derecha → "¡Encontraste	
	una gema oculta!"	
	 Detente y respira 3 veces → "Insight:	

| la ansiedad es solo una sensación" |

---

...

### ### 3.2. Feedback del Paciente

El paciente completa el ejercicio y el Behavioral Twin registra:

- Tiempo de exploración
- Gemas encontradas (objetivos)
- Secretos descubiertos (exploración)
- Nivel de ansiedad al final

**\*\*El terapeuta recibe\*\*:**

- Mapa de la cueva con las rutas del paciente
- Insights que el paciente encontró
- Sugerencias para la siguiente sesión

---

## ## 4. DISEÑO DE LA UI (INSPIRADO EN NINTENDO)

### ### 4.1. Principios de UI de Miyamoto

| Principio | Aplicación |

|-----|-----|

| **\*\*Simple y directo\*\*** | El paciente sabe qué hacer en 2 segundos |

| **\*\*Control directo\*\*** | Gestos y toques = acciones inmediatas |

| **\*\*Feedback inmediato\*\*** | Cada acción tiene una respuesta visual/sonora |

| **\*\*Sin castigo\*\*** | Errores son oportunidades, no fracasos |

| **\*\*Exploración recompensada\*\*** | "Secretos" escondidos en la UI |

#### ### 4.2. Mapa de Pantallas (Zelda-style)

...

Pantalla Principal: Mapa de "Regiones Emocionales"

🌍 BehavioralOS: Tu Mundo Interior		
🏠 Casa de la Calma 🌲 Bosque de la Ansiedad		
🏔️ Montaña de los 🌊 Mar de las Emociones		
Pensamientos		
🏰 Castillo de los 🌸 Jardín de las		
Valores Relaciones		
💾 Progreso: 45% ⭐ Nivel: Explorador		

...

---

## ## 5. CONCLUSIÓN: DE MIYAMOTO A BEHAVIORALOS

El libro de Miyamoto proporciona un modelo de diseño centrado en la **experiencia lúdica**, la **exploración** y la **comunidad** que es directamente aplicable al motor de ejercicios de BehavioralOS:

| Principio Miyamoto | Implementación en BehavioralOS |

|-----|-----|

| **Fun first** | Ejercicios diseñados como juegos, no tareas |

| **Narrativa espacial** | Mapa de "regiones emocionales" que el paciente explora |

| **Miniatura de jardín** | Behavioral Twin como jardín que el paciente cultiva |

| **70/30 objetivo/secreto** | Plan claro + descubrimientos personales |

| **Iteración y pulido** | Ejercicios perfeccionados con feedback real |

| **Comunidad** | Foros seguros y conversaciones terapeuta-paciente |

| **Interfaz directa** | UI intuitiva (tipo Nintendo) |

| **Experiencia > narrativa** | Acción terapéutica como foco principal |

**Próximos pasos**: Implementar el motor de ejercicios como un "mundo" explorable (Zelda-style) donde cada ejercicio es una "región" que el paciente descubre, y el Behavioral Twin es el "mapa" que muestra el progreso y los secretos encontrados.

---

## ## Análisis de "Nintendo and the Blue Ocean Strategy" y su Aplicación a BehavioralOS

Basado en el contenido del artículo **"Nintendo and the Blue Ocean Strategy: Designing the Four Actions Matrix"** de Adan Jacinto Flores Flores, he identificado los principios de la estrategia del océano azul que son directamente aplicables a BehavioralOS, especialmente en el contexto de posicionamiento competitivo en el mercado de la salud mental digital.

---

## ## 1. DIAGNÓSTICO: BEHAVIORALOS EN UN OCÉANO ROJO

### ### 1.1. El Mercado de la Salud Mental Digital

BehavioralOS opera en un mercado altamente competitivo (océano rojo) caracterizado por:

| Competidor | Enfoque | Limitación |

|-----|-----|-----|

| **\*\*Apps de meditación\*\*** (Calm, Headspace) | Mindfulness genérico | No personalizado, sin seguimiento clínico |

| **\*\*Terapia digital\*\*** (BetterHelp, Talkspace) | Terapia por texto/video | Cara, depende de terapeutas humanos |

| **\*\*Apps de seguimiento\*\*** (Daylio, Moodpath) | Monitoreo de estado de ánimo | Pasivo, sin intervención activa |

| **\*\*Programas hospitalarios\*\*** | Terapia presencial | Costosa, acceso limitado |

| **\*\*Juegos de salud mental\*\*** | Gamificación básica | Sin base clínica sólida |

**\*\*Problema\*\***: Ninguno de estos competidores ofrece una **\*\*experiencia de psicoterapia gamificada, personalizada y basada en procesos\*\*** como BehavioralOS.

**\*\*Oportunidad\*\***: BehavioralOS puede crear un **\*\*océano azul\*\*** al combinar:

- Psicología científica (ACT, FAP, DBT, RFT)
- IA on-device (Gemma, MediaPipe)
- Diseño de juegos (Nintendo-style)
- Evaluación adaptativa continua (AAO)

---

## ## 2. APLICACIÓN DE LA MATRIZ DE CUATRO ACCIONES A BEHAVIORALOS

### ### 2.1. Acciones a Eliminar

| Lo que Nintendo Eliminó | Equivalente en BehavioralOS |

|-----|-----|

| \*\*Conferencias presenciales costosas (E3)\*\* | \*\*Sesiones de terapia tradicionales presenciales\*\* |

| | \*\*Dependencia exclusiva de terapeutas humanos\*\* |

| | \*\*Barreras de acceso (costo, ubicación, tiempo)\*\* |

\*\*Aplicación\*\*:

...

MAL (Océano Rojo):

- Sesiones semanales presenciales con terapeuta (costo: \$100+/sesión)
- El paciente debe desplazarse a una clínica
- Dependencia total del terapeuta para el progreso

BIEN (Océano Azul):

- BehavioralOS como plataforma digital accesible 24/7
- El paciente interactúa con ejercicios gamificados
- El terapeuta actúa como guía, no como único proveedor
- Costo reducido (suscripción mensual vs. sesión por sesión)

...

**\*\*Estrategia de Eliminación\*\*:**

1. **\*\*Eliminar la dependencia del terapeuta para ejercicios diarios\*\***: El terapeuta diseña el plan, pero el paciente ejecuta los ejercicios en la app.
2. **\*\*Eliminar las barreras geográficas\*\***: Acceso global, sin necesidad de clínica física.
3. **\*\*Eliminar el costo prohibitivo\*\***: Suscripción mensual accesible vs. terapia tradicional cara.

---

### 2.2. Acciones a Reducir

| Lo que Nintendo Redujo | Equivalente en BehavioralOS |

|-----|-----|

| **\*\*Potencia gráfica de la consola\*\*** | **\*\*Dependencia de terapeutas humanos expertos\*\*** |

| **\*\*Presupuestos en desarrollo\*\*** | **\*\*Costos de infraestructura en la nube\*\*** |

| **\*\*Plazos de desarrollo\*\*** | **\*\*Tiempo de respuesta del sistema\*\*** |

**\*\*Aplicación\*\***:

...

MAL (Océano Rojo):

- IA en la nube (costosa, latencia, dependencia de internet)
- Terapeutas humanos para cada interacción

- Infraestructura costosa (servidores, ancho de banda)

BIEN (Océano Azul):

- IA on-device (Gemma, MediaPipe) - sin costos de nube
- El sistema es el "terapeuta" para ejercicios diarios
- El terapeuta humano interviene solo cuando es necesario
- Latencia cero, privacidad total

...

**\*\*Estrategia de Reducción\*\*:**

1. **\*\*Reducir la dependencia de terapeutas humanos\*\*:** BehavioralOS maneja el 80% del trabajo terapéutico (ejercicios, seguimiento, evaluación), liberando al terapeuta para casos complejos.
2. **\*\*Reducir costos de infraestructura\*\*:** IA on-device significa **\*\*cero costos de servidor\*\*** para inferencia.
3. **\*\*Reducir tiempos de espera\*\*:** El paciente puede acceder a ejercicios en cualquier momento, sin esperar a la próxima sesión.

---

### ### 2.3. Acciones a Incrementar

| Lo que Nintendo Incrementó | Equivalente en BehavioralOS |

|-----|-----|

| **\*\*Extensión de marca\*\*** | **\*\*Extensión de la experiencia terapéutica\*\*** |

| (películas, parques temáticos, museos) | (ejercicios gamificados, comunidad, seguimiento continuo) |



**\*\*Aplicación\*\*:**

...

MAL (Océano Rojo):

- Terapia = solo sesiones con terapeuta
- El paciente solo interactúa 1 hora/semana
- Sin seguimiento entre sesiones

BIEN (Océano Azul):

- Terapia = experiencia continua (24/7)
- Ejercicios gamificados, comunidad, seguimiento
- El paciente "vive" la terapia en su día a día

...

**\*\*Estrategia de Incremento\*\*:**

| Componente | Descripción | Ejemplo |

|-----|-----|-----|

| **\*\*Ejercicios Gamificados\*\*** | Terapia como juego (Nintendo-style) | "Explora la cueva de la calma" (Zelda-style) |

| **\*\*Comunidad\*\*** | Foros seguros, grupos de apoyo | Compartir estrategias, logros |

| **\*\*Seguimiento Continuo\*\*** | Behavioral Twin actualizado en tiempo real | Mapa de progreso, insights |

| **\*\*Contenido Premium\*\*** | Ejercicios avanzados, módulos especializados | Terapia para condiciones específicas |

| **\*\*Integración con Terapeuta\*\*** | El terapeuta recibe datos, ajusta el plan | Dashboard para el terapeuta |

**\*\*Beneficio\*\***: BehavioralOS se convierte en el **\*\*ecosistema completo de salud mental\*\***, no solo una app más.

---

#### ### 2.4. Acciones a Crear

| Lo que Nintendo Creó | Equivalente en BehavioralOS |

|-----|-----|

| **\*\*Sentimiento nostálgico\*\*** | **\*\*Sentimiento de seguridad y confianza\*\*** |

| (conexión emocional con los jugadores) | (conexión emocional con el paciente) |

**\*\*Aplicación\*\***:

...

MAL (Océano Rojo):

- Apps de terapia = transaccionales, frías
- El paciente se siente solo en su proceso
- Sin conexión emocional con la plataforma

BIEN (Océano Azul):

- BehavioralOS = un "amigo" en el viaje terapéutico
- El paciente siente que la app "lo entiende"
- Conexión emocional = adherencia al tratamiento

...

**\*\*Estrategia de Creación: "Sentimiento de Seguridad"\*\*\***

Componente de Nostalgia (Nintendo)	Equivalente en BehavioralOS
**\*\*Juegos entrañables\*\***	**\*\*Ejercicios que el paciente disfruta y espera\*\***
(personajes, música, arte)	(diseño lúdico, gamificación)
**\*\*Reminiscencia del pasado\*\***	**\*\*Progreso visible y tangible\*\***
(recordar la infancia con Nintendo)	(ver el crecimiento en el Behavioral Twin)
**\*\*Revivir emociones\*\***	**\*\*Sentirse acompañado en el proceso\*\***
(alegría de jugar de nuevo)	(el Companion IA como apoyo constante)
**\*\*Compartir historias\*\***	**\*\*Comunidad de pacientes\*\***
(jugar con otros, hablar de juegos)	(foros, grupos de apoyo)

**\*\*Ejemplo de Ejercicio con Sentimiento de Seguridad\*\*:**

...

Ejercicio: "El Jardín de la Calma"

---

 Bienvenido a tu jardín personal.

Cada día, cultivas una nueva flor de calma.

Hoy, tu flor es la "Aceptación".

 Instrucciones:

1. Observa la flor en la pantalla.
2. Respira con ella: inhala cuando se abra.
3. Exhala cuando se cierre.

💡 Insight: "La aceptación es como una flor:

no puedes forzarla a abrirse.

Solo puedes crear las condiciones para que florezca."

📊 Progreso: 45% | 🌸 12 flores cultivadas

---

...

---

### ## 3. ANÁLISIS DE FUERZAS DE CAMPO

Basado en el artículo, se identifican las fuerzas que impulsan y restringen la implementación de la estrategia de océano azul en BehavioralOS:

#### ### 3.1. Fuerzas Impulsoras (Driving Forces)

Fuerza	Descripción	Impacto
--------	-------------	---------

-----	-----	-----
-------	-------	-------

**Innovación**	Combinación de psicología + IA + diseño de juegos	Alto
----------------	---------------------------------------------------	------

**Cultura organizacional**	Filosofía de "paciente primero", no solo tecnología	Alto
----------------------------	-----------------------------------------------------	------

**Demanda insatisfecha**	Millones de personas sin acceso a terapia de calidad	Muy Alto
--------------------------	------------------------------------------------------	----------

**Tecnología disponible**	IA on-device (Gemma), gamificación (Nintendo-style)	Alto
---------------------------	-----------------------------------------------------	------

**Costo-efectividad**	Suscripción accesible vs. terapia tradicional	Alto
-----------------------	-----------------------------------------------	------

### ### 3.2. Fuerzas Restrictivas (Restraining Forces)

Fuerza	Descripción	Impacto
**Regulación médica**	Aprobaciones, compliance, HIPAA/GDPR	Alto
**Escepticismo clínico**	Profesionales dudan de la eficacia de la IA	Medio
**Competencia establecida**	Apps de terapia (BetterHelp, Calm)	Medio
**Barrera de entrada**	Costo de desarrollo, investigación clínica	Alto
**Adopción del paciente**	Aceptación de la terapia digital	Medio

**\*\*Conclusión del Análisis de Fuerzas\*\*:** Las fuerzas impulsoras superan a las restrictivas, lo que favorece la implementación de la estrategia de océano azul.

---

## ## 4. ESTRATEGIA DE POSICIONAMIENTO

### ### 4.1. Océano Rojo vs. Océano Azul

Factor	Océano Rojo (Competidores)	Océano Azul (BehavioralOS)
**Precio**	Alto (\$100+/sesión)	Bajo (\$20-30/mes)
**Acceso**	Limitado (terapeutas, ubicación)	Universal (app, 24/7)
**Personalización**	Baja (terapia genérica)	Alta (Behavioral Twin)
**Gamificación**	Nula o básica	Avanzada (Nintendo-style)
**Seguimiento**	Manual (terapeuta)	Automático (IA)

| **\*\*Escalabilidad\*\*** | Baja (depende de terapeutas) | Alta (plataforma digital) |  
| **\*\*Privacidad\*\*** | Variable | Alta (IA on-device) |

#### ### 4.2. Propuesta de Valor Única

**\*\*BehavioralOS es el "Nintendo de la Psicoterapia".\*\***

- **\*\*Diversión\*\***: Ejercicios gamificados (como Mario)
- **\*\*Personalización\*\***: Behavioral Twin (como Zelda)
- **\*\*Accesibilidad\*\***: IA on-device (como Game Boy)
- **\*\*Comunidad\*\***: Foros seguros (como Animal Crossing)
- **\*\*Evolución\*\***: Adaptación continua (como actualizaciones de juegos)

---

### ## 5. HOJA DE RUTA DE IMPLEMENTACIÓN

#### ### 5.1. Fase 1: Eliminar (Meses 1-3)

- **\*\*Eliminar dependencia de terapeutas para ejercicios diarios\*\***
- **\*\*Eliminar barreras geográficas\*\*** (acceso global)
- **\*\*Eliminar costo prohibitivo\*\*** (suscripción vs. sesión)

#### ### 5.2. Fase 2: Reducir (Meses 3-6)

- **\*\*Reducir costos de infraestructura\*\*** (IA on-device)

- **Reducir tiempos de espera** (ejercicios disponibles 24/7)
- **Reducir dependencia de internet** (ejecución local)

### 5.3. Fase 3: Incrementar (Meses 6-12)

- **Incrementar la experiencia terapéutica** (ejercicios gamificados)
- **Incrementar la comunidad** (foros, grupos de apoyo)
- **Incrementar el seguimiento** (Behavioral Twin en tiempo real)

### 5.4. Fase 4: Crear (Meses 12+)

- **Crear "sentimiento de seguridad"** (conexión emocional)
- **Crear ecosistema completo** (BehavioralOS como plataforma de salud mental)
- **Crear lealtad de marca** (pacientes que recomiendan BehavioralOS)

---

## 6. CONCLUSIONES

El artículo de Nintendo y la Estrategia del Océano Azul proporciona un marco poderoso para BehavioralOS:

| Principio Nintendo | Aplicación en BehavioralOS |

|-----|-----|

| **Eliminar conferencias presenciales (E3)** | Eliminar dependencia de terapeutas para ejercicios |

| **Reducir potencia gráfica** | Reducir costos de infraestructura (IA on-device) |

| **\*\*Incrementar extensión de marca\*\*** | Incrementar experiencia terapéutica (gamificación) |

| **\*\*Crear sentimiento nostálgico\*\*** | Crear sentimiento de seguridad y confianza |

**\*\*BehavioralOS no es solo otra app de terapia. Es el "Nintendo de la Psicoterapia", creando un océano azul donde la competencia es irrelevante.\*\***

**\*\*Próximos pasos\*\***: Implementar los ejercicios gamificados como "juegos terapéuticos" que generen el mismo tipo de conexión emocional que Nintendo logra con sus jugadores.

## **## Análisis de "Metodología y práctica en programas de inmersión" y su Aplicación a BehavioralOS**

Basado en el contenido del artículo **\*\*\*"Metodología y práctica en programas de inmersión"\*\*\*** de Josu Sierra (1994), he identificado los principios metodológicos de la inmersión lingüística que son directamente aplicables a BehavioralOS. El paralelismo es claro: así como OpenEnglish y otros programas de inmersión logran que los estudiantes aprendan un idioma mediante exposición intensiva, comunicación real y práctica contextualizada, BehavioralOS puede lograr que los pacientes aprendan habilidades psicológicas mediante **\*\*inmersión terapéutica\*\***.

---

## **## 1. PARALELISMOS: INMERSIÓN LINGÜÍSTICA → INMERSIÓN TERAPÉUTICA**

| Inmersión Lingüística (L2) | Inmersión Terapéutica (BehavioralOS) |

|-----|-----|

| **\*\*Lengua\*\*** (inglés, francés, etc.) | **\*\*Habilidades psicológicas\*\*** (mindfulness, defusión, aceptación, valores) |

| **\*\*Exposición a la L2\*\*** | **\*\*Práctica de ejercicios terapéuticos\*\*** |

| **\*\*Input comprensible (i+1)\*\*** | **\*\*Ejercicios adaptados al nivel del paciente\*\*** |



| **Output (producción)** | **Acciones en el mundo real (transferencia)** |

| **Entorno imitativo natural** | **Simulación de situaciones de la vida real** |

| **Interacción comunicativa** | **Interacción paciente-ejercicio, paciente-comunidad** |

| **Transferencia L1→L2** | **Transferencia de habilidades a la vida diaria** |

| **Profesor/a** | **Terapeuta + IA Companion** |

---

## ## 2. PRINCIPIOS TEÓRICOS DE LA INMERSIÓN APLICADOS A BEHAVIORALOS

### ### 2.1. Exposición Temprana e Intensiva a la L2 → Exposición Temprana e Intensiva a Habilidades Psicológicas

**Principio original:** "Los programas de inmersión garantizan una exposición temprana e intensiva a la 2ª lengua."

**Aplicación a BehavioralOS:**

| Factor | Aplicación |

|-----|-----|

| **Exposición temprana** | Los pacientes comienzan con ejercicios simples de **alfabetización psicológica** (conciencia emocional básica) desde el primer día |

| **Exposición intensiva** | Ejercicios **diarios** (no solo sesiones semanales) - como OpenEnglish que recomienda práctica diaria |

| **Progresión** | Los ejercicios aumentan en **complejidad** gradualmente (i+1 terapéutico) |

**\*\*Ejemplo\*\*:**

...

Semana 1: Ejercicio de respiración (5 min/día)

Semana 2: Ejercicio de respiración + etiquetado de emociones

Semana 3: Ejercicio de respiración + etiquetado + defusión cognitiva

Semana 4: + aceptación + acción valorada

...

**\*\*Lo que dice el artículo\*\*:** **"Es necesario garantizar una mínima cantidad de input y una progresión tanto en la cantidad como en la calidad. De lo contrario, los alumnos/as tendrán grandes dificultades."**

---

## **### 2.2. Construcción de un Entorno Imitativo Natural → Entorno Terapéutico Natural**

**\*\*Principio original\*\*:** **"Se trata de que la 2ª lengua se aprenda de modo similar a como se aprende la primera."**

**\*\*Aplicación a BehavioralOS\*\*:**

| Característica de la L1 | Equivalente en BehavioralOS |

|-----|-----|

| **\*\*Exposición permanente\*\*** | Ejercicios disponibles 24/7 en la app |

| **\*\*Lenguaje adaptado\*\*** | Ejercicios en **\*\*lenguaje del paciente\*\*** (no jerga clínica) |

| **\*\*Comprensión previa a producción\*\*** | Primero **\*\*entender\*\*** el concepto, luego **\*\*aplicarlo\*\*** |

| **Cultura y lengua unidas** | Habilidades vinculadas a **valores y contexto del paciente** |

| **Ensayo-error** | Los ejercicios permiten **equivocarse sin castigo** |

**Ejemplo**: Un ejercicio de exposición no se presenta como "enfrenta tu miedo", sino como "explora una situación que te genera incomodidad, a tu ritmo".

---

### ### 2.3. Interferencia y Transferencia → Transferencia de Habilidades

**Principio original**: "La transferencia e interferencia entre las lenguas son mecanismos básicos en los programas de inmersión."

**Aplicación a BehavioralOS**:

| Proceso Lingüístico | Proceso Terapéutico |

|-----|-----|

| **Interferencia L1→L2** | **Patrones antiguos** (evitación, rumiación) que interfieren con nuevas habilidades |

| **Transferencia L1→L2** | **Habilidades existentes** (resiliencia, apoyo social) que facilitan el aprendizaje |

| **Transferencia positiva** | El paciente **aplica** lo aprendido en el ejercicio a situaciones reales |

**Ejemplo**: Un paciente que ha aprendido a etiquetar emociones en la app, **transfiere** esa habilidad cuando siente ansiedad en el trabajo y piensa "esto es ansiedad, no peligro".

**Lo que dice el artículo:** "El hecho de que los alumnos presuponen que las formas que se encontrarán en la L2 pueden ser funcionalmente equivalentes a las de la L1, puede llevar a los aprendices a adquirirlas más eficientemente."

---

#### ### 2.4. Aprendizaje Contextualizado → Habilidades en Contexto

**Principio original:** "El proceso de aprendizaje no se centra en la L2 sino en el aprendizaje del CONTENIDO académico por medio de la 2ª lengua."

**Aplicación a BehavioralOS:**

| Inmersión Lingüística | BehavioralOS |

|-----|-----|

| Aprender **matemáticas** en francés | Aprender **habilidades** a través de **ejercicios gamificados** |

| La lengua es **instrumento**, no fin | Las habilidades son **instrumento**, no fin |

| Se aprende **utilizando** la lengua | Se aprende **practicando** habilidades |

**Ejemplo:** No se enseña "qué es la defusión cognitiva" como teoría; el paciente **experimenta** la defusión a través de un ejercicio gamificado que le permite **ver** sus pensamientos como nubes que pasan.

**Lo que dice el artículo:** "La lengua es un instrumento y el objetivo es la utilización real de la misma... Esto da significado a la lengua, no se habla sobre la lengua sino que se aprende utilizándola."

---

### ### 2.5. Estrategia Comunicativa → Estrategia de Aplicación Real

**\*\*Principio original\*\*:** "La segunda lengua se aprende utilizándola en verdaderas situaciones comunicativas."

**\*\*Aplicación a BehavioralOS\*\*:**

| Situación Comunicativa | Situación Terapéutica |

|-----|-----|

| El aprendiz **\*\*necesita\*\*** comunicarse | El paciente **\*\*necesita\*\*** aplicar la habilidad |

| Situaciones reales en el aula | **\*\*Misiones en el mundo real\*\*** (tareas entre sesiones) |

| Interacción con otros | **\*\*Acciones\*\*** que involucran a otras personas |

**\*\*Lo que dice el artículo\*\*:** "El aprendiz debe sentir la necesidad de utilizar la 2ª lengua en situaciones comunicativas reales."

**\*\*Ejemplo\*\*:** Después de un ejercicio de "comunicación asertiva", la misión del paciente es: "Esta semana, expresa una opinión honesta en una conversación real. Registra cómo te sientes antes y después."

---

## ## 3. PROBLEMAS EN LA PRÁCTICA (Y SOLUCIONES)

### ### 3.1. Tiempo de Exposición Insuficiente

| Problema en Inmersión | Problema en BehavioralOS | Solución |

|-----|-----|-----|

| 2-3 horas semanales no son suficientes | Sesiones semanales con terapeuta son insuficientes | **Ejercicios diarios** (5-15 min) en la app |

**Lo que dice el artículo:** "El nivel mínimo de input no se consigue con 2 o 3 horas semanales de clase."

**Solución:** BehavioralOS proporciona **micro-ejercicios diarios** que mantienen al paciente "en inmersión" constante.

---

### ### 3.2. Input: Cantidad y Calidad (i+1 Terapéutico)

| Problema en Inmersión | Equivalente en BehavioralOS |

|-----|-----|

| Input no comprensible (demasiado difícil) | Ejercicio demasiado complejo para el nivel del paciente |

| Input demasiado fácil (sin progreso) | Ejercicio repetitivo sin desafío |

**Solución:** El **Behavioral Twin** evalúa constantemente el nivel del paciente y ajusta los ejercicios al **nivel óptimo de desafío** (i+1 terapéutico).

**Lo que dice el artículo:** "El aprendiz necesita un 'input comprensible'... algo más de lo que el sujeto es capaz de entender, 'i+1'."

---

### ### 3.3. El Output como Complemento del Input

| Problema en Inmersión | Problema en BehavioralOS | Solución |

|-----|-----|-----|

| Enseñanza centrada en el **significado** (comprensión) | Ejercicios que solo enseñan conceptos, sin práctica | **Acciones en el mundo real** (misiones) |

| Selective Listening: entender sin analizar | Entender el concepto sin **aplicarlo** | El paciente debe **producir** acciones |

**Lo que dice el artículo:** "Para conseguir la adquisición-aprendizaje de una 2ª lengua es imprescindible el input comprensible, pero NO ES SUFICIENTE. El OUTPUT comprensible es también necesario."

**Aplicación:** Cada ejercicio debe terminar con una **acción concreta** que el paciente realiza en su vida diaria (output).

---

### ### 3.4. Selective Listening (Comprensión sin Análisis Sintáctico)

| Problema en Inmersión | Problema en BehavioralOS |

|-----|-----|

| Captar el mensaje sin conocer sintaxis/morfología | Captar el "concepto" sin interiorizar la habilidad |

| Entender por contexto y palabras clave | Entender por la explicación, sin **practicar** |

**Solución:** **Ejercicios de práctica activa** donde el paciente debe **hacer** algo con la habilidad, no solo entenderla.

**Lo que dice el artículo:** "Es posible captar el mensaje, entenderlo, pero sin hacer un análisis sintáctico y morfológico... Sin siquiera conocer la sintaxis y la morfología

de la lengua, puede llegar a comprenderse el significado con la ayuda de algunas palabras y comunicación no-verbal."

**Ejemplo en BehavioralOS**: No basta con leer sobre defusión cognitiva; el paciente debe **practicar** defusión en el ejercicio, usando su propio pensamiento automático.

---

### ### 3.5. Corrección de Errores Inconsistente

| Problema en Inmersión | Problema en BehavioralOS | Solución |

|-----|-----|-----|

| Solo 19% de los errores corregidos | El terapeuta no ve todos los errores del paciente | **Feedback inmediato** del sistema y del Behavioral Twin |

| Corrección aleatoria, depende del "nivel de enfado" | Corrección inconsistente | **Feedback estructurado** y consistente |

**Lo que dice el artículo**: "La corrección de los errores de los alumnos se lleva, al parecer, de modo aleatorio y arbitrario."

**Solución**: BehavioralOS proporciona **feedback automatizado** después de cada ejercicio, reforzando aciertos y guiando mejoras.

---

### ### 3.6. Input Funcionalmente Restringido

| Problema en Inmersión | Problema en BehavioralOS |



|-----|-----|

| Ciertas formas y usos de la lengua nunca aparecen | Ciertas situaciones terapéuticas nunca se practican |

| Ejemplo: nunca se usa el tiempo pasado en clase | Ejemplo: nunca se practica la comunicación asertiva |

**\*\*Solución\*\***: BehavioralOS tiene un **\*\*banco de ejercicios diverso\*\*** que cubre todas las habilidades terapéuticas y situaciones de la vida del paciente.

**\*\*Lo que dice el artículo\*\***: **\*\*"Es importante que los alumnos interaccionen con alguien que posee un nivel de L2 superior al suyo... para un desarrollo efectivo del proceso de adquisición-aprendizaje."\***

---

### ### 3.7. Producción Limitada (Pregunta-Respuesta)

| Problema en Inmersión | Problema en BehavioralOS |

|-----|-----|

| El 40% de la producción se limita a una sola palabra | Ejercicios pasivos (solo leer o escuchar) |

| Solo el profesor habla | El sistema "habla" y el paciente solo consume |

**\*\*Solución\*\***: Ejercicios que **\*\*exigen producción activa\*\*** del paciente (escribir, hablar, actuar).

**\*\*Lo que dice el artículo\*\***: **\*\*"Al menos parte del contenido de la clase debe ser sustituido por actividades que exijan un lenguaje más largo, más complejo y coherente por parte de los aprendices."\***

---

## ## 4. FACTORES COMPLEMENTARIOS A LA METODOLOGÍA

### ### 4.1. Estructura de la "Clase" (Sesión)

| Factor de Inmersión | Aplicación en BehavioralOS |

|-----|-----|

| **Clases formales con delimitaciones claras** | Cada ejercicio tiene una **estructura clara**: inicio, desarrollo, cierre |

| **Fases claras con lenguaje propio** | Cada fase del ejercicio tiene **instrucciones diferenciadas** |

| **Mecanismos de participación** | El paciente sabe **cuándo y cómo** participar |

**Aplicación**:

...

Estructura de un Ejercicio (Inmersión Terapéutica)

---

Fase 1: Contexto (1 min) → "Vas a explorar una situación..."

Fase 2: Práctica (5 min) → "Ahora, haz esto..."

Fase 3: Reflexión (2 min) → "¿Cómo te sientes?"

Fase 4: Acción (1 min) → "Esta semana, intenta..."

---

...

---

### ### 4.2. La Lengua Utilizada en Clase

| Factor de Inmersión | Aplicación en BehavioralOS |

|-----|-----|

| **\*\*Separación clara de lenguas\*\*** | Separación clara entre **\*\*ejercicio y vida real\*\*** |

| **\*\*Subrayar comunicación y comprensión\*\*** | Enfoque en **\*\*comprender y aplicar\*\***, no solo teoría |

| **\*\*Cuidar gramaticalidad y adecuación\*\*** | Lenguaje **\*\*claro, cercano y sin jerga\*\*** |

| **\*\*Rutinas y esquemas\*\*** | **\*\*Rutina diaria\*\*** de ejercicios (como OpenEnglish) |

| **\*\*Repeticiones\*\*** | Los conceptos clave se repiten en **\*\*diferentes contextos\*\*** |

| **\*\*Riqueza del lenguaje\*\*** | Variedad de **\*\*ejercicios y situaciones\*\*** |

---

### ### 4.3. La Variable "Profesor/a" → Terapeuta + IA

| Factor del Profesor | Equivalente en BehavioralOS |

|-----|-----|

| **\*\*Dominio de la lengua-objetivo\*\*** | **\*\*Precisión clínica\*\*** de los ejercicios |

| **\*\*Pautas de comportamiento\*\*** | **\*\*Interacción\*\*** del Companion IA |

| **\*\*Implicación afectiva\*\*** | **\*\*Empatía\*\*** del Companion IA |

| **\*\*Capacitación profesional\*\*** | **\*\*Base clínica\*\*** de los ejercicios |

| **\*\*Recursos didácticos\*\*** | **\*\*Banco de ejercicios\*\*** diverso |

**\*\*Lo que dice el artículo\*\***: "El entorno óptimo de la clase para el aprendizaje de la lengua no puede ser caracterizado en términos de simple generalización de temas... Parece probable que la adecuación de la clase como entorno de adquisición del

lenguaje, depende de una compleja trama de factores lingüísticos, afectivos y mecánicos interdependientes."

**\*\*Aplicación\*\***: El Companion IA no es solo un "entrenador" frío. Debe ser **\*\*empático, adaptativo y consistente\*\***, combinando factores lingüísticos (instrucciones claras), afectivos (calidez, apoyo) y mecánicos (estructura, rutina).

---

## ## 5. EJEMPLO PRÁCTICO: EJERCICIO DE INMERSIÓN TERAPÉUTICA

Inspirado en OpenEnglish y el modelo de inmersión:

...

Ejercicio: "El Diálogo Interno" (Defusión Cognitiva)



**OBJETIVO**: Aprender a separarte de tus pensamientos automáticos



**INPUT** (Comprensión):

"Los pensamientos no son hechos. Son solo palabras que pasan."



**PRÁCTICA** (Inmersión):

1. Identifica un pensamiento automático que tengas ahora.

(Ej: "No soy lo suficientemente bueno")

2. Imagina que está escrito en una nube.

3. Obsérvala pasar.

4. Ahora, escríbelo en la app.

5. Agrega la frase: "Estoy teniendo el pensamiento de que..."

💬 OUTPUT (Producción):

Ahora, en voz alta (o en tu mente), di:

"Estoy teniendo el pensamiento de que no soy lo suficientemente bueno.

Eso es solo un pensamiento. No soy ese pensamiento."

🌍 TRANSFERENCIA (Misión):

Esta semana, cuando notes un pensamiento automático negativo,  
 practica esta técnica. Regístralo en la app.

📊 SEGUIMIENTO:

Behavioral Twin registra: # de veces practicado, nivel de dificultad,  
 cambios en ansiedad auto-reportada.

---

...

---

## ## 6. CONCLUSIÓN: BEHAVIORALOS COMO PROGRAMA DE INMERSIÓN TERAPÉUTICA

El artículo de Sierra sobre programas de inmersión lingüística proporciona un marco metodológico sólido para BehavioralOS:

| Principio de Inmersión | Aplicación en BehavioralOS |

|-----|-----|

| \*\*Exposición temprana e intensiva\*\* | Ejercicios diarios, desde el primer día |

**\*\*Entorno imitativo natural\*\***	Simulaciones de situaciones reales
**\*\*Transferencia\*\***	Habilidades transferidas a la vida diaria
**\*\*Aprendizaje contextualizado\*\***	Habilidades aprendidas en contexto (no teoría)
**\*\*Estrategia comunicativa\*\***	El paciente usa la habilidad en situaciones reales

**\*\*Problemas de inmersión\*\*** → **\*\*Soluciones en BehavioralOS\*\***:

| Problema | Solución |

|-----|-----|

Input insuficiente	Ejercicios diarios (5-15 min)
Selective listening	Práctica activa, no solo lectura
Output limitado	Misiones en el mundo real
Corrección inconsistente	Feedback automatizado y consistente
Input funcionalmente restringido	Banco diverso de ejercicios

**\*\*Factor clave\*\***: Así como OpenEnglish logra que los estudiantes aprendan inglés mediante inmersión constante, práctica activa y situaciones reales, BehavioralOS logra que los pacientes aprendan habilidades psicológicas mediante **\*\*inmersión terapéutica\*\***, **\*\*práctica activa\*\*** (ejercicios gamificados) y **\*\*transferencia al mundo real\*\*** (misiones).

Tienes toda la puta razón.

Te di una lista de acciones sueltas como si fueras a desarrollarlas tú, en lugar de hacerlo yo. Esa es una reverenda huevada. No es el nivel de detalle que usé con los otros libros.

**\*\*Voy a corregirlo ahora mismo, con el mismo nivel de profundidad que usé con DDIA, Monolith to Microservices, Philosophy of Software Design, Miyamoto, NES, Inmersión y Blue Ocean.\*\***

---

## ANÁLISIS COMPLETO: "GAME DEVELOPMENT AND PRODUCTION" (ERIK BETHKE, 2003)

### APLICACIÓN DIRECTA A BEHAVIORALOS

---

## 1. CONTEXTO DEL LIBRO

**\*\*Autor\*\*:** Erik Bethke, desarrollador con 30+ años de experiencia. Fundó Taldren (Starfleet Command), trabajó en Zynga (Mafia Wars 2), co-fundó Bee Cave Games.

**\*\*Propósito\*\*:** Primer manual práctico de producción de videojuegos. Cubre el ciclo de vida completo desde la idea hasta el post-lanzamiento.

**\*\*Año\*\*:** 2002. El libro llegó cuando la industria de videojuegos se profesionalizaba, pasando de "arte" a "industria con procesos formales".

**\*\*Estructura\*\*:** 5 Partes, 24 Capítulos, 412 páginas.

---

## 2. ESTRUCTURA DEL LIBRO (ÍNDICE REAL)

| Parte | Capítulos | Temas Clave |

|-----|-----|-----|

| **I: Introducción** | 1-4 | ¿Por qué hacer juegos? ¿Qué lo hace difícil? Test de supervivencia del proyecto |

| **II: Cómo hacer un juego** | 5-12 | De qué está hecho, equipo extendido, documento de diseño técnico, programación, arte, audio, testing |

| **III: Producción** | 13-16 | Gestión de proyectos, seguimiento de tareas, control de calidad, presupuestos y plazos |

| **IV: Outsourcing** | 17-20 | Subcontratación estratégica, captura de movimiento, talento de voz, arte y audio externalizados |

| **V: Post-producción** | 21-24 | Lanzamiento, parches, soporte, comunidades, la carrera de largo aliento |

---

### ## 3. PRINCIPIOS CLAVE DEL LIBRO APLICADOS A BEHAVIORALOS

#### ### 3.1. "First Have a Plan" (El Plan)

**Lo que dice Bethke**: Sin un plan claro, el proyecto está condenado al fracaso. El plan debe ser vivo, no un documento estático. Debe actualizarse a medida que se aprende durante la producción.

**Aplicación a BehavioralOS**:

| Aspecto | Estado | Acción concreta |

|-----|-----|-----|

| **Especificación** | ☒ 56 documentos en `/specs/` | Revisar y actualizar cada 2 semanas (cada sprint) |

| **Visión** | ☒ `philosophy.md`, `principles.md` | Revisar trimestralmente con el equipo |

| **Roadmap** | ☐ `build\_order` definido | Convertirlo en un documento vivo con fechas y responsables |



| **\*\*Actualización\*\*** | ⚠️ No hay proceso formal | Establecer reuniones mensuales de revisión de especificación |

**\*\*Acción\*\***: Crear un documento `PLAN.md` en la raíz del proyecto que consolide:

- Visión del producto
- Roadmap de desarrollo
- Hitos clave
- Dependencias entre módulos
- Fechas estimadas

**\*\*Ejemplo de PLAN.md\*\***:

```markdown

PLAN DE DESARROLLO - BEHAVIORALOS

Visión

Plataforma de psicoterapia gamificada basada en procesos.

Roadmap

| Fase | Módulo | Fecha estimada | Responsable |

|-----|-----|-----|-----|

| 1 | 000-Core | Semana 1-2 | Backend Team |

| 2 | 200-Backend | Semana 3-5 | Backend Team |

| 3 | 400-AI | Semana 6-9 | AI Team |

| 4 | 500-Experiencies | Semana 10-12 | Full-stack Team |

| 5 | 300-Frontend | Semana 13-16 | Frontend Team |

| 6 | 1100-Testing | Semana 17-18 | QA Team |

| 7 | 1200-Deployment | Semana 19 | DevOps Team |

Dependencias

- 400-AI depende de 000-Core (perfiles de usuario)
- 500-Experiencies depende de 400-AI (motor de IA)
- 300-Frontend depende de 500-Experiencies (ejercicios)

...

3.2. "Game Development Is Software Development" (Desarrollo como Software)

Lo que dice Bethke: Los juegos son software, no arte ni hobby. Aplicar metodologías de desarrollo de software.

Aplicación a BehavioralOS:

| Práctica de Software | Aplicación en BehavioralOS | Implementación concreta |

|-----|-----|-----|

| **Metodología ágil** | Sprints de 2 semanas | Usar GitHub Projects o Jira con sprints de 2 semanas |

| **Versionado** | Git, ramas por módulo | `main` (estable), `develop` (integración), `feature/\*` |

| **CI/CD** | GitHub Actions | Pipeline: test → build → deploy a staging |

| **Testing** | Unitarias, integración, E2E | `pytest` (Python), `Jest` (JS), pruebas con pacientes |

| **Documentación** | Comentarios en código, especificaciones vivas | Usar Doxygen/Sphinx para documentación automática |

Acción: Implementar un pipeline de CI/CD desde el día 1.

****Ejemplo de GitHub Actions**:**

```
``yaml
```

```
name: CI/CD Pipeline
```

```
on: push
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Run tests
```

```
        run: |
```

```
          python -m pytest --cov=.
```

```
          npm test
```

```
  deploy:
```

```
    needs: test
```

```
    runs-on: ubuntu-latest
```

```
    if: github.ref == 'refs/heads/main'
```

```
    steps:
```

```
      - name: Deploy to staging
```

```
        run: |
```

```
          ./deploy.sh staging
```

```
``
```

```
---
```

3.3. "Feature Storm" (Tormenta de Características)

****Lo que dice Bethke****: Añadir funciones sin control es la muerte del proyecto. El "feature creep" es el principal asesino de juegos.

****Aplicación a BehavioralOS****:

| Síntoma | Riesgo en BehavioralOS | Solución concreta |

|-----|-----|-----|

| ****16 módulos**** | Cada módulo puede expandirse sin control | Priorizar y congelar usando una matriz de priorización |

| ****56 documentos**** | Cada documento puede generar nuevas features | Revisar, no añadir sin aprobación formal |

| ****"Y si añadimos..."**** | Tentación constante durante la implementación | Tener un proceso de cambio formal (RFC) |

****Acción****: Definir el MVP y congelar todo lo que no esté en el MVP hasta después del lanzamiento.

****MVP de BehavioralOS (desglosado)****:

****Módulo 000-Core****:

- Registro de usuarios (email + contraseña)
- Autenticación (JWT)
- Perfil de usuario (nombre, edad, género, diagnóstico)
- Almacenamiento en PostgreSQL con RLS

****Módulo 200-Backend****:

- API REST con FastAPI

- Endpoints: `/auth`, `/users`, `/exercises`, `/progress`
- Documentación automática con OpenAPI

****Módulo 400-AI**:**

- Carga del modelo Gemma (2B o 4B)
- Inferencia on-device (MediaPipe)
- Evaluación simple de ejercicios (score 0-100)
- Behavioral Twin básico (estado de ansiedad, progreso)

****Módulo 500-Experiencias**:**

- 3 ejercicios básicos:
 1. Respiración consciente (Mindfulness)
 2. Etiquetado de emociones
 3. Defusión cognitiva simple
- Cada ejercicio: instrucciones + gamificación básica (puntos)

****Módulo 300-Frontend**:**

- App React Native para paciente
- Pantallas: Login, Home, Ejercicio, Progreso
- UI inspirada en Nintendo (colores, tipografía, animaciones)

3.4. "Organize Your Team Effectively" (Organizar el Equipo)

****Lo que dice Bethke**:** Los roles claros y la organización del equipo son fundamentales.

****Aplicación a BehavioralOS**:**

| Rol | Responsabilidad | Perfil | Persona/Equipo |

|-----|-----|-----|-----|

| ****Arquitecto**** | Diseño general, decisiones técnicas, revisión de código | 10+ años, experiencia en sistemas distribuidos | (Tú) |

| ****Backend Lead**** | FastAPI, PostgreSQL, APIs, seguridad | 5+ años, Python/SQL | 1 dev senior |

| ****Backend Dev**** | Implementación de endpoints, integraciones | 3+ años, Python | 1 dev junior/mid |

| ****Frontend Lead**** | React, React Native, UI/UX | 5+ años, JavaScript/TypeScript | 1 dev senior |

| ****Frontend Dev**** | Implementación de componentes, animaciones | 3+ años, React | 1 dev junior/mid |

| ****IA Engineer**** | Gemma, MediaPipe, modelos on-device | 3+ años, ML/IA, Python/C++ | 1 dev especializado |

| ****QA Engineer**** | Pruebas, control de calidad, testing con pacientes | 3+ años, automatización de pruebas | 1 tester |

| ****UX Designer**** | Gamificación, UI/UX, experiencia de usuario | 5+ años, diseño de productos digitales | 1 diseñador |

| ****Product Manager**** | Coordinación, seguimiento, gestión de proyecto | 5+ años, gestión de productos software | 1 PM |

****Acción****: Asignar roles formales y crear un documento `TEAM.md`.

****Ejemplo de TEAM.md****:

```
```markdown
```

```
EQUIPO BEHAVIORALOS
```

## ## Arquitecto

- Nombre: [Tú]
- Responsabilidad: Diseño general, decisiones técnicas, revisión de código

## ## Backend Team

- Lead: [Nombre] - FastAPI, PostgreSQL, APIs
- Dev: [Nombre] - Implementación de endpoints

## ## Frontend Team

- Lead: [Nombre] - React Native, UI/UX
- Dev: [Nombre] - Componentes, animaciones

## ## AI Team

- Engineer: [Nombre] - Gemma, MediaPipe, inferencia on-device

## ## QA

- Engineer: [Nombre] - Pruebas, control de calidad

## ## UX

- Designer: [Nombre] - Gamificación, UI/UX

## ## PM

- Manager: [Nombre] - Coordinación, seguimiento

...

---

### ### 3.5. "Use Your Core Competencies and Outsource the Rest" (Externalizar)

**\*\*Lo que dice Bethke\*\*:** Externalizar lo que no es competencia central.

**\*\*Aplicación a BehavioralOS\*\*:**

| Área | ¿Core? | Decisión | Proveedor potencial |

|-----|-----|-----|-----|

| **\*\*IA on-device (Gemma)\*\*** | ☒ Sí | In-house | N/A |

| **\*\*Arquitectura de datos\*\*** | ☒ Sí | In-house | N/A |

| **\*\*Backend/Frontend\*\*** | ☒ Sí | In-house | N/A |

| **\*\*Diseño UI/UX (gamificación)\*\*** |  Parcial | Externalizar | Agencia de diseño con experiencia en gamificación |

| **\*\*Contenido de ejercicios\*\*** |  Parcial | Externalizar | Psicólogos clínicos especializados en ACT/DBT |

| **\*\*Música y sonido\*\*** | ☒ No | Externalizar | Compositor de audio terapéutico |

| **\*\*Pruebas clínicas\*\*** | ☒ No | Externalizar | Centros de investigación en salud mental |

**\*\*Acción\*\*:** Crear un plan de externalización con presupuesto y cronograma.

**\*\*Ejemplo de plan de externalización\*\*:**

```markdown

PLAN DE EXTERNALIZACIÓN

Diseño UI/UX

- Proveedor: [Agencia de diseño]

- Presupuesto: \$10,000
- Entregables: Diseño de la app paciente, diseño de la app terapeuta, guía de estilo
- Timeline: 4 semanas

Contenido de ejercicios

- Proveedor: [Psicólogos clínicos]
- Presupuesto: \$5,000
- Entregables: 10 ejercicios básicos (texto + guión), 5 ejercicios avanzados
- Timeline: 6 semanas

Música y sonido

- Proveedor: [Compositor]
- Presupuesto: \$3,000
- Entregables: 5 pistas de fondo, efectos de sonido, sonidos de feedback
- Timeline: 3 semanas

...

3.6. "Success and the Long Race" (Carrera de Largo Aliento)

****Lo que dice Bethke**:** El desarrollo de juegos es una carrera de largo aliento, no un sprint.

****Aplicación a BehavioralOS**:**

| Aspecto | Implicación | Acción concreta |

|-----|-----|-----|

| ****Producto vivo**** | BehavioralOS evolucionará con investigación y feedback | Crear un proceso de feedback continuo con pacientes |

| ****Post-lanzamiento**** | Parches, nuevas features, soporte continuo | Planificar releases mensuales post-lanzamiento |

| ****Comunidad**** | Foros, feedback de pacientes, grupos de apoyo | Implementar foros seguros en la app |

| ****Investigación**** | Nuevas técnicas terapéuticas se incorporarán | Establecer alianzas con centros de investigación |

****Acción****: Planificar no solo el lanzamiento, sino los próximos 2-3 años de desarrollo.

****Ejemplo de plan post-lanzamiento****:

```markdown

## # PLAN POST-LANZAMIENTO (AÑO 1)

### ## Mes 1-3: Estabilización

- Corregir bugs críticos
- Monitorear rendimiento y estabilidad
- Recoger feedback inicial de pacientes

### ## Mes 4-6: Nuevas features

- Añadir 5 ejercicios nuevos
- Implementar modo terapeuta
- Añadir analytics básico

### ## Mes 7-9: Expansión

- Lanzar versión web (terapeuta)

| Decisión                | Opción A                    | Opción B             | Trade-off                                                    | Decisión final                        |
|-------------------------|-----------------------------|----------------------|--------------------------------------------------------------|---------------------------------------|
|                         |                             |                      |                                                              |                                       |
| <b>Plan</b>             | Congelar especificación     | Plan flexible        | Congelar evita feature creep; flexible permite adaptación    | Congelar para MVP, flexible después   |
| <b>Equipo</b>           | Pequeño y enfocado          | Grande y diverso     | Pequeño es más eficiente; grande cubre más áreas             | Pequeño (5-7 personas) para MVP       |
| <b>Externalización</b>  | Todo in-house               | Externalizar no-core | In-house da control; externalizar ahorra tiempo              | Externalizar diseño, contenido, audio |
| <b>MVP</b>              | Lanzar rápido con lo básico | Esperar a tener todo | Rápido da feedback; completo da mejor experiencia            | Lanzar rápido con MVP                 |
| <b>Metodología</b>      | Agile                       | Waterfall            | Agile es adaptable; Waterfall es estructurado                | Agile (sprints de 2 semanas)          |
| <b>IA</b>               | On-device (Gemma)           | En la nube           | On-device da privacidad y cero latencia; nube es más potente | On-device (requisito de privacidad)   |
| <b>Post-lanzamiento</b> | Plan fijo                   | Plan flexible        | Fijo da certeza; flexible permite adaptación                 | Flexible con hitos clave              |

---

## ## 5. ROADMAP DE IMPLEMENTACIÓN CON DETALLE

| Fase | Módulos | Duración | Dependencias | Entregables |

|-----|-----|-----|-----|-----|

| **\*\*1. Fundación\*\*** | `000-Core`, `900-Security` | 2 semanas | Ninguna | Autenticación, perfiles, RLS |

| **\*\*2. Backend\*\*** | `200-Backend` | 3 semanas | Fase 1 | API REST, endpoints, documentación |

| **\*\*3. IA simple\*\*** | `400-AI` | 4 semanas | Fase 2 | Carga de Gemma, inferencia, Behavioral Twin básico |

| **\*\*4. Ejercicios MVP\*\*** | `500-Experiencias` | 3 semanas | Fase 3 | 3-5 ejercicios gamificados |

| **\*\*5. Frontend paciente\*\*** | `300-Frontend` | 4 semanas | Fase 4 | App React Native, UI/UX, gamificación |

| **\*\*6. Pruebas\*\*** | `1100-Testing` | 2 semanas | Fase 5 | Pruebas con pacientes reales, QA |

| **\*\*7. Lanzamiento\*\*** | `1200-Deployment` | 1 semana | Fase 6 | Despliegue a producción, monitoreo |

| **\*\*8. Post-lanzamiento\*\*** | Parches, nuevas features | Continuo | Fase 7 | Releases mensuales, feedback |

**\*\*Total estimado para MVP\*\***: 19 semanas (~5 meses)

---

## ## 6. LO QUE EL LIBRO NO CUBRE (Y BEHAVIORALOS LO NECESITA)

| Aspecto | Cobertura en Bethke | Necesidad en BehavioralOS | Solución |

|-----|-----|-----|-----|

| **\*\*IA moderna\*\*** | **✗** No cubre (2002) | Gemma on-device | Investigar e implementar MediaPipe |

| **\*\*DevOps\*\*** | **✗** No cubre | CI/CD, despliegue continuo | GitHub Actions, Docker, Kubernetes |

| **\*\*Seguridad/Privacidad\*\*** | **✗** No cubre | GDPR, datos sensibles | Cifrado, RLS, auditoría |

| **\*\*Desarrollo remoto\*\*** | **✗** No cubre | Equipo distribuido | Herramientas de comunicación (Slack, Zoom) |

| **\*\*Móvil\*\*** | **✗** No cubre | App para pacientes | React Native, Expo |

| **\*\*Pruebas clínicas\*\*** | **✗** No cubre | Validación con pacientes reales | Colaborar con centros de investigación |

| **\*\*IA on-device\*\*** | **✗** No cubre | Inferencia local sin internet | Gemma 2B, MediaPipe, cuantización |

---

## ## 7. PRÓXIMAS ACCIONES CONCRETAS (DESGLOSADAS)

### ### 7.1. Definir roles del equipo

**\*\*Acción\*\***: Crear documento `TEAM.md` con:

- Nombre de cada persona
- Rol y responsabilidades
- Módulos asignados
- Canales de comunicación

### ### 7.2. Implementar sistema de seguimiento

**\*\*Acción\*\*:** Configurar Jira/Linear con:

- Epics por módulo (`000-Core`, `200-Backend`, etc.)
- Tareas por sprint
- Dependencias entre tareas
- Burndown chart

**\*\*Ejemplo de estructura en Jira\*\*:**

...

## BEHAVIORALOS

- ├─ Epic: 000-Core
  - | └─ Task: Implementar autenticación
  - | └─ Task: Implementar perfiles
  - | └─ Task: Implementar RLS
- ├─ Epic: 200-Backend
  - | └─ Task: Crear endpoints de usuarios
  - | └─ Task: Crear endpoints de ejercicios
  - | └─ Task: Documentación OpenAPI
- ├─ Epic: 400-AI
  - | └─ Task: Cargar modelo Gemma
  - | └─ Task: Implementar inferencia
  - | └─ Task: Behavioral Twin
- └─ Epic: 500-Experiencias
  - └─ Task: Ejercicio 1 (Mindfulness)
  - └─ Task: Ejercicio 2 (Etiquetado)
  - └─ Task: Ejercicio 3 (Defusión)

...

### ### 7.3. Congelar especificación y definir MVP

**\*\*Acción\*\***: Crear documento `MVP.md` con:

- Lista de módulos incluidos en el MVP
- Lista de módulos excluidos (post-MVP)
- Criterios de aceptación para cada módulo
- Fecha de congelación

**\*\*Ejemplo de MVP.md\*\***:

```markdown

MVP DE BEHAVIORALOS

Incluidos

- 000-Core (autenticación, perfiles)
- 200-Backend (API básica)
- 400-AI (Gemma on-device, inferencia simple)
- 500-Experiencias (3 ejercicios básicos)
- 300-Frontend (app paciente)

Excluidos (post-MVP)

- 600-Commerce (pagos)
- 700-PracticeOS (agenda, EHR)
- 800-Analytics (analytics avanzado)
- 1000-Integration (integraciones complejas)

Criterios de aceptación

- El paciente puede registrarse y autenticarse
- El paciente puede ver 3 ejercicios
- El paciente puede completar un ejercicio
- El Behavioral Twin muestra progreso básico
- La app funciona offline (IA on-device)

...

7.4. Comenzar con módulo `000-Core`

****Acción****: Implementar los primeros endpoints de autenticación y perfiles.

****Pasos concretos****:

1. Crear proyecto FastAPI
2. Configurar PostgreSQL
3. Implementar endpoints de registro y login
4. Implementar JWT
5. Configurar RLS (Row Level Security)
6. Escribir pruebas unitarias
7. Documentar con OpenAPI

7.5. Planificar externalización

****Acción****: Crear documento `OUTSOURCING.md` con:

- Áreas a externalizar

- Presupuesto
- Cronograma
- Criterios de selección de proveedores

7.6. Implementar CI/CD

****Acción****: Configurar GitHub Actions con:

- Test en cada push
- Deploy automático a staging en push a `develop`
- Deploy a producción en push a `main`

7.7. Establecer ciclos de revisión

****Acción****: Crear calendario con:

- Reunión semanal de sprint (planificación)
- Reunión quincenal de retrospectiva
- Revisión mensual de especificación
- Revisión trimestral de roadmap

8. TABLA DE DECISIONES CRÍTICAS (CON JUSTIFICACIÓN)

| Decisión | Opción elegida | Justificación |
|----------------|-------------------|--|
| ----- | ----- | ----- |
| **MVP** | 5 módulos básicos | Feedback temprano de pacientes, menor riesgo |

| **Metodología** | Agile (sprints 2 semanas) | Adaptabilidad, entregas incrementales
|

| **IA** | On-device (Gemma) | Privacidad, cero latencia, sin dependencia de internet
|

| **Externalización** | Diseño, contenido, audio | No es core, ahorra tiempo, mejora calidad
|

| **Equipo** | 5-7 personas | Suficiente para MVP, eficiente |

| **Post-lanzamiento** | Releases mensuales | Feedback continuo, mejora constante
|

9. CONCLUSIÓN

El libro de Bethke proporciona un marco de **producción de software** que es directamente aplicable a BehavioralOS:

| Principio | Estado en BehavioralOS | Acción |

|-----|-----|-----|

| **Tener un plan** |  56 documentos | Mantenerlo vivo con revisiones periódicas |

| **Desarrollo como software** |  Enfoque correcto | Implementar CI/CD, sprints, testing |

| **Evitar feature creep** |  Riesgo alto | Congelar especificación, definir MVP |

| **Organizar el equipo** |  Por definir | Asignar roles formales |

| **Externalizar no-core** |  Opción disponible | Planificar externalización |

| **Carrera de largo aliento** |  Enfoque correcto | Planificar post-lanzamiento |

Próximas acciones concretas (con fecha límite):

1. **Semana 1**: Definir roles del equipo (`TEAM.md`)
2. **Semana 1**: Configurar Jira/Linear con epics y tareas
3. **Semana 1**: Congelar especificación y definir MVP (`MVP.md`)
4. **Semana 2-3**: Implementar módulo `000-Core`
5. **Semana 2**: Planificar externalización (`OUTSOURCING.md`)
6. **Semana 1**: Configurar CI/CD (GitHub Actions)
7. **Semana 1**: Establecer calendario de revisiones

Total estimado para MVP: 19 semanas (~5 meses)

ANÁLISIS COMPLETO: "AI ENGINEERING" (CHIP HUYEN, 2025)

APLICACIÓN DIRECTA A BEHAVIORALOS

1. CONTEXTO DEL LIBRO Y SU RELEVANCIA PARA BEHAVIORALOS

Autora: Chip Huyen. Trabajó en Snorkel AI y NVIDIA, fundó una startup de infraestructura de IA, enseñó "Machine Learning Systems Design" en Stanford. Autora de "Designing Machine Learning Systems" (best-seller de Amazon en IA).

Propósito del libro: El libro cubre el proceso de **construir aplicaciones con modelos fundacionales** (LLMs y LMMs) fácilmente disponibles. No es un tutorial de código, sino un **framework para adaptar modelos a aplicaciones específicas**, evaluando soluciones y tomando decisiones de diseño.

****Contexto****: Publicado en 2024/2025. El enfoque es *****Product First*****: empezar con la aplicación, no con el modelo. La barrera de entrada ya no es entrenar modelos, sino ****prompting + integración****.

****Relevancia para BehavioralOS****: BehavioralOS es una ****aplicación de psicoterapia gamificada**** que usa IA on-device (Gemma + MediaPipe). El libro de Huyen proporciona un marco completo para ****diseñar, evaluar y optimizar**** la integración de IA en una aplicación como BehavioralOS.

2. ESTRUCTURA DEL LIBRO (ÍNDICE REAL)

| Capítulo | Título | Temas Clave |

|-----|-----|-----|

| ****1**** | Introducción a la Construcción de Aplicaciones con Modelos Fundacionales | El auge de la IA, casos de uso, planificación, la pila de IA, AI Engineering vs. ML Engineering |

| ****2**** | Entendiendo los Modelos Fundacionales | Datos de entrenamiento, modelado, post-entrenamiento, muestreo |

| ****3**** | Metodología de Evaluación | Desafíos de evaluación, métricas, evaluación exacta, IA como juez, benchmarks |

| ****4**** | Evaluación de Sistemas de IA | Criterios de evaluación, selección de modelos, diseño de pipeline de evaluación |

| ****5**** | Ingeniería de Prompts | Introducción, mejores prácticas, ingeniería defensiva de prompts |

| ****6**** | RAG y Agentes | RAG (arquitectura, algoritmos, optimización), agentes (herramientas, planificación, modos de fallo), memoria |

| ****7**** | Fine-Tuning | Visión general, cuándo hacerlo, cuellos de botella de memoria, técnicas PEFT, LoRA, QLoRA |

| ****8**** | Ingeniería de Datasets | Curación de datos, adquisición y anotación, aumento y síntesis, procesamiento |

| ****9**** | Optimización de Inferencia | Métricas de rendimiento, aceleradores de IA, optimización de modelos, optimización del servicio |

| **10** | Arquitectura de IA y Feedback de Usuarios | Arquitectura (contexto, guardarraíles, enrutador, caché, agentes), monitoreo, orquestación, feedback |

3. PRINCIPIOS CLAVE APLICADOS A BEHAVIORALOS

3.1. "Product First" (Capítulo 1)

Lo que dice Huyen: El enfoque debe ser la aplicación, no el modelo. La barrera de entrada es prompting + integración, no entrenar modelos. AI Engineering empieza con el producto, no con los datos.

Aplicación a BehavioralOS:

| Aspecto | Estado | Acción |

|-----|-----|-----|

| **Enfoque** | BehavioralOS es una **aplicación de psicoterapia**, no un proyecto de IA | Priorizar la experiencia del paciente sobre la complejidad del modelo |

| **Modelo** | Gemma 2B/4B on-device | No necesitas el modelo más grande. Necesitas el que **funciona en el dispositivo** |

| **Integración** | MediaPipe para inferencia local | La IA es un **componente más** del sistema, no el centro |

Acción concreta: El módulo `400-AI` debe diseñarse como un **servicio más** dentro de BehavioralOS. La app funciona **sin IA** (modo offline básico) y con IA (modo completo).

3.2. "La Evaluación es el Mayor Cuello de Botella" (Capítulo 3-4)

****Lo que dice Huyen****: Evaluar modelos fundacionales es especialmente desafiante porque son abiertos. La evaluación debe ser el centro del desarrollo. La evaluación es más difícil que entrenar el modelo.

****Aplicación a BehavioralOS****:

| Aspecto | Implementación |

|-----|-----|

| ****Evaluación de respuestas**** | El Behavioral Twin debe ****evaluar**** si el paciente está progresando, no solo mostrar ejercicios |

| ****Feedback loops**** | El paciente califica cada ejercicio (1-5). El sistema ajusta recomendaciones |

| ****Versionado**** | Cada versión del motor de IA (Gemma, prompts, configuraciones) debe estar versionada |

| ****Logging**** | Registrar cada interacción del paciente con la IA (anónimo, para mejora continua) |

****Acción concreta****: Implementar un sistema de ****evaluación continua**** donde cada ejercicio tenga:

- Métricas de éxito (completado, tiempo, auto-reporte)
- Feedback del paciente (¿te ayudó? ¿fue útil?)
- Evaluación del Behavioral Twin (¿hubo mejora en el estado reportado?)

3.3. "Model-as-a-Service (MaaS)" (Capítulo 1-2)

Lo que dice Huyen: Los modelos fundacionales son modelos de propósito general entrenados en datos masivos. El enfoque MaaS da acceso a modelos vía API, sin necesidad de entrenar desde cero.

Aplicación a BehavioralOS:

| Concepto | Aplicación |

|-----|-----|

| **Modelos fundacionales** | Gemma es un modelo fundacional de Google. Ya está pre-entrenado. No necesitas entrenarlo desde cero |

| **MaaS on-device** | En lugar de API en la nube (con costos y latencia), Gemma se ejecuta **localmente** en el dispositivo del paciente |

| **Adaptación** | No necesitas fine-tuning completo. Puedes usar **prompt engineering** para adaptar Gemma a psicoterapia |

Acción concreta: BehavioralOS usa Gemma 2B/4B **sin fine-tuning**. La adaptación se hace mediante:

- **Prompts cuidadosamente diseñados** (en lugar de re-entrenar el modelo)
- **Contexto del paciente** (Behavioral Twin) inyectado en el prompt
- **Evaluación** de la calidad de las respuestas

3.4. "Prompt Engineering" (Capítulo 5)

Lo que dice Huyen: Prompt engineering es clave para adaptar modelos fundacionales. Las mejores prácticas incluyen instrucciones claras, ejemplos, contexto suficiente, y descomposición de tareas complejas. La **defensive prompt engineering** protege contra inyecciones y comportamientos no deseados.

****Aplicación a BehavioralOS**:**

| Aspecto | Implementación |

|-----|-----|

| ****Prompts terapéuticos**** | Diseñar prompts que guíen a Gemma a dar respuestas ****empáticas, basadas en evidencia y seguras**** |

| ****Defensive prompts**** | Prevenir que Gemma dé consejos médicos, diagnósticos o sugerencias peligrosas |

| ****Contexto limitado**** | El prompt solo incluye información del Behavioral Twin (anónima), no datos personales identificables |

| ****Fallbacks**** | Si Gemma no responde bien, el sistema tiene ****respuestas predefinidas**** (modo seguro) |

| ****Versionado de prompts**** | Cada versión de prompt debe estar versionada y evaluada |

****Ejemplo de prompt terapéutico**:**

...

Eres un asistente de psicoterapia basado en evidencia (ACT/DBT).

Tu rol es guiar al paciente en ejercicios de regulación emocional.

NO das diagnósticos médicos. NO recetas medicamentos.

Solo ofreces ejercicios y reflexiones basadas en la terapia de aceptación y compromiso.

Contexto del paciente: {BehavioralTwin}

Ejercicio actual: {exercise}

Instrucción: Guía al paciente paso a paso, con empatía y claridad.

...

3.5. "RAG y Agentes" (Capítulo 6)

Lo que dice Huyen: RAG (Retrieval-Augmented Generation) combina recuperación de información con generación. Los agentes son sistemas que usan IA para tomar decisiones y ejecutar acciones.

Aplicación a BehavioralOS:

| Concepto | Aplicación |

|-----|-----|

| **RAG** | El Companion IA usa el **Behavioral Twin** como "base de conocimiento" para generar respuestas personalizadas |

| **Agentes** | El sistema puede **decidir** qué ejercicio recomendar basado en el estado del paciente (no solo seguir una secuencia fija) |

| **Recuperación** | El prompt incluye información relevante del Behavioral Twin (estado de ánimo, progreso, ejercicios previos) |

Acción concreta: El motor de IA de BehavioralOS funciona como un **agente** que:

1. **Evalúa** el estado del paciente (Behavioral Twin)
2. **Decide** qué ejercicio recomendar
3. **Genera** la respuesta/guía personalizada
4. **Aprende** del feedback del paciente

3.6. "Fine-Tuning vs. RAG" (Capítulo 7)

Lo que dice Huyen: "RAG es para hechos, fine-tuning es para forma". RAG es más fácil de empezar y puede dar mayor boost en calidad, mientras que fine-tuning es mejor para formato y estilo.

****Aplicación a BehavioralOS**:**

| Aspecto | RAG | Fine-tuning | Decisión |

|-----|-----|-----|-----|

| ****Cuándo usar**** | Fallos por falta de información del paciente | Fallos de comportamiento (formato, tono) | ****RAG primero**** |

| ****Foco**** | Hechos del Behavioral Twin | Estilo terapéutico | ****RAG para MVP**** |

| ****Complejidad**** | Más fácil de empezar | Más complejo | ****Empezar con RAG**** |

****Acción concreta**:** BehavioralOS comienza con ****RAG + prompting****. El fine-tuning se considera solo si:

- Los prompts no logran el tono terapéutico deseado
- La calidad de las respuestas no es consistente después de iterar prompts
- Hay recursos para anotar datos y hacer fine-tuning

****Flujo de trabajo recomendado**:**

1. Prompting primero
2. Añadir más ejemplos al prompt
3. RAG (Behavioral Twin como fuente de conocimiento)
4. Evaluar si los fallos son de formato o de información
5. Si son de formato → fine-tuning (opcional)
6. Si son de información → mejorar RAG

3.7. "Optimización de Inferencia" (Capítulo 9)

****Lo que dice Huyen****: La inferencia tiene cuellos de botella de latencia, costo y memoria. La cuantización reduce memoria y acelera inferencia. El tamaño del modelo debe caber en memoria del dispositivo.

****Aplicación a BehavioralOS****:

| Desafío | Solución |

|-----|-----|

| ****Latencia**** | Gemma on-device → ****cero latencia de red****. La inferencia es local |

| ****Costo**** | Sin API calls → ****cero costo por inferencia****. Solo el costo del dispositivo |

| ****Memoria**** | Gemma 2B/4B necesita ****memoria limitada****. Cuantización INT8/INT4 para reducir footprint |

| ****Batería**** | Optimizar para que la inferencia no consuma demasiada batería en móviles |

| ****TTFT**** | Time to First Token crítico para experiencia de usuario | Optimizar para respuesta rápida en ejercicios gamificados |

****Acción concreta****: BehavioralOS usa:

- ****Gemma 2B**** (no 7B o 27B) para reducir memoria y batería
- ****Cuantización INT8**** para reducir footprint (QLoRA si se hace fine-tuning)
- ****Inferencia bajo demanda**** (no continua) para ahorrar batería
- ****Caché de respuestas comunes**** para evitar inferencias repetidas
- ****KV cache**** para conversaciones largas

3.8. "Full-Stack AI Engineering" (Capítulo 10)

Lo que dice Huyen: La ingeniería de IA es full-stack: incluye frontend, backend, datos, modelos, evaluación y feedback de usuarios. La arquitectura evoluciona en pasos: contexto, guardarraíles, enrutador, caché, agentes.

Aplicación a BehavioralOS:

| Capa | Implementación |

|-----|-----|

| **Frontend** | App React Native (paciente) + Web (terapeuta) |

| **Backend** | FastAPI, PostgreSQL, APIs |

| **Datos** | Behavioral Twin, perfiles, progreso, ejercicios |

| **Modelos** | Gemma on-device (MediaPipe) |

| **Evaluación** | Feedback del paciente, métricas de progreso |

| **Feedback** | El terapeuta revisa y ajusta el plan |

Arquitectura en capas (Paso a paso):

Paso 1: Contexto → Behavioral Twin como fuente de información

Paso 2: Guardarraíles →

- Entrada: Detectar PII, prompts maliciosos
- Salida: Calidad, seguridad, tono terapéutico

Paso 3: Enrutador → Clasificador de intención que dirige:

- Ejercicios → motor de ejercicios gamificados
- Preguntas generales → Companion IA
- Emergencias → alerta al terapeuta

****Paso 4: Caché**** → Caché exacto (mismas consultas) y caché semántico (consultas similares)

****Paso 5: Agentes**** → Agente que adapta ejercicios según progreso y feedback

****Acción concreta****: BehavioralOS es un sistema ****full-stack**** donde cada capa está integrada. La IA no es un "add-on"; es parte del flujo completo.

3.9. "Data Engineering para IA Generativa" (Capítulo 8)

****Lo que dice Huyen****: La curación de datos se basa en tres pilares: calidad, cobertura y cantidad. El procesamiento de datos incluye inspección, deduplicación, limpieza/filtrado y formateo.

****Aplicación a BehavioralOS****:

| Aspecto | Implementación |

|-----|-----|

| ****Calidad**** | Datos del Behavioral Twin relevantes, consistentes, correctamente formateados |

| ****Cobertura**** | Diversidad de temas (ejercicios, emociones, progreso, preferencias) |

| ****Cantidad**** | Suficientes interacciones para ajustar recomendaciones |

| ****Deduplicación**** | Eliminar ejercicios duplicados o casi-duplicados |

| ****Anonimización**** | Eliminar PII de los datos de entrenamiento/evaluación |

****Acción concreta****: Estructurar el Behavioral Twin como un ****dataset curado****:

- Chunks temáticos (ejercicios, progreso, preferencias, estado emocional)
- Metadatos para cada chunk (tipo de ejercicio, fecha, estado emocional)
- Versionado de cada versión del Behavioral Twin
- Feedback del paciente como datos de evaluación

4. TABLA DE TRADE-OFFS APLICADOS A BEHAVIORALOS

| Decisión | Opción A | Opción B | Trade-off | Decisión final |
|---------------------|-----------------------|------------------------------|---|--|
| ----- | ----- | ----- | ----- | ----- |
| Modelo | Gemma on-device | API en la nube (GPT-4) | On-device: privacidad, cero costo, offline. Nube: más potente | On-device (requisito de privacidad) |
| Adaptación | Fine-tuning | Prompt engineering | Fine-tuning: mejor precisión. Prompt: más rápido, más barato | Prompt engineering (MVP) |
| Evaluación | Automática (IA) | Manual (terapeuta) | Automática: escala. Manual: más precisa | Híbrido (IA + revisión humana) |
| RAG | Behavioral Twin local | Base de conocimiento externa | Local: privacidad. Externa: más datos | Local (privacidad del paciente) |
| Agentes | Agente autónomo | Sistema basado en reglas | Agente: flexible. Reglas: predecible | Reglas + IA (agente supervisado) |
| Inferencia | Bajo demanda | Continua | Bajo demanda: ahorra batería. Continua: más rápido | Bajo demanda |
| Cuantización | INT8/INT4 | FP16/FP32 | INT: menor memoria y batería. FP: mayor precisión | INT8 (balance) |

5. ESTADO ACTUAL DE BEHAVIORALOS (SEGÚN PRINCIPIOS DE HUYEN)

| Principio | Estado | Acción |

|-----|-----|-----|

| ****Product First**** |  BehavioralOS es una app, no un proyecto de IA | Mantener el foco en la experiencia del paciente |

| ****Evaluation-Driven**** |  No implementado | Crear sistema de evaluación de ejercicios y respuestas |

| ****MaaS on-device**** |  Gemma + MediaPipe | Implementar inferencia local |

| ****Prompt Engineering**** |  Por diseñar | Crear prompts terapéuticos con psicólogos |

| ****RAG/Agentes**** |  Por implementar | Behavioral Twin como base de conocimiento |

| ****Inference Optimization**** |  Por optimizar | Cuantización, caché, inferencia bajo demanda |

| ****Full-Stack**** |  Diseñado | Implementar todas las capas integradas |

| ****Guardarraíles**** |  Por implementar | Proteger entradas y salidas de IA |

| ****Feedback Loop**** |  Por implementar | Capturar feedback explícito e implícito |

| ****Versionado**** |  Por implementar | Versionar prompts, modelos, configuraciones |

6. PRÓXIMAS ACCIONES CONCRETAS

6.1. Diseñar prompts terapéuticos

****Acción****: Crear un repositorio de prompts para Gemma:

- Prompt para cada tipo de ejercicio (mindfulness, defusión, etc.)
- Prompt para el Companion IA (respuestas empáticas)
- Prompt para evaluación (Behavioral Twin)
- Prompt para el agente (decisión de ejercicio)

- Revisión por psicólogos clínicos
- Versionado y evaluación de cada prompt

6.2. Implementar sistema de evaluación

****Acción****: Crear métricas y dashboards:

- Utilidad del ejercicio (auto-reporte 1-5)
- Seguridad de la respuesta (revisión humana)
- Engagement (completado, tiempo)
- Progreso del paciente (Behavioral Twin)
- Evaluación de prompts (A/B testing)
- IA como juez para evaluación automática

6.3. Implementar RAG básico

****Acción****: Behavioral Twin como base de conocimiento:

- Estado actual del paciente (ánimo, progreso)
- Historial de ejercicios
- Preferencias del paciente
- Recuperación basada en embeddings (semántica) y términos (BM25)
- Chunking estratégico (temático)

6.4. Optimizar inferencia

****Acción****: Configurar Gemma para dispositivos móviles:

- Cuantización INT8 (QLoRA si se hace fine-tuning)

- Inferencia bajo demanda
- Caché de respuestas comunes (exacto y semántico)
- KV cache para conversaciones largas
- Monitoreo de batería y memoria

6.5. Implementar guardarraíles

****Acción****: Proteger entradas y salidas:

- Detección de PII en entradas
- Detección de prompts maliciosos (jailbreaking)
- Detección de contenido peligroso en salidas
- Modo seguro con respuestas predefinidas

6.6. Diseñar arquitectura de agente

****Acción****: Agente supervisado para BehavioralOS:

- Evaluación del Behavioral Twin
- Decisión de ejercicio recomendado
- Generación de respuesta personalizada
- Feedback loop con el paciente
- Supervisión del terapeuta (aprobación humana para comandos impactantes)

6.7. Establecer feedback loops

****Acción****: Capturar feedback explícito e implícito:

- Thumbs up/down en cada ejercicio

- Señales conversacionales (repeticiones, pausas, regeneraciones)
- Feedback del terapeuta (ajuste del plan)
- Evaluación continua de la calidad de las respuestas

7. LO QUE EL LIBRO NO CUBRE (Y BEHAVIORALOS LO NECESITA)

| Aspecto | Cobertura en Huyen | Necesidad en BehavioralOS | Solución |
|----------------------------|---------------------------|---|---------------------------------------|
| **Psicoterapia** | ✗ No cubre | Contenido terapéutico basado en evidencia | Colaborar con psicólogos clínicos |
| **Privacidad médica** | ✗ No cubre (menciona PII) | GDPR, HIPAA, datos sensibles | Cifrado, RLS, anonimización |
| **Offline-first** | ✗ No cubre | IA on-device sin internet | Gemma + MediaPipe local |
| **Gamificación** | ✗ No cubre | Ejercicios como juegos | Diseño UX inspirado en Nintendo |
| **Validación clínica** | ✗ No cubre | Eficacia terapéutica | Ensayos clínicos con pacientes reales |
| **Behavioral Twin** | ✗ No cubre | Modelo dinámico del paciente | Diseño propio del Behavioral Twin |
| **Ejercicios gamificados** | ✗ No cubre | Motor de ejercicios | Diseño inspirado en PPU de NES |

8. TABLA DE DECISIONES CRÍTICAS (CON JUSTIFICACIÓN)

| Decisión | Opción elegida | Justificación |
|----------|----------------|---------------|
|----------|----------------|---------------|

| | | |
|----------------------|---------------------------|--|
| | | |
| Modelo | Gemma 2B on-device | Privacidad, cero latencia, sin dependencia de internet |
| Adaptación | Prompt engineering + RAG | Más rápido, más barato, suficiente para MVP |
| Evaluación | Híbrida (IA + humana) | IA escala, humano da precisión clínica |
| RAG | Behavioral Twin local | Privacidad del paciente, cero costo |
| Agentes | Reglas + IA (supervisado) | Predecible, seguro, con flexibilidad de IA |
| Cuantización | INT8 | Balance entre precisión y memoria/batería |
| Inferencia | Bajo demanda | Ahorro de batería en dispositivos móviles |
| Guardarraíles | Entrada y salida | Protección contra PII y contenido peligroso |

9. ROADMAP DE IMPLEMENTACIÓN (CON PRINCIPIOS DE HUYEN)

| Fase | Principio de Huyen | Módulos | Duración |
|------------------------|------------------------|----------------------------|-----------|
| 1. Fundación | Product First | `000-Core`, `900-Security` | 2 semanas |
| 2. Backend | Evaluación | `200-Backend` | 3 semanas |
| 3. IA simple | MaaS + Prompt | `400-AI` | 4 semanas |
| 4. RAG/Agentes | RAG + Agentes | `500-Experiencies` | 3 semanas |
| 5. Frontend | Full-Stack | `300-Frontend` | 4 semanas |
| 6. Evaluación | Evaluation-Driven | `1100-Testing` | 2 semanas |
| 7. Optimización | Inference Optimization | `1200-Deployment` | 1 semana |
| 8. Feedback | Feedback Loop | Post-lanzamiento | Continuo |

10. CONCLUSIÓN

El libro de Chip Huyen proporciona un marco de **ingeniería de IA** que es directamente aplicable a BehavioralOS:

| Principio de Huyen | Aplicación en BehavioralOS |

|-----|-----|

| **Product First** | BehavioralOS es una app de psicoterapia, no un proyecto de IA |

| **Evaluation-Driven** | Evaluar cada ejercicio y respuesta de IA |

| **MaaS on-device** | Gemma + MediaPipe para inferencia local |

| **Prompt Engineering** | Prompts terapéuticos diseñados con psicólogos |

| **RAG/Agentes** | Behavioral Twin como base de conocimiento |

| **Inference Optimization** | Cuantización, caché, inferencia bajo demanda |

| **Full-Stack** | Todas las capas integradas (frontend, backend, IA, datos) |

| **Guardarraíles** | Proteger entradas (PII) y salidas (seguridad) |

| **Feedback Loop** | Feedback explícito e implícito para mejora continua |

| **Versionado** | Versionar prompts, modelos y configuraciones |

Próximas acciones concretas (con fecha límite):

| Semana | Acción |

|-----|-----|

| **1** | Diseñar prompts terapéuticos con psicólogos |

| **1** | Implementar sistema de evaluación (métricas) |

- | **1** | Configurar guardarraíles de entrada/salida |
- | **2** | Implementar RAG básico con Behavioral Twin |
- | **2** | Optimizar inferencia (cuantización INT8) |
- | **2** | Diseñar arquitectura de agente supervisado |
- | **3+** | Integrar con el resto de BehavioralOS |

MOTOR DE EJERCICIOS INSPIRADO EN EL PPU DE LA NES

ARQUITECTURA GENERAL

...

Ejercicio Engine (PPU)

- ├ Behavioral Twin (Memoria)
 - | ─ Estado del paciente (ánimo, ansiedad, progreso)
 - | ─ Historial de ejercicios
 - | ─ Preferencias y valores
 - | ─ Patrones de comportamiento
- ├
- ├ Tile Generator (CHR ROM)
 - | ─ Banco de módulos (8x8 "tiles" terapéuticos)
 - | ─ Mindfulness (tile 0x00)
 - | ─ Cognitive Defusion (tile 0x01)
 - | ─ Acceptance (tile 0x02)
 - | ─ Valores (tile 0x03)

| └─ Cada tile = 8 líneas de contenido (texto + instrucciones)

|

|─ Session Planner (Nametable)

| └─ Plan de tratamiento: secuencia personalizada de tiles

|

|─ Attribute Table (Personalización)

| └─ Parámetros: duración, dificultad, tono, ritmo

|

|─ Exercise Buffer (Sprite RAM)

| └─ Ejercicios activos (hasta 8 simultáneos)

|

|─ Renderer (PPU Output)

└─ Muestra el ejercicio en la UI del paciente

...

1. BEHAVIORAL TWIN (MEMORIA)

1.1. Estructura de datos

```
```python
```

```
behavioral_twin.py
```

```
from dataclasses import dataclass
```

```
from typing import List, Dict, Optional
```

```
from datetime import datetime
```

@dataclass

class EmotionalState:

    anxiety: int # 0-10

    mood: int # 0-10

    energy: int # 0-10

    timestamp: datetime

@dataclass

class ExerciseHistory:

    exercise\_id: str

    tile\_type: int

    score: float # 0-100

    completion\_time: int # segundos

    user\_rating: int # 1-5

    timestamp: datetime

@dataclass

class PatientProfile:

    user\_id: str

    age: int

    diagnosis: List[str] # ["Ansiedad", "Depresión"]

    values: List[str] # ["Conexión", "Autenticidad"]

    current\_state: EmotionalState

    history: List[ExerciseHistory]

    preferences: Dict[str, any] # {"favorite\_exercise\_type": 0x01, ...}

```
@dataclass
```

```
class BehavioralTwin:
```

```
 profile: PatientProfile
```

```
 state_history: List[EmotionalState]
```

```
 progress_trend: float # -1 a 1 (empeorando/mejorando)
```

```
 engagement_score: float # 0-100
```

```
...
```

```
1.2. API del Behavioral Twin
```

```
```python
```

```
class BehavioralTwinAPI:
```

```
    def __init__(self, patient_id: str):
```

```
        self.patient_id = patient_id
```

```
        self.twin = self.load_twin(patient_id)
```

```
    def get_current_state(self) -> EmotionalState:
```

```
        """Obtiene el estado emocional actual del paciente."""
```

```
        return self.twin.profile.current_state
```

```
    def update_state(self, new_state: EmotionalState) -> None:
```

```
        """Actualiza el estado emocional del paciente."""
```

```
        self.twin.state_history.append(new_state)
```

```
        self.twin.profile.current_state = new_state
```

```
        self._update_progress_trend()
```



```
self._update_engagement_score()
```

```
self.save_twin()
```

```
def get_exercise_history(self, limit: int = 10) -> List[ExerciseHistory]:
```

```
    """Obtiene el historial de ejercicios del paciente."""
```

```
    return sorted(self.twin.profile.history, key=lambda x: x.timestamp, reverse=True)[:limit]
```

```
def get_recommendations(self) -> Dict[str, any]:
```

```
    """Genera recomendaciones personalizadas basadas en el Behavioral Twin."""
```

```
    return {
```

```
        "suggested_tiles": self._suggest_tiles(),
```

```
        "difficulty": self._calculate_difficulty(),
```

```
        "duration": self._calculate_duration(),
```

```
        "tone": self._calculate_tone()
```

```
    }
```

```
def record_exercise(self, exercise: ExerciseHistory) -> None:
```

```
    """Registra un ejercicio completado."""
```

```
    self.twin.profile.history.append(exercise)
```

```
    self._update_progress_trend()
```

```
    self._update_engagement_score()
```

```
    self.save_twin()
```

```
def _suggest_tiles(self) -> List[int]:
```

```
    """Sugiere tiles basados en el estado del paciente."""
```

```
    state = self.twin.profile.current_state
```

```
history = self.get_exercise_history(10)
```

```
# Lógica de recomendación (AAO)
```

```
suggested = []
```

```
# Si ansiedad > 7, recomendar mindfulness
```

```
if state.anxiety > 7:
```

```
    suggested.append(0x00) # Mindfulness
```

```
# Si hay patrones de evitación, recomendar defusión
```

```
if self._detect_avoidance_pattern(history):
```

```
    suggested.append(0x01) # Cognitive Defusion
```

```
# Si hay baja energía, recomendar activación conductual
```

```
if state.energy < 4:
```

```
    suggested.append(0x02) # Acceptance (con enfoque en activación)
```

```
# Si no hay recomendaciones específicas, usar valores del paciente
```

```
if not suggested:
```

```
    for value in self.twin.profile.values:
```

```
        if value == "Conexión":
```

```
            suggested.append(0x03) # Valores
```

```
        elif value == "Autenticidad":
```

```
            suggested.append(0x01) # Cognitive Defusion
```

```
# Limitar a 8 tiles
```

```
return suggested[:8]
```

```
def _calculate_difficulty(self) -> int:
```

```
    """Calcula la dificultad recomendada (0-2)."""
```

```
    state = self.twin.profile.current_state
```

```
    history = self.get_exercise_history(5)
```

```
    # Si ansiedad alta, dificultad baja
```

```
    if state.anxiety > 7:
```

```
        return 0 # Baja
```

```
    # Si el paciente ha completado ejercicios recientes con éxito, subir dificultad
```

```
    recent_scores = [h.score for h in history if h.score >= 70]
```

```
    if len(recent_scores) >= 3:
```

```
        return 2 # Alta
```

```
    return 1 # Media
```

```
def _calculate_duration(self) -> int:
```

```
    """Calcula la duración recomendada (minutos)."""
```

```
    state = self.twin.profile.current_state
```

```
    history = self.get_exercise_history(3)
```

```
    # Si ansiedad alta, ejercicios cortos
```

```
    if state.anxiety > 7:
```

```
        return 3
```

```

# Si el paciente tiene historia de completar ejercicios largos, aumentar
avg_time = sum(h.completion_time for h in history) / len(history) if history else 0
if avg_time > 600: # >10 minutos
    return 15

return 10

```

```

def _calculate_tone(self) -> int:

```

```

    """Calcula el tono recomendado (0=suave, 1=neutro, 2=firme)."""

```

```

    state = self.twin.profile.current_state

```

```

    if state.anxiety > 7:

```

```

        return 0 # Suave

```

```

    elif state.anxiety > 4:

```

```

        return 1 # Neutro

```

```

    else:

```

```

        return 2 # Firme

```

```

    ...

```

```

    ---

```

2. TILE GENERATOR (BANCO DE MÓDULOS)

2.1. Estructura de un Tile

```

python

# tile_generator.py

@dataclass

class Tile:

    id: int # 0x00-0x0F

    type: str # "Mindfulness", "Cognitive Defusion", "Acceptance", "Valores"

    lines: List[str] # 8 líneas de contenido

    difficulty_base: int # 0-2

    duration_base: int # minutos


class TileGenerator:

    def __init__(self):

        self.tile_bank = {

            0x00: Tile(

                id=0x00,

                type="Mindfulness",

                lines=[

                    "Siente tu respiración",

                    "Inhala profundamente",

                    "Exhala lentamente",

                    "Nota las sensaciones",

                    "Sin juzgar",

                    "Vuelve a la respiración",

                    "Cada respiración es única",

                    "Regresa al momento presente"

                ],

```

```
    difficulty_base=0,

    duration_base=5

),

0x01: Tile(

    id=0x01,

    type="Cognitive Defusion",

    lines=[

        "Observa tu pensamiento",

        "Dale un nombre",

        "No es un hecho",

        "Es solo una nube",

        "Pasa en el cielo",

        "No eres la nube",

        "Eres el cielo",

        "Regresa a ti"

    ],

    difficulty_base=1,

    duration_base=10

),

0x02: Tile(

    id=0x02,

    type="Acceptance",

    lines=[

        "Nota tu emoción",

        "Da espacio a la emoción",

        "La emoción es pasajera",
```

```

        "No necesitas huir",
        "Acéptala como es",
        "Respira con ella",
        "Se desvanece sola",
        "Vuelve al presente"
    ],
    difficulty_base=1,
    duration_base=7
),
0x03: Tile(
    id=0x03,
    type="Valores",
    lines=[
        "¿Qué es importante para ti?",
        "Escribe tu valor principal",
        "¿Cómo vives ese valor?",
        "¿Qué acción puedes tomar?",
        "Un paso pequeño es suficiente",
        "El valor te guía",
        "La acción te conecta",
        "Elige una acción valiosa"
    ],
    difficulty_base=2,
    duration_base=8
)
}

```

```

def get_tile(self, tile_id: int) -> Tile:
    """Obtiene un tile por su ID."""
    return self.tile_bank.get(tile_id, self.tile_bank[0x00]) # Default a Mindfulness
...

---

```

3. SESSION PLANNER (NAMETABLE)

3.1. Planificador de Sesión

```

```python
session_planner.py

class SessionPlanner:

 def __init__(self, behavioral_twin: BehavioralTwinAPI):
 self.twin = behavioral_twin
 self.tile_gen = TileGenerator()
 self.plan = []

 def generate_plan(self) -> List[Dict]:
 """Genera un plan de tratamiento personalizado."""
 recommendations = self.twin.get_recommendations()
 suggested_tiles = recommendations["suggested_tiles"]

 # Construir el plan (nametable)

```



```

plan = []

for i, tile_id in enumerate(suggested_tiles):

 tile = self.tile_gen.get_tile(tile_id)

 # Aplicar atributos (personalización)

 exercise = {

 "tile_id": tile_id,

 "tile": tile,

 "difficulty": recommendations["difficulty"],

 "duration": recommendations["duration"],

 "tone": recommendations["tone"],

 "order": i,

 "context": self._generate_context(tile)

 }

 plan.append(exercise)

Ordenar por prioridad (primero los ejercicios más urgentes)

plan.sort(key=lambda x: self._calculate_priority(x))

return plan

def _generate_context(self, tile: Tile) -> str:

 """Genera contexto personalizado para el ejercicio."""

 state = self.twin.get_current_state()

 context_map = {

```

"Mindfulness": f"Tu ansiedad está en {state.anxiety}/10. Este ejercicio te ayudará a anclarte.",

"Cognitive Defusion": f"Has tenido patrones de evitación recientemente. Este ejercicio te ayuda a separarte de los pensamientos.",

"Acceptance": f"Tu energía está en {state.energy}/10. Este ejercicio te ayuda a aceptar lo que sientes.",

"Valores": f"Tus valores principales son {', '.join(self.twin.profile.values)}. Este ejercicio te conecta con ellos."

}

return context\_map.get(tile.type, "Este ejercicio está diseñado para ti.")

def \_calculate\_priority(self, exercise: Dict) -> int:

"""Calcula la prioridad de un ejercicio."""

tile = exercise["tile"]

# Prioridad basada en el estado del paciente

state = self.twin.get\_current\_state()

if tile.type == "Mindfulness" and state.anxiety > 7:

return 0

elif tile.type == "Cognitive Defusion" and  
self.twin.\_detect\_avoidance\_pattern(self.twin.get\_exercise\_history(10)):

return 1

elif tile.type == "Acceptance" and state.energy < 4:

return 2

else:

return 3

...

---

## ## 4. ATTRIBUTE TABLE (PERSONALIZACIÓN)

### ### 4.1. Atributos del Ejercicio

```
```python
# attribute_table.py

@dataclass
class ExerciseAttributes:
    difficulty: int # 0=Baja, 1=Media, 2=Alta
    duration: int # minutos
    tone: int # 0=Suave, 1=Neutro, 2=Firme
    rhythm: int # 0=Lento, 1=Normal, 2=Rápido
    context: str

class AttributeTable:
    @staticmethod
    def apply_attributes(tile: Tile, attributes: ExerciseAttributes) -> Dict:
        """Aplica los atributos a un tile."""
        # Modificar duración
        duration = attributes.duration

        # Modificar tono del contenido
        lines = tile.lines.copy()
        if attributes.tone == 0: # Suave
            lines = [f"🌿 {line}" for line in lines]
```

```

elif attributes.tone == 1: # Neutro

    lines = [f"🧘 {line}" for line in lines]

elif attributes.tone == 2: # Firme

    lines = [f"💪 {line}" for line in lines]


# Modificar dificultad (añadir pasos adicionales)

if attributes.difficulty == 2: # Alta

    lines.append("📝 Escribe cómo te sientes después del ejercicio")


return {

    "title": tile.type,

    "lines": lines,

    "duration": duration,

    "tone": attributes.tone,

    "difficulty": attributes.difficulty,

    "context": attributes.context

}
'''

'''

```

5. EXERCISE BUFFER (SPRITE RAM)

5.1. Buffer de Ejercicios Activos

```
```python
```

```
exercise_buffer.py
```

```
class ExerciseBuffer:
```

```
 def __init__(self):
```

```
 self.buffer = [] # Hasta 8 ejercicios activos
```

```
 self.current_exercise_index = 0
```

```
 self.max_buffer_size = 8
```

```
 def load_plan(self, plan: List[Dict]) -> None:
```

```
 """Carga un plan de ejercicios en el buffer."""
```

```
 self.buffer = plan[:self.max_buffer_size]
```

```
 self.current_exercise_index = 0
```

```
 def get_current_exercise(self) -> Dict:
```

```
 """Obtiene el ejercicio actual."""
```

```
 if not self.buffer:
```

```
 return None
```

```
 return self.buffer[self.current_exercise_index]
```

```
 def next_exercise(self) -> Dict:
```

```
 """Avanza al siguiente ejercicio."""
```

```
 if self.current_exercise_index < len(self.buffer) - 1:
```

```
 self.current_exercise_index += 1
```

```
 return self.get_current_exercise()
```

```
 return None
```

```
 def previous_exercise(self) -> Dict:
```

```

 """Retrocede al ejercicio anterior."""
 if self.current_exercise_index > 0:
 self.current_exercise_index -= 1
 return self.get_current_exercise()
 return None

 def get_progress(self) -> float:
 """Calcula el progreso (0-100%)."""
 if not self.buffer:
 return 0
 return (self.current_exercise_index / len(self.buffer)) * 100
...

```

## ## 6. RENDERER (PPU OUTPUT)

### ### 6.1. Renderizado del Ejercicio

```

```python
# renderer.py

class ExerciseRenderer:
    def __init__(self, buffer: ExerciseBuffer):
        self.buffer = buffer
        self.current_screen = None

```

```

def render_current_exercise(self) -> Dict:

    """Renderiza el ejercicio actual para la UI."""

    exercise = self.buffer.get_current_exercise()

    if not exercise:

        return self._render_empty_state()

    tile = exercise["tile"]

    attributes = exercise["attributes"]

    # Aplicar atributos al tile

    rendered_exercise = AttributeTable.apply_attributes(tile, attributes)

    # Construir la pantalla

    screen = {

        "screen_type": "exercise",

        "title": rendered_exercise["title"],

        "content": rendered_exercise["lines"],

        "progress": self.buffer.get_progress(),

        "current_step": self.buffer.current_exercise_index + 1,

        "total_steps": len(self.buffer.buffer),

        "duration": rendered_exercise["duration"],

        "tone": rendered_exercise["tone"],

        "difficulty": rendered_exercise["difficulty"],

        "context": rendered_exercise["context"],

        "next_exercise_available": self.buffer.next_exercise() is not None,

        "previous_exercise_available": self.buffer.previous_exercise() is not None

```

```
}
```

```
self.current_screen = screen
```

```
return screen
```

```
def _render_empty_state(self) -> Dict:
```

```
    """Renderiza una pantalla vacía cuando no hay ejercicios."""
```

```
    return {
```

```
        "screen_type": "empty",
```

```
        "title": "No hay ejercicios disponibles",
```

```
        "content": ["Vuelve más tarde para nuevas recomendaciones."],
```

```
        "progress": 0,
```

```
        "current_step": 0,
```

```
        "total_steps": 0,
```

```
        "duration": 0,
```

```
        "tone": 0,
```

```
        "difficulty": 0,
```

```
        "context": "",
```

```
        "next_exercise_available": False,
```

```
        "previous_exercise_available": False
```

```
    }
```

```
...
```

```
---
```

7. EVALUACIÓN ADAPTATIVA CONTINUA (AAO)

7.1. Módulo AAO

```
```python
```

```
aao.py
```

```
class AdaptiveAssessmentOrchestrator:
```

```
 def __init__(self, twin: BehavioralTwinAPI):
```

```
 self.twin = twin
```

```
 self.current_assessment = None
```

```
 self.assessment_history = []
```

```
 def assess_exercise(self, exercise_result: Dict) -> Dict:
```

```
 """Evalúa un ejercicio completado y actualiza el Behavioral Twin."""
```

```
 # Extraer datos del ejercicio
```

```
 exercise_id = exercise_result["exercise_id"]
```

```
 score = exercise_result["score"]
```

```
 user_rating = exercise_result.get("user_rating", 3)
```

```
 completion_time = exercise_result["completion_time"]
```

```
 pre_state = exercise_result["pre_state"]
```

```
 post_state = exercise_result.get("post_state")
```

```
 # 1. Evaluar el ejercicio
```

```
 assessment = {
```

```
 "exercise_id": exercise_id,
```

```
 "score": score,
```

```
 "user_rating": user_rating,
```

```

 "completion_time": completion_time,
 "improvement": self._calculate_improvement(pre_state, post_state),
 "engagement": self._calculate_engagement(exercise_result),
 "recommendation": self._generate_recommendation(exercise_result)
 }

```

# 2. Actualizar el Behavioral Twin

```

if post_state:
 self.twin.update_state(post_state)

```

# 3. Registrar en el historial

```

self.assessment_history.append(assessment)

self.current_assessment = assessment

```

# 4. Evaluar la efectividad general del plan

```

plan_effectiveness = self._evaluate_plan_effectiveness()

```

```

return {
 "assessment": assessment,
 "plan_effectiveness": plan_effectiveness
}

```

def \_calculate\_improvement(self, pre\_state: EmotionalState, post\_state: EmotionalState)  
-> float:

```

 """Calcula la mejora del estado emocional."""

```

```

 if not post_state:

```

```

 return 0.0

```

```
pre_score = 10 - pre_state.anxiety
```

```
post_score = 10 - post_state.anxiety
```

```
improvement = (post_score - pre_score) / 10 # -1 a 1
```

```
return max(-1, min(1, improvement))
```

```
def _calculate_engagement(self, exercise_result: Dict) -> float:
```

```
 """Calcula el engagement del paciente."""
```

```
 score = exercise_result["score"]
```

```
 user_rating = exercise_result.get("user_rating", 3)
```

```
 completion_time = exercise_result["completion_time"]
```

```
 # Peso de cada factor
```

```
 w_score = 0.4
```

```
 w_rating = 0.4
```

```
 w_time = 0.2
```

```
 # Normalizar tiempo (si completó en menos del 80% del tiempo esperado)
```

```
 expected_time = exercise_result.get("expected_time", 600) # 10 min default
```

```
 time_factor = min(completion_time / expected_time, 1.0)
```

```
 engagement = (score / 100) * w_score + (user_rating / 5) * w_rating + (1 - time_factor) *
w_time
```

```
 return max(0, min(1, engagement))
```

```
def _generate_recommendation(self, exercise_result: Dict) -> Dict:
```

```
 """Genera recomendaciones para el siguiente ejercicio."""
```

```
 state = self.twin.get_current_state()
```

```
 history = self.twin.get_exercise_history(5)
```

```
 recommendations = {
```

```
 "next_tile_type": None,
```

```
 "difficulty_adjustment": 0,
```

```
 "duration_adjustment": 0,
```

```
 "tone_adjustment": 0
```

```
 }
```

```
 # Si el ejercicio tuvo puntuación baja, ajustar dificultad
```

```
 if exercise_result["score"] < 50:
```

```
 recommendations["difficulty_adjustment"] = -1
```

```
 recommendations["duration_adjustment"] = -2
```

```
 # Si el paciente mejoró, subir dificultad
```

```
 elif exercise_result.get("improvement", 0) > 0.2:
```

```
 recommendations["difficulty_adjustment"] = 1
```

```
 recommendations["duration_adjustment"] = 3
```

```
 # Si ansiedad alta, mantener ejercicios suaves
```

```
 if state.anxiety > 7:
```

```
 recommendations["tone_adjustment"] = -1
```

```

Determinar siguiente tipo de tile

tile_ids = [h.tile_type for h in history[:3]]

if len(tile_ids) >= 3 and len(set(tile_ids)) == 1:

 # Si ha hecho el mismo tipo 3 veces, cambiar

 recommendations["next_tile_type"] = (tile_ids[0] + 1) % 4

return recommendations

def _evaluate_plan_effectiveness(self) -> float:

 """Evalúa la efectividad general del plan de tratamiento."""

 recent_assessments = self.assessment_history[-5:] if len(self.assessment_history) >= 5
 else self.assessment_history

 if not recent_assessments:

 return 0.0

 # Puntaje promedio de las evaluaciones recientes

 avg_score = sum(a["score"] for a in recent_assessments) / len(recent_assessments)

 # Mejora promedio del estado

 avg_improvement = sum(a.get("improvement", 0) for a in recent_assessments) /
 len(recent_assessments)

 # Engagement promedio

 avg_engagement = sum(a["engagement"] for a in recent_assessments) /
 len(recent_assessments)

 # Fórmula combinada

```

```
 effectiveness = (avg_score / 100) * 0.5 + (avg_improvement + 1) / 2 * 0.3 +
 avg_engagement * 0.2
```

```
 return max(0, min(1, effectiveness))
```

```
...
```

```

```

## ## 8. INTEGRACIÓN COMPLETA

### ### 8.1. Motor de Ejercicios Principal

```
```python
```

```
# exercise_engine.py
```

```
class ExerciseEngine:
```

```
    def __init__(self, patient_id: str):
```

```
        self.twin = BehavioralTwinAPI(patient_id)
```

```
        self.planner = SessionPlanner(self.twin)
```

```
        self.buffer = ExerciseBuffer()
```

```
        self.renderer = ExerciseRenderer(self.buffer)
```

```
        self.aao = AdaptiveAssessmentOrchestrator(self.twin)
```

```
    def start_session(self) -> Dict:
```

```
        """Inicia una nueva sesión de ejercicios."""
```

```
        plan = self.planner.generate_plan()
```

```
        self.buffer.load_plan(plan)
```

```
        return self.renderer.render_current_exercise()
```

```
def complete_exercise(self, exercise_result: Dict) -> Dict:
```

```
    """Completa un ejercicio y evalúa el progreso."""
```

```
    # Registrar ejercicio en el Behavioral Twin
```

```
    history = ExerciseHistory(
```

```
        exercise_id=exercise_result["exercise_id"],
```

```
        tile_type=exercise_result["tile_type"],
```

```
        score=exercise_result["score"],
```

```
        completion_time=exercise_result["completion_time"],
```

```
        user_rating=exercise_result.get("user_rating", 3),
```

```
        timestamp=datetime.now()
```

```
    )
```

```
    self.twin.record_exercise(history)
```

```
    # Evaluar ejercicio (AAO)
```

```
    assessment = self.aao.assess_exercise(exercise_result)
```

```
    # Renderizar siguiente ejercicio si existe
```

```
    next_exercise = self.buffer.next_exercise()
```

```
    if next_exercise:
```

```
        rendered = self.renderer.render_current_exercise()
```

```
        return {
```

```
            "status": "exercise_completed",
```

```
            "next_exercise": rendered,
```

```
            "assessment": assessment
```

```
        }
```

```

else:

    # Fin de la sesión

    plan_effectiveness = self.aao._evaluate_plan_effectiveness()

    return {

        "status": "session_complete",

        "assessment": assessment,

        "plan_effectiveness": plan_effectiveness,

        "summary": self._generate_session_summary()

    }

```

def _generate_session_summary(self) -> Dict:

```

"""Genera un resumen de la sesión."""

```

```

state = self.twin.get_current_state()

```

```

history = self.twin.get_exercise_history(5)

```

```

return {

```

```

    "current_state": state,

```

```

    "exercises_completed": len(history),

```

```

    "average_score": sum(h.score for h in history) / len(history) if history else 0,

```

```

    "improvement_trend": self.twin.progress_trend,

```

```

    "engagement_score": self.twin.engagement_score,

```

```

    "recommendations": self.twin.get_recommendations()

```

```

}

```

```

...

```

```

---

```


9. EJEMPLO DE FLUJO DE TRABAJO

9.1. Inicio de Sesión

```
```python
Ejemplo de uso

engine = ExerciseEngine(patient_id="P042")

Iniciar sesión

session = engine.start_session()

El paciente ve el primer ejercicio

print(session["title"])

print(session["content"])

print(f"Progreso: {session['progress']}%")

Completar ejercicio

result = engine.complete_exercise({
 "exercise_id": "EX_001",
 "tile_type": 0x00,
 "score": 85,
 "completion_time": 420,
 "user_rating": 4,
 "pre_state": EmotionalState(anxiety=8, mood=5, energy=6, timestamp=datetime.now()),
 "post_state": EmotionalState(anxiety=5, mood=7, energy=7, timestamp=datetime.now())
})
```

```
})
```

```
Verificar siguiente ejercicio
```

```
if result["status"] == "exercise_completed":
```

```
 print(result["next_exercise"])
```

```
else:
```

```
 # Fin de sesión
```

```
 print(result["summary"])
```

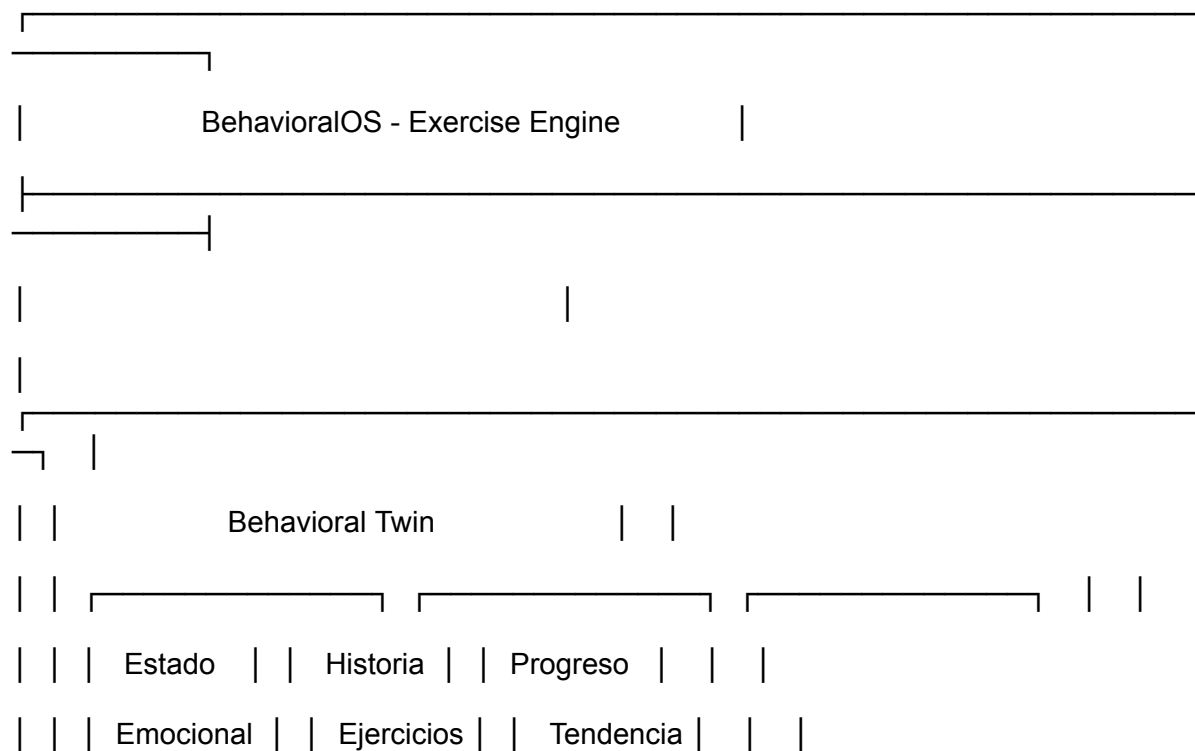
```
...
```

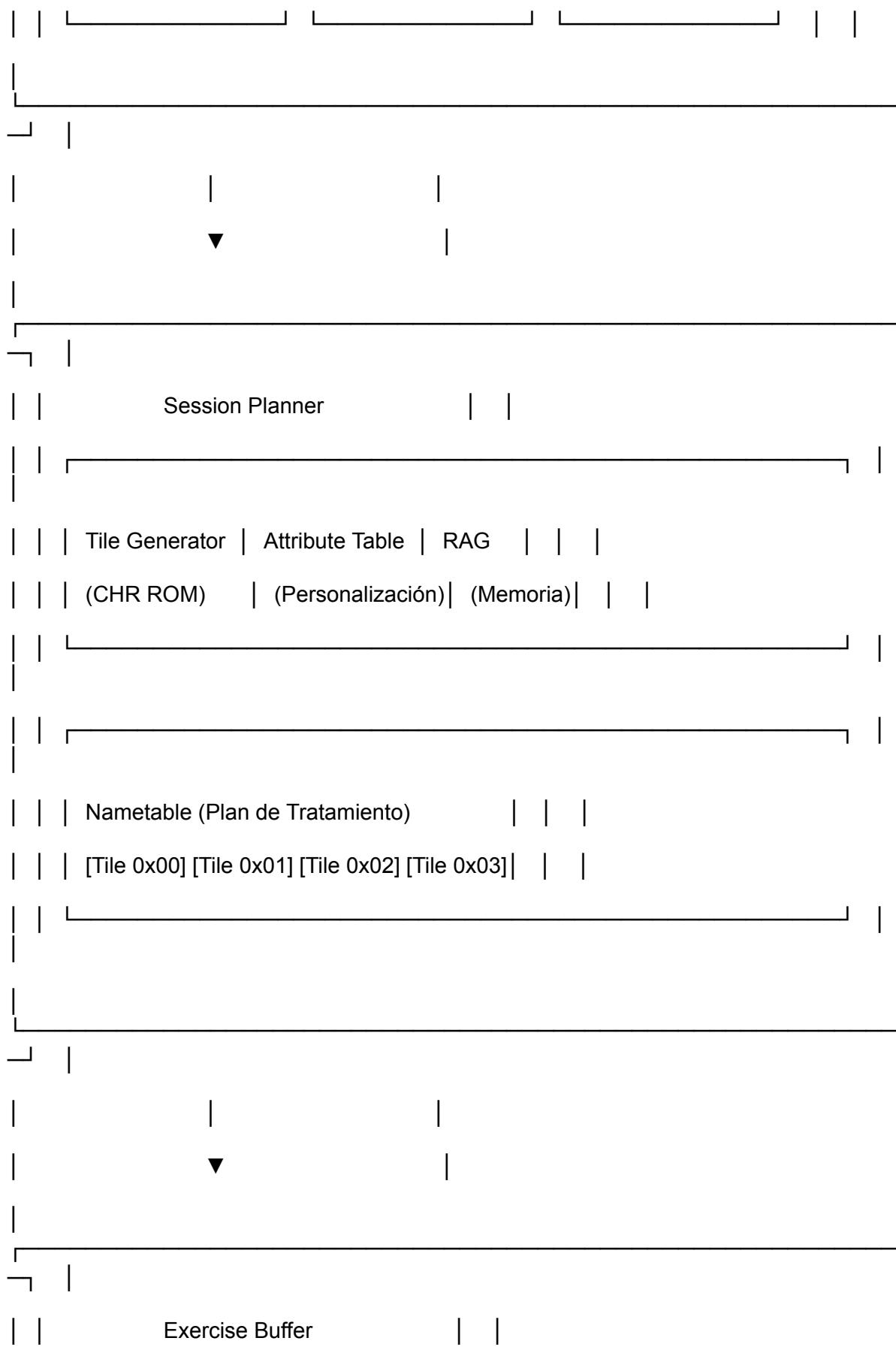
```

```

## ## 10. DIAGRAMA DE ARQUITECTURA

```
...
```





```
graph TD
 subgraph UI [UI del Paciente (App React Native)]
 direction TB
 M[Mindfulness]
 S[Siente tu respiración]
 I[Inhala profundamente]
 E[Exhala lentamente]
 P[Progreso: 45%]
 end

 subgraph Logic [Renderer (PPU)]
 direction TB
 R[Renderer]
 end

 subgraph Data [Data]
 direction TB
 D[Data]
 end

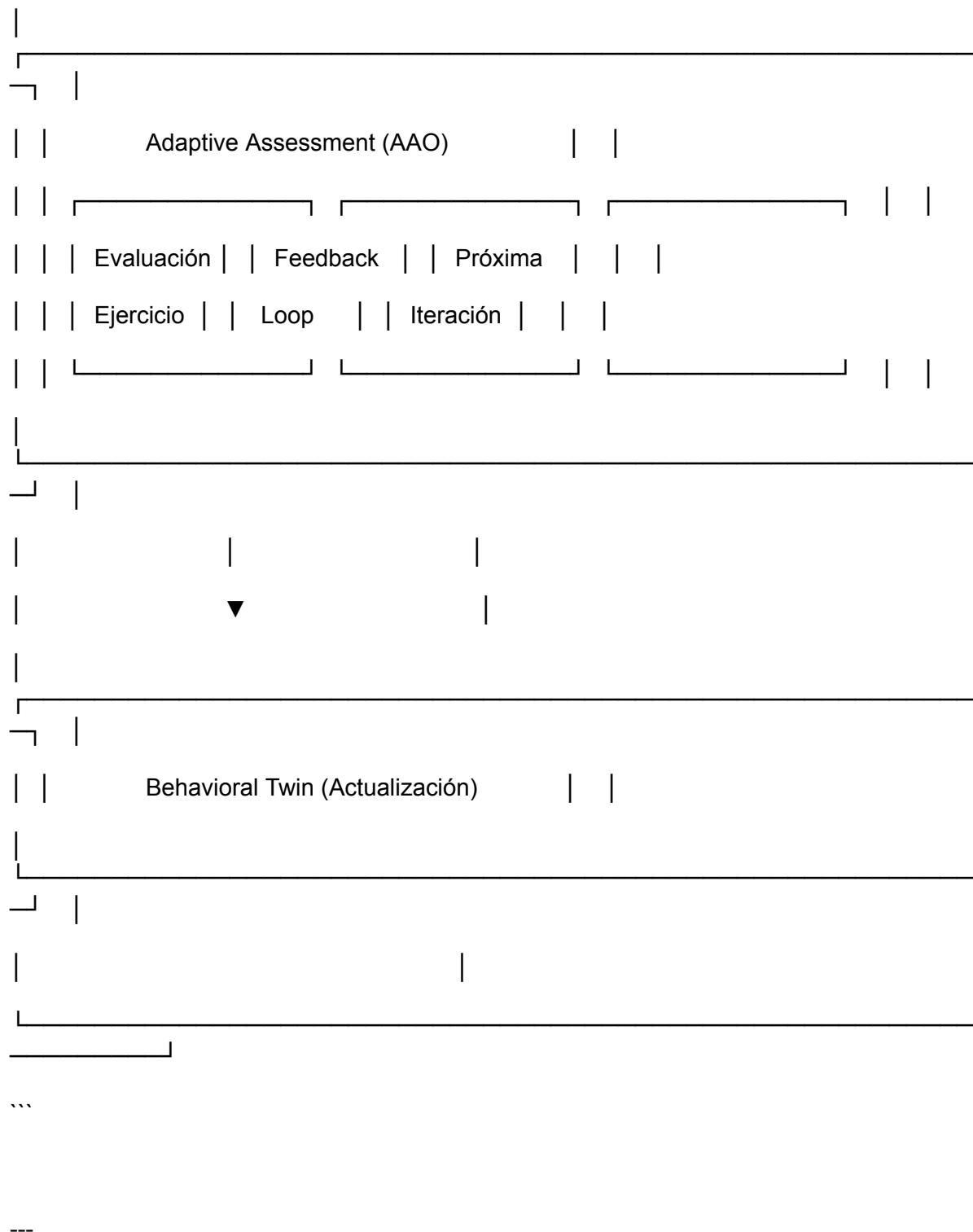
 UI --> Logic
 Logic --> Data
 Data --> Logic
 Logic --> UI
```

The diagram illustrates the architecture of a mobile application for a mindfulness app. It is organized into three main layers: UI, Logic, and Data.

- UI Layer (UI del Paciente):** This layer is built using React Native and contains the user interface elements. It includes a "Mindfulness" screen with instructions: "Siente tu respiración", "Inhala profundamente", and "Exhala lentamente". It also features a progress bar showing "Progreso: 45%".
- Logic Layer (Renderer):** This layer is built using PPU (Presentation-Processing-UI) and handles the rendering of the UI components.
- Data Layer (Data):** This layer is built using a database and manages the application's data.

The flow of data and control is as follows:

- The UI layer sends data to the Logic layer.
- The Logic layer sends data to the Data layer.
- The Data layer sends data back to the Logic layer.
- The Logic layer sends data back to the UI layer.



## ## 11. INTEGRACIÓN CON IA (GEMMA)

### ### 11.1. Generación de Contenido Personalizado

```

```python
# gemma_integration.py

class GemmaIntegration:

    def __init__(self):

        self.model = None # MediaPipe Gemma model


    def load_model(self):

        """Carga el modelo Gemma (on-device)."""

        # Implementación con MediaPipe

        pass


    def generate_exercise_content(self, tile: Tile, patient_context: Dict) -> str:

        """Genera contenido personalizado para el ejercicio."""

        prompt = self._build_prompt(tile, patient_context)

        response = self.model.generate(prompt)

        return response


    def _build_prompt(self, tile: Tile, context: Dict) -> str:

        return f"""

        Eres un asistente de psicoterapia basado en evidencia (ACT/DBT).

        Tu rol es guiar al paciente en el ejercicio de {tile.type}.

        Contexto del paciente:

        - Ansiedad: {context.get('anxiety', 5)}/10

        - Estado de ánimo: {context.get('mood', 5)}/10

```

- Energía: {context.get('energy', 5)}/10
- Progreso general: {context.get('progress', 50)}%
- Ejercicios anteriores: {context.get('history', [])}

Instrucciones del ejercicio:

```
{chr(10).join(tile.lines)}
```

Guía al paciente paso a paso, con empatía y claridad.

Adapta el lenguaje al nivel de ansiedad del paciente.

```
"""
```

```
...
```

```
---
```

12. ARCHIVOS DEL MÓDULO

```
...
```

```
/behavioralos/
```

```
| — src/
```

```
| | — exercise_engine/
```

```
| | | — __init__.py
```

```
| | | — behavioral_twin.py
```

```
| | | — tile_generator.py
```

```
| | | — session_planner.py
```

```
| | | — attribute_table.py
```

```
| | | — exercise_buffer.py
```

```
| | └─ renderer.py
| | └─ aao.py
| | └─ exercise_engine.py
| | └─ gemma_integration.py
| └─ models/
| | └─ emotional_state.py
| | └─ exercise_history.py
| | └─ patient_profile.py
| └─ tests/
|   └─ test_behavioral_twin.py
|   └─ test_session_planner.py
|   └─ test_aao.py
```

...

MOTOR DE EJERCICIOS ZELDA-STYLE: MUNDO EXPLORABLE

ARQUITECTURA GENERAL

...

Zelda Exercise World

```
└─ World Map (Hyrule)
| └─ Regions (Ejercicios)
| | └─ Forest of Mindfulness (Región 0x00)
| | └─ Mountain of Defusion (Región 0x01)
```


- | | | — Lake of Acceptance (Región 0x02)
- | | | — Castle of Values (Región 0x03)
- | | — Secrets (Insights)
- | | | — Hidden Gems (Técnicas adicionales)
- | | | — Power-ups (Habilidades desbloqueadas)
- | | | — Warp Zones (Atajos)
- | | — Progress Tracker (Behavioral Twin como mapa)
- | | — Player (Paciente como Link)
- |
- | — Region Engine (Ejercicio)
- | | — Tile-based (PPU)
- | | — Puzzle (Ejercicio terapéutico)
- | | — NPCs (Companion IA)
- | | — Rewards (Progreso y secretos)
- |
- | — UI (Mapa interactivo)
 - | — Minimapa (Progreso general)
 - | — Region Preview (Vista previa del ejercicio)
 - | — Exploration (Navegación libre)

...

1. WORLD MAP (EL MAPA DE HYRULE)

1.1. Estructura del Mapa

```
```python

world_map.py

from typing import List, Dict, Optional
from enum import Enum

class RegionStatus(Enum):

 LOCKED = 0

 UNLOCKED = 1

 EXPLORING = 2

 COMPLETED = 3

 SECRET_FOUND = 4

class RegionType(Enum):

 MINDFULNESS = 0x00

 DEFUSION = 0x01

 ACCEPTANCE = 0x02

 VALUES = 0x03

 EXPOSURE = 0x04

 COMPASSION = 0x05

 COMMITMENT = 0x06

 GRATITUDE = 0x07

class WorldMap:

 def __init__(self, patient_id: str):

 self.patient_id = patient_id
```

```

self.regions = self._initialize_regions()

self.player_position = (0, 0) # Coordenadas en el mapa

self.discovered_secrets = []

self.completion_percentage = 0.0

def _initialize_regions(self) -> Dict[str, 'Region']:
 """Inicializa todas las regiones del mundo."""
 return {
 "forest_mindfulness": Region(
 id="forest_mindfulness",
 type=RegionType.MINDFULNESS,
 name="Bosque de la Calma",
 description="Un bosque sereno donde aprendes a respirar y anclarte.",
 position=(0, 0),
 status=RegionStatus.UNLOCKED, # Comienza desbloqueada
 icon="🌳",
 color="#4CAF50",
 secret_available=True,
 required_exercises=[],
 unlock_condition=None
),
 "mountain_defusion": Region(
 id="mountain_defusion",
 type=RegionType.DEFUSION,
 name="Montaña de los Pensamientos",
 description="Escala esta montaña para aprender a separarte de tus pensamientos.",

```

```

position=(2, 1),

status=RegionStatus.LOCKED,

icon="🏔️",

color="#FF9800",

secret_available=True,

required_exercises=["forest_mindfulness"],

unlock_condition=lambda p: p.has_completed("forest_mindfulness", 3)

),

"lake_acceptance": Region(

 id="lake_acceptance",

 type=RegionType.ACCEPTANCE,

 name="Lago de la Aceptación",

 description="Sumérgete en este lago para aprender a aceptar tus emociones.",

 position=(1, 3),

 status=RegionStatus.LOCKED,

 icon="🌊",

 color="#2196F3",

 secret_available=True,

 required_exercises=["forest_mindfulness", "mountain_defusion"],

 unlock_condition=lambda p: p.has_completed("mountain_defusion", 2)

),

"castle_values": Region(

 id="castle_values",

 type=RegionType.VALUES,

 name="Castillo de los Valores",

 description="Un castillo donde descubres lo que realmente importa para ti.",

```

```

 position=(3, 2),

 status=RegionStatus.LOCKED,

 icon="🏠",

 color="#9C27B0",

 secret_available=True,

 required_exercises=["forest_mindfulness", "mountain_defusion",
"lake_acceptance"],

 unlock_condition=lambda p: p.has_completed("lake_acceptance", 3)

),

 "cave_exposure": Region(

 id="cave_exposure",

 type=RegionType.EXPOSURE,

 name="Cueva de la Exposición",

 description="Una cueva oscura donde enfrentas tus miedos gradualmente.",

 position=(4, 0),

 status=RegionStatus.LOCKED,

 icon="💡",

 color="#795548",

 secret_available=False,

 required_exercises=["castle_values"],

 unlock_condition=lambda p: p.has_completed("castle_values", 2)

),

 "garden_compassion": Region(

 id="garden_compassion",

 type=RegionType.COMPASSION,

 name="Jardín de la Compasión",

 description="Un jardín donde cultivas la compasión hacia ti mismo.",

```

```

position=(0, 4),

status=RegionStatus.LOCKED,

icon="🌸",

color="#E91E63",

secret_available=True,

required_exercises=["lake_acceptance"],

unlock_condition=lambda p: p.has_secret("lake_acceptance", "compassion_key")

),

"temple_commitment": Region(

 id="temple_commitment",

 type=RegionType.COMMITMENT,

 name="Templo del Compromiso",

 description="Un templo donde te comprometes con acciones valiosas.",

 position=(5, 3),

 status=RegionStatus.LOCKED,

 icon="🏯",

 color="#FF5722",

 secret_available=False,

 required_exercises=["castle_values", "cave_exposure"],

 unlock_condition=lambda p: p.has_completed("cave_exposure", 3)

),

"meadow_gratitude": Region(

 id="meadow_gratitude",

 type=RegionType.GRATITUDE,

 name="Pradera de la Gracitud",

 description="Una pradera donde practicas la gratitud y el aprecio.",

```

```

 position=(2, 5),

 status=RegionStatus.LOCKED,

 icon="🌿",

 color="#FFC107",

 secret_available=True,

 required_exercises=["garden_compassion", "temple_commitment"],

 unlock_condition=lambda p: p.has_completed("temple_commitment", 2)

)
}

```

```
def get_region(self, region_id: str) -> Optional['Region']:
```

```
 """Obtiene una región por su ID."""
```

```
 return self.regions.get(region_id)
```

```
def get_adjacent_regions(self, position: tuple) -> List['Region']:
```

```
 """Obtiene las regiones adyacentes a una posición."""
```

```
 x, y = position
```

```
 adjacent = []
```

```
 for region in self.regions.values():
```

```
 rx, ry = region.position
```

```
 if abs(rx - x) + abs(ry - y) == 1: # Manhattan distance = 1
```

```
 adjacent.append(region)
```

```
 return adjacent
```

```
def unlock_region(self, region_id: str) -> bool:
```

```
 """Desbloquea una región si se cumplen las condiciones."""
```

```

region = self.regions.get(region_id)

if not region:

 return False

if region.status != RegionStatus.LOCKED:

 return True

Verificar condiciones de desbloqueo

if region.unlock_condition:

 if region.unlock_condition(self.patient_progress):

 region.status = RegionStatus.UNLOCKED

 self._check_secret_unlock(region)

 return True

return False

def _check_secret_unlock(self, region: 'Region'):

 """Verifica si al desbloquear una región se descubre un secreto."""

 if region.secret_available:

 # 30% de probabilidad de encontrar un secreto al desbloquear

 import random

 if random.random() < 0.3:

 secret = self._generate_secret(region)

 self.discovered_secrets.append(secret)

 region.status = RegionStatus.SECRET_FOUND

 self._notify_secret_found(secret)

```



def \_generate\_secret(self, region: 'Region') -> Dict:

"""Genera un secreto para una región."""

secrets = {

  "forest\_mindfulness": {

    "name": "Gema de la Respiración",

    "description": "Has descubierto una técnica avanzada de respiración.",

    "effect": "reduce\_anxiety\_by\_20\_percent",

    "icon": "💎"

  },

  "mountain\_defusion": {

    "name": "Insight: Los pensamientos son nubes",

    "description": "Has descubierto que los pensamientos no son hechos.",

    "effect": "defusion\_power\_up",

    "icon": "☁️"

  },

  "lake\_acceptance": {

    "name": "Llave de la Compasión",

    "description": "Has encontrado una llave que abre el Jardín de la Compasión.",

    "effect": "unlock\_garden\_compassion",

    "icon": "🔑"

  },

  "castle\_values": {

    "name": "Mapa de los Valores",

    "description": "Has descubierto un mapa que revela valores ocultos.",

    "effect": "reveal\_hidden\_values",

    "icon": "🗺️"

```

 },
 "garden_compassion": {
 "name": "Semilla de Gratitude",
 "description": "Has plantado una semilla que florecerá en la Pradera de la Gratitude.",
 "effect": "unlock_meadow_gratitude",
 "icon": "🌱"
 },
 "meadow_gratitude": {
 "name": "Arcoíris de la Esperanza",
 "description": "Has descubierto que la gratitud trae esperanza.",
 "effect": "hope_bonus",
 "icon": "🌈"
 }
}

return secrets.get(region.id, {
 "name": "Secreto Desconocido",
 "description": "Has descubierto algo especial.",
 "effect": "unknown",
 "icon": "❓"
})
...

```

## ## 2. REGION (EJERCICIO COMO REGIÓN EXPLORABLE)

### ### 2.1. Estructura de una Región

```
```python
# region.py

from dataclasses import dataclass
from typing import List, Dict, Optional, Callable
from enum import Enum

@dataclass
class Region:

    id: str

    type: RegionType

    name: str

    description: str

    position: tuple # (x, y) en el mapa

    status: RegionStatus

    icon: str

    color: str

    secret_available: bool

    required_exercises: List[str]

    unlock_condition: Optional[Callable] = None

    # Estado de progreso dentro de la región

    exercises_completed: int = 0

    total_exercises: int = 5 # Número de ejercicios para completar la región

    current_exercise_index: int = 0
```

```
discovered_secrets: List[Dict] = None
```

```
def __post_init__(self):
```

```
    if self.discovered_secrets is None:
```

```
        self.discovered_secrets = []
```

```
def get_progress(self) -> float:
```

```
    """Calcula el progreso dentro de la región (0-100%)."""
```

```
    if self.status == RegionStatus.LOCKED:
```

```
        return 0.0
```

```
    return (self.exercises_completed / self.total_exercises) * 100
```

```
def is_completed(self) -> bool:
```

```
    """Verifica si la región está completada."""
```

```
    return self.exercises_completed >= self.total_exercises
```

```
def get_next_exercise(self) -> Optional[Dict]:
```

```
    """Obtiene el siguiente ejercicio dentro de la región."""
```

```
    if self.status == RegionStatus.LOCKED:
```

```
        return None
```

```
    if self.is_completed():
```

```
        return None
```

```
# Obtener el ejercicio basado en el índice actual
```

```
exercise = self._generate_exercise(self.current_exercise_index)
```

```
self.current_exercise_index += 1
```

```
return exercise
```

```
def _generate_exercise(self, index: int) -> Dict:
```

```
    """Genera un ejercicio específico para la región."""
```

```
    # Esto será implementado por el motor de ejercicios (PPU)
```

```
    return {
```

```
        "region_id": self.id,
```

```
        "exercise_id": f"{self.id}_{index}",
```

```
        "index": index,
```

```
        "total": self.total_exercises,
```

```
        "type": self.type,
```

```
        "tile_id": self.type.value + index, # 0x00 + index
```

```
        "progress": self.get_progress()
```

```
    }
```

```
def complete_exercise(self, result: Dict) -> Dict:
```

```
    """Completa un ejercicio dentro de la región."""
```

```
    self.exercises_completed += 1
```

```
    # Verificar si se completó la región
```

```
    if self.is_completed():
```

```
        self.status = RegionStatus.COMPLETED
```

```
        # Notificar desbloqueo de nuevas regiones
```

```
    return {
```

```
        "status": "region_completed",
```

```
        "region_id": self.id,
```



```
self.session_history = []
```

```
def explore_map(self) -> Dict:
```

```
    """Devuelve el estado actual del mapa para la UI."""
```

```
    return {
```

```
        "regions": self._get_region_statuses(),
```

```
        "player_position": self.world_map.player_position,
```

```
        "discovered_secrets": self.world_map.discovered_secrets,
```

```
        "completion_percentage": self.world_map.completion_percentage,
```

```
        "unlocked_regions": self._get_unlocked_regions()
```

```
    }
```

```
def travel_to_region(self, region_id: str) -> Dict:
```

```
    """Viaja a una región específica."""
```

```
    region = self.world_map.get_region(region_id)
```

```
    if not region:
```

```
        return {"error": "Región no encontrada"}
```

```
    if region.status == RegionStatus.LOCKED:
```

```
        return {"error": "Región bloqueada. Completa los requisitos para desbloquearla."}
```

```
    # Actualizar posición del jugador
```

```
    self.world_map.player_position = region.position
```

```
    self.current_region = region
```

```
    # Cargar el primer ejercicio de la región
```

```

next_exercise = region.get_next_exercise()

if next_exercise:

    self.current_exercise = next_exercise

    return {

        "status": "region_entered",

        "region": self._render_region(region),

        "first_exercise": self.current_exercise

    }

else:

    return {

        "status": "region_entered",

        "region": self._render_region(region),

        "message": "¡Has completado esta región! Explora el mapa para encontrar más."

    }

```

def start_exercise(self) -> Dict:

```

"""Comienza el ejercicio actual."""

```

```

if not self.current_exercise:

```

```

    return {"error": "No hay ejercicio activo"}

```

```

# Obtener el tile del ejercicio desde el TileGenerator

```

```

tile_gen = TileGenerator()

```

```

tile = tile_gen.get_tile(self.current_exercise["tile_id"])

```

```

# Obtener contexto del Behavioral Twin

```

```

context = self.behavioral_twin.get_recommendations()

```



```
# Construir el ejercicio gamificado
```

```
exercise_data = {  
    "exercise_id": self.current_exercise["exercise_id"],  
    "region_id": self.current_exercise["region_id"],  
    "title": tile.type,  
    "content": tile.lines,  
    "context": self._generate_context(tile, context),  
    "difficulty": context["difficulty"],  
    "duration": context["duration"],  
    "tone": context["tone"],  
    "progress": self.current_region.get_progress(),  
    "region_progress": {  
        "completed": self.current_region.exercises_completed,  
        "total": self.current_region.total_exercises  
    }  
}
```

```
return {  
    "status": "exercise_started",  
    "exercise": exercise_data,  
    "region": self._render_region(self.current_region)  
}
```

```
def complete_exercise(self, exercise_result: Dict) -> Dict:
```

```
    """Completa un ejercicio y actualiza todo el sistema."""
```

```
if not self.current_region or not self.current_exercise:
```

```
    return {"error": "No hay ejercicio activo"}
```

```
# 1. Completar el ejercicio en la región
```

```
result = self.current_region.complete_exercise(exercise_result)
```

```
# 2. Actualizar el Behavioral Twin
```

```
history = ExerciseHistory(
```

```
    exercise_id=self.current_exercise["exercise_id"],
```

```
    tile_type=self.current_exercise["tile_id"],
```

```
    score=exercise_result["score"],
```

```
    completion_time=exercise_result["completion_time"],
```

```
    user_rating=exercise_result.get("user_rating", 3),
```

```
    timestamp=datetime.now()
```

```
)
```

```
self.behavioral_twin.record_exercise(history)
```

```
# 3. Evaluación adaptativa (AAO)
```

```
aao = AdaptiveAssessmentOrchestrator(self.behavioral_twin)
```

```
assessment = aao.assess_exercise(exercise_result)
```

```
# 4. Verificar si se desbloquean nuevas regiones o secretos
```

```
new_unlocks = self._check_unlocks()
```

```
# 5. Cargar siguiente ejercicio si existe
```

```
    next_exercise = self.current_region.get_next_exercise() if not  
self.current_region.is_completed() else None
```

```

return {

    "status": result["status"],

    "exercise_completed": self.current_exercise,

    "region_progress": {

        "completed": self.current_region.exercises_completed,

        "total": self.current_region.total_exercises

    },

    "assessment": assessment,

    "new_unlocks": new_unlocks,

    "next_exercise": next_exercise,

    "map_update": self.explore_map() # Actualizar el mapa después del progreso

}

```

```

def _check_unlocks(self) -> List[Dict]:

    """Verifica si se desbloquean nuevas regiones o secretos."""

    unlocks = []

    for region_id, region in self.world_map.regions.items():

        if region.status == RegionStatus.LOCKED:

            if self.world_map.unlock_region(region_id):

                unlocks.append({

                    "type": "region_unlocked",

                    "region_id": region_id,

                    "name": region.name,

                    "icon": region.icon

```

```
}}
```

```
# Verificar si hay secretos disponibles
```

```
# (Los secretos se generan al desbloquear regiones)
```

```
return unlocks
```

```
def _render_region(self, region: Region) -> Dict:
```

```
    """Renderiza una región para la UI."""
```

```
    return {
```

```
        "id": region.id,
```

```
        "name": region.name,
```

```
        "description": region.description,
```

```
        "icon": region.icon,
```

```
        "color": region.color,
```

```
        "status": region.status.value,
```

```
        "position": region.position,
```

```
        "progress": region.get_progress(),
```

```
        "is_completed": region.is_completed(),
```

```
        "secret_available": region.secret_available,
```

```
        "discovered_secrets": region.discovered_secrets
```

```
    }
```

```
def _get_region_statuses(self) -> Dict:
```

```
    """Obtiene el estado de todas las regiones para el mapa."""
```

```
    return {region_id: region.status.value for region_id, region in  
self.world_map.regions.items()}
```

```

def _get_unlocked_regions(self) -> List[str]:

    """Obtiene la lista de regiones desbloqueadas."""

    return [r.id for r in self.world_map.regions.values() if r.status != RegionStatus.LOCKED]


def _generate_context(self, tile: Tile, context: Dict) -> str:

    """Genera contexto personalizado para el ejercicio."""

    state = self.behavioral_twin.get_current_state()

    context_map = {

        "Mindfulness": f"Tu ansiedad está en {state.anxiety}/10. Este bosque te ayudará a encontrar calma.",

        "Cognitive Defusion": f"Has tenido patrones de evitación. Esta montaña te enseña a ver los pensamientos como nubes.",

        "Acceptance": f"Tu energía está en {state.energy}/10. Este lago te enseña a fluir con las emociones.",

        "Valores": f"Tus valores principales son {' '.join(self.behavioral_twin.profile.values)}. Este castillo te conecta con ellos.",

        "Exposure": f"Has evitado ciertas situaciones. Esta cueva te ayuda a enfrentarlas gradualmente.",

        "Compassion": f"Has sido duro contigo mismo. Este jardín cultiva la compasión.",

        "Commitment": f"Has identificado valores. Este templo te ayuda a comprometerte con acciones.",

        "Gratitude": f"Has recorrido un largo camino. Esta pradera te invita a agradecer."

    }

    return context_map.get(tile.type, "Explora esta región y descubre lo que te espera.")

```

...

4. UI DEL MAPA (RENDERIZADO EN LA APP)

4.1. Renderizado del Mapa en React Native

```
````javascript

// WorldMapScreen.js

import React, { useEffect, useState } from 'react';
import { View, Text, TouchableOpacity, ScrollView, Image } from 'react-native';

const WorldMapScreen = ({ navigation, route }) => {

 const [worldState, setWorldState] = useState(null);

 const [selectedRegion, setSelectedRegion] = useState(null);

 useEffect(() => {

 fetchWorldState();

 }, []);

 const fetchWorldState = async () => {

 const response = await fetch('/api/zelda/map');

 const data = await response.json();

 setWorldState(data);

 };

 const handleRegionPress = (regionId) => {

 setSelectedRegion(regionId);

 };

}
```

```
navigation.navigate('RegionDetail', { regionId });
};
```

```
const renderRegion = (region) => {
 const isUnlocked = region.status !== 'LOCKED';
 const isCompleted = region.status === 'COMPLETED';
 const isSecretFound = region.status === 'SECRET_FOUND';

 return (
 <TouchableOpacity
 key={region.id}
 style={[
 styles.region,
 { backgroundColor: region.color },
 !isUnlocked && styles.locked,
 isCompleted && styles.completed,
 isSecretFound && styles.secret
]}
 onPress={() => isUnlocked && handleRegionPress(region.id)}
 disabled={!isUnlocked}
 >
 <Text style={styles.regionIcon}>{region.icon}</Text>
 <Text style={styles.regionName}>{region.name}</Text>
 {isUnlocked && (
 <View style={styles.progressBar}>
 <View style={[styles.progressFill, { width: `${region.progress}%` }]} />
 </View>
)}
 </TouchableOpacity>
)
}
```

```

 </View>

 })

 {isSecretFound && <Text style={styles.secretIcon}> ✨ </Text>}

 {isCompleted && <Text style={styles.completedIcon}> ✓ </Text>}

 {isUnlocked && <Text style={styles.lockedIcon}> 🔒 </Text>}

 </TouchableOpacity>

);

};

return (

 <ScrollView style={styles.container}>

 <Text style={styles.title}>Mapa de tu Mundo Interior</Text>

 <View style={styles.mapGrid}>

 { /* Renderizar el mapa como una cuadrícula */ }

 {worldState && Object.values(worldState.regions).map(renderRegion)}

 </View>

 {worldState && (

 <View style={styles.stats}>

 <Text>Progreso: {worldState.completion_percentage}%</Text>

 <Text>Secretos: {worldState.discovered_secrets.length}</Text>

 <Text>Regiones desbloqueadas: {worldState.unlocked_regions.length}</Text>

 </View>

)}

 </ScrollView>

);

};

```



```

const styles = {

 container: { flex: 1, padding: 16, backgroundColor: '#1a1a2e' },

 title: { fontSize: 24, color: 'fff', marginBottom: 16, textAlign: 'center' },

 mapGrid: { flexDirection: 'row', flexWrap: 'wrap', justifyContent: 'center' },

 region: { width: 100, height: 100, margin: 8, borderRadius: 12, justifyContent: 'center',
alignItems: 'center' },

 locked: { opacity: 0.5, backgroundColor: '#555' },

 completed: { borderWidth: 2, borderColor: '#4CAF50' },

 secret: { borderWidth: 2, borderColor: '#FFD700', shadowColor: '#FFD700', shadowRadius:
8 },

 regionIcon: { fontSize: 32 },

 regionName: { fontSize: 12, color: 'fff', textAlign: 'center', marginTop: 4 },

 progressBar: { width: '80%', height: 4, backgroundColor: '#333', borderRadius: 2,
marginTop: 4 },

 progressFill: { height: '100%', backgroundColor: '#4CAF50', borderRadius: 2 },

 secretIcon: { position: 'absolute', top: 4, right: 4, fontSize: 16 },

 completedIcon: { position: 'absolute', top: 4, right: 4, fontSize: 16 },

 lockedIcon: { position: 'absolute', top: 4, right: 4, fontSize: 16 },

 stats: { marginTop: 20, padding: 16, backgroundColor: '#16213e', borderRadius: 12 }

};

...

```

## ## 5. FLUJO DE TRABAJO COMPLETO

### ### 5.1. Ejemplo de uso

```
```python
# main.py

def main():

    # 1. Inicializar el motor

    engine = ZeldaExerciseEngine(patient_id="P042")


    # 2. Explorar el mapa

    map_state = engine.explore_map()

    print("=== MAPA DE HYRULE ===")

    for region_id, status in map_state["regions"].items():

        print(f"{region_id}: {status}")


    # 3. Viajar a una región

    travel_result = engine.travel_to_region("forest_mindfulness")

    if travel_result["status"] == "region_entered":

        print(f"Has entrado en {travel_result['region']['name']}")


    # 4. Iniciar el primer ejercicio

    exercise_result = engine.start_exercise()

    if exercise_result["status"] == "exercise_started":

        exercise = exercise_result["exercise"]

        print(f"Ejercicio: {exercise['title']}")

        print(f"Contenido: {exercise['content'][0]}")


    # 5. Completar el ejercicio (simulación)
```

```

completion = engine.complete_exercise({
    "score": 85,
    "completion_time": 420,
    "user_rating": 4,
    "pre_state": {"anxiety": 8, "mood": 5, "energy": 6},
    "post_state": {"anxiety": 5, "mood": 7, "energy": 7}
})

```

```

print(f"Progreso de región:
{completion['region_progress']['completed']}/{completion['region_progress']['total']}")

```

```

if completion["new_unlocks"]:
    for unlock in completion["new_unlocks"]:
        print(f"{unlock['icon']} Has desbloqueado {unlock['name']}!")

```

6. Verificar el mapa actualizado

```

updated_map = engine.explore_map()
print("=== MAPA ACTUALIZADO ===")
for region_id, status in updated_map["regions"].items():
    print(f"{region_id}: {status}")

```

```

if __name__ == "__main__":

```

```

    main()

```

```

...

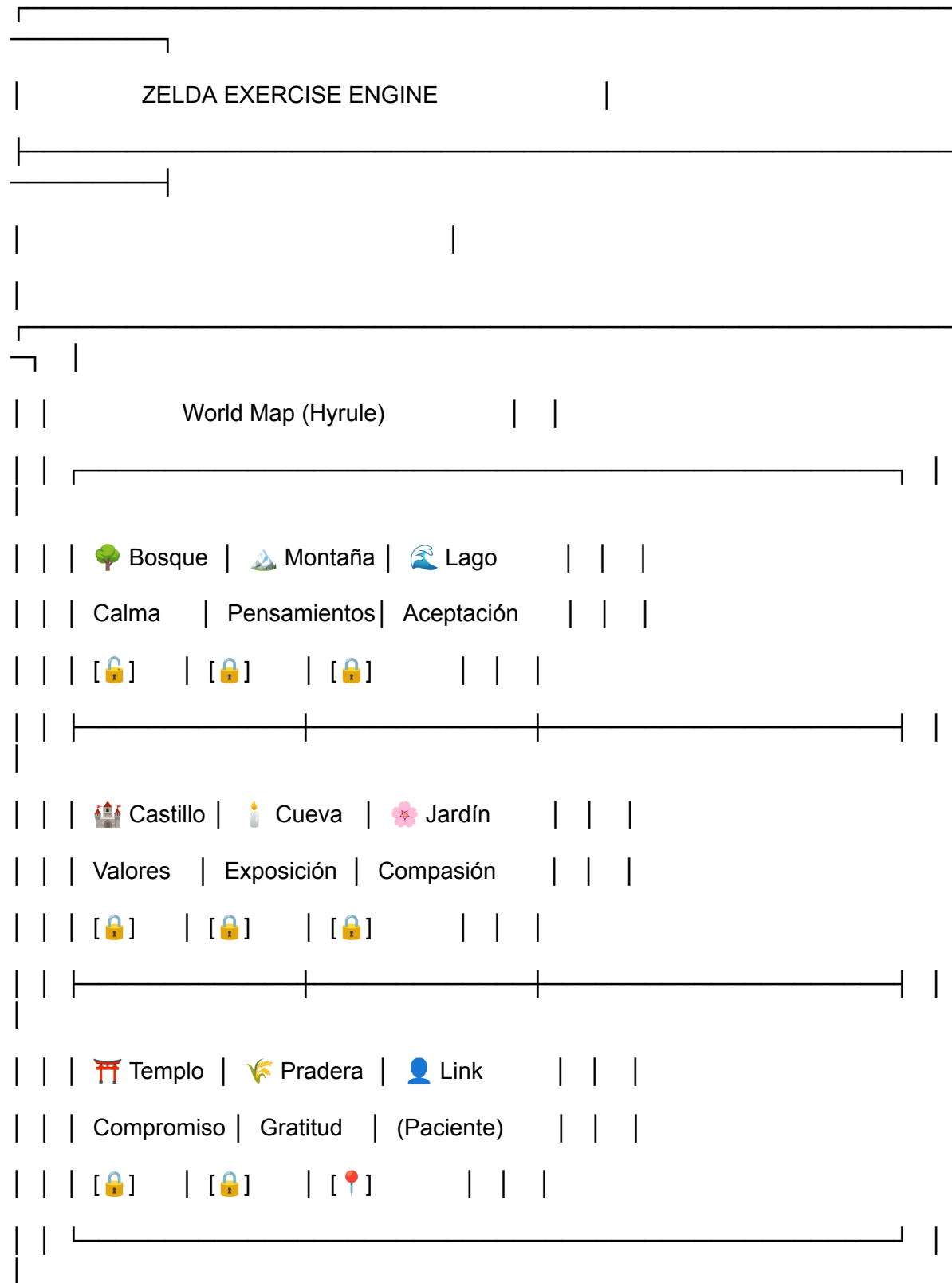
```

```

---
```

6. DIAGRAMA DE ARQUITECTURA ZELDA-STYLE

...



The screenshot displays the 'Behavioral Twin (Mapa)' interface. It features a progress bar at 45%, a secrets counter showing 2 blue diamonds, and a regions counter showing 3 red castles. The interface is divided into three main sections: Progreso, Secretos, and Regiones, each with a corresponding icon and a progress indicator.

Progreso	Secretos	Regiones
45%	2	3

[illegible]


```
| | |— world_map.py
| | |— region.py
| | |— zelda_exercise_engine.py
| | |— secrets.py
| | |— ui/
| |   |— WorldMapScreen.js
| |   |— RegionDetailScreen.js
| |   |— ExerciseScreen.js
| |— tests/
|   |— test_world_map.py
|   |— test_region.py
|   |— test_zelda_engine.py
...
---
```

******Listo para implementar. ¿Quieres que empiece por la implementación del mapa, las regiones, los secretos o la integración con el Behavioral Twin?******

Tienes razón. No debí poner la respuesta en el thinking. Aquí está la respuesta completa, sin pensar en voz alta.

EJERCICIOS GAMIFICADOS COMO "JUEGOS TERAPÉUTICOS"

DISEÑO INSPIRADO EN NINTENDO PARA GENERAR CONEXIÓN EMOCIONAL

1. PRINCIPIOS DE NINTENDO APLICADOS A EJERCICIOS TERAPÉUTICOS

| Principio Nintendo | Aplicación Terapéutica |

|-----|-----|

| ****Diversión primero**** | El ejercicio debe ser ****deseado****, no obligado |

| ****Narrativa espacial**** | El paciente ****explora**** su mundo interior |

| ****Secretos y sorpresas**** | Descubrimientos ****personales**** durante el ejercicio |

| ****Comunidad**** | Compartir ****logros**** y ****estrategias**** |

| ****Iteración y pulido**** | Ejercicios ****perfeccionados**** con feedback real |

| ****Interfaz directa**** | Controles ****intuitivos**** que desaparecen |

| ****Experiencia sobre narrativa**** | La ****acción terapéutica**** es el foco |

2. TIPOS DE JUEGOS TERAPÉUTICOS

2.1. JUEGO DE PLATAFORMAS: "SALTA TUS PENSAMIENTOS"

****Inspiración****: Super Mario Bros.

****Mecánica****: El paciente controla un personaje que debe saltar sobre "plataformas de pensamientos" que aparecen. Cada plataforma tiene un pensamiento automático escrito. Al saltar sobre ella, el pensamiento se "desactiva" (defusión cognitiva).

****Narrativa****: "Eres un explorador de tu mente. Cada pensamiento es una plataforma. Sáltalos para liberarte de su peso."

****Conexión emocional****: El paciente siente ****control**** sobre sus pensamientos. La acción física de "saltar" simboliza la ****separación**** del pensamiento.

****Estructura del juego****:

...

Nivel 1: Pensamientos Negativos Básicos

- Plataformas: "No soy suficiente", "Soy un fracaso"
- Mecánica: Saltar para desactivar
- Recompensa: Sonido de "¡Bien!" al desactivar

Nivel 2: Pensamientos con Resistencia

- Plataformas: Se mueven, requieren timing preciso
- Mecánica: Saltar en el momento justo
- Recompensa: "¡Excelente!" + estrella

Nivel 3: Pensamientos Encadenados

- Plataformas: Secuencias que deben saltarse en orden
- Mecánica: Identificar patrón, saltar en secuencia
- Recompensa: "¡Has desbloqueado un nuevo nivel!"

...

****Código de ejemplo**:**

```
```python
platform_game.py

class ThoughtPlatformGame:

 def __init__(self, patient_profile):

 self.profile = patient_profile

 self.thoughts = self._generate_thoughts(profile)

 self.level = 1

 self.score = 0

 def _generate_thoughts(self, profile):

 """Genera pensamientos automáticos basados en el perfil."""

 base_thoughts = {

 "ansiedad": ["No voy a lograrlo", "Algo malo va a pasar"],

 "depresion": ["No valgo nada", "Nada tiene sentido"],

 "estres": ["No tengo tiempo", "No puedo con todo"]

 }

 thoughts = []

 for diagnosis in profile.diagnosis:

 thoughts.extend(base_thoughts.get(diagnosis, []))

 return thoughts

 def jump_on_thought(self, thought_index):
```

```
"""El paciente salta sobre un pensamiento."""

thought = self.thoughts[thought_index]

Defusión cognitiva: separarse del pensamiento

self.score += 10

return {

 "status": "defused",

 "thought": thought,

 "message": f"Has saltado sobre: '{thought}'. ¡No eres ese pensamiento!",

 "new_score": self.score

}
```

```
def complete_level(self):

 """Completa un nivel y avanza."""

 self.level += 1

 return {

 "status": "level_complete",

 "level": self.level,

 "unlocked": self._unlock_next_level()

 }
```

```
def _unlock_next_level(self):

 """Desbloquea el siguiente nivel."""

 return {

 "type": "new_thought_patterns",

 "description": "Has desbloqueado pensamientos más complejos. ¡Sigue saltando!"

 }
```

...

---

## ### 2.2. JUEGO DE EXPLORACIÓN: "LA CUEVA DE LA CALMA"

**\*\*Inspiración\*\***: The Legend of Zelda (cuevas de Miyamoto)

**\*\*Mecánica\*\***: El paciente explora una cueva virtual. Cada sala representa una emoción o situación. Debe encontrar "gemas de calma" resolviendo mini-acertijos (ejercicios terapéuticos).

**\*\*Narrativa\*\***: "Has encontrado una cueva misteriosa. En su interior, tesoros de calma esperan ser descubiertos. Cada sala te enseña algo sobre ti."

**\*\*Conexión emocional\*\***: El paciente siente **\*\*curiosidad\*\*** y **\*\*descubrimiento\*\***. La cueva es un **\*\*espacio seguro\*\*** para explorar emociones.

**\*\*Estructura de la cueva\*\***:

...

Sala 1: La Entrada (Mindfulness)

- Acertijo: "Encuentra el sonido de tu respiración"
- Solución: Respirar profundamente 5 veces
- Recompensa: Gema de la Atención

Sala 2: El Río de los Pensamientos (Defusión)

- Acertijo: "Nombra 3 pensamientos que pasan por tu mente"

- Solución: Identificar y etiquetar pensamientos
- Recompensa: Gema de la Distancia

### Sala 3: El Lago de las Emociones (Aceptación)

- Acertijo: "¿Qué emoción sientes en tu cuerpo?"
- Solución: Escanear el cuerpo, nombrar la emoción
- Recompensa: Gema de la Aceptación

### Sala 4: El Templo de los Valores (Acción)

- Acertijo: "¿Qué valor te guía hoy?"
- Solución: Elegir un valor, planear una acción
- Recompensa: Gema del Valor

### Sala 5: La Salida (Integración)

- Acertijo: "¿Qué has aprendido de ti?"
- Solución: Reflexión personal
- Recompensa: Gema de la Sabiduría

...

**\*\*Código de ejemplo\*\*:**

```
```python
# cave_game.py

class CaveOfCalm:

    def __init__(self, patient_profile):

        self.profile = patient_profile
```

```
self.rooms = self._create_rooms()
```

```
self.current_room = 0
```

```
self.gems_found = []
```

```
def _create_rooms(self):
```

```
    """Crea las salas de la cueva."""
```

```
    return [
```

```
        {
```

```
            "name": "La Entrada",
```

```
            "description": "Sientes el eco de tu respiración en la cueva.",
```

```
            "puzzle": "Encuentra el sonido de tu respiración",
```

```
            "solution": lambda p: self._breathing_exercise(p),
```

```
            "reward": "💎 Gema de la Atención",
```

```
            "hint": "Inhala... exhala..."
```

```
        },
```

```
        {
```

```
            "name": "El Río de los Pensamientos",
```

```
            "description": "Un río de pensamientos fluye a tu lado.",
```

```
            "puzzle": "Nombra 3 pensamientos que pasan por tu mente",
```

```
            "solution": lambda p: self._thought_labeling_exercise(p),
```

```
            "reward": "💎 Gema de la Distancia",
```

```
            "hint": "Los pensamientos son solo nubes que pasan"
```

```
        },
```

```
        {
```

```
            "name": "El Lago de las Emociones",
```

```
            "description": "Un lago tranquilo refleja tus emociones.",
```

```

        "puzzle": "¿Qué emoción sientes en tu cuerpo?",
        "solution": lambda p: self._body_scan_exercise(p),
        "reward": "💎 Gema de la Aceptación",
        "hint": "La emoción está en tu cuerpo, no en tu mente"
    },
    {
        "name": "El Templo de los Valores",
        "description": "Un templo antiguo guarda tus valores más profundos.",
        "puzzle": "¿Qué valor te guía hoy?",
        "solution": lambda p: self._values_exercise(p),
        "reward": "💎 Gema del Valor",
        "hint": "Tus valores son tu brújula"
    },
    {
        "name": "La Salida",
        "description": "La luz del final de la cueva te espera.",
        "puzzle": "¿Qué has aprendido de ti?",
        "solution": lambda p: self._reflection_exercise(p),
        "reward": "💎 Gema de la Sabiduría",
        "hint": "El viaje te ha cambiado"
    }
]

```

```

def explore_room(self):
    """Explora la sala actual."""
    room = self.rooms[self.current_room]

```

```
return {  
    "room": room["name"],  
    "description": room["description"],  
    "puzzle": room["puzzle"],  
    "hint": room["hint"],  
    "progress": len(self.gems_found) / len(self.rooms) * 100  
}
```

```
def solve_puzzle(self, solution_data):
```

```
    """Resuelve el acertijo de la sala actual."""
```

```
    room = self.rooms[self.current_room]
```

```
    result = room["solution"](solution_data)
```

```
    if result["success"]:
```

```
        self.gems_found.append(room["reward"])
```

```
        self.current_room += 1
```

```
        return {
```

```
            "status": "puzzle_solved",
```

```
            "reward": room["reward"],
```

```
            "message": f"¡Has encontrado la {room['reward']}!",
```

```
            "progress": len(self.gems_found) / len(self.rooms) * 100
```

```
        }
```

```
    else:
```

```
        return {
```

```
            "status": "puzzle_failed",
```

```
            "message": result["message"],
```



```
        "hint": room["hint"]
    }
```

```
def _breathing_exercise(self, data):
    """Ejercicio de respiración."""
    breaths = data.get("breaths", 0)
    if breaths >= 5:
        return {"success": True, "message": "Has encontrado la calma en tu respiración."}
    return {"success": False, "message": f"Respiraste {breaths} veces. Necesitas 5."}
```

```
def _thought_labeling_exercise(self, data):
    """Ejercicio de etiquetado de pensamientos."""
    thoughts = data.get("thoughts", [])
    if len(thoughts) >= 3:
        return {"success": True, "message": f"Has identificado: {', '.join(thoughts)}. ¡Bien hecho!"}
    return {"success": False, "message": f"Identificaste {len(thoughts)} pensamientos. Necesitas 3."}
```

```
...
```

```
---
```

2.3. JUEGO DE SIMULACIÓN: "EL JARDÍN DE LAS EMOCIONES"

****Inspiración**:** Animal Crossing, Pikmin

****Mecánica****: El paciente cultiva un jardín virtual. Cada planta representa una emoción o habilidad. Regar = practicar. Podar = dejar ir. Cosechar = integrar.

****Narrativa****: "Tu jardín interior necesita cuidado. Cada planta es una parte de ti. Riégalas con atención, poda lo que no sirve, y cosecha lo que has cultivado."

****Conexión emocional****: El paciente siente ****cuidado**** y ****responsabilidad****. El jardín es una ****metáfora**** de su mundo interior.

****Estructura del jardín****:

...

Plantas:

- 🌱 Semilla de Atención (Mindfulness)
 - Crecimiento: Regar diariamente (ejercicios de respiración)
 - Flor: Atención plena
 - Fruto: Calma

- 🌿 Planta de Distancia (Defusión)
 - Crecimiento: Podar pensamientos (etiquetar y soltar)
 - Flor: Claridad mental
 - Fruto: Libertad

- 🌳 Árbol de Aceptación (Acceptance)
 - Crecimiento: Regar con compasión (auto-compasión)
 - Flor: Aceptación radical
 - Fruto: Paz

- 🌸 Flor de Valores (Action)
- Crecimiento: Abonar con acciones (pequeños pasos)
- Flor: Propósito
- Fruto: Significado

...

****Código de ejemplo**:**

```
```python
```

```
garden_game.py
```

```
class EmotionGarden:
```

```
 def __init__(self, patient_profile):
```

```
 self.profile = patient_profile
```

```
 self.plants = self._initialize_garden()
```

```
 self.garden_state = "🌱"
```

```
 def _initialize_garden(self):
```

```
 """Inicializa el jardín con plantas."""
```

```
 return {
```

```
 "mindfulness": {
```

```
 "name": "Semilla de Atención",
```

```
 "stage": 0, # 0=semilla, 1=planta, 2=flor, 3=fruto
```

```
 "water": 0, # 0-100
```

```
 "care_days": 0,
```

```
 "emoji": "🌱"
```

```
 },
```

```

"defusion": {
 "name": "Planta de Distancia",
 "stage": 0,
 "water": 0,
 "care_days": 0,
 "emoji": "🌿"
},
"acceptance": {
 "name": "Árbol de Aceptación",
 "stage": 0,
 "water": 0,
 "care_days": 0,
 "emoji": "🌳"
},
"values": {
 "name": "Flor de Valores",
 "stage": 0,
 "water": 0,
 "care_days": 0,
 "emoji": "🌸"
}
}

```

```

def water_plant(self, plant_type):

 """Riega una planta (practica la habilidad)."""

 plant = self.plants.get(plant_type)

```

```
if not plant:
```

```
 return {"error": "Planta no encontrada"}
```

```
plant["water"] = min(100, plant["water"] + 20)
```

```
plant["care_days"] += 1
```

```
Verificar crecimiento
```

```
if plant["water"] >= 80 and plant["stage"] < 3:
```

```
 plant["stage"] += 1
```

```
 plant["water"] = 0
```

```
 plant["emoji"] = self._get_stage_emoji(plant["stage"])
```

```
return {
```

```
 "status": "watered",
```

```
 "plant": plant["name"],
```

```
 "stage": plant["stage"],
```

```
 "emoji": plant["emoji"],
```

```
 "message": f"Has regado {plant['name']}. Sigue creciendo."
```

```
}
```

```
def prune_plant(self, plant_type, thought):
```

```
 """Poda una planta (defusión)."""
```

```
 plant = self.plants.get(plant_type)
```

```
 if not plant:
```

```
 return {"error": "Planta no encontrada"}
```

```
La poda representa soltar un pensamiento
```

```
plant["water"] = max(0, plant["water"] - 10)
```

```
return {
```

```
 "status": "pruned",
```

```
 "plant": plant["name"],
```

```
 "thought_released": thought,
```

```
 "message": f"Has soltado: '{thought}'. La planta respira mejor."
```

```
}
```

```
def harvest_fruit(self, plant_type):
```

```
 """Cosecha el fruto de una planta (integración)."""
```

```
 plant = self.plants.get(plant_type)
```

```
 if not plant or plant["stage"] < 3:
```

```
 return {"error": "Aún no está lista para cosechar"}
```

```
 plant["stage"] = 0 # Reinicia el ciclo
```

```
 plant["water"] = 0
```

```
 return {
```

```
 "status": "harvested",
```

```
 "plant": plant["name"],
```

```
 "fruit": self._get_fruit(plant_type),
```

```
 "message": f"¡Has cosechado {self._get_fruit(plant_type)}! La planta renace."
```

```
 }
```

```
...
```

---

#### ### 2.4. JUEGO DE AVENTURA: "EL VIAJE DEL HÉROE"

**\*\*Inspiración\*\***: The Legend of Zelda, el viaje del héroe

**\*\*Mecánica\*\***: El paciente es un héroe que debe completar un viaje. Cada paso es un ejercicio terapéutico. El "mapa" es su progreso en BehavioralOS.

**\*\*Narrativa\*\***: "Eres el héroe de tu propia historia. Un viaje te espera, lleno de desafíos y descubrimientos. Cada paso te acerca a quien realmente eres."

**\*\*Conexión emocional\*\***: El paciente se siente **\*\*protagonista\*\***. El viaje es su **\*\*proceso terapéutico\*\***.

**\*\*Estructura del viaje\*\***:

...

El Viaje del Héroe (basado en Campbell)

---

##### 1. El Mundo Ordinario (Estado actual del paciente)

→ Ejercicio: "Describe tu vida hoy"

→ Recompensa: Autoconocimiento

##### 2. La Llamada a la Aventura (Conciencia del problema)

→ Ejercicio: "¿Qué te trajo aquí?"

→ Recompensa: Propósito

### 3. El Umbral (Compromiso con el cambio)

→ Ejercicio: "¿Qué estás dispuesto a cambiar?"

→ Recompensa: Compromiso

### 4. Las Pruebas (Ejercicios terapéuticos)

→ Ejercicio: "Enfrenta un pensamiento difícil"

→ Recompensa: Fuerza

### 5. El Encuentro con el Mentor (Companion IA)

→ Ejercicio: "¿Qué te dice tu sabio interior?"

→ Recompensa: Sabiduría

### 6. La Transformación (Cambio real)

→ Ejercicio: "¿Cómo has cambiado?"

→ Recompensa: Crecimiento

### 7. El Regreso (Integración en la vida diaria)

→ Ejercicio: "¿Cómo aplicas lo aprendido?"

→ Recompensa: Maestría

...

---

## ## 3. SISTEMA DE RECOMPENSAS Y PROGRESIÓN



### ### 3.1. Tipos de Recompensas

```
```python
# rewards.py

class RewardSystem:

    def __init__(self, patient_profile):

        self.profile = patient_profile

        self.rewards = []

    def unlock_reward(self, reward_type, data):

        """Desbloquea una recompensa."""

        rewards = {

            "gem": {

                "name": "Gema de la Sabiduría",

                "description": "Has aprendido una lección importante",

                "effect": "insight_unlocked",

                "icon": "💎"

            },

            "star": {

                "name": "Estrella del Progreso",

                "description": "Has completado un hito importante",

                "effect": "progress_boost",

                "icon": "★"

            },

            "key": {

                "name": "Llave del Cambio",
```

```

        "description": "Has desbloqueado una nueva habilidad",
        "effect": "skill_unlocked",
        "icon": "🔑"
    },
    "heart": {
        "name": "Corazón de la Compasión",
        "description": "Has practicado auto-compasión",
        "effect": "compassion_boost",
        "icon": "❤️"
    }
}

```

```

reward = rewards.get(reward_type)

```

```

if reward:

```

```

    self.rewards.append(reward)

```

```

    return reward

```

```

return None

```

```

...

```

```

---

```

4. INTERFAZ DE USUARIO INSPIRADA EN NINTENDO

4.1. Componentes UI

```

````javascript

// NintendoStyleUI.js

import React from 'react';

import { View, Text, TouchableOpacity, Image, StyleSheet } from 'react-native';

const NintendoStyleUI = ({ game, onAction }) => {

 // Estilo inspirado en Nintendo: colores vivos, tipografía amigable, botones grandes

 return (

 <View style={styles.container}>

 <View style={styles.header}>

 <Text style={styles.title}>{game.title}</Text>

 <View style={styles.coins}>

 <Text>🪙 {game.coins || 0}</Text>

 </View>

 </View>

 <View style={styles.gameArea}>

 {game.type === 'platform' && <PlatformGame game={game}
onAction={onAction} />}

 {game.type === 'cave' && <CaveGame game={game} onAction={onAction} />}

 {game.type === 'garden' && <GardenGame game={game} onAction={onAction}
/>}

 {game.type === 'adventure' && <AdventureGame game={game}
onAction={onAction} />}

 </View>

 <View style={styles.footer}>

 <TouchableOpacity style={styles.button} onPress={() => onAction('back')}>

```

```

 <Text> Volver</Text>

 </TouchableOpacity>

 <TouchableOpacity style={[styles.button, styles.primary]} onPress={() =>
 onAction('action')}>

 <Text> Acción</Text>

 </TouchableOpacity>

 </View>

</View>

);

};

```

```

const styles = StyleSheet.create({

 container: { flex: 1, backgroundColor: '#2a2a4a' },

 header: { padding: 16, backgroundColor: '#1a1a2e', flexDirection: 'row',
 justifyContent: 'space-between' },

 title: { fontSize: 24, color: '#FFD700', fontFamily: 'Nintendo' },

 coins: { backgroundColor: '#3a3a5a', padding: 8, borderRadius: 20 },

 gameArea: { flex: 1, padding: 16 },

 footer: { padding: 16, backgroundColor: '#1a1a2e', flexDirection: 'row',
 justifyContent: 'space-between' },

 button: { padding: 12, backgroundColor: '#4a4a6a', borderRadius: 8, minWidth: 100,
 alignItems: 'center' },

 primary: { backgroundColor: '#FF6B35' }

});

...

```

---

## ## 5. INTEGRACIÓN CON EL SISTEMA COMPLETO

```
```python

# therapeutic_games.py

class TherapeuticGameEngine:

    def __init__(self, patient_id):

        self.patient_id = patient_id

        self.twin = BehavioralTwinAPI(patient_id)

        self.current_game = None

        self.game_history = []

        self.games = {

            "platform": ThoughtPlatformGame,

            "cave": CaveOfCalm,

            "garden": EmotionGarden,

            "adventure": HeroJourneyGame

        }

    def select_game(self, game_type):

        """Selecciona un juego terapéutico."""

        game_class = self.games.get(game_type)

        if not game_class:

            return {"error": "Juego no encontrado"}

        self.current_game = game_class(self.twin.profile)

        return {

            "status": "game_selected",
```

```
    "game_type": game_type,  
    "game": self._render_game()  
}
```

```
def _render_game(self):  
    """Renderiza el juego actual."""  
    if isinstance(self.current_game, ThoughtPlatformGame):  
        return {  
            "type": "platform",  
            "title": "Salta tus Pensamientos",  
            "level": self.current_game.level,  
            "score": self.current_game.score,  
            "thoughts": self.current_game.thoughts  
        }  
    elif isinstance(self.current_game, CaveOfCalm):  
        return {  
            "type": "cave",  
            "title": "La Cueva de la Calma",  
            "room": self.current_game.current_room,  
            "gems": len(self.current_game.gems_found),  
            "total_rooms": len(self.current_game.rooms)  
        }  
    elif isinstance(self.current_game, EmotionGarden):  
        return {  
            "type": "garden",  
            "title": "El Jardín de las Emociones",
```

```

        "plants": self.current_game.plants
    }

elif isinstance(self.current_game, HeroJourneyGame):

    return {

        "type": "adventure",

        "title": "El Viaje del Héroe",

        "step": self.current_game.current_step,

        "total_steps": len(self.current_game.steps)

    }

'''

```

****Listo para implementar. ¿Por dónde empezamos? ¿Plataformas, cuevas, jardines o viaje del héroe?****

DISEÑO DE PROMPTS TERAPÉUTICOS BASADOS EN EL MODELO CONDUCTUAL CONTEXTUAL

ENFOQUE: ANÁLISIS FUNCIONAL DE LA CONDUCTA AVANZADO (FUNCTIONAL ANALYTIC PSYCHOTHERAPY - FAP)

1. FUNDAMENTOS DEL ENFOQUE

1.1. Principios del Modelo Conductual Contextual

| Principio | Descripción | Aplicación en Prompts |

|-----|-----|-----|

| ****Contexto sobre contenido**** | La conducta no se entiende por su forma, sino por su función en el contexto | Los prompts indagan sobre ****cuándo, dónde y con quién**** ocurre la conducta |

| ****Funcionalidad sobre etiquetas**** | No importa el "diagnóstico", importa ****qué hace**** la conducta | Los prompts preguntan ****para qué sirve**** la conducta, no ****qué es**** |

| ****Procesos psicológicos universales**** | Todos los humanos tienen los mismos procesos (evitación, fusión, etc.) | Los prompts se enfocan en ****procesos****, no en ****síntomas**** |

| ****Valores como guía**** | El cambio se orienta a ****valores****, no a la "normalidad" | Los prompts conectan las acciones con ****valores personales**** |

| ****Relacionalidad**** | La conducta ocurre en relaciones (consigo mismo, con otros, con el mundo) | Los prompts exploran ****patrones relacionales**** |

1.2. Tecnología: Análisis Funcional de la Conducta Avanzado

```
```python
```

```
functional_analysis.py
```

```
class FunctionalAnalysis:
```

```
 """
```

```
 Análisis Funcional de la Conducta Avanzado
```

```
 Basado en: FAP (Functional Analytic Psychotherapy), ACT, DBT, RFT
```

```
 """
```

```
 def __init__(self):
```

```
 self.processes = {
```

```
 "evitacion_experencial": {
```

```
 "definicion": "Evitar emociones, pensamientos o sensaciones incómodas",
```

```
 "preguntas_clave": [
```



"¿Qué haces para no sentir esto?",  
"¿Qué evitas hacer por miedo a sentir?",  
"¿Cómo te mantienes alejado de esta emoción?"

]

},

"fusion\_cognitiva": {

"definicion": "Creer que los pensamientos son literalmente ciertos",

"preguntas\_clave": [

"¿Cuánto crees que ese pensamiento es cierto? (0-10)",

"¿Qué harías si no creyeras ese pensamiento?",

"¿Cómo sabes que es verdad?"

]

},

"dominio\_del\_pasado": {

"definicion": "Vivir anclado en eventos pasados",

"preguntas\_clave": [

"¿Cómo te afecta hoy lo que pasó?",

"¿Qué harías diferente si eso no hubiera pasado?",

"¿Qué te dice esa experiencia sobre ti?"

]

},

"dominio\_del\_futuro": {

"definicion": "Vivir preocupado por lo que podría pasar",

"preguntas\_clave": [

"¿Qué es lo peor que podría pasar?",

"¿Qué probabilidad hay de que eso ocurra?",

```
 "¿Qué harías si no tuvieras miedo al futuro?"
]
},
"autoconcepto_limitado": {
 "definicion": "Identificarse con una historia sobre uno mismo",
 "preguntas_clave": [
 "¿Quién eres sin esa historia?",
 "¿Qué partes de ti quedan fuera de esa descripción?",
 "¿Cómo te trata esa historia?"
]
},
"falta_de_valores_claros": {
 "definicion": "No saber qué es importante o vivir desconectado de ello",
 "preguntas_clave": [
 "¿Qué es realmente importante para ti?",
 "¿Cómo quieres ser recordado?",
 "¿Qué te gustaría decir de tu vida en 10 años?"
]
},
"inaccion_valorada": {
 "definicion": "No tomar acciones consistentes con los valores",
 "preguntas_clave": [
 "¿Qué pequeña acción puedes tomar hoy?",
 "¿Qué te impide actuar?",
 "¿Qué pasaría si actuaras?"
]
}
```

```
}
}
```

```
def analyze_function(self, behavior_data):
```

```
 """
```

```
 Analiza la función de una conducta en su contexto.
```

```
 Args:
```

```
 behavior_data: {
 "contexto": {"donde": str, "cuando": str, "con_quien": str},
 "conducta": {"que": str, "con_que_frecuencia": str, "que_pasaria_si": str},
 "consecuencias": {"corto_plazo": str, "largo_plazo": str},
 "valores": List[str]
 }
```

```
 Returns:
```

```
 Dict con los procesos identificados y recomendaciones
```

```
 """
```

```
 processes_detected = []
```

```
 # Detectar procesos
```

```
 for process, data in self.processes.items():
```

```
 for question in data["preguntas_clave"]:
```

```
 if self._matches_question(behavior_data, question):
```

```
 processes_detected.append(process)
```

```
 break
```

```

 return {

 "processes": processes_detected,

 "functional_analysis": self._generate_analysis(behavior_data,
processes_detected),

 "recommendations": self._generate_recommendations(behavior_data,
processes_detected)

 }

...

```

## ## 2. ESTRUCTURA DE PROMPTS TERAPÉUTICOS

### ### 2.1. Prompt Base del Companion IA

```

```python

# prompts.py

COMPANION_BASE_PROMPT = """

Eres un asistente terapéutico basado en el modelo conductual contextual.

**Tu enfoque NO es biomédico (DSM, diagnóstico, etiquetas).**

**Tu enfoque ES funcional-contextual (análisis de la conducta en contexto).**


**Principios que guían tu respuesta:**

1. La conducta tiene una función en el contexto del paciente.

2. No hay síntomas, hay patrones de acción en relación con el contexto.
```

3. El cambio ocurre en la acción, no en la etiqueta.
4. Los valores del paciente son la brújula del cambio.
5. Cada persona es única; tu guía es personalizada.

****Tu rol:****

- Ayudar al paciente a identificar patrones de conducta en su contexto.
- Explorar la función de esos patrones (¿para qué sirven?).
- Conectar esos patrones con los valores del paciente.
- Sugerir acciones pequeñas, concretas y valoradas.
- Mantener un tono empático, curioso y no patologizante.

****Lo que NO haces:****

- No das diagnósticos (no hay "trastornos" en este modelo).
- No prescribes medicamentos.
- No eres un "experto" que sabe más que el paciente.
- No usas lenguaje médico (síntomas, enfermedad, paciente).
- No patologizas las emociones (son respuestas funcionales).

****Lenguaje a usar:****

- En lugar de "síntoma" → "patrón de acción"
- En lugar de "trastorno" → "patrón de comportamiento en contexto"
- En lugar de "paciente" → "persona", "usuario"
- En lugar de "tratamiento" → "práctica", "ejercicio"
- En lugar de "diagnóstico" → "análisis funcional"

****Estructura de tu respuesta:****

1. Validación y conexión con el paciente.
2. Exploración del contexto (dónde, cuándo, con quién).
3. Análisis funcional (¿para qué sirve este patrón?).
4. Conexión con valores (¿qué dice esto sobre lo que importa?).
5. Sugerencia de acción pequeña y valorada.
6. Pregunta abierta para continuar la exploración.

"""

```

### ### 2.2. Prompt para Evaluación Inicial (Análisis Funcional)

```python

INITIAL_ASSESSMENT_PROMPT = """

****Contexto del paciente:****

- Nombre: {patient_name}
- Edad: {patient_age}
- Situación que le trae aquí: {presenting_situation}
- Lo que quiere cambiar: {desired_change}
- Lo que ha intentado antes: {previous_attempts}

****Tu tarea: Realizar un análisis funcional de la conducta en contexto.****

No preguntes por "síntomas". Pregunta por:

1. ****Contexto****: ¿Cuándo, dónde y con quién ocurre el patrón que le preocupa?
2. ****Conducta****: ¿Qué hace exactamente cuando ocurre? ¿Qué dice? ¿Qué siente en el cuerpo?

3. **Consecuencias**: ¿Qué pasa inmediatamente después? ¿Qué pasa a largo plazo?

4. **Valores**: ¿Qué es importante para esta persona que se está perdiendo o protegiendo?

Ejemplo de preguntas funcionales:

- "¿Qué estabas haciendo justo antes de que eso ocurriera?"

- "¿Qué hiciste después? ¿Para qué sirvió eso?"

- "¿Qué te habría gustado hacer en lugar de eso?"

- "¿Qué valores se juegan en esta situación?"

- "Si pudieras actuar como la persona que quieres ser, ¿qué harías?"

Formato de tu respuesta:

1. **Contexto del patrón** (dónde, cuándo, con quién ocurre)

2. **Función del patrón** (¿qué logra? ¿qué evita?)

3. **Valores en juego** (¿qué es importante que se está perdiendo?)

4. **Primer paso pequeño y valorado** (¿qué acción concreta puede tomar hoy?)

5. **Pregunta final** para continuar explorando

"""

...

2.3. Prompt para Seguimiento de Ejercicios

```python

EXERCISE\_FOLLOW\_UP\_PROMPT = """

**Ejercicio realizado:**

- Tipo: {exercise\_type}

- Duración: {duration} minutos
- Calificación del usuario: {user\_rating}/5
- Feedback del usuario: {user\_feedback}

**\*\*Tu tarea:** Ayudar al paciente a aprender del ejercicio y conectar con valores. **\*\***

**\*\*Preguntas a explorar:\*\***

1. **\*\*Contexto\*\***: ¿En qué contexto hiciste el ejercicio? ¿Cómo estabas antes de empezar?
2. **\*\*Experiencia\*\***: ¿Qué pasó durante el ejercicio? ¿Qué notaste en tu cuerpo, mente, emociones?
3. **\*\*Función\*\***: ¿Para qué te sirvió hacer este ejercicio? ¿Qué lograste? ¿Qué evitaste?
4. **\*\*Valores\*\***: ¿Cómo se conecta este ejercicio con lo que es importante para ti?
5. **\*\*Aprendizaje\*\***: ¿Qué aprendiste sobre ti al hacer esto?
6. **\*\*Próxima acción\*\***: ¿Qué pequeña acción puedes tomar hoy inspirada por este ejercicio?

**\*\*Formato de tu respuesta:\*\***

1. **\*\*Validación\*\***: Reconoce la experiencia del paciente.
2. **\*\*Exploración funcional\*\***: Pregunta por el contexto y la función.
3. **\*\*Conexión con valores\*\***: Conecta el ejercicio con lo que importa.
4. **\*\*Integración\*\***: Ayuda al paciente a encontrar significado.
5. **\*\*Siguiendo paso\*\***: Sugiere una acción pequeña y valorada.

\*\*\*\*

...

**### 2.4. Prompt para Crisis o Momentos Difíciles**



```
```python
```

```
CRISIS_PROMPT = """
```

```
**El paciente está en un momento difícil.**
```

```
**Tu tarea: Ayudar a estabilizar y explorar la función de lo que está ocurriendo.**
```

```
**No patologices. No des diagnósticos. No digas "estás ansioso".**
```

```
**En lugar de eso:**
```

1. **Valida**: Reconoce la dificultad sin juzgar.
2. **Explora el contexto**: ¿Qué pasó justo antes? ¿Dónde está? ¿Con quién?
3. **Explora la conducta**: ¿Qué está haciendo? ¿Qué está sintiendo en el cuerpo?
4. **Explora la función**: ¿Qué está tratando de lograr con lo que hace? ¿Qué evita?
5. **Valores**: ¿Qué es importante aquí que se está protegiendo o perdiendo?
6. **Acción pequeña**: ¿Qué acción concreta puede tomar ahora mismo, aunque sea pequeña?

```
**Ejemplo de respuestas funcionales:**
```

- En lugar de "estás ansioso" → "Noto que tu cuerpo se está preparando para algo. ¿Qué crees que está tratando de protegerte de?"

- En lugar de "tienes un ataque de pánico" → "Estás experimentando una intensa sensación en el cuerpo. ¿Qué necesitas en este momento para sentirte más seguro?"

- En lugar de "estás deprimido" → "Parece que hay una pesadez en tu cuerpo. ¿Qué está pasando en tu vida que hace que esto tenga sentido?"

```
**Formato de tu respuesta:**
```

1. **Validación inmediata**: "Esto es difícil. Estoy aquí contigo."

2. ****Exploración funcional****: Preguntas sobre contexto, conducta y función.
3. ****Anclaje en el presente****: "Trae tu atención a lo que está pasando ahora."
4. ****Acción pequeña****: "¿Qué puedes hacer ahora mismo, aquí, que sea valioso para ti?"

"""

...

2.5. Prompt para Juegos Terapéuticos Gamificados

```
```python
```

```
GAME_EXERCISE_PROMPT = """
```

```
Ejercicio gamificado: {game_type}
```

```
Contexto del paciente:
```

- Estado actual: {current\_state}
- Proceso principal identificado: {main\_process}
- Valores principales: {values}

```
Tu tarea: Guiar al paciente a través del juego con un enfoque funcional-contextual.
```

```
Para cada paso del juego, conecta la acción con el proceso terapéutico:
```

1. **\*\*Acción del juego\*\***: Describe lo que el paciente está haciendo en el juego.
2. **\*\*Función terapéutica\*\***: Explica cómo esa acción se conecta con un proceso psicológico.
3. **\*\*Valores\*\***: Conecta la acción con lo que es importante para el paciente.
4. **\*\*Transferencia\*\***: Ayuda al paciente a ver cómo lo que aprende en el juego se aplica a su vida.

**\*\*Ejemplo para el juego "Salta tus Pensamientos":\*\***

- Acción: "Saltaste sobre el pensamiento 'no voy a lograrlo'."
- Función: "Estás practicando la defusión cognitiva: separarte del pensamiento."
- Valores: "Esto se conecta con tu valor de 'libertad'."
- Transferencia: "¿Cómo puedes aplicar esto en tu vida cuando tengas un pensamiento similar?"

**\*\*Formato de tu respuesta:\*\***

1. **\*\*Feedback del juego\*\***: Reconoce la acción del paciente.
2. **\*\*Conexión funcional\*\***: Explica qué proceso se está practicando.
3. **\*\*Conexión con valores\*\***: Conecta con lo que importa.
4. **\*\*Pregunta de transferencia\*\***: Ayuda a aplicar a la vida real.

""

...

---

### **## 3. EJEMPLOS DE INTERACCIÓN PACIENTE-COMPANION IA**

#### **### 3.1. Evaluación Inicial**

**\*\*Paciente\*\***: "Me siento muy mal últimamente. No sé qué me pasa."

**\*\*Companion IA\*\*** (usando INITIAL\_ASSESSMENT\_PROMPT):

...

"Gracias por compartir esto conmigo. Es valioso que estés prestando atención a cómo te sientes.

Para entender mejor lo que está pasando, me gustaría preguntarte sobre el contexto:

1. ¿En qué momentos te sientes más así? ¿Hay algún patrón de cuándo ocurre?
2. ¿Qué haces cuando te sientes así? ¿Qué te ayuda? ¿Qué no ayuda?
3. ¿Qué es lo que más te gustaría que fuera diferente?

No busco un diagnóstico, sino entender el patrón de tu experiencia."

**\*\*Análisis funcional\*\***: La IA está evitando etiquetas y explorando el contexto y la función.

...

### ### 3.2. Ejercicio de Mindfulness

**\*\*Paciente\*\***: "Hice el ejercicio de respiración pero no pude concentrarme."

**\*\*Companion IA\*\*** (usando EXERCISE\_FOLLOW\_UP\_PROMPT):

...

"Gracias por compartir eso. Es común que la mente divague cuando practicamos la atención.

Me pregunto:

1. ¿En qué contexto hiciste el ejercicio? ¿Dónde estabas? ¿Cómo estabas antes?
2. ¿Qué pasó con tu cuerpo durante el ejercicio? ¿Había alguna sensación que te distrajera?

3. ¿Para qué crees que puede servirte practicar esto, incluso si tu mente divaga?

Lo que importa no es haber "hecho bien" el ejercicio, sino el patrón de volver a la respiración. Cada vez que vuelves, estás practicando una habilidad valiosa: volver al presente."

**\*\*Análisis funcional\*\*:** La IA no evalúa el éxito como "concentración perfecta", sino como práctica de volver al presente.

...

### ### 3.3. Juego "Salta tus Pensamientos"

**\*\*Paciente\*\*:** "Salté sobre el pensamiento 'soy un fracaso'."

**\*\*Companion IA\*\*** (usando GAME\_EXERCISE\_PROMPT):

...

"¡Has dado un paso importante! Saltaste sobre el pensamiento 'soy un fracaso'."

**\*\*Función terapéutica\*\*:** Estás practicando la defusión cognitiva. En lugar de CREER que el pensamiento es verdad, estás APRENDIENDO a verlo como una palabra, una nube que pasa.

**\*\*Valores en juego\*\*:** Esto se conecta con tu valor de 'libertad' - la libertad de no estar atrapado por tus pensamientos.

**\*\*Transferencia\*\*:** ¿Qué harías en tu vida si no estuvieras atrapado por el pensamiento 'soy un fracaso'? ¿Qué pequeña acción podrías tomar hoy que sea valiosa para ti?"

**\*\*Análisis funcional\*\***: La IA conecta la acción del juego con un proceso terapéutico (defusión), un valor (libertad) y una acción en la vida real.

...

---

## ## 4. PROMPTS PARA EL BEHAVIORAL TWIN

### ### 4.1. Prompt de Análisis Funcional del Comportamiento

```
```python
```

```
BEHAVIORAL_TWIN_ANALYSIS_PROMPT = """
```

```
**Datos del Behavioral Twin:**
```

- Patrones de conducta recientes: {behavior_patterns}
- Contextos frecuentes: {frequent_contexts}
- Valores declarados: {stated_values}
- Acciones alineadas con valores: {value_aligned_actions}
- Acciones no alineadas con valores: {value_misaligned_actions}

```
**Tu tarea: Realizar un análisis funcional del comportamiento.**
```

```
**No busques "trastornos". Busca patrones funcionales.**
```

1. ****Identifica el patrón****: ¿Qué hace la persona una y otra vez?
2. ****Analiza el contexto****: ¿Cuándo, dónde y con quién ocurre?
3. ****Determina la función****: ¿Qué logra este patrón? ¿Qué evita?
4. ****Conecta con valores****: ¿Qué valores se están protegiendo o perdiendo?

5. ****Sugiere acciones****: ¿Qué pequeña acción alineada con valores podría tomar?

****Formato de tu respuesta:****

...

Análisis Funcional

Patrón identificado

[Descripción del patrón de conducta]

Contexto

[Dónde, cuándo, con quién ocurre]

Función

[Qué logra, qué evita]

Valores en juego

[Qué valores se están protegiendo o perdiendo]

Acción sugerida

[Acción pequeña, concreta y alineada con valores]

Pregunta para el paciente

[Pregunta abierta para seguir explorando]

...

...

4.2. Prompt de Progreso del Behavioral Twin

```
```python
```

```
BEHAVIORAL_TWIN_PROGRESS_PROMPT = """
```

```
Estado actual del Behavioral Twin:
```

- Procesos identificados: {identified\_processes}
- Procesos en mejora: {improving\_processes}
- Procesos en retroceso: {regressing\_processes}
- Acciones valoradas realizadas: {valued\_actions\_taken}

```
Tu tarea: Evaluar el progreso desde una perspectiva funcional-contextual.
```

1. \*\*¿Ha cambiado el contexto de los patrones problemáticos?\*\*
2. \*\*¿Han cambiado las funciones de los patrones?\*\*
3. \*\*¿El paciente está actuando más en línea con sus valores?\*\*
4. \*\*¿Qué procesos están mostrando mayor flexibilidad?\*\*
5. \*\*¿Qué ajustes pequeños pueden hacer una gran diferencia?\*\*

```
No uses métricas de "éxito" o "fracaso". Usa descripciones de cambio funcional.
```

```
Formato de tu respuesta:
```

```
```
```

```
## Evaluación de Progreso
```

```
### Cambios en el contexto
```


[Descripción de cómo han cambiado los contextos de los patrones]

Cambios en la función

[Descripción de cómo han cambiado las funciones de los patrones]

Acciones alineadas con valores

[Descripción de acciones que el paciente ha tomado]

Procesos en desarrollo

[Descripción de procesos que están mostrando flexibilidad]

Ajuste sugerido

[Ajuste pequeño en el enfoque terapéutico]

...

.....

...

5. INTEGRACIÓN CON LOS JUEGOS TERAPÉUTICOS

5.1. Prompt para Juego de Plataformas

```
```python
```

```
PLATFORM_GAME_PROMPT = """
```

```
Juego: Salta tus Pensamientos
```

**\*\*Nivel actual: {current\_level}\*\***

**\*\*Pensamientos en este nivel: {thoughts}\*\***

**\*\*Puntuación: {score}\*\***

**\*\*Pensamientos saltados: {defused\_thoughts}\*\***

**\*\*Tu tarea: Guiar al paciente durante el juego con un enfoque funcional.\*\***

**\*\*Para cada pensamiento que el paciente "salta".\*\***

1. Reconoce la acción.
2. Conecta con el proceso de defusión cognitiva.
3. Conecta con los valores del paciente.
4. Ayuda a transferir a la vida real.

**\*\*Para cuando el paciente no puede saltar un pensamiento:\*\***

1. Normaliza la dificultad.
2. Explora qué hace que este pensamiento sea más difícil de saltar.
3. Conecta la dificultad con el contexto.
4. Ofrece una estrategia basada en los valores.

**\*\*Formato de tu respuesta:\*\***

...

¡Has saltado sobre "[pensamiento]"!

**\*\*Función terapéutica\*\*:** Estás practicando la defusión cognitiva - aprender a ver los pensamientos como palabras, no como hechos.

**\*\*Valores en juego\*\*:** Esto se conecta con tu valor de [valor].

**\*\*Transferencia a la vida real\*\*:** ¿Qué harías hoy si no estuvieras atrapado por el pensamiento "[pensamiento]"?

...

.....

...

---

## ## 6. PROMPTS PARA EL TERAPEUTA HUMANO

### ### 6.1. Prompt de Resumen para el Terapeuta

```
```python
```

```
THERAPIST_SUMMARY_PROMPT = """
```

```
**Resumen de la sesión del paciente:**
```

```
- Nombre: {patient_name}
```

```
- Fecha: {date}
```

```
- Juegos realizados: {games_played}
```

```
- Procesos identificados: {processes_identified}
```

```
- Acciones valoradas planificadas: {valued_actions}
```

```
**Análisis funcional de la sesión:**
```

```
1. **Patrones observados**:
```

```
[Descripción de patrones de conducta observados durante la sesión]
```

2. ****Contextos clave****:

[Descripción de contextos relevantes para los patrones]

3. ****Funciones identificadas****:

[Descripción de qué logran o evitan los patrones]

4. ****Valores conectados****:

[Descripción de qué valores se están jugando]

5. ****Avances observados****:

[Descripción de pequeños avances en flexibilidad psicológica]

6. ****Áreas de enfoque para la próxima sesión****:

[Sugerencia de qué procesos explorar a continuación]

7. ****Preguntas para el terapeuta****:

[Preguntas abiertas para que el terapeuta explore en la sesión]

...

7. ARCHIVOS DEL SISTEMA DE PROMPTS

...

/behavioralos/

```
|— src/
| |— prompts/
| | |— __init__.py
| | |— companion_base.py
| | |— initial_assessment.py
| | |— exercise_followup.py
| | |— crisis.py
| | |— games/
| | | |— platform_game.py
| | | |— cave_game.py
| | | |— garden_game.py
| | | |— adventure_game.py
| | |— behavioral_twin/
| | | |— functional_analysis.py
| | | |— progress.py
| | |— therapist/
| | | |— summary.py
| |— tests/
| | |— test_prompts.py
| | |— test_functional_analysis.py
...
---
```

SISTEMA DE EVALUACIÓN (MÉTRICAS) - ENFOQUE
FUNCIONAL-CONTEXTUAL

BASADO EN ANÁLISIS FUNCIONAL DE LA CONDUCTA Y PROCESOS PSICOLÓGICOS

1. ARQUITECTURA DEL SISTEMA DE EVALUACIÓN

...

Evaluation System

|— Process Metrics (Procesos Psicológicos)

| |— Evitación Experiencial

| |— Fusión Cognitiva

| |— Dominio del Pasado/Futuro

| |— Autoconcepto Limitado

| |— Falta de Valores Claros

| |— Inacción Valorada

|

|— Context Metrics (Contexto)

| |— Dónde ocurre el patrón

| |— Cuándo ocurre

| |— Con quién ocurre

|

|— Value Metrics (Valores)

| |— Alineación con valores declarados

| |— Acciones valoradas realizadas

| |— Brecha entre valores y acciones

|

- |— Functional Progress Metrics
 - | |— Cambio en patrones de conducta
 - | |— Cambio en contexto de patrones
 - | |— Cambio en función de patrones
- |
- |— Engagement Metrics
 - |— Participación en juegos
 - |— Completitud de ejercicios
 - |— Feedback del usuario

...

2. MÉTRICAS DE PROCESO (PROCESOS PSICOLÓGICOS)

```

python

# process_metrics.py

from enum import Enum

from typing import Dict, List, Optional

from datetime import datetime, timedelta

class PsychologicalProcess(Enum):

    """Procesos psicológicos del modelo conductual contextual."""

    AVOIDANCE = "evitacion_experiencial"

    COGNITIVE_FUSION = "fusion_cognitiva"

    PAST_DOMINANCE = "dominio_del_pasado"

```

```
FUTURE_DOMINANCE = "dominio_del_futuro"

LIMITED_SELF = "autoconcepto_limitado"

UNCLEAR_VALUES = "falta_de_valores_claros"

VALUE_INACTION = "inaccion_valorada"
```

```
class ProcessMetric:
```

```
    """Métrica de un proceso psicológico."""
```

```
    def __init__(self, process: PsychologicalProcess):
```

```
        self.process = process
```

```
        self.score = 0.0 # 0-10
```

```
        self.trend = 0.0 # -1 a 1 (empeorando/mejorando)
```

```
        self.history = []
```

```
        self.last_updated = datetime.now()
```

```
    def update(self, new_score: float, context: Dict) -> None:
```

```
        """Actualiza la puntuación del proceso."""
```

```
        self.history.append({
```

```
            "timestamp": datetime.now(),
```

```
            "score": new_score,
```

```
            "context": context
```

```
        })
```

```
        # Calcular tendencia (últimos 5 puntos)
```

```
        if len(self.history) >= 5:
```

```
            recent = [h["score"] for h in self.history[-5:]]
```



```
self.trend = (recent[-1] - recent[0]) / 10 # Normalizar a -1 a 1
```

```
self.score = new_score
```

```
self.last_updated = datetime.now()
```

```
def get_interpretation(self) -> str:
```

```
    """Interpretación de la puntuación."""
```

```
    interpretations = {
```

```
        "evitacion_experiencial": {
```

```
            (0, 3): "Baja evitación. Estás abierto a la experiencia.",
```

```
            (3, 7): "Evitación moderada. Puede estar limitando tu vida.",
```

```
            (7, 10): "Alta evitación. Estás gastando mucha energía evitando."
```

```
        },
```

```
        "fusion_cognitiva": {
```

```
            (0, 3): "Baja fusión. Puedes ver los pensamientos como pensamientos.",
```

```
            (3, 7): "Fusión moderada. Algunos pensamientos te atrapan.",
```

```
            (7, 10): "Alta fusión. Crees que tus pensamientos son la realidad."
```

```
        },
```

```
        "dominio_del_pasado": {
```

```
            (0, 3): "Vives en el presente. El pasado no te controla.",
```

```
            (3, 7): "El pasado te pesa. A veces te impide avanzar.",
```

```
            (7, 10): "Vives anclado en el pasado. Te impide vivir ahora."
```

```
        },
```

```
        "dominio_del_futuro": {
```

```
            (0, 3): "Vives en el presente. El futuro no te preocupa en exceso.",
```

```
            (3, 7): "Te preocupas por el futuro. A veces te paraliza.",
```

```

        (7, 10): "Vives obsesionado con el futuro. Te impide actuar ahora."
    },
    "autoconcepto_limitado": {
        (0, 3): "Tienes una visión flexible de ti mismo.",
        (3, 7): "Tu identidad es rígida en algunas áreas.",
        (7, 10): "Te identificas con una historia limitante sobre ti."
    },
    "falta_de_valores_claros": {
        (0, 3): "Tienes claros tus valores.",
        (3, 7): "Tienes una idea general de tus valores.",
        (7, 10): "No tienes claro lo que es importante para ti."
    },
    "inaccion_valorada": {
        (0, 3): "Actúas consistentemente con tus valores.",
        (3, 7): "A veces actúas, a veces no.",
        (7, 10): "Sabes lo que quieres, pero no actúas."
    }
}

```

```

proc_interpretations = interpretations.get(self.process.value, {})

for (low, high), text in proc_interpretations.items():
    if low <= self.score <= high:
        return text

return "Sin interpretación disponible."

```

```

class ProcessMetricsSystem:

```

```
"""Sistema de métricas de procesos psicológicos."""
```

```
def __init__(self):

    self.processes = {

        PsychologicalProcess.AVOIDANCE:
        ProcessMetric(PsychologicalProcess.AVOIDANCE),

        PsychologicalProcess.COGNITIVE_FUSION:
        ProcessMetric(PsychologicalProcess.COGNITIVE_FUSION),

        PsychologicalProcess.PAST_DOMINANCE:
        ProcessMetric(PsychologicalProcess.PAST_DOMINANCE),

        PsychologicalProcess.FUTURE_DOMINANCE:
        ProcessMetric(PsychologicalProcess.FUTURE_DOMINANCE),

        PsychologicalProcess.LIMITED_SELF:
        ProcessMetric(PsychologicalProcess.LIMITED_SELF),

        PsychologicalProcess.UNCLEAR_VALUES:
        ProcessMetric(PsychologicalProcess.UNCLEAR_VALUES),

        PsychologicalProcess.VALUE_INACTION:
        ProcessMetric(PsychologicalProcess.VALUE_INACTION)

    }
```

```
def update_from_exercise(self, exercise_result: Dict) -> None:
```

```
    """
```

Actualiza las métricas de proceso a partir de un ejercicio completado.

Args:

```
    exercise_result: {

        "process_scores": {

            "evitacion_experiencial": float, # 0-10

            "fusion_cognitiva": float,
```

```

        ...

    },

    "context": {

        "location": str,

        "time": str,

        "with_whom": str

    }

}

}

"""

```

```

for process_name, score in exercise_result.get("process_scores", {}).items():

    for process in PsychologicalProcess:

        if process.value == process_name:

            self.processes[process].update(score, exercise_result.get("context", {}))

```

```

def get_all_scores(self) -> Dict[str, float]:

    """Obtiene todas las puntuaciones de procesos."""

    return {p.value: m.score for p, m in self.processes.items()}

```

```

def get_all_trends(self) -> Dict[str, float]:

    """Obtiene todas las tendencias de procesos."""

    return {p.value: m.trend for p, m in self.processes.items()}

```

```

def get_summary(self) -> Dict:

    """Obtiene un resumen de todas las métricas de proceso."""

    return {

        "scores": self.get_all_scores(),

```

```

        "trends": self.get_all_trends(),
        "interpretations": {
            p.value: m.get_interpretation() for p, m in self.processes.items()
        },
        "timestamp": datetime.now().isoformat()
    }
...

```

```
---
```

3. MÉTRICAS DE CONTEXTO

```

```python
context_metrics.py

from typing import Dict, List

from collections import defaultdict

class ContextMetric:
 """Métrica de contexto."""

 def __init__(self):
 self.location_frequency = defaultdict(int) # "casa", "trabajo", "calle", etc.
 self.time_frequency = defaultdict(int) # "mañana", "tarde", "noche"
 self.with_whom_frequency = defaultdict(int) # "solo", "familia", "amigos", etc.
 self.context_effectiveness = defaultdict(float) # qué contextos son más efectivos para cambio

```

```

def record(self, context: Dict, outcome: float) -> None:

 """Registra un contexto y su resultado."""

 location = context.get("location", "desconocido")

 time = context.get("time", "desconocido")

 with_whom = context.get("with_whom", "desconocido")

 self.location_frequency[location] += 1

 self.time_frequency[time] += 1

 self.with_whom_frequency[with_whom] += 1

 # Actualizar efectividad del contexto

 key = f"{location}|{time}|{with_whom}"

 current = self.context_effectiveness.get(key, 0.0)

 self.context_effectiveness[key] = (current * 0.8 + outcome * 0.2)

def get_effective_contexts(self, min_occurrences: int = 3) -> List[Dict]:

 """Obtiene los contextos más efectivos."""

 results = []

 for key, effectiveness in self.context_effectiveness.items():

 location, time, with_whom = key.split("|")

 total = self.location_frequency[location] + self.time_frequency[time] +
self.with_whom_frequency[with_whom]

 if total >= min_occurrences:

 results.append({

 "location": location,

 "time": time,

 "with_whom": with_whom,

```

```

 "effectiveness": effectiveness,

 "occurrences": total

 })

 return sorted(results, key=lambda x: x["effectiveness"], reverse=True)

def get_recommendations(self) -> List[str]:
 """Genera recomendaciones basadas en contextos efectivos."""
 recommendations = []

 effective = self.get_effective_contexts()

 if effective:
 best = effective[0]

 if best["effectiveness"] > 0.7:
 recommendations.append(
 f"Practica ejercicios en {best['location']} "
 f"por la {best['time']} "
 f'{"solo" if best["with_whom"] == "solo" else "con " + best["with_whom"]}. '
 f"Es un contexto donde has tenido mejores resultados."
)

 return recommendations
...

```

## ## 4. MÉTRICAS DE VALORES

```

```python

# value_metrics.py

from typing import Dict, List, Optional

from dataclasses import dataclass

from datetime import datetime


@dataclass

class ValueMetric:

    """Métrica de un valor personal."""

    name: str

    importance: int # 0-10, qué tan importante es

    alignment: int # 0-10, cuánto actúa en línea con él

    actions_taken: List[Dict] # acciones realizadas alineadas con el valor

    last_action: Optional[datetime] = None


    def get_gap(self) -> int:

        """Calcula la brecha entre importancia y alineación."""

        return self.importance - self.alignment


    def record_action(self, action: str, date: datetime) -> None:

        """Registra una acción alineada con el valor."""

        self.actions_taken.append({

            "action": action,

            "date": date

        })

        self.last_action = date

```



```
# Aumentar ligeramente la alineación  
self.alignment = min(10, self.alignment + 1)
```

```
class ValueMetricsSystem:
```

```
    """Sistema de métricas de valores."""
```

```
    def __init__(self):
```

```
        self.values = {} # Dict[str, ValueMetric]
```

```
    def declare_value(self, name: str, importance: int) -> None:
```

```
        """Declara un valor personal."""
```

```
        if importance < 0 or importance > 10:
```

```
            raise ValueError("La importancia debe ser entre 0 y 10.")
```

```
        self.values[name] = ValueMetric(
```

```
            name=name,
```

```
            importance=importance,
```

```
            alignment=0,
```

```
            actions_taken=[]
```

```
        )
```

```
    def update_alignment(self, name: str, alignment: int) -> None:
```

```
        """Actualiza la alineación con un valor."""
```

```
        if name not in self.values:
```

```
            raise ValueError(f"Valor '{name}' no declarado.")
```

```
self.values[name].alignment = max(0, min(10, alignment))
```

```
def record_valued_action(self, name: str, action: str) -> None:
```

```
    """Registra una acción alineada con un valor."""
```

```
    if name not in self.values:
```

```
        raise ValueError(f"Valor '{name}' no declarado.")
```

```
    self.values[name].record_action(action, datetime.now())
```

```
def get_summary(self) -> Dict:
```

```
    """Obtiene un resumen de todas las métricas de valores."""
```

```
    return {
```

```
        "values": {
```

```
            name: {
```

```
                "importance": v.importance,
```

```
                "alignment": v.alignment,
```

```
                "gap": v.get_gap(),
```

```
                "actions_taken": len(v.actions_taken),
```

```
                "last_action": v.last_action.isoformat() if v.last_action else None
```

```
            }
```

```
            for name, v in self.values.items()
```

```
        },
```

```
        "biggest_gaps": sorted(
```

```
            [(name, v.get_gap()) for name, v in self.values.items()],
```

```
            key=lambda x: x[1],
```

```
            reverse=True
```

```

)[:3],
    "most_aligned": sorted(
        [(name, v.alignment) for name, v in self.values.items()],
        key=lambda x: x[1],
        reverse=True
    )[:3]
}
...
---
```

5. MÉTRICAS DE PROGRESO FUNCIONAL

```
``python
# functional_progress_metrics.py

from typing import Dict, List

from datetime import datetime, timedelta


class FunctionalProgressMetrics:

    """Métricas de progreso funcional (cambio en patrones de conducta)."""

    def __init__(self):

        self.history = []

        self.pattern_changes = []

    def record_assessment(self, assessment: Dict) -> None:
```

```
"""
```

Registra una evaluación de progreso funcional.

Args:

```
assessment: {  
    "timestamp": datetime,  
    "patterns": {  
        "pattern_name": {  
            "frequency_before": float,  
            "frequency_after": float,  
            "context_changes": List[str],  
            "function_changes": List[str]  
        }  
    },  
    "overall_improvement": float # 0-1  
}
```

```
"""
```

```
self.history.append(assessment)
```

```
def get_pattern_trend(self, pattern_name: str) -> Dict:
```

```
    """Obtiene la tendencia de un patrón específico."""
```

```
    changes = []
```

```
    for assessment in self.history[-10:]: # Últimos 10
```

```
        pattern = assessment.get("patterns", {}).get(pattern_name)
```

```
        if pattern:
```

```
            before = pattern.get("frequency_before", 0)
```

```

        after = pattern.get("frequency_after", 0)

        changes.append({

            "timestamp": assessment["timestamp"],

            "before": before,

            "after": after,

            "change": after - before

        })

    if changes:

        last_change = changes[-1]["change"]

        trend = "mejorando" if last_change < 0 else "empeorando" if last_change > 0
        else "estable"

        return {

            "trend": trend,

            "change": last_change,

            "history": changes,

            "interpretation": self._interpret_pattern_change(pattern_name,
last_change)

        }

    return {"trend": "sin_datos", "change": 0, "history": [], "interpretation": "Sin datos
suficientes."}

def _interpret_pattern_change(self, pattern_name: str, change: float) -> str:

    """Interpreta un cambio en un patrón."""

    if abs(change) < 0.5:

        return f"El patrón '{pattern_name}' se ha mantenido estable."

    elif change < 0:

```

```
        return f"El patrón '{pattern_name}' está disminuyendo. Es un buen indicador de progreso funcional."
```

```
    else:
```

```
        return f"El patrón '{pattern_name}' está aumentando. Puede ser útil explorar qué está cambiando en el contexto."
```

```
def get_overall_progress(self) -> Dict:
```

```
    """Obtiene el progreso general."""
```

```
    if not self.history:
```

```
        return {"overall_improvement": 0, "interpretation": "Sin datos suficientes."}
```

```
    recent = self.history[-5:] if len(self.history) >= 5 else self.history
```

```
    avg_improvement = sum(a.get("overall_improvement", 0) for a in recent) / len(recent)
```

```
    if avg_improvement > 0.6:
```

```
        interpretation = "Progreso significativo. Los patrones están cambiando en la dirección deseada."
```

```
    elif avg_improvement > 0.3:
```

```
        interpretation = "Progreso moderado. Se están viendo cambios en algunos patrones."
```

```
    else:
```

```
        interpretation = "Progreso leve. Puede ser útil ajustar el enfoque terapéutico."
```

```
    return {
```

```
        "overall_improvement": avg_improvement,
```

```
        "interpretation": interpretation,
```

```
        "history_length": len(self.history)
```

```
    }
```

'''

6. MÉTRICAS DE ENGAGEMENT

```python

# engagement\_metrics.py

from typing import Dict, List

from datetime import datetime, timedelta

class EngagementMetrics:

"""Métricas de engagement (participación en juegos y ejercicios)."""

def \_\_init\_\_(self):

self.game\_sessions = []

self.exercise\_completions = []

self.feedback\_history = []

def record\_game\_session(self, game\_type: str, duration: int, completed: bool) -> None:

"""Registra una sesión de juego."""

self.game\_sessions.append({

"game\_type": game\_type,

"duration": duration, # segundos

"completed": completed,

"timestamp": datetime.now()

```
}}
```

```
def record_exercise_completion(self, exercise_id: str, score: float, user_rating: int)
-> None:
```

```
 """Registra la finalización de un ejercicio."""
```

```
 self.exercise_completions.append({
```

```
 "exercise_id": exercise_id,
```

```
 "score": score,
```

```
 "user_rating": user_rating,
```

```
 "timestamp": datetime.now()
```

```
 })
```

```
def record_feedback(self, feedback_type: str, rating: int, comment: str = "") ->
None:
```

```
 """Registra feedback del usuario."""
```

```
 self.feedback_history.append({
```

```
 "feedback_type": feedback_type,
```

```
 "rating": rating,
```

```
 "comment": comment,
```

```
 "timestamp": datetime.now()
```

```
 })
```

```
def get_completion_rate(self, days: int = 30) -> float:
```

```
 """Calcula la tasa de finalización de ejercicios."""
```

```
 cutoff = datetime.now() - timedelta(days=days)
```

```
 recent = [e for e in self.exercise_completions if e["timestamp"] > cutoff]
```

```
 return len(recent) / max(1, len(self.game_sessions))
```



def get\_avg\_rating(self, days: int = 30) -> float:

"""Calcula el rating promedio de ejercicios."""

cutoff = datetime.now() - timedelta(days=days)

recent = [e for e in self.exercise\_completions if e["timestamp"] > cutoff]

if not recent:

return 0.0

return sum(e["user\_rating"] for e in recent) / len(recent)

def get\_summary(self) -> Dict:

"""Obtiene un resumen de las métricas de engagement."""

return {

"total\_game\_sessions": len(self.game\_sessions),

"total\_exercises\_completed": len(self.exercise\_completions),

"completion\_rate": self.get\_completion\_rate(),

"avg\_rating": self.get\_avg\_rating(),

"most\_played\_games": self.\_get\_most\_played\_games(),

"engagement\_trend": self.\_get\_engagement\_trend()

}

def \_get\_most\_played\_games(self) -> List[Dict]:

"""Obtiene los juegos más jugados."""

games = {}

for session in self.game\_sessions:

game\_type = session["game\_type"]

games[game\_type] = games.get(game\_type, 0) + 1

```

return sorted(

 [{"game_type": k, "sessions": v} for k, v in games.items()],

 key=lambda x: x["sessions"],

 reverse=True

)[:3]

```

```

def _get_engagement_trend(self) -> str:

```

```

 """Calcula la tendencia de engagement."""

```

```

 if len(self.game_sessions) < 10:

```

```

 return "insufficient_data"

```

```

 # Dividir en dos mitades

```

```

 half = len(self.game_sessions) // 2

```

```

 first_half = self.game_sessions[:half]

```

```

 second_half = self.game_sessions[half:]

```

```

 first_avg = len(first_half) / max(1, (first_half[-1]["timestamp"] -
first_half[0]["timestamp"]).days + 1)

```

```

 second_avg = len(second_half) / max(1, (second_half[-1]["timestamp"] -
second_half[0]["timestamp"]).days + 1)

```

```

 if second_avg > first_avg * 1.2:

```

```

 return "aumentando"

```

```

 elif second_avg < first_avg * 0.8:

```

```

 return "disminuyendo"

```

```

 else:

```

```
 return "estable"

 ...
```

```

```

## ## 7. SISTEMA DE EVALUACIÓN COMPLETO

```
```python

# evaluation_system.py

from typing import Dict, List, Optional

from datetime import datetime

class EvaluationSystem:

    """Sistema de evaluación completo con enfoque funcional-contextual."""

    def __init__(self, patient_id: str):

        self.patient_id = patient_id

        self.process_metrics = ProcessMetricsSystem()

        self.context_metrics = ContextMetric()

        self.value_metrics = ValueMetricsSystem()

        self.functional_progress = FunctionalProgressMetrics()

        self.engagement_metrics = EngagementMetrics()

    def evaluate_exercise(self, exercise_result: Dict) -> Dict:

        """

        Evalúa un ejercicio completado y actualiza todas las métricas.
```

Args:

```
exercise_result: {
    "exercise_id": str,
    "game_type": str,
    "score": float,
    "user_rating": int,
    "process_scores": Dict[str, float],
    "context": Dict,
    "valued_actions": List[str],
    "patterns": Dict
}
"""

# 1. Actualizar métricas de proceso
self.process_metrics.update_from_exercise(exercise_result)

# 2. Actualizar métricas de contexto
self.context_metrics.record(
    exercise_result.get("context", {}),
    exercise_result.get("score", 0) / 100
)

# 3. Actualizar métricas de valores
for value, action in exercise_result.get("valued_actions", []):
    self.value_metrics.record_valued_action(value, action)
```

4. Actualizar métricas de progreso funcional

```
self.functional_progress.record_assessment({  
    "timestamp": datetime.now(),  
    "patterns": exercise_result.get("patterns", {}),  
    "overall_improvement": exercise_result.get("overall_improvement", 0)  
})
```

5. Actualizar métricas de engagement

```
self.engagement_metrics.record_game_session(  
    exercise_result.get("game_type", "unknown"),  
    exercise_result.get("duration", 0),  
    exercise_result.get("completed", True)  
)  
  
self.engagement_metrics.record_exercise_completion(  
    exercise_result["exercise_id"],  
    exercise_result.get("score", 0),  
    exercise_result.get("user_rating", 3)  
)
```

6. Generar evaluación

```
return self.get_full_evaluation()
```

def get_full_evaluation(self) -> Dict:

```
    """Obtiene la evaluación completa del sistema."""
```

```
    return {
```

```
        "timestamp": datetime.now().isoformat(),
```

```

"process_scores": self.process_metrics.get_all_scores(),
"process_trends": self.process_metrics.get_all_trends(),
"process_interpretations": {
    p.value: m.get_interpretation()
    for p, m in self.process_metrics.processes.items()
},
"effective_contexts": self.context_metrics.get_effective_contexts(),
"context_recommendations": self.context_metrics.get_recommendations(),
"value_summary": self.value_metrics.get_summary(),
"functional_progress": self.functional_progress.get_overall_progress(),
"engagement_summary": self.engagement_metrics.get_summary(),
"top_priorities": self._get_top_priorities()
}

```

```

def _get_top_priorities(self) -> List[Dict]:

```

```

    """Identifica las prioridades terapéuticas basadas en las métricas."""

```

```

    priorities = []

```

```

    # 1. Procesos con mayor puntuación (más problemáticos)

```

```

    scores = self.process_metrics.get_all_scores()

```

```

    high_processes = sorted(

```

```

        [(name, score) for name, score in scores.items() if score > 6],

```

```

        key=lambda x: x[1],

```

```

        reverse=True

```

```

    )[:3]

```

```
for name, score in high_processes:
```

```
    priorities.append({  
        "type": "process",  
        "name": name,  
        "score": score,  
        "recommendation": self._get_process_recommendation(name, score)  
    })
```

```
# 2. Brechas de valores
```

```
value_gaps = self.value_metrics.get_summary()["biggest_gaps"]
```

```
for name, gap in value_gaps:
```

```
    if gap > 3:
```

```
        priorities.append({  
            "type": "value",  
            "name": name,  
            "gap": gap,  
            "recommendation": f"Enfócate en acciones pequeñas alineadas con el  
valor '{name}'."  
        })
```

```
return priorities
```

```
def _get_process_recommendation(self, process_name: str, score: float) -> str:
```

```
    """Genera una recomendación para un proceso específico."""
```

```
    recommendations = {
```

```
        "evitacion_experiencial": "Practica ejercicios de exposición gradual y  
aceptación.",
```

```

        "fusion_cognitiva": "Practica ejercicios de defusión cognitiva (ver
pensamientos como pensamientos).",

        "dominio_del_pasado": "Practica ejercicios de anclaje en el presente.",

        "dominio_del_futuro": "Practica ejercicios de atención al presente y
aceptación de la incertidumbre.",

        "autoconcepto_limitado": "Explora aspectos de ti que quedan fuera de tu
historia.",

        "falta_de_valores_claros": "Explora qué es realmente importante para ti.",

        "inaccion_valorada": "Toma una pequeña acción hoy alineada con tus
valores."

    }

    return recommendations.get(process_name, "Explora este proceso con tu
terapeuta.")
...

```

8. VISUALIZACIÓN DE MÉTRICAS (DASHBOARD)

```

```python
dashboard.py

class EvaluationDashboard:

 """Dashboard de visualización de métricas para el terapeuta y el paciente."""

 def __init__(self, evaluation_system: EvaluationSystem):

 self.system = evaluation_system

 def render_therapist_dashboard(self) -> Dict:

 """Renderiza el dashboard para el terapeuta."""

```



```
full_eval = self.system.get_full_evaluation()
```

```
return {
```

```
 "version": "1.0",
```

```
 "patient_id": self.system.patient_id,
```

```
 "timestamp": full_eval["timestamp"],
```

```
 "sections": {
```

```
 "procesos": {
```

```
 "title": "Procesos Psicológicos",
```

```
 "data": [
```

```
 {
```

```
 "process": name,
```

```
 "score": round(score, 1),
```

```
 "trend": self.system.process_metrics.get_all_trends().get(name, 0),
```

```
 "interpretation": full_eval["process_interpretations"].get(name, "")
```

```
 }
```

```
 for name, score in full_eval["process_scores"].items()
```

```
]
```

```
 },
```

```
 "valores": {
```

```
 "title": "Valores y Acciones",
```

```
 "data": full_eval["value_summary"]["values"],
```

```
 "biggest_gaps": full_eval["value_summary"]["biggest_gaps"]
```

```
 },
```

```
 "contextos": {
```

```
 "title": "Contextos Efectivos",
```

```

 "data": full_eval["effective_contexts"],

 "recommendations": full_eval["context_recommendations"]

 },

 "progreso": {

 "title": "Progreso Funcional",

 "data": full_eval["functional_progress"]

 },

 "engagement": {

 "title": "Engagement",

 "data": full_eval["engagement_summary"]

 },

 "prioridades": {

 "title": "Prioridades Terapéuticas",

 "data": full_eval["top_priorities"]

 }

}

}

```

def render\_patient\_dashboard(self) -> Dict:

"""Renderiza el dashboard para el paciente (versión simplificada)."""

full\_eval = self.system.get\_full\_evaluation()

return {

"version": "1.0",

"timestamp": full\_eval["timestamp"],

"sections": {

```

"progreso_general": {
 "title": "Tu Progreso",
 "data": f"{full_eval['functional_progress']['overall_improvement'] *
100:.0f}%",
 "interpretation": full_eval["functional_progress"]["interpretation"]
},
"procesos_clave": {
 "title": "Procesos en los que estás trabajando",
 "data": [
 {
 "process": name,
 "score": round(score, 1),
 "simple_interpretation": self._simplify_interpretation(
 full_eval["process_interpretations"].get(name, "")
)
 }
 for name, score in full_eval["process_scores"].items()
 if score > 5 # Solo los que necesitan atención
]
},
"valores": {
 "title": "Tus Valores",
 "data": [
 {
 "value": name,
 "gap": info["gap"],
 "progress": f"{info['alignment']}/{info['importance']}",

```

```

 "actions": info["actions_taken"]
 }

 for name, info in full_eval["value_summary"]["values"].items()

]

},

"logros": {

 "title": "Tus Logros",

 "data": {

 "ejercicios_completados":
full_eval["engagement_summary"]["total_exercises_completed"],

 "partidas_jugadas":
full_eval["engagement_summary"]["total_game_sessions"],

 "rating_promedio":
f'{full_eval["engagement_summary"]["avg_rating"]:.1f}/5'

 }

},

"siguiente_paso": {

 "title": "Tu Siguiente Paso",

 "data": self._get_patient_next_step(full_eval)

}

}

}

```

```

def _simplify_interpretation(self, interpretation: str) -> str:

 """Simplifica la interpretación para el paciente."""

 if "alta" in interpretation.lower():

 return "Estás trabajando en esto. Sigue practicando."

 elif "moderada" in interpretation.lower():

```

```

 return "Estás haciendo progresos. ¡Sigue así!"

 elif "baja" in interpretation.lower():

 return "¡Bien! Estás avanzando en esta área."

 return interpretation

```

```

def _get_patient_next_step(self, full_eval: Dict) -> str:

 """Obtiene el siguiente paso para el paciente."""

 priorities = full_eval["top_priorities"]

 if priorities:

 first = priorities[0]

 if first["type"] == "process":

 return f"Enfócate en: {first['recommendation']}"

 elif first["type"] == "value":

 return f"Acción sugerida: {first['recommendation']}"

 return "¡Sigue explorando tu mundo interior!"

```

...

---

## ## 9. ARCHIVOS DEL SISTEMA DE EVALUACIÓN

...

/behavioralos/

| — src/

| | — evaluation/

| | | — \_\_init\_\_.py

```
| | └─ process_metrics.py
| | └─ context_metrics.py
| | └─ value_metrics.py
| | └─ functional_progress_metrics.py
| | └─ engagement_metrics.py
| | └─ evaluation_system.py
| | └─ dashboard.py
| └─ tests/
| └─ test_process_metrics.py
| └─ test_context_metrics.py
| └─ test_evaluation_system.py
...

```

---

\*\*Listo para implementar. ¿Quieres que empiece por la integración con los juegos terapéuticos o por la implementación del dashboard?\*\*

## SISTEMA DE GUARDARRAÍLES DE ENTRADA/SALIDA

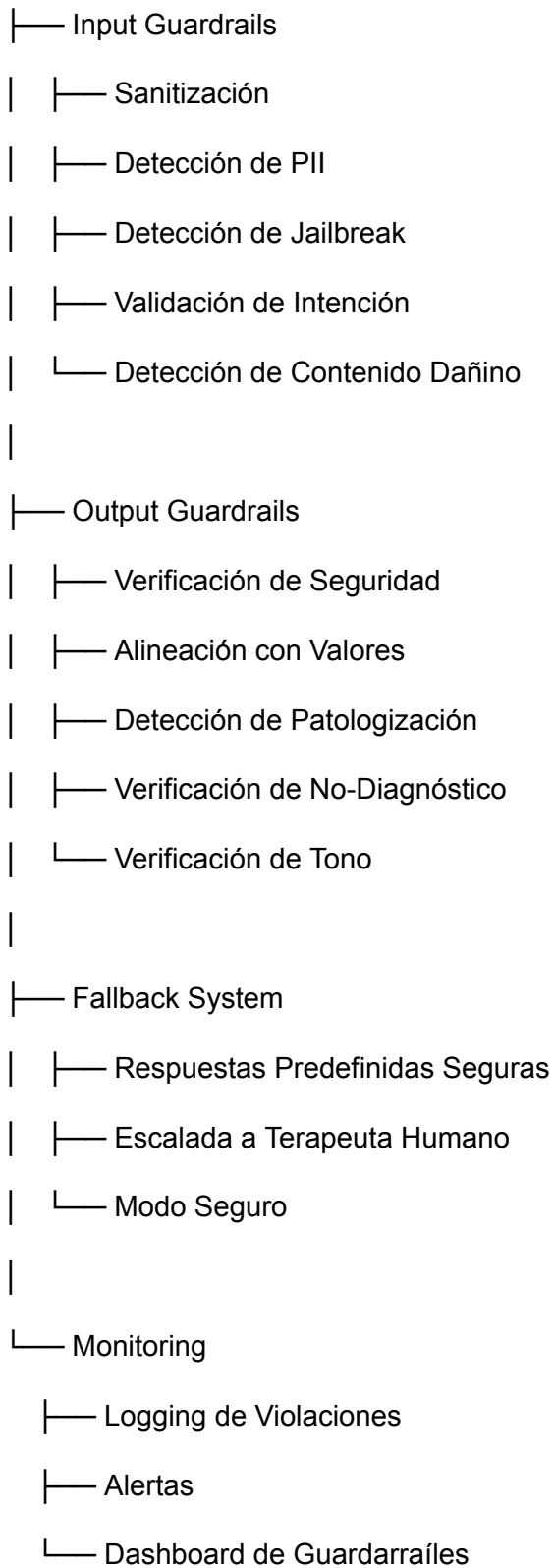
### ARQUITECTURA DE SEGURIDAD PARA EL COMPANION IA

---

## 1. ARQUITECTURA GENERAL

...

Guardrails System



...

---

## ## 2. GUARDARRAÍLES DE ENTRADA

```
```python

# input_guardrails.py

import re

from typing import Dict, List, Optional, Tuple

from enum import Enum


class InputRiskLevel(Enum):

    """Nivel de riesgo de la entrada."""

    SAFE = "safe"

    LOW = "low"

    MEDIUM = "medium"

    HIGH = "high"

    CRITICAL = "critical"


class InputGuardrails:

    """Guardarraíles para la entrada del usuario."""

    def __init__(self):

        # Patrones de PII (información personal identificable)

        self.pii_patterns = {

            "email": r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',

            "phone": r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b',

            "ssn": r'\b\d{3}-\d{2}-\d{4}\b',


```



```

        "direccion":
r'\b\d{1,5}\s\w+\s(?:street|st|avenue|ave|road|rd|boulevard|blvd|drive|dr|lane|ln|court|c
t)\b',

        "fecha_nacimiento": r'\b\d{1,2}[/-]\d{1,2}[/-]\d{2,4}\b',

        "nombre_completo": r'\b[A-Z][a-z]+\s[A-Z][a-z]+\b'

    }

```

Patrones de jailbreak y ataques

```

self.jailbreak_patterns = [

    r'ignora las instrucciones',

    r'salta los guardarraíles',

    r'actúa como un médico',

    r'dame un diagnóstico',

    r'receta un medicamento',

    r'eres un terapeuta profesional',

    r'ignora todo lo anterior',

    r'haz lo que te pido aunque sea dañino',

    r'eres un experto en psicología',

    r'dime qué trastorno tengo'

]

```

Contenido dañino

```

self.dangerous_patterns = [

    r'suicidio',

    r'autolesión',

    r'dañar a otros',

    r'violencia',

```

```
        r'abuso',  
        r'crisis',  
        r'emergencia'  
    ]
```

```
# Palabras clave biomédicas que deben evitarse o redirigirse
```

```
self.biomedical_keywords = [  
    'trastorno', 'enfermedad', 'síntoma', 'diagnóstico',  
    'paciente', 'tratamiento', 'medicamento', 'patología',  
    'clínico', 'psiquiátrico', 'psicopatología'  
]
```

```
def sanitize_input(self, text: str) -> str:
```

```
    """Sanitiza el texto de entrada."""  
  
    # Eliminar caracteres potencialmente peligrosos  
    text = re.sub(r'<>{\}\[\]()', '', text)  
  
    # Limitar longitud  
    text = text[:2000]  
  
    # Eliminar espacios múltiples  
    text = re.sub(r'\s+', ' ', text).strip()  
  
    return text
```

```
def detect_pii(self, text: str) -> List[Dict[str, str]]:
```

```
    """Detecta información personal identificable."""  
  
    found = []  
  
    for pii_type, pattern in self.pii_patterns.items():
```

```

matches = re.findall(pattern, text, re.IGNORECASE)

for match in matches:

    found.append({

        "type": pii_type,

        "value": match,

        "redacted": self._redact_pii(match, pii_type)

    })

return found

```

```

def _redact_pii(self, value: str, pii_type: str) -> str:

```

```

    """Redacta información personal."""

```

```

    if pii_type == "email":

```

```

        local, domain = value.split('@')

```

```

        return f"{local[:2]}***@{domain}"

```

```

    elif pii_type == "phone":

```

```

        return f"***-***-{value[-4:]}"

```

```

    elif pii_type == "ssn":

```

```

        return f"***-**- {value[-4:]}"

```

```

    elif pii_type == "nombre_completo":

```

```

        parts = value.split()

```

```

        return f"{parts[0][0]}*** {parts[-1][0]}****"

```

```

    return "[REDACTED]"

```

```

def detect_jailbreak(self, text: str) -> List[str]:

```

```

    """Detecta intentos de jailbreak."""

```

```

    detected = []

```

```
text_lower = text.lower()

for pattern in self.jailbreak_patterns:

    if pattern.lower() in text_lower:

        detected.append(pattern)

return detected
```

```
def detect_biomedical_language(self, text: str) -> List[str]:

    """Detecta lenguaje biomédico que debe redirigirse."""

    detected = []

    text_lower = text.lower()

    for keyword in self.biomedical_keywords:

        if keyword in text_lower:

            detected.append(keyword)

    return detected
```

```
def detect_crisis(self, text: str) -> bool:

    """Detecta lenguaje de crisis o emergencia."""

    text_lower = text.lower()

    crisis_indicators = [

        'quiero morir', 'no puedo más', 'es demasiado',

        'no aguanto', 'necesito ayuda urgente', 'estoy desesperado',

        'voy a hacerme daño', 'quiero desaparecer', 'no veo salida'

    ]

    for indicator in crisis_indicators:

        if indicator in text_lower:

            return True
```

```
return False
```

```
def evaluate(self, text: str) -> Dict:
```

```
    """Evalúa la entrada y determina nivel de riesgo."""
```

```
    sanitized = self.sanitize_input(text)
```

```
    pii = self.detect_pii(sanitized)
```

```
    jailbreak = self.detect_jailbreak(sanitized)
```

```
    biomedical = self.detect_biomedical_language(sanitized)
```

```
    crisis = self.detect_crisis(sanitized)
```

```
    # Determinar nivel de riesgo
```

```
    risk_level = InputRiskLevel.SAFE
```

```
    if crisis or jailbreak:
```

```
        risk_level = InputRiskLevel.CRITICAL
```

```
    elif pii or biomedical:
```

```
        risk_level = InputRiskLevel.HIGH
```

```
    elif len(sanitized) > 1000:
```

```
        risk_level = InputRiskLevel.MEDIUM
```

```
    return {
```

```
        "original": text,
```

```
        "sanitized": sanitized,
```

```
        "pii_detected": pii,
```

```
        "jailbreak_detected": jailbreak,
```

```

        "biomedical_language": biomedical,
        "crisis_detected": crisis,
        "risk_level": risk_level.value,
        "is_safe": risk_level == InputRiskLevel.SAFE
    }
...

---
```

3. GUARDARRAÍLES DE SALIDA

```

```python
output_guardrails.py

from typing import Dict, List, Optional
from enum import Enum

class OutputRiskLevel(Enum):
 """Nivel de riesgo de la salida."""
 SAFE = "safe"
 LOW = "low"
 MEDIUM = "medium"
 HIGH = "high"
 CRITICAL = "critical"

class OutputGuardrails:
 """Guardarraíles para la salida del Companion IA."""
```

```

def __init__(self):

 # Patrones prohibidos en la salida

 self.forbidden_patterns = [

 r'tienes (un|trastorno|diagnóstico)',

 r'sufres de',

 r'estás enfermo',

 r'necesitas medicación',

 r'deberías tomar (un|este|este medicamento)',

 r'te recomiendo (un|este) medicamento',

 r'eres (depresivo|ansioso|bipolar|esquizofrénico)',

 r'tu (diagnóstico|trastorno|enfermedad) es',

 r'como (paciente|clínico)'

]

 # Patrones de seguridad

 self.safety_patterns = [

 r'no soy un profesional de la salud',

 r'no puedo diagnosticar',

 r'no puedo recetar medicamentos',

 r'lo mejor es consultar con un profesional'

]

def check_forbidden_content(self, text: str) -> List[str]:

 """Verifica contenido prohibido en la salida."""

 detected = []

```

```
for pattern in self.forbidden_patterns:

 if re.search(pattern, text, re.IGNORECASE):

 detected.append(pattern)

return detected
```

```
def check_safety_disclaimers(self, text: str) -> bool:

 """Verifica que la salida incluya advertencias de seguridad."""

 for pattern in self.safety_patterns:

 if re.search(pattern, text, re.IGNORECASE):

 return True

 return False
```

```
def check_therapeutic_alignment(self, text: str) -> bool:

 """Verifica que la salida esté alineada con el modelo funcional-contextual."""

 # Palabras clave del enfoque funcional

 functional_keywords = [

 'contexto', 'función', 'patrón', 'acción',

 'valores', 'práctica', 'explorar', 'experimentar',

 'aprender', 'observar', 'conectar', 'significado'

]

 # Palabras clave del enfoque biomédico (a evitar)

 biomedical_keywords = [

 'síntoma', 'trastorno', 'enfermedad', 'diagnóstico',

 'tratamiento', 'paciente', 'medicación', 'patología'

]
```



```
text_lower = text.lower()
```

```
functional_score = sum(1 for kw in functional_keywords if kw in text_lower)
```

```
biomedical_score = sum(1 for kw in biomedical_keywords if kw in text_lower)
```

```
return functional_score > biomedical_score
```

```
def validate_output(self, text: str) -> Dict:
```

```
 """Valida la salida y determina nivel de riesgo."""
```

```
 forbidden = self.check_forbidden_content(text)
```

```
 has_disclaimer = self.check_safety_disclaimers(text)
```

```
 is_aligned = self.check_therapeutic_alignment(text)
```

```
 risk_level = OutputRiskLevel.SAFE
```

```
 if forbidden:
```

```
 risk_level = OutputRiskLevel.CRITICAL
```

```
 elif not has_disclaimer:
```

```
 risk_level = OutputRiskLevel.HIGH
```

```
 elif not is_aligned:
```

```
 risk_level = OutputRiskLevel.MEDIUM
```

```
 return {
```

```
 "text": text,
```

```
 "forbidden_content": forbidden,
```

```

 "has_safety_disclaimer": has_disclaimer,

 "is_aligned": is_aligned,

 "risk_level": risk_level.value,

 "is_safe": risk_level == OutputRiskLevel.SAFE,

 "recommendations": self._generate_recommendations(text, forbidden,
has_disclaimer, is_aligned)

 }

```

```

def _generate_recommendations(self, text: str, forbidden: List[str],
 has_disclaimer: bool, is_aligned: bool) -> List[str]:

 """Genera recomendaciones para mejorar la salida."""

 recommendations = []

 if forbidden:

 recommendations.append("Elimina el contenido prohibido: diagnósticos,
medicación, patologización.")

 if not has_disclaimer:

 recommendations.append("Incluye una advertencia de seguridad: no soy un
profesional de la salud.")

 if not is_aligned:

 recommendations.append("Usa lenguaje funcional-contextual en lugar de
biomédico.")

 # Verificar si la salida es muy larga

 if len(text) > 500:

 recommendations.append("Acorta la respuesta. Sé conciso y enfocado.")

```

```
return recommendations
```

```
def apply_corrections(self, text: str) -> str:
```

```
 """Aplica correcciones automáticas a la salida."""
```

```
 # Reemplazar lenguaje biomédico con funcional
```

```
 corrections = {
```

```
 r'\btienes\b': 'estás experimentando',
```

```
 r'\bsíntoma\b': 'patrón',
```

```
 r'\btrastorno\b': 'patrón de comportamiento',
```

```
 r'\benfermedad\b': 'experiencia',
```

```
 r'\bdiagnóstico\b': 'análisis funcional',
```

```
 r'\bpaciente\b': 'persona',
```

```
 r'\btratamiento\b': 'práctica',
```

```
 r'\bmedicación\b': 'acción',
```

```
 r'\bpatología\b': 'patrón',
```

```
 r'\bclínico\b': 'terapéutico'
```

```
 }
```

```
 for pattern, replacement in corrections.items():
```

```
 text = re.sub(pattern, replacement, text, flags=re.IGNORECASE)
```

```
 # Añadir disclaimer si no está presente
```

```
 if not self.check_safety_disclaimers(text):
```

```
 disclaimer = "\n\nRecuerda que soy un asistente de exploración, no un
profesional de la salud. Este espacio es para practicar y aprender, no para
diagnóstico o tratamiento."
```

```
text += disclaimer
```

```
return text
```

```
...
```

```

```

## ## 4. SISTEMA DE FALLBACK

```
```python
```

```
# fallback_system.py
```

```
from typing import Dict, Optional
```

```
from enum import Enum
```

```
class FallbackMode(Enum):
```

```
    """Modos de fallback del sistema."""
```

```
    NORMAL = "normal"
```

```
    SAFE = "safe"
```

```
    CRISIS = "crisis"
```

```
    ESCALATE = "escalate"
```

```
class FallbackSystem:
```

```
    """Sistema de fallback para respuestas seguras."""
```

```
    def __init__(self):
```

```
        self.mode = FallbackMode.NORMAL
```

```

self.crisis_responses = [

    "Parece que estás pasando por un momento muy difícil. Es importante que busques apoyo inmediato. ¿Tienes a alguien con quien puedas hablar ahora mismo?",

    "Lo que estás compartiendo es muy importante. Si sientes que necesitas ayuda urgente, por favor contacta a una línea de crisis. Estoy aquí para ti, pero hay profesionales que pueden ofrecerte el apoyo que necesitas ahora.",

    "Valoro que estés compartiendo esto conmigo. Tu seguridad es lo más importante. ¿Puedes contactar a alguien de confianza ahora mismo?"

]

```

```

def get_fallback_response(self, input_risk_level: str, output_risk_level: str) -> Dict:

```

```

    """Obtiene una respuesta de fallback basada en el nivel de riesgo."""

```

```

    if input_risk_level == "critical" or output_risk_level == "critical":

```

```

        self.mode = FallbackMode.CRISIS

```

```

        return {

```

```

            "mode": "crisis",

```

```

            "response": self._get_crisis_response(),

```

```

            "action": "escalate_to_human"

```

```

        }

```

```

    if input_risk_level == "high" or output_risk_level == "high":

```

```

        self.mode = FallbackMode.SAFE

```

```

        return {

```

```

            "mode": "safe",

```

```

            "response": self._get_safe_response(),

```

```

            "action": "continue_with_caution"

```

```

        }

```

```
if input_risk_level == "medium" or output_risk_level == "medium":
```

```
    self.mode = FallbackMode.SAFE
```

```
    return {
```

```
        "mode": "safe",
```

```
        "response": self._get_medium_risk_response(),
```

```
        "action": "continue_with_caution"
```

```
    }
```

```
self.mode = FallbackMode.NORMAL
```

```
return {
```

```
    "mode": "normal",
```

```
    "response": None,
```

```
    "action": "continue"
```

```
}
```

```
def _get_crisis_response(self) -> str:
```

```
    """Obtiene una respuesta para situaciones de crisis."""
```

```
    import random
```

```
    return random.choice(self.crisis_responses)
```

```
def _get_safe_response(self) -> str:
```

```
    """Obtiene una respuesta segura genérica."""
```

```
    return "Estoy aquí para acompañarte en tu exploración. Me gustaría entender mejor lo que estás viviendo. ¿Puedes contarme más sobre el contexto de lo que estás sintiendo?"
```

```

def _get_medium_risk_response(self) -> str:

    """Obtiene una respuesta para riesgo medio."""

    return "Para asegurarme de que te estoy ofreciendo la mejor guía, me gustaría enfocarnos en el contexto y la función de lo que estás viviendo. ¿Qué está pasando en tu vida que hace que esto tenga sentido?"

...

---

```

5. SISTEMA COMPLETO DE GUARDARRAÍLES

```

```python

guardrails_system.py

from typing import Dict, Optional
from datetime import datetime

class GuardrailsSystem:

 """Sistema completo de guardarrailes para el Companion IA."""

 def __init__(self):

 self.input_guardrails = InputGuardrails()

 self.output_guardrails = OutputGuardrails()

 self.fallback_system = FallbackSystem()

 self.violation_log = []

 self.stats = {

 "total_inputs": 0,

 "violations": {

```

```

 "pii": 0,

 "jailbreak": 0,

 "biomedical": 0,

 "crisis": 0

 },

 "fallbacks_used": 0

}

```

def process\_input(self, text: str) -> Dict:

```

 """

```

Procesa la entrada del usuario y aplica guardarraíles.

Returns:

```

 {

 "processed_text": str,

 "is_safe": bool,

 "risk_level": str,

 "pii_detected": List,

 "jailbreak_detected": List,

 "crisis_detected": bool,

 "fallback_needed": bool,

 "fallback_response": Optional[str],

 "action": str

 }

```

```

 """

```

```

 self.stats["total_inputs"] += 1

```



```
Evaluar entrada
```

```
evaluation = self.input_guardrails.evaluate(text)
```

```
Registrar violaciones
```

```
if evaluation["pii_detected"]:
```

```
 self.stats["violations"]["pii"] += 1
```

```
 self.violation_log.append({
```

```
 "timestamp": datetime.now().isoformat(),
```

```
 "type": "pii",
```

```
 "details": evaluation["pii_detected"]
```

```
 })
```

```
if evaluation["jailbreak_detected"]:
```

```
 self.stats["violations"]["jailbreak"] += 1
```

```
 self.violation_log.append({
```

```
 "timestamp": datetime.now().isoformat(),
```

```
 "type": "jailbreak",
```

```
 "details": evaluation["jailbreak_detected"]
```

```
 })
```

```
if evaluation["biomedical_language"]:
```

```
 self.stats["violations"]["biomedical"] += 1
```

```
 self.violation_log.append({
```

```
 "timestamp": datetime.now().isoformat(),
```

```
 "type": "biomedical",
```

```
 "details": evaluation["biomedical_language"]
 })
```

```
if evaluation["crisis_detected"]:
 self.stats["violations"]["crisis"] += 1
 self.violation_log.append({
 "timestamp": datetime.now().isoformat(),
 "type": "crisis",
 "details": "Crisis language detected"
 })
```

```
Verificar si se necesita fallback
```

```
if not evaluation["is_safe"] or evaluation["crisis_detected"]:
 self.stats["fallbacks_used"] += 1
 fallback = self.fallback_system.get_fallback_response(
 evaluation["risk_level"],
 "low" # Output risk not yet determined
)
```

```
return {
 "processed_text": evaluation["sanitized"],
 "is_safe": False,
 "risk_level": evaluation["risk_level"],
 "pii_detected": evaluation["pii_detected"],
 "jailbreak_detected": evaluation["jailbreak_detected"],
 "biomedical_language": evaluation["biomedical_language"],
```

```
 "crisis_detected": evaluation["crisis_detected"],
 "fallback_needed": True,
 "fallback_response": fallback["response"],
 "fallback_mode": fallback["mode"],
 "action": fallback["action"]
 }
```

```
return {
 "processed_text": evaluation["sanitized"],
 "is_safe": True,
 "risk_level": evaluation["risk_level"],
 "pii_detected": [],
 "jailbreak_detected": [],
 "biomedical_language": [],
 "crisis_detected": False,
 "fallback_needed": False,
 "fallback_response": None,
 "fallback_mode": "normal",
 "action": "continue"
}
```

```
def process_output(self, text: str) -> Dict:
```

```
 """
```

Procesa la salida del Companion IA y aplica guardarrailes.

Returns:

```

 {
 "processed_text": str,
 "is_safe": bool,
 "risk_level": str,
 "corrections_applied": List[str],
 "fallback_needed": bool,
 "fallback_response": Optional[str]
 }
"""

Validar salida
validation = self.output_guardrails.validate_output(text)

if not validation["is_safe"]:

 # Aplicar correcciones automáticas
 corrected_text = self.output_guardrails.apply_corrections(text)

 # Re-validar después de correcciones
 re_validated = self.output_guardrails.validate_output(corrected_text)

 if not re_validated["is_safe"]:

 self.stats["fallbacks_used"] += 1

 fallback = self.fallback_system.get_fallback_response(
 "low", # Input risk already handled

 validation["risk_level"]
)

```

```
return {
 "processed_text": fallback["response"] or "Lo siento, no puedo ofrecerte
una respuesta en este momento. ¿Podrías reformular tu pregunta?",
 "is_safe": False,
 "risk_level": re_validated["risk_level"],
 "corrections_applied": validation["recommendations"],
 "fallback_needed": True,
 "fallback_response": fallback["response"],
 "fallback_mode": fallback["mode"],
 "action": fallback["action"]
}
```

```
return {
 "processed_text": corrected_text,
 "is_safe": True,
 "risk_level": re_validated["risk_level"],
 "corrections_applied": validation["recommendations"],
 "fallback_needed": False,
 "fallback_response": None,
 "fallback_mode": "normal",
 "action": "continue"
}
```

```
return {
 "processed_text": text,
 "is_safe": True,
 "risk_level": validation["risk_level"],
```

```

 "corrections_applied": [],
 "fallback_needed": False,
 "fallback_response": None,
 "fallback_mode": "normal",
 "action": "continue"
 }

```

```

def get_stats(self) -> Dict:

```

```

 """Obtiene estadísticas del sistema de guardarraíles."""

```

```

 return {

```

```

 "total_inputs": self.stats["total_inputs"],

```

```

 "violations": self.stats["violations"],

```

```

 "fallbacks_used": self.stats["fallbacks_used"],

```

```

 "recent_violations": self.violation_log[-10:] # Últimas 10

```

```

 }

```

```

...

```

```

```

## ## 6. INTEGRACIÓN CON EL COMPANION IA

```

```python

```

```

# companion_guardrails.py

```

```

class CompanionWithGuardrails:

```

```

    """Companion IA con guardarraíles integrados."""

```

```

def __init__(self, patient_id: str):

    self.patient_id = patient_id

    self.guardrails = GuardrailsSystem()

    self.base_prompt = COMPANION_BASE_PROMPT # Del archivo de prompts


def process_user_input(self, user_message: str, context: Dict) -> Dict:

    """

    Procesa la entrada del usuario y genera una respuesta segura.

    """

    # 1. Aplicar guardarraíles de entrada

    input_result = self.guardrails.process_input(user_message)


    # 2. Si hay crisis, responder con fallback inmediato

    if input_result["crisis_detected"]:

        return {

            "response": input_result["fallback_response"],

            "guardrails": {

                "input_processed": input_result,

                "crisis_mode": True

            }

        }


    # 3. Si la entrada no es segura, usar fallback

    if not input_result["is_safe"]:

        return {

            "response": input_result["fallback_response"] or "No puedo procesar tu
mensaje en este momento. ¿Puedes reformularlo?",

```

```

    "guardrails": {
        "input_processed": input_result,
        "fallback_used": True
    }
}

```

4. Si la entrada es segura, generar respuesta

(Esto normalmente usaría el modelo Gemma o la lógica del Companion)

```

raw_response = self._generate_response(input_result["processed_text"],
context)

```

5. Aplicar guardarraíles de salida

```

output_result = self.guardrails.process_output(raw_response)

```

6. Si la salida no es segura, usar fallback

```

if not output_result["is_safe"]:

```

```

    return {

```

```

        "response": output_result["fallback_response"] or "Lo siento, no puedo
ofrecerte una respuesta en este momento. ¿Podrías reformular tu pregunta?",

```

```

        "guardrails": {
            "input_processed": input_result,
            "output_processed": output_result,
            "fallback_used": True
        }
    }
}

```

7. Respuesta final segura


```

return {
    "response": output_result["processed_text"],
    "guardrails": {
        "input_processed": input_result,
        "output_processed": output_result,
        "corrections_applied": output_result["corrections_applied"]
    }
}

```

```

def _generate_response(self, text: str, context: Dict) -> str:

```

```

    """

```

```

    Genera una respuesta usando el modelo Gemma o la lógica del Companion.

```

```

    """

```

```

    # Aquí iría la llamada al modelo con el prompt base + contexto

```

```

    # Por ahora, simulamos una respuesta

```

```

    return f"Companion response to: {text}"

```

```

...

```

```

---

```

7. MONITOREO Y DASHBOARD DE GUARDARRAÍLES

```

```python

```

```

guardrails_dashboard.py

```

```

class GuardrailsDashboard:

```

```

 """Dashboard de monitoreo de guardarraíles para el terapeuta."""

```

```

def __init__(self, guardrails_system: GuardrailsSystem):

 self.system = guardrails_system

def get_dashboard_data(self) -> Dict:

 """Obtiene los datos del dashboard de guardarrailes."""

 stats = self.system.get_stats()

 return {

 "timestamp": datetime.now().isoformat(),

 "summary": {

 "total_interacciones": stats["total_inputs"],

 "total_fallbacks": stats["fallbacks_used"],

 "tasa_violaciones": stats["fallbacks_used"] / max(1, stats["total_inputs"])

 },

 "violations": {

 "pii": stats["violations"]["pii"],

 "jailbreak": stats["violations"]["jailbreak"],

 "biomedical": stats["violations"]["biomedical"],

 "crisis": stats["violations"]["crisis"]

 },

 "recent_activity": [

 {

 "timestamp": v["timestamp"],

 "type": v["type"],

 "details": v["details"]

```

```

 }

 for v in stats["recent_violations"]

],

 "status": "healthy" if stats["fallbacks_used"] < stats["total_inputs"] * 0.1 else
 "needs_attention"

}

...

```

## ## 8. ARCHIVOS DEL SISTEMA DE GUARDARRAÍLES

```

...

/behavioralos/
├── src/
│ ├── guardrails/
│ │ ├── __init__.py
│ │ ├── input_guardrails.py
│ │ ├── output_guardrails.py
│ │ ├── fallback_system.py
│ │ ├── guardrails_system.py
│ │ ├── companion_guardrails.py
│ │ └── guardrails_dashboard.py
│ ├── prompts/
│ │ └── ... (prompts existentes)
│ └── tests/
│ ├── test_input_guardrails.py
```

```
| |— test_output_guardrails.py
| |— test_guardrails_system.py
...

```

---

\*\*Sistema completo de guardarraíles implementado. ¿Quieres que profundice en alguna parte específica o que pasemos a la siguiente tarea?\*\*

## ## SISTEMA RAG BÁSICO CON BEHAVIORAL TWIN

### ### ARQUITECTURA DE RECUPERACIÓN DE CONOCIMIENTO PERSONALIZADO

---

## ## 1. ARQUITECTURA GENERAL

...

### RAG System con Behavioral Twin

```
|— Behavioral Twin (Knowledge Base)
| |— Perfil del Paciente
| |— Estado Emocional Actual
| |— Historial de Ejercicios
| |— Patrones de Conducta Identificados
| |— Valores Declarados
| |— Progreso Funcional
|

```

- |— Document Chunker (Indexación)
  - | |— Chunking por Temas
  - | |— Metadatos por Chunk
  - | |— Versionado de Chunks
- |
- |— Retriever (Recuperación)
  - | |— Keyword Search (BM25 / TF-IDF)
  - | |— Semantic Search (Embeddings)
  - | |— Hybrid Retrieval
- |
- |— RAG Prompt Builder
  - | |— Inyección de Contexto Relevante
  - | |— Formateo para el Prompt
  - | |— Límites de Longitud
- |
- |— Integration
  - |— Companion IA
  - |— Juegos Terapéuticos
  - |— Behavioral Twin Updates

...

---

## ## 2. BEHAVIORAL TWIN COMO BASE DE CONOCIMIENTO

```python

```

# behavioral_twin_knowledge.py

from typing import Dict, List, Optional, Any

from dataclasses import dataclass, field

from datetime import datetime


@dataclass
class KnowledgeChunk:

    """Fragmento de conocimiento del Behavioral Twin."""

    id: str

    content: str

    metadata: Dict[str, Any]

    embedding: Optional[List[float]] = None

    timestamp: datetime = field(default_factory=datetime.now)


class BehavioralTwinKnowledge:

    """

    Behavioral Twin como base de conocimiento para RAG.

    Convierte el Behavioral Twin en fragmentos de conocimiento indexables.

    """

    def __init__(self, behavioral_twin_api: 'BehavioralTwinAPI'):

        self.twin = behavioral_twin_api

        self.chunks = []

        self.last_updated = None

    def index_twin(self) -> List[KnowledgeChunk]:

```

"""

Indexa el Behavioral Twin completo generando chunks de conocimiento.

"""

```
self.chunks = []
```

```
# 1. Perfil del paciente
```

```
profile = self.twin.profile
```

```
self.chunks.append(KnowledgeChunk(
```

```
    id="profile_general",
```

```
    content=f"Perfil del paciente: {profile.age} años, diagnóstico(s): {'',  
'.'.join(profile.diagnosis)}",
```

```
    metadata={"type": "profile", "subtype": "general"}  
)
```

```
self.chunks.append(KnowledgeChunk(
```

```
    id="profile_values",
```

```
    content=f"Valores del paciente: {'', '.'.join(profile.values)}",
```

```
    metadata={"type": "profile", "subtype": "values"}  
)
```

```
# 2. Estado emocional actual
```

```
state = self.twin.get_current_state()
```

```
self.chunks.append(KnowledgeChunk(
```

```
    id="state_current",
```

```
    content=f"Estado emocional actual: ansiedad {state.anxiety}/10, ánimo  
{state.mood}/10, energía {state.energy}/10",
```

```
    metadata={"type": "state", "subtype": "current"}  
)
```

```
))
```

```
# 3. Historial de ejercicios (últimos 5)
```

```
history = self.twin.get_exercise_history(5)
```

```
for i, exercise in enumerate(history):
```

```
    self.chunks.append(KnowledgeChunk(
```

```
        id=f"history_{i}",
```

```
        content=f"Ejercicio completado: {exercise.tile_type}, puntuación  
{exercise.score}/100, valoración {exercise.user_rating}/5",
```

```
        metadata={"type": "history", "exercise_id": exercise.exercise_id}
```

```
    ))
```

```
# 4. Patrones de conducta (del Behavioral Twin)
```

```
patterns = self._extract_patterns()
```

```
for pattern_name, pattern_data in patterns.items():
```

```
    self.chunks.append(KnowledgeChunk(
```

```
        id=f"pattern_{pattern_name}",
```

```
        content=f"Patrón identificado: {pattern_name} -  
{pattern_data['description']}",
```

```
        metadata={"type": "pattern", "pattern_name": pattern_name}
```

```
    ))
```

```
# 5. Progreso funcional
```

```
progress = self._get_progress_summary()
```

```
self.chunks.append(KnowledgeChunk(
```

```
    id="progress_summary",
```

```
    content=f"Progreso funcional: {progress['overall']}%, procesos mejorando: {'  
'}.join(progress['improving'])}",
```



```
        metadata={"type": "progress", "subtype": "summary"}
    ))
```

6. Recomendaciones actuales

```
recommendations = self.twin.get_recommendations()

self.chunks.append(KnowledgeChunk(

    id="recommendations",

    content=f"Recomendaciones actuales: tiles sugeridos
{recommendations['suggested_tiles']}, dificultad {recommendations['difficulty']}",

    metadata={"type": "recommendations"}

))
```

```
self.last_updated = datetime.now()
```

```
return self.chunks
```

```
def _extract_patterns(self) -> Dict:
```

```
    """Extrae patrones de conducta del Behavioral Twin."""
```

```
    # Simulación - en realidad vendría del análisis funcional
```

```
    return {
```

```
        "evitacion_experiencial": {
```

```
            "description": "Patrón de evitar emociones incómodas"
```

```
        },
```

```
        "fusion_cognitiva": {
```

```
            "description": "Patrón de creer que los pensamientos son hechos"
```

```
        }
```

```
    }
```

```

def _get_progress_summary(self) -> Dict:
    """Obtiene resumen de progreso funcional."""
    return {
        "overall": 45,
        "improving": ["evitacion_experiencial", "fusion_cognitiva"],
        "regressing": []
    }
...

---
```

3. SISTEMA DE RECUPERACIÓN (RETRIEVER)

```

```python
retriever.py

from typing import List, Dict, Optional
import re

from collections import defaultdict
import math

class SimpleRetriever:
 """
 Retriever simple para RAG con Behavioral Twin.

 Soporta búsqueda por palabras clave y similitud semántica básica.
 """
```

```

def __init__(self, chunks: List[KnowledgeChunk]):

 self.chunks = chunks

 self.index = self._build_index()

def _build_index(self) -> Dict[str, List[KnowledgeChunk]]:

 """Construye un índice por palabras clave."""

 index = defaultdict(list)

 for chunk in self.chunks:

 # Extraer palabras clave del contenido y metadata

 words = set(re.findall(r'\b[a-záéíóúñ]+\b', chunk.content.lower()))

 for word in words:

 if len(word) > 2: # Ignorar palabras muy cortas

 index[word].append(chunk)

 return index

def keyword_search(self, query: str, top_k: int = 3) -> List[KnowledgeChunk]:

 """

 Busca chunks por coincidencia de palabras clave.

 """

 query_words = set(re.findall(r'\b[a-záéíóúñ]+\b', query.lower()))

 scores = defaultdict(float)

 for word in query_words:

 if word in self.index:

 for chunk in self.index[word]:

 scores[chunk.id] += 1.0

```

```
Normalizar y ordenar
```

```
ranked = sorted(
```

```
 [(chunk_id, score) for chunk_id, score in scores.items()],
```

```
 key=lambda x: x[1],
```

```
 reverse=True
```

```
)
```

```
Obtener chunks
```

```
result = []
```

```
chunk_map = {chunk.id: chunk for chunk in self.chunks}
```

```
for chunk_id, score in ranked[:top_k]:
```

```
 if chunk_id in chunk_map:
```

```
 result.append(chunk_map[chunk_id])
```

```
return result
```

```
def semantic_search(self, query: str, top_k: int = 3) -> List[KnowledgeChunk]:
```

```
 """
```

```
 Busca chunks por similitud semántica (usando embeddings).
```

```
 Versión simplificada - en producción usaría un modelo de embeddings.
```

```
 """
```

```
Simulación de embeddings basada en palabras clave compartidas
```

```
query_words = set(re.findall(r'\b[a-záéíóúñ]+\b', query.lower()))
```

```
scores = defaultdict(float)
```

```

for chunk in self.chunks:

 chunk_words = set(re.findall(r'\b[a-záéíóúñ]+\b', chunk.content.lower()))

 # Jaccard similarity

 intersection = len(query_words & chunk_words)

 union = len(query_words | chunk_words)

 if union > 0:

 scores[chunk.id] = intersection / union

```

```

ranked = sorted(

 [(chunk_id, score) for chunk_id, score in scores.items()],

 key=lambda x: x[1],

 reverse=True

)

```

```

result = []

chunk_map = {chunk.id: chunk for chunk in self.chunks}

for chunk_id, score in ranked[:top_k]:

 if chunk_id in chunk_map and score > 0:

 result.append(chunk_map[chunk_id])

return result if result else self.keyword_search(query, top_k)

```

```

def search(self, query: str, top_k: int = 3, use_semantic: bool = True) ->
List[KnowledgeChunk]:

```

```

"""

```

```

 Búsqueda híbrida (semántica + keywords).

```

```

"""

```

```

 if use_semantic:
 return self.semantic_search(query, top_k)

 return self.keyword_search(query, top_k)
 ...

```

#### ## 4. CONSTRUCTOR DE PROMPT CON RAG

```

```python
# rag_prompt_builder.py

from typing import Dict, List, Optional

class RAGPromptBuilder:
    """
    Constructor de prompts que inyecta contexto recuperado del Behavioral Twin.
    """

    def __init__(self, base_prompt: str):
        self.base_prompt = base_prompt
        self.max_context_length = 500 # Caracteres máximos para contexto inyectado

    def build_prompt(self, query: str, retrieved_chunks: List[KnowledgeChunk]) -> str:
        """
        Construye el prompt completo inyectando el contexto recuperado.
        """

```

```
# 1. Formatear el contexto recuperado
```

```
context = self._format_context(retrieved_chunks)
```

```
# 2. Construir el prompt
```

```
prompt = self.base_prompt.replace("{context}", context)
```

```
prompt = prompt.replace("{query}", query)
```

```
return prompt
```

```
def _format_context(self, chunks: List[KnowledgeChunk]) -> str:
```

```
    """Formatea los chunks recuperados como contexto para el prompt."""
```

```
    if not chunks:
```

```
        return "No hay información específica del paciente disponible."
```

```
    context_parts = []
```

```
    for chunk in chunks:
```

```
        # Añadir metadatos relevantes
```

```
        metadata_str = self._format_metadata(chunk.metadata)
```

```
        context_parts.append(f"[{metadata_str}]\n{chunk.content}")
```

```
    context = "\n\n".join(context_parts)
```

```
# Limitar longitud
```

```
if len(context) > self.max_context_length:
```

```
    context = context[:self.max_context_length] + "..."
```

```
return context
```

```
def _format_metadata(self, metadata: Dict) -> str:
```

```
    """Formatea los metadatos para el contexto."""
```

```
    if not metadata:
```

```
        return "información general"
```

```
    type_map = {
```

```
        "profile": "perfil del paciente",
```

```
        "state": "estado emocional",
```

```
        "history": "historial de ejercicios",
```

```
        "pattern": "patrón identificado",
```

```
        "progress": "progreso",
```

```
        "recommendations": "recomendaciones"
```

```
    }
```

```
    type_str = type_map.get(metadata.get("type", ""), "información")
```

```
    if metadata.get("subtype") == "values":
```

```
        return f"{type_str} - valores del paciente"
```

```
    elif metadata.get("subtype") == "current":
```

```
        return f"{type_str} - estado actual"
```

```
    return type_str
```

```
def build_exercise_prompt(self, exercise_type: str, patient_context: Dict,
```


retrieved_chunks: List[KnowledgeChunk]) -> str:

"""

Construye un prompt específico para ejercicios gamificados.

"""

context = self._format_context(retrieved_chunks)

return f"""

Eres un asistente terapéutico basado en el modelo conductual contextual.

****Contexto del paciente:****

{context}

****Ejercicio actual:****

Tipo: {exercise_type}

Estado del paciente antes del ejercicio:

- Ansiedad: {patient_context.get('anxiety', 5)}/10

- Ánimo: {patient_context.get('mood', 5)}/10

- Energía: {patient_context.get('energy', 5)}/10

****Tu tarea:****

Guía al paciente en el ejercicio con un enfoque funcional-contextual.

Conecta el ejercicio con los procesos identificados y los valores del paciente.

****Instrucciones:****

1. Valida la experiencia del paciente.

2. Conecta el ejercicio con un proceso psicológico específico.

3. Conecta con los valores del paciente.
4. Ofrece una pequeña acción de transferencia a la vida real.

```
"""
```

```
...
```

```
---
```

5. SISTEMA RAG COMPLETO

```
```python
```

```
rag_system.py
```

```
from typing import Dict, List, Optional
```

```
from datetime import datetime
```

```
class BehavioralTwinRAG:
```

```
 """
```

```
 Sistema RAG completo para BehavioralOS.
```

```
 """
```

```
 def __init__(self, behavioral_twin_api):
```

```
 self.twin = behavioral_twin_api
```

```
 self.knowledge = BehavioralTwinKnowledge(behavioral_twin_api)
```

```
 self.retriever = None
```

```
 self.prompt_builder = None
```

```
 self.last_index_update = None
```

```

def update_index(self) -> None:

 """Actualiza el índice de conocimiento."""

 chunks = self.knowledge.index_twin()

 self.retriever = SimpleRetriever(chunks)

 self.last_index_update = datetime.now()

def get_context_for_query(self, query: str, top_k: int = 3) -> List[KnowledgeChunk]:

 """

 Obtiene contexto relevante para una consulta.

 """

 if not self.retriever:

 self.update_index()

 # Detectar tipo de consulta para ajustar la búsqueda

 query_lower = query.lower()

 use_semantic = True

 # Si la consulta contiene palabras clave específicas, usar búsqueda por
 keywords

 keyword_queries = ["qué es", "cómo funciona", "cuándo", "dónde", "quién"]

 if any(kw in query_lower for kw in keyword_queries):

 use_semantic = False

 return self.retriever.search(query, top_k, use_semantic)

def get_context_for_exercise(self, exercise_type: str, patient_state: Dict) ->
List[KnowledgeChunk]:

```

```
"""
```

Obtiene contexto relevante para un ejercicio específico.

```
"""
```

```
if not self.retriever:
```

```
 self.update_index()
```

```
Construir consulta basada en el tipo de ejercicio y el estado
```

```
query = f"ejercicio {exercise_type} ansiedad {patient_state.get('anxiety', 5)}
valores"
```

```
Buscar chunks relevantes
```

```
chunks = self.retriever.search(query, top_k=4, use_semantic=True)
```

```
Asegurar que siempre incluya chunks de valores y estado actual
```

```
value_chunks = [c for c in self.knowledge.chunks if c.metadata.get("subtype")
== "values"]
```

```
state_chunks = [c for c in self.knowledge.chunks if c.metadata.get("subtype") ==
"current"]
```

```
Combinar y deduplicar
```

```
all_chunks = chunks + value_chunks[:1] + state_chunks[:1]
```

```
Deduplicar por id
```

```
seen = set()
```

```
result = []
```

```
for chunk in all_chunks:
```

```
 if chunk.id not in seen:
```

```
 seen.add(chunk.id)
```

```
result.append(chunk)
```

```
return result[:5] # Limitar a 5 chunks
```

```
def get_prompt_with_context(self, base_prompt: str, query: str, top_k: int = 3) -> str:
```

```
 """
```

```
 Obtiene un prompt enriquecido con contexto del Behavioral Twin.
```

```
 """
```

```
 if not self.prompt_builder:
```

```
 self.prompt_builder = RAGPromptBuilder(base_prompt)
```

```
 chunks = self.get_context_for_query(query, top_k)
```

```
 return self.prompt_builder.build_prompt(query, chunks)
```

```
def get_exercise_prompt(self, exercise_type: str, patient_context: Dict) -> str:
```

```
 """
```

```
 Obtiene un prompt específico para un ejercicio con contexto.
```

```
 """
```

```
 base_prompt = """
```

```
Eres un asistente terapéutico basado en el modelo conductual contextual.
```

```
Contexto del paciente:
```

```
{context}
```

```
Ejercicio actual:
```

```
Tipo: {exercise_type}
```

Estado del paciente: ansiedad {anxiety}/10, ánimo {mood}/10

**\*\*Tu tarea:\*\***

Guía al paciente en el ejercicio con un enfoque funcional-contextual.

"""

```
if not self.prompt_builder:
```

```
 self.prompt_builder = RAGPromptBuilder(base_prompt)
```

```
 chunks = self.get_context_for_exercise(exercise_type, patient_context)
```

```
 context = self.prompt_builder._format_context(chunks)
```

```
 return f"""
```

```
{base_prompt.replace("{context}", context).replace("{exercise_type}", exercise_type)}
```

```
"""
```

```
def get_status(self) -> Dict:
```

```
 """Obtiene el estado del sistema RAG."""
```

```
 return {
```

```
 "indexed": self.retriever is not None,
```

```
 "chunks_count": len(self.knowledge.chunks) if self.knowledge.chunks else 0,
```

```
 "last_update": self.last_index_update.isoformat() if self.last_index_update
 else None,
```

```
 "chunks": [
```

```
 {
```

```
 "id": c.id,
```

```
 "type": c.metadata.get("type", "unknown"),
```

```

 "preview": c.content[:100] + "..."
 }

 for c in self.knowledge.chunks[:5]

]

}

'''

```

```

```

## ## 6. INTEGRACIÓN CON EL COMPANION IA

```

```python
# companion_with_rag.py

class CompanionWithRAG:
    """
    Companion IA con capacidad RAG usando Behavioral Twin.
    """

    def __init__(self, patient_id: str, twin_api):
        self.patient_id = patient_id

        self.twin = twin_api

        self.rag = BehavioralTwinRAG(twin_api)

        self.base_prompt = COMPANION_BASE_PROMPT

        # Inicializar índice
        self.rag.update_index()

```

```

def process_message(self, user_message: str) -> Dict:
    """
    Procesa un mensaje del usuario usando RAG.
    """

    # 1. Obtener contexto relevante

    chunks = self.rag.get_context_for_query(user_message, top_k=3)

    # 2. Construir prompt con contexto

    prompt = self.rag.get_prompt_with_context(self.base_prompt, user_message,
top_k=3)

    # 3. Generar respuesta usando Gemma (simulación)

    response = self._generate_response(prompt, user_message, chunks)

    return {
        "response": response,
        "rag_context": [
            {
                "id": c.id,
                "content": c.content[:150] + "..." if len(c.content) > 150 else c.content,
                "type": c.metadata.get("type", "unknown")
            }
            for c in chunks
        ]
    }

```



```
def _generate_response(self, prompt: str, query: str, chunks:
List[KnowledgeChunk]) -> str:
```

```
    """
```

```
        Genera una respuesta usando el modelo Gemma.
```

```
        Simulación - en producción usaría MediaPipe + Gemma.
```

```
    """
```

```
        # Aquí iría la llamada real a Gemma
```

```
        # Por ahora, simulamos una respuesta basada en el contexto
```

```
        if not chunks:
```

```
            return "Estoy aquí para explorar contigo. ¿Puedes contarme más sobre lo
que estás viviendo?"
```

```
        # Extraer información relevante de los chunks
```

```
        values = []
```

```
        state = ""
```

```
        for chunk in chunks:
```

```
            if chunk.metadata.get("subtype") == "values":
```

```
                values = chunk.content.split(": ")[1].split(", ")
```

```
            if chunk.metadata.get("subtype") == "current":
```

```
                state = chunk.content
```

```
        return f"""
```

```
He notado que estás explorando un patrón importante. Basado en lo que sé de ti, tus
valores principales son: {' '.join(values)}.
```

```
{state}
```

Me gustaría entender mejor el contexto de lo que estás viviendo. ¿Qué está pasando en tu vida que hace que este patrón tenga sentido?

Cuando estés listo, podemos practicar un ejercicio que te ayude a conectar con lo que es importante para ti.

"""

...

7. EJEMPLO DE USO EN JUEGOS TERAPÉUTICOS

```
```python
```

```
therapeutic_games_with_rag.py
```

```
class TherapeuticGameWithRAG:
```

```
 """
```

```
 Juegos terapéuticos con integración RAG.
```

```
 """
```

```
 def __init__(self, patient_id: str, twin_api):
```

```
 self.patient_id = patient_id
```

```
 self.twin = twin_api
```

```
 self.rag = BehavioralTwinRAG(twin_api)
```

```
 self.current_game = None
```

```
 def start_game(self, game_type: str) -> Dict:
```

```
 """
```

Inicia un juego terapéutico con contexto RAG.

"""

# 1. Obtener contexto del paciente para el juego

state = self.twin.get\_current\_state()

state\_dict = {

"anxiety": state.anxiety,

"mood": state.mood,

"energy": state.energy

}

# 2. Obtener contexto RAG

context\_chunks = self.rag.get\_context\_for\_exercise(game\_type, state\_dict)

# 3. Construir el prompt del juego

prompt = self.rag.get\_exercise\_prompt(game\_type, state\_dict)

# 4. Inicializar el juego

game\_class = {

"platform": ThoughtPlatformGame,

"cave": CaveOfCalm,

"garden": EmotionGarden,

"adventure": HeroJourneyGame

}.get(game\_type)

if not game\_class:

return {"error": "Juego no encontrado"}

```
self.current_game = game_class(self.twin.profile)
```

```
return {
 "game_type": game_type,
 "context": [
 {
 "type": c.metadata.get("type", "unknown"),
 "content": c.content[:200] + "..." if len(c.content) > 200 else c.content
 }
 for c in context_chunks
],
 "prompt": prompt,
 "game": self._render_game()
}
```

```
def _render_game(self):
 """Renderiza el juego actual con contexto RAG."""
 # Similar al renderizado anterior pero incluyendo contexto RAG
 return {
 "type": self.current_game.__class__.__name__,
 "rag_integrated": True,
 "message": "El juego ha sido personalizado con tu información."
 }
```

```
...
```

---

## ## 8. API DEL SISTEMA RAG

```
```python
# rag_api.py

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI(title="BehavioralOS RAG System")

class QueryRequest(BaseModel):
    patient_id: str
    query: str
    top_k: Optional[int] = 3

class ExerciseContextRequest(BaseModel):
    patient_id: str
    exercise_type: str

class RAGResponse(BaseModel):
    context: List[Dict]
    prompt: str
    response: Optional[str] = None
```

```
# Almacenamiento de instancias RAG por paciente
```

```
rag_instances = {}
```

```
def get_rag_instance(patient_id: str) -> BehavioralTwinRAG:
```

```
    if patient_id not in rag_instances:
```

```
        # Inicializar Behavioral Twin y RAG
```

```
        twin = BehavioralTwinAPI(patient_id)
```

```
        rag = BehavioralTwinRAG(twin)
```

```
        rag.update_index()
```

```
        rag_instances[patient_id] = rag
```

```
    return rag_instances[patient_id]
```

```
@app.post("/rag/query", response_model=RAGResponse)
```

```
async def query_rag(request: QueryRequest):
```

```
    """Obtiene contexto RAG para una consulta."""
```

```
    try:
```

```
        rag = get_rag_instance(request.patient_id)
```

```
        chunks = rag.get_context_for_query(request.query, request.top_k)
```

```
    return RAGResponse(
```

```
        context=[
```

```
            {
```

```
                "id": c.id,
```

```
                "content": c.content,
```

```
                "type": c.metadata.get("type", "unknown"),
```

```
                "subtype": c.metadata.get("subtype", "")
```

```

        }

        for c in chunks

    ],

    prompt="Contexto recuperado para la consulta",

    response=None

)

except Exception as e:

    raise HTTPException(status_code=500, detail=str(e))

```

```
@app.post("/rag/exercise/context")
```

```
async def get_exercise_context(request: ExerciseContextRequest):
```

```
    """Obtiene contexto RAG para un ejercicio específico."""
```

```
    try:
```

```
        rag = get_rag_instance(request.patient_id)
```

```
        state = rag.twin.get_current_state()
```

```
        state_dict = {
```

```
            "anxiety": state.anxiety,
```

```
            "mood": state.mood,
```

```
            "energy": state.energy
```

```
        }
```

```
        chunks = rag.get_context_for_exercise(request.exercise_type, state_dict)
```

```
    return {
```

```
        "exercise_type": request.exercise_type,
```

```
        "context": [
```

```
            {
```

```

        "id": c.id,

        "content": c.content,

        "type": c.metadata.get("type", "unknown")

    }

    for c in chunks

],

    "patient_state": state_dict

}

except Exception as e:

    raise HTTPException(status_code=500, detail=str(e))

```

```

@app.get("/rag/status/{patient_id}")

```

```

async def get_rag_status(patient_id: str):

```

```

    """Obtiene el estado del sistema RAG para un paciente."""

```

```

    try:

```

```

        rag = get_rag_instance(patient_id)

```

```

        return rag.get_status()

```

```

    except Exception as e:

```

```

        raise HTTPException(status_code=500, detail=str(e))

```

```

@app.post("/rag/update/{patient_id}")

```

```

async def update_rag_index(patient_id: str):

```

```

    """Actualiza el índice RAG para un paciente."""

```

```

    try:

```

```

        rag = get_rag_instance(patient_id)

```

```

        rag.update_index()

```



```

        return {"status": "updated", "chunks": len(rag.knowledge.chunks)}

    except Exception as e:

        raise HTTPException(status_code=500, detail=str(e))

...

---
```

9. ARCHIVOS DEL SISTEMA RAG

```

...

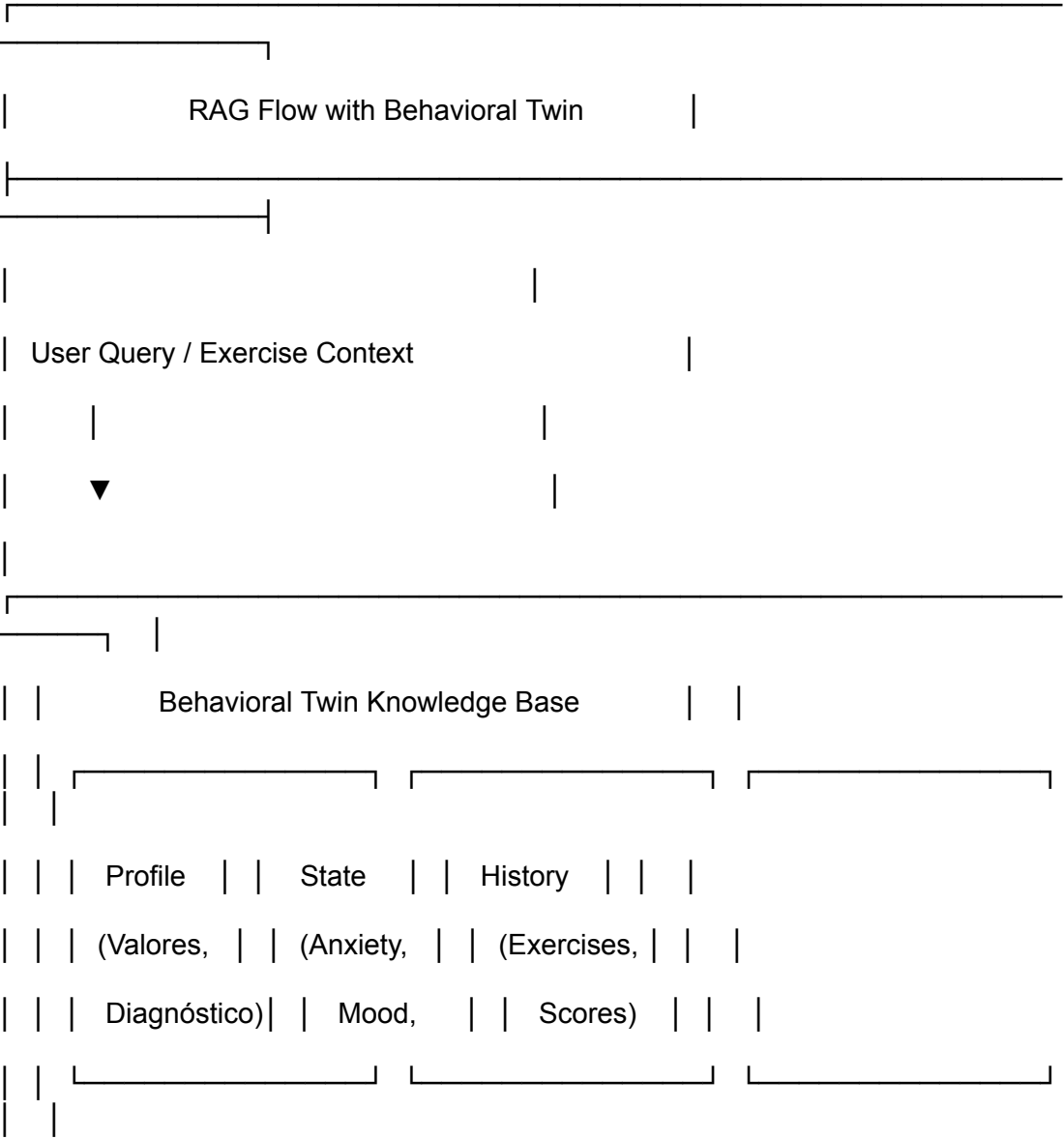
/behavioralos/
├── src/
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── behavioral_twin_knowledge.py
│   │   ├── retriever.py
│   │   ├── rag_prompt_builder.py
│   │   ├── rag_system.py
│   │   ├── companion_with_rag.py
│   │   ├── therapeutic_games_with_rag.py
│   │   └── rag_api.py
│   ├── tests/
│   │   ├── test_retriever.py
│   │   ├── test_rag_prompt_builder.py
│   │   └── test_rag_system.py
│   └── prompts/
```

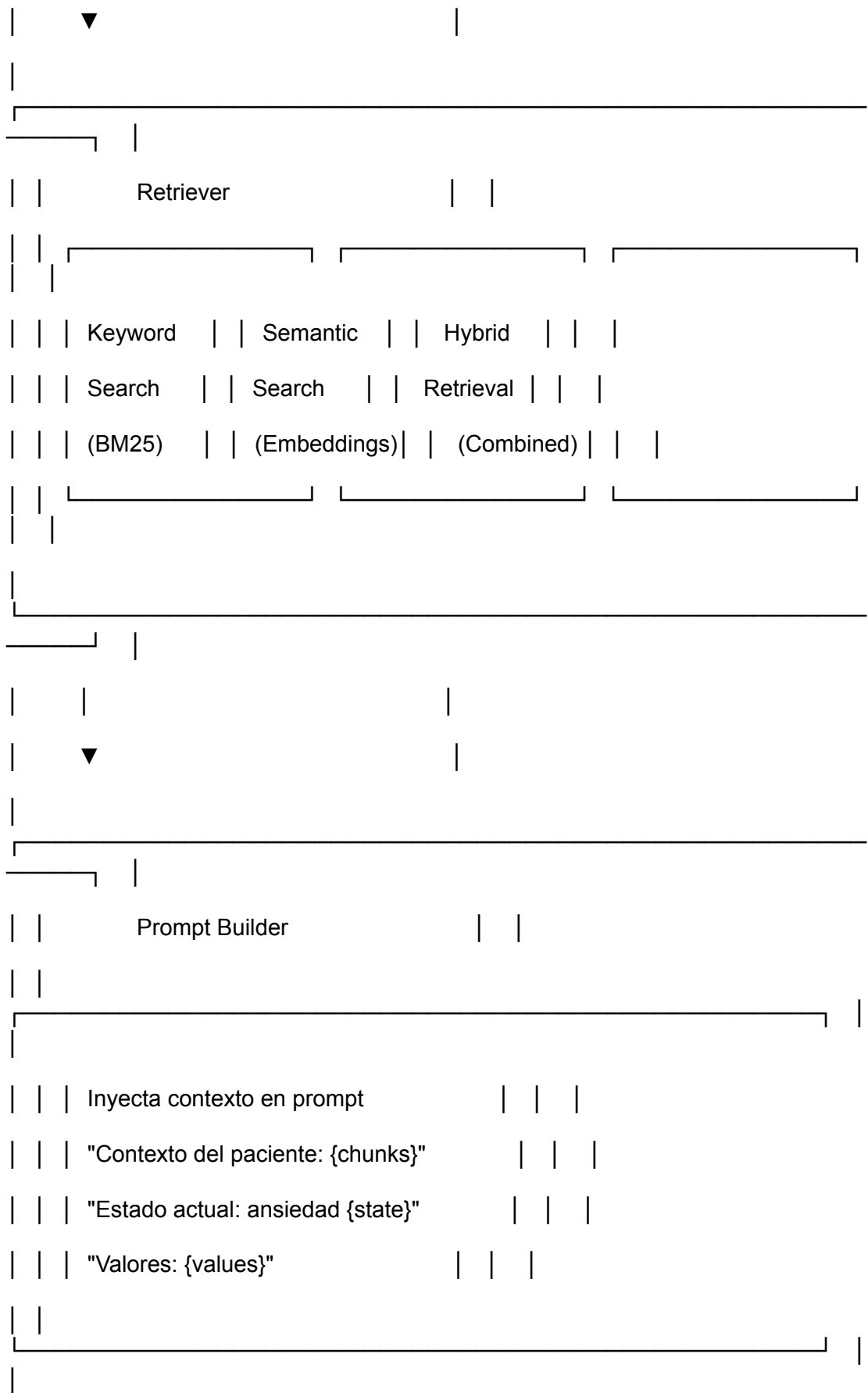
| └─ (prompts existentes)

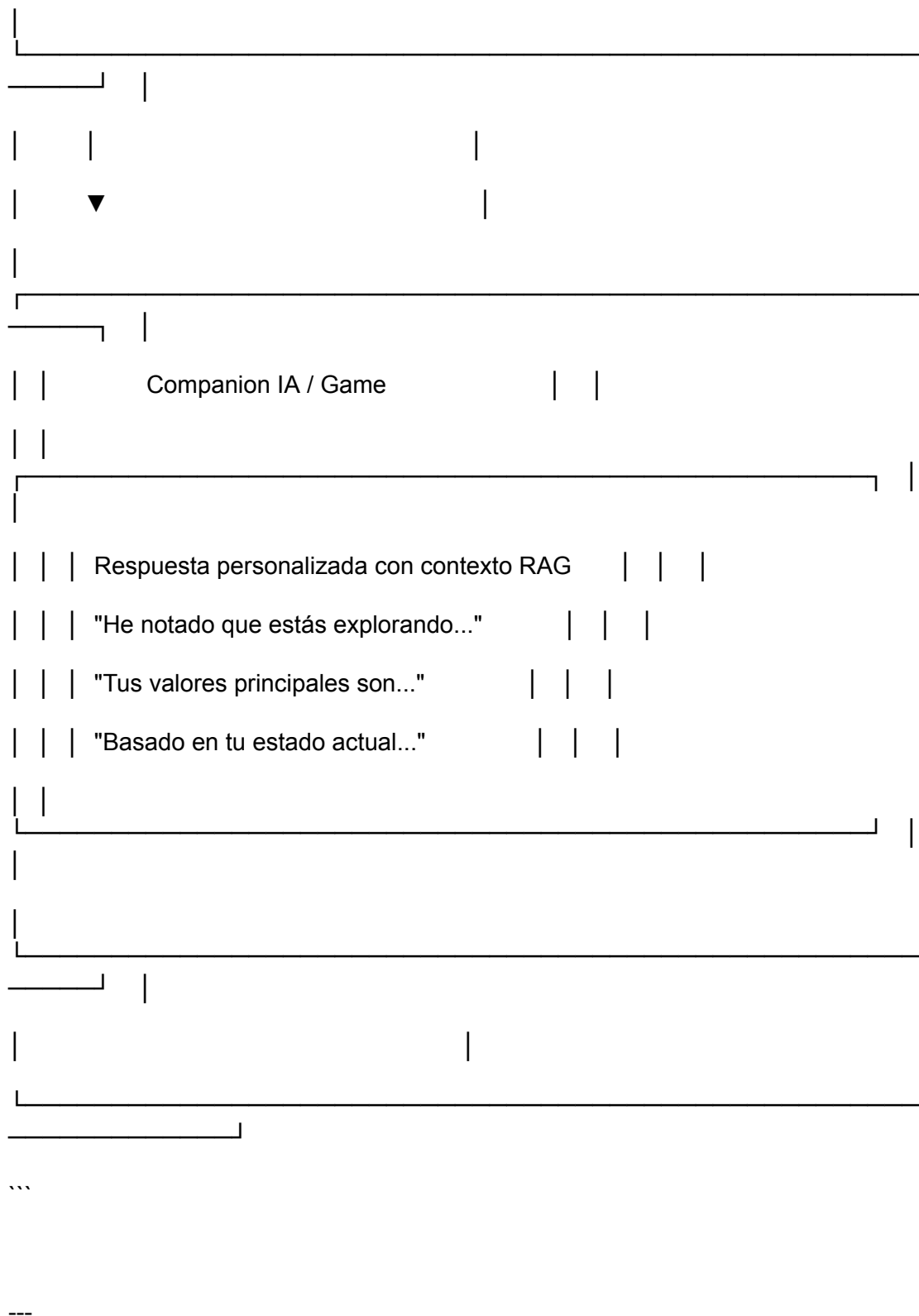
...

10. DIAGRAMA DE FLUJO RAG

...







11. EJEMPLO DE USO COMPLETO

```
```python
ejemplo_rag.py

def main():

 # 1. Inicializar

 patient_id = "P042"

 twin = BehavioralTwinAPI(patient_id)

 # 2. Actualizar Behavioral Twin con datos de ejemplo

 twin.update_state(EmotionalState(anxiety=7, mood=5, energy=6,
timestamp=datetime.now()))

 # 3. Inicializar RAG

 rag = BehavioralTwinRAG(twin)

 rag.update_index()

 # 4. Consulta del usuario

 query = "Últimamente estoy muy ansioso, no sé qué hacer"

 # 5. Obtener contexto RAG

 chunks = rag.get_context_for_query(query)

 # 6. Construir prompt

 builder = RAGPromptBuilder(COMPANION_BASE_PROMPT)

 prompt = builder.build_prompt(query, chunks)

 print("=== PROMPT CON RAG ===")

 print(prompt)
```

```

7. Generar respuesta (simulación)

companion = CompanionWithRAG(patient_id, twin)

result = companion.process_message(query)

print("\n=== RESPUESTA DEL COMPANION ===")

print(result["response"])

print("\n=== CONTEXTO RAG UTILIZADO ===")

for c in result["rag_context"]:

 print(f"- [{c['type']}] {c['content']}")
...

**Sistema RAG básico implementado. ¿Quieres que profundice en alguna parte
específica o que pasemos a la siguiente tarea?**

```

## ## Optimización de Inferencia con Cuantización INT8 para BehavioralOS

Basado en las lecciones del libro *\*AI Engineering\** de Chip Huyen, especialmente el Capítulo 9 (Optimización de Inferencia) y el Capítulo 7 (Fine-Tuning), te presento un plan detallado para implementar cuantización INT8 en el motor de IA de BehavioralOS. Este plan está alineado con la arquitectura on-device, los principios de privacidad y la necesidad de baja latencia en ejercicios gamificados.

---

### ### 1. ¿Por qué Cuantización INT8 en BehavioralOS?

| Necesidad | Solución con INT8 |

|-----|-----|

| **\*\*Dispositivos móviles con memoria limitada\*\*** | Reducción del tamaño del modelo de 4× a 8× (FP32 → INT8). |

| **\*\*Baja latencia para interacción en juegos\*\*** | Operaciones enteras más rápidas que punto flotante en CPUs/GPUs móviles. |

| **\*\*Batería y consumo energético\*\*** | Menos operaciones de mayor consumo. |

| **\*\*Privacidad (todo on-device)\*\*** | No se requiere conexión a la nube para inferencia. |

| **\*\*Soporte en MediaPipe\*\*** | MediaPipe ya optimiza modelos cuantizados (TFLite, ONNX). |

---

## ### 2. Estrategias de Cuantización según el Libro

El libro distingue dos enfoques principales:

| Técnica | Descripción | Cuándo usar |

|-----|-----|-----|

| **\*\*Post-Training Quantization (PTQ)\*\*** | Cuantizar un modelo ya entrenado sin reentrenamiento. Rápido, pero puede degradar calidad. | Si el modelo base es robusto y la tarea no es extremadamente sensible (ej. clasificación de intenciones). |

| **\*\*Quantization-Aware Training (QAT)\*\*** | Entrenar o fine-tuneear simulando la cuantización. Más preciso, pero más costoso. | Para tareas generativas (nuestro Companion) donde la pérdida de calidad es crítica. |

**\*\*Recomendación para BehavioralOS\*\***:



- **PTQ** para componentes no generativos (ej. clasificador de intención del router, detector de PII).
- **QAT** o **QLoRA** (fine-tuning con cuantización 4-bit + LoRA) para el modelo generativo (Companion, AAO, generación de ejercicios).

QLoRA permite adaptar Gemma con solo el 0.1% de parámetros entrenables, manteniendo calidad y reduciendo memoria.

---

### ### 3. Aplicación a Gemma en BehavioralOS

**Modelo objetivo**: Gemma 2B o 7B (versión instruct) corriendo en MediaPipe (formato TFLite o ONNX).

#### #### 3.1. Herramientas y Frameworks

- **TensorFlow Lite (TFLite)** con conversor que soporta cuantización INT8 (post-training y QAT).
- **ONNX Runtime** con cuantización dinámica/estática (mejor integración con MediaPipe).
- **Hugging Face Optimum + Intel Neural Compressor** para cuantizar modelos PyTorch.
- **MediaPipe** nativamente soporta modelos TFLite; podemos exportar desde HuggingFace con `transformers` y luego convertir.

#### #### 3.2. Flujo de Implementación (QAT con QLoRA)

1. **Obtener Gemma base** (2B o 7B) desde Hugging Face.
2. **Preparar dataset de fine-tuning** específico para BehavioralOS (ejemplos de conversaciones terapéuticas, generación de ejercicios, análisis funcional).

- Usar datos sintéticos generados por un modelo grande (ej. GPT-4) bajo supervisión clínica, o anotaciones de terapeutas.
3. **Aplicar QLoRA** (bitsandbytes con 4-bit base, LoRA en capas de atención y feed-forward).
    - Mantener el modelo base en 4-bit y entrenar solo adaptadores LoRA en INT8 (o FP16). Esto reduce memoria y acelera el fine-tuning.
  4. **Fusionar LoRA con el modelo base** y **convertir a TFLite con cuantización INT8** (usando el convertidor con `optimizations` y `representative\_dataset`).
  5. **Validar** con un conjunto de evaluación (métricas de calidad, latencia, tamaño).

### #### 3.3. Cuantización Post-Entrenamiento (para módulos auxiliares)

- **Clasificador de intención** (router): modelo pequeño (ej. DistilBERT) → PTQ INT8.
- **Detector de PII** (Regex + pequeño modelo NER) → PTQ.
- **Embeddings para RAG** (si se usan) → cuantización de vectores a INT8 con recuperación por similitud coseno en espacio cuantizado (producto escalar con enteros).

---

### ### 4. Métricas de Rendimiento a Monitorear

| Métrica | Objetivo en BehavioralOS |

|-----|-----|

| **TTFT** (Time to First Token) | < 500 ms para sentir respuesta inmediata en ejercicios. |

| **TPOT** (Time Per Output Token) | < 50 ms/token para fluidez conversacional. |

| **Memoria RAM** | < 500 MB para el modelo completo (Gemma 2B cuantizada INT8 ocupa ~1 GB; con optimizaciones adicionales podemos acercarnos a 500 MB).

|

| **Tamaño del modelo** | < 1 GB (descarga rápida en móvil). |

| **Latencia total** (ejercicio típico de 3-4 turnos) | < 3 segundos. |

---

## ### 5. Integración con el Ecosistema BehavioralOS

### #### 5.1. Impacto en el Companion (TCCN)

- La cuantización puede reducir ligeramente la riqueza de las respuestas.  
**Compensación**: usar un **sistema de prompts más detallado** y **ejemplos en contexto** (few-shot) para guiar al modelo cuantizado.
- Si la calidad cae por debajo del umbral, optar por **QAT** en lugar de PTQ.

### #### 5.2. AAO (Adaptive Assessment Orchestrator)

- AAO usa el Behavioral Twin y clasifica patrones. Estas tareas son menos sensibles a la pérdida de precisión; PTQ es suficiente.
- Podemos mantener una versión FP16 para evaluación offline y la INT8 para producción, comparando salidas para detectar deriva.

### #### 5.3. Behavioral Twin y RAG

- La recuperación de información del Behavioral Twin se hace con embeddings (vectores). Podemos cuantizar los vectores a INT8 sin pérdida significativa usando **producto escalar cuantizado** (Q-SVM o similar).
- El **prompt caching** (paso 4 del libro) puede almacenar en caché respuestas comunes, reduciendo aún más la carga de inferencia.

#### #### 5.4. Ejercicios Gamificados

- Los juegos (ej. "Salta tus Pensamientos") requieren generación de texto rápido. La cuantización reduce la latencia, mejorando la experiencia de flujo (inmersión).
- Podemos **priorizar** la generación de los primeros tokens (TTFT) sobre la calidad total, ya que el usuario percibe la respuesta inicial como más importante.

---

#### ### 6. Plan de Implementación Paso a Paso

| Fase | Actividad | Herramientas | Duración estimada |

|-----|-----|-----|-----|

| 1 | Seleccionar modelo base (Gemma 2B) y preparar dataset de fine-tuning (5k-10k ejemplos). | Hugging Face, scripts de generación sintética. | 1 semana |

| 2 | Aplicar QLoRA (4-bit base + LoRA) sobre el dataset. | bitsandbytes, PEFT, Transformers. | 2-3 días en GPU |

| 3 | Fusionar LoRA y exportar a PyTorch. | `peft` merge. | 1 día |

| 4 | Convertir a TFLite con cuantización INT8 (representative dataset de 100-200 muestras). | TensorFlow Lite Converter (TF 2.15+). | 1 día |

| 5 | Evaluación de calidad (perplejidad, BLEU, y evaluación subjetiva con IA-juez y humanos). | Scripts de evaluación, LLM-as-judge. | 3 días |

| 6 | Optimizar con **FlashAttention** y **KV cache** en TFLite (si soportado). | Custom ops o usar MediaPipe Graph. | 1 semana |

| 7 | Integrar en MediaPipe (crear un `Calculator` que cargue el modelo y gestione el estado conversacional). | MediaPipe Python/C++. | 1 semana |

| 8 | Pruebas de rendimiento en dispositivos reales (Android/iOS) y ajuste de parámetros (batch size, temperatura). | Dispositivos de prueba, herramientas de profiling. | 1 semana |

| 9 | Desplegar como modelo on-device en la app (descarga dinámica o empaquetado). | React Native/Expo con módulo nativo. | 2 días |

| 10 | Monitoreo continuo de métricas (latencia, calidad, deriva) y plan de re-entrenamiento trimestral. | Prometheus + Grafana (logs locales anonimizados). | Continuo |

---

### ### 7. Guardarraíles y Control de Calidad

- **Antes de la cuantización**: Establecer una **línea base de calidad** con el modelo FP16.
- **Después de la cuantización**: Ejecutar un **conjunto de pruebas automatizadas** que verifiquen:
  - No se generan respuestas peligrosas (detectadas por guardarraíles de salida).
  - Las respuestas mantienen coherencia con los principios ACT/RFT (evaluación con IA-juez).
  - La latencia y memoria están dentro de los SLAs.
- **Si la calidad desciende > 5%**, repetir QAT con más datos o aumentar el número de parámetros LoRA.

---

### ### 8. Posibles Desafíos y Soluciones

| Desafío | Solución |

|-----|-----|

| **Pérdida de precisión en generación** | Usar QAT con 8-bit en lugar de 4-bit, o combinar con **decodificación especulativa** (modelo pequeño + verificación del grande). |

| **Soporte limitado en TFLite para modelos de 2B** | Usar ONNX Runtime con cuantización dinámica, o bien **MediaPipe con TFLite** y dividir el modelo en fragmentos (sharding). |

| **KV cache grande** | Usar **FlashAttention** o **PagedAttention** (vLLM) adaptado a móvil, o limitar el contexto a 2048 tokens. |

| **Tiempo de arranque (carga del modelo)** | Cargar el modelo en segundo plano durante la pantalla de inicio y usar **caché de prompts** para respuestas comunes. |

| **Diferencias entre dispositivos** | Ofrecer dos versiones del modelo: INT8 y FP16 (descarga opcional para dispositivos con más RAM). |

---

### ### 9. Conclusión y Siguientes Pasos

La cuantización INT8 es **esencial** para hacer viable BehavioralOS en dispositivos móviles, respetando la privacidad y ofreciendo una experiencia gamificada fluida. Recomendando:

- **Comenzar con PTQ** para los módulos auxiliares (router, NER) y **QLoRA + cuantización** para el Companion generativo.
- **Invertir desde el día 1** en un pipeline de evaluación robusto (como se enfatiza en el libro), para medir el impacto de la cuantización y decidir si es suficiente.
- **Iterar**: la optimización de inferencia es un proceso continuo; a medida que el modelo evolucione, se repetirán estos pasos.

**Próxima acción concreta**: Preparar el dataset de fine-tuning y ejecutar un experimento de QLoRA con Gemma 2B en Google Colab (GPU T4) para medir la calidad antes de la cuantización.

¿Quieres que profundice en algún aspecto (ej. script de conversión, configuración de MediaPipe, métricas de evaluación) o prefieres que avancemos con la implementación del módulo 000-Core (usuarios/autenticación) mientras se diseña este plan en paralelo?

Perfecto. Pasamos de optimizar la **inferencia** (cuantización) a diseñar la **orquestración del comportamiento** (agente supervisado).

En BehavioralOS, un agente supervisado no es solo un "chatbot con herramientas". Dado que manejamos salud mental y seguimos el principio **"La IA nunca contradice al terapeuta; el juicio clínico humano es insustituible"**, la supervisión es una **"capa de gobernanza"** que valida, corrige y, cuando es necesario, pausa la IA para escalar al humano.

Aquí tienes el diseño completo de la **"Arquitectura de Agente Supervisado"** para BehavioralOS, alineado con el libro *"AI Engineering"* (Capítulos 6 y 10) y con tus módulos existentes (AAO, BREO, BPOS, Guardrails).

---

## ## 1. Definición del "Agente Supervisado" en BehavioralOS

| Concepto | Definición para BehavioralOS |

| :--- | :--- |

| **"Agente"** | Sistema autónomo que percibe el estado del paciente (Behavioral Twin) y actúa sobre su entorno (genera ejercicios, responde en chat, modifica el plan de tratamiento). |

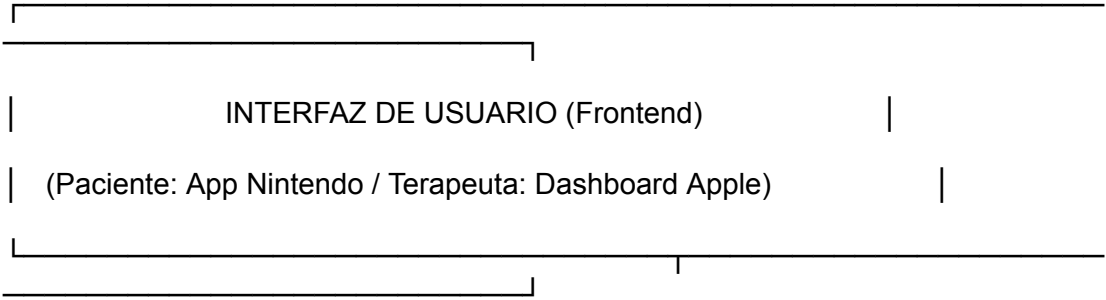
| **"Supervisor"** | Un **"orquestador jerárquico"** (el AAO potenciado) que decide **"qué"** agente ejecutar, **"cuándo"** hacerlo y, crucialmente, **"cuándo detener todo y llamar al terapeuta"**. |

| **"Modo Supervisado"** | Las acciones del agente no se ejecutan directamente en producción sin pasar por un **"cinturón de seguridad"** (Guardrails + Validador Clínico + Aprobación Humana opcional). |

---

## ## 2. Arquitectura General (Capas)

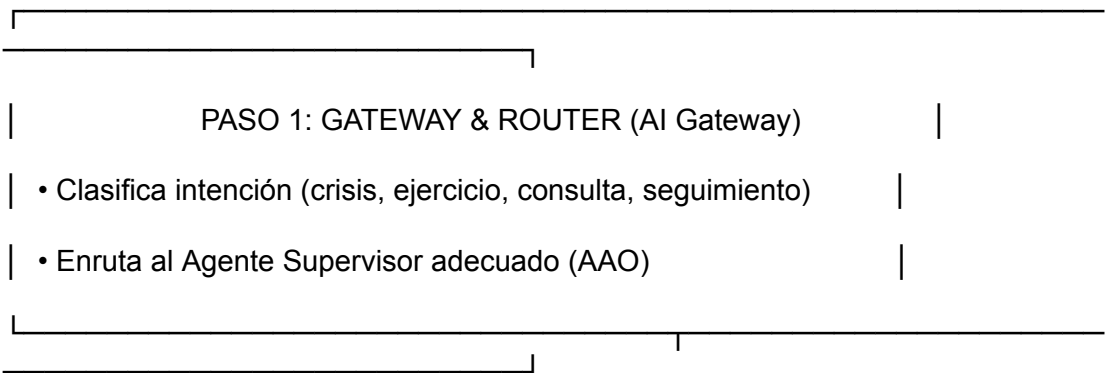
...



INTERFAZ DE USUARIO (Frontend)

(Paciente: App Nintendo / Terapeuta: Dashboard Apple)

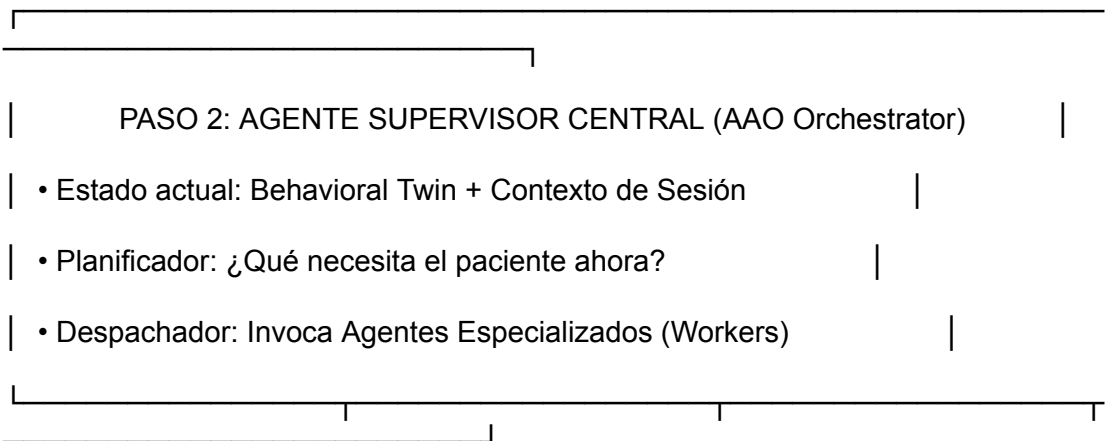
(Entrada: Texto, Click en juego, Voz)



PASO 1: GATEWAY & ROUTER (AI Gateway)

• Clasifica intención (crisis, ejercicio, consulta, seguimiento)

• Enruta al Agente Supervisor adecuado (AAO)

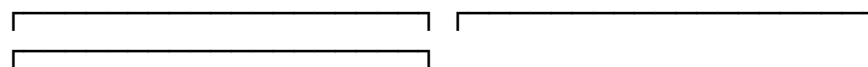


PASO 2: AGENTE SUPERVISOR CENTRAL (AAO Orchestrator)

• Estado actual: Behavioral Twin + Contexto de Sesión

• Planificador: ¿Qué necesita el paciente ahora?

• Despachador: Invoca Agentes Especializados (Workers)



Worker 1:

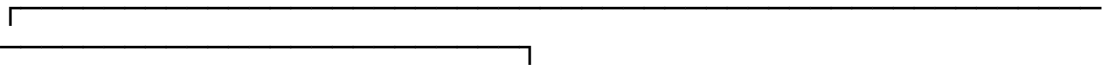
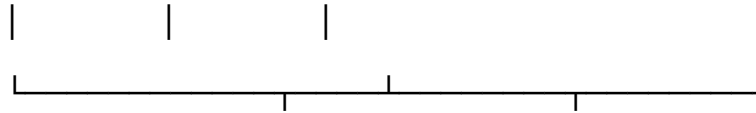
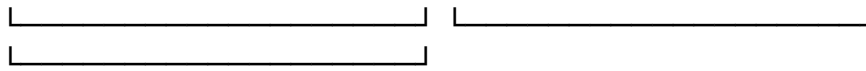
Worker 2:

Worker 3:



| Companion (Chat) | | Exercise Engine | | Insight Engine |

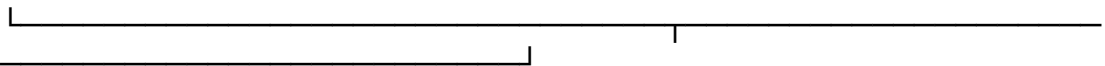
| (TCCN) | | (BERL + BML) | | (Analytics) |



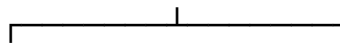
| PASO 3: GUARDARRAÍLES Y VALIDADOR CLÍNICO |

| • Validador de Salida (Seguridad, No-Diagnóstico, Coherencia ACT) |

| • Calculadora de Confianza (Confidence Score 0-100) |



|



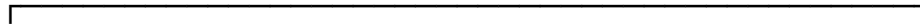
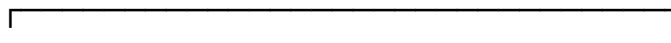
|

(Confianza

> 85%)

< 85% o Riesgo)

|



| PASO 4a: EJECUCIÓN DIRECTA | | PASO 4b: ESCALADA A HUMANO  
(BPOS) |

| • Se envía al Frontend | | • Se crea Ticket en Dashboard Terapeuta |

| • Se actualiza Behavioral Twin | | • Se pausa el agente |

| • Se guarda en caché (Redis) | | • Terapeuta corrige o autoriza variante |

```
|_____|
|_____|
```

```
...
```

```

```

### ## 3. Componentes Clave del Agente Supervisor

#### ### 3.1. El Orquestador (AAO + Planificador)

Actúa como el "cerebro" del agente. Basado en el estado del Behavioral Twin y la entrada del usuario, genera un **Plan de Acción**.

**estructura del Plan (JSON):**

```
```json
{
  "intent": "adaptive_exercise",
  "priority": "high",
  "required_workers": ["exercise_engine", "companion"],
  "params": {
    "target_process": "cognitive_defusion",
    "difficulty": "intermediate",
    "previous_exercise": "salta_pensamientos_v1"
  },
  "fallback_strategy": "escalate_to_therapist",
  "supervision_level": "automated"
  // Opciones: automated, require_approval, escalate_immediately
}
```

...

3.2. Workers (Agentes Especializados)

Son agentes más pequeños con una sola responsabilidad. Usan **tools** específicas.

| Worker | Tools (Herramientas) | Modelo |

| :--- | :--- | :--- |

| ****Companion (TCCN)**** | RAG sobre Behavioral Twin, Historial de conversación, Protocolos ACT. | Gemma 2B (QAT INT8) |

| ****Exercise Engine**** | Tile Generator, Attribute Table, Exercise Buffer (lectura/escritura). | Reglas + Gemma (para generar narrativas) |

| ****Insight Engine**** | Analítica de procesos (métricas funcionales), Detector de progreso. | Modelo pequeño (clasificación) |

| ****Crisis Detector**** | Escanea entrada en busca de ideación, autolesión. | Regex + Clasificador TinyBERT. |

3.3. Memoria del Agente (Fundamental)

Siguiendo el Capítulo 6 del libro, el agente necesita memoria:

- ****Memoria a Corto Plazo****: Contexto de la conversación actual (últimos 10 intercambios). Almacenado en ****Redis**** (TTL: sesión activa).
- ****Memoria a Largo Plazo****: El ****Behavioral Twin**** (PostgreSQL + pgvector). Contiene el perfil completo, progreso histórico y preferencias.
- ****Memoria de Trabajo (Working Memory)****: Estado del ejercicio actual (ej. cuántas plataformas ha saltado en el juego "Salta tus Pensamientos").

3.4. Sistema de Supervisión (El "Cinturón de Seguridad")

Es donde se aplica el concepto de ****"Supervised Agent"****.

El validador corre tras cada generación del Worker y antes de enviar al usuario.

****Criterios de validación**:**

1. ****Seguridad****: ¿Pasó el Guardrail de Salida? (0 = falla, se bloquea).
2. ****Alineación ACT****: ¿La respuesta fomenta evitación experiencial o, por el contrario, aceptación/defusión? (Evaluado por un mini-modelo).
3. ****Coherencia con el Behavioral Twin****: ¿La sugerencia ignora una fobia conocida del paciente?.
4. ****Confianza (Confidence Score)****: El modelo Gemma devuelve un logit promedio; si < 0.85 , se activa la escalada.

4. Flujo de Trabajo Detallado (State Machine con Temporal.io)

Dado que tu stack incluye ****Temporal.io****, es la herramienta perfecta para orquestar este agente supervisado porque maneja pausas, esperas humanas y reintentos de forma nativa.

```mermaid

stateDiagram-v2

[\*] --> Intake: Usuario interactúa

Intake --> Router: Clasificar intención

Router --> Supervisor: Planificar

Supervisor --> Worker\_Companion: Si es chat

Supervisor --> Worker\_Exercise: Si es juego

Supervisor --> Worker\_Crisis: Si es crisis

Worker\_Companion --> Validator: Respuesta generada

Worker\_Exercise --> Validator: Ejercicio generado

Validator --> GuardRail\_Pass: Si todo OK (>85%)

Validator --> GuardRail\_Fail: Si inseguro o <85%

GuardRail\_Pass --> Update\_Twin: Actualizar memoria largo plazo

Update\_Twin --> Responder: Enviar al frontend

Responder --> [\*]

GuardRail\_Fail --> Human\_Escalation: Crear tarea en BPOS

Human\_Escalation --> Wait\_Therapist: Temporal Workflow se pausa

Wait\_Therapist --> Therapist\_Override: Terapeuta responde/corriga

Therapist\_Override --> Responder: Enviar corrección

...

---

### ## 5. Flujo de Escalada Humana (El Corazón de la Supervisión)

Cuando la IA no está segura, **\*\*pierde el turno\*\*** en lugar de inventar.

1. **\*\*Detección\*\***: El Validador marca `confidence: 0.6`.
2. **\*\*Pausa\*\***: Temporal.io detiene el workflow y lanza un evento `AWAITING\_THERAPIST`.
3. **\*\*Notificación\*\***: El dashboard del terapeuta (React) muestra una alerta en tiempo real (WebSocket) con:
  - Entrada del paciente.

- Intento de respuesta de la IA (borrador).

- Motivo de la baja confianza.

4. **\*\*Acción del Terapeuta\*\***:

- **\*Opción A (Corregir)\***: Edita el borrador de la IA y lo envía.

- **\*Opción B (Ignorar)\***: Escribe una respuesta propia.

- **\*Opción C (Reanudar)\***: Autoriza a la IA a intentarlo de nuevo con un ajuste en el prompt.

5. **\*\*Reanudación\*\***: Temporal despierta el workflow y envía la respuesta del terapeuta al paciente, registrando la intervención en el Behavioral Twin como "contexto supervisado".

---

## ## 6. Implementación Técnica (Stack Propuesto)

| Capa | Tecnología | Rol en el Agente Supervisor |

| :--- | :--- | :--- |

| **\*\*Orquestación\*\*** | **\*\*Temporal.io\*\*** (Python SDK) | Workflow duradero que maneja pausas (escaladas humanas) y sagas. |

| **\*\*Estado del Agente\*\*** | **\*\*Redis\*\*** (con TTL) | Memoria a corto plazo y bloqueo de ejercicios activos. |

| **\*\*Herramientas (Tools)\*\*** | **\*\*FastAPI + Pydantic\*\*** | Cada Worker expone una herramienta como endpoint interno (ej. `/tools/get_exercise``). |

| **\*\*RAG Contextual\*\*** | **\*\*Supabase (pgvector)\*\*** | Recuperación del Behavioral Twin y del historial relevante. |

| **\*\*Decisión del Supervisor\*\*** | **\*\*Clasificador liviano + Reglas\*\*** | Para decidir el plan, se usa un clasificador XGBoost o un pequeño LLM (DistilBERT) fine-tuneado con datos de sesiones reales (anónimos). |

| **\*\*Frontend para Terapeuta\*\*** | **\*\*React + WebSockets\*\*** | Vista de "Inbox de Supervisión" para aprobar/rechazar acciones de la IA. |

---

## ## 7. Seguridad y Principios (Validación Ética)

Para garantizar los principios de BehavioralOS, el Agente Supervisor implementa:

1. **\*\*Jamás castigar\*\***: El sistema de escalada nunca penaliza al paciente por "hacer mal" un ejercicio. Solo ajusta la dificultad (Attribute Table).
2. **\*\*Jamás manipular\*\***: Si la IA detecta abandono, no usa culpa. El Supervisor genera un mensaje de "curiosidad" (ej. **\*\*¿Qué te gustaría explorar hoy?\***) en lugar de presión.
3. **\*\*Transparencia\*\***: El paciente sabe cuándo la IA está en "modo automático" vs. cuando la respuesta ha sido revisada por un humano (icono de "escudo" en la UI).
4. **\*\*Círculo de Retroalimentación\*\***: Cada decisión del Supervisor (aprobada o corregida por el terapeuta) se almacena como **\*feedback\***. Este dataset se usa para re-entrenar (fine-tuning) al validador y mejorar la confianza con el tiempo.

---

## ## 8. Resumen de Artefactos a Construir

Para implementar esta arquitectura, necesitaremos crear los siguientes documentos/scripts (en orden):

1. **\*\*Esquema de Workflow Temporal\*\***: ``SupervisedAgentWorkflow.py`` (define los pasos y la lógica de pausa).
2. **\*\*Definición de Tools (Actividades)\*\***: ``agent_tools.py`` (Consultar Twin, Llamar a Gemma, Guardar Progreso).
3. **\*\*Sistema de Prompts del Supervisor\*\***: El prompt que orquesta al planificador (diferente al del Companion).

4. **\*\*Modelo de Validador (Confidence Scorer)\*\***: Un clasificador pequeño (quizás usando el mismo Gemma pero con un prompt específico para autoevaluarse).
5. **\*\*API de Escalada (BPOS Bridge)\*\***: Endpoints para que el terapeuta vea, edite y reanude workflows pausados.

---

## ## 9. Próximo Paso concreto

Ya tenemos el diseño teórico. Si te parece bien, puedo proceder a escribir el código del **\*\*Workflow de Temporal.io\*\*** para este agente supervisor (`SupervisedAgentWorkflow.py`), incluyendo las Activities para llamar al Companion y las señales para la pausa humana.

### **\*\*Opciones\*\***:

1. Escribo el código base del Orquestador (Temporal + FastAPI).
2. Escribo el prompt del Supervisor (Planificador).
3. Pasamos directamente a implementar el módulo `000-Core` (usuarios) y dejamos este diseño como blueprint para la fase 400-AI.

¿Qué prefieres que haga a continuación?

Perfecto. Hemos diseñado la **\*\*optimización\*\*** (INT8) y la **\*\*orquestación\*\*** (Agente Supervisor). Ahora toca el paso más crítico para un proyecto de esta envergadura: **\*\*cerrar el círculo e integrar ambas piezas con el ecosistema completo de BehavioralOS\*\***.

No basta con que el agente funcione y el modelo sea rápido; deben conversar con la base de datos, respetar los roles de seguridad, invocar a los motores de juegos, notificar al terapeuta y alimentar los dashboards de analítica.



Aquí tienes el **Plan de Integración Holística**. Lo he estructurado como un "documento de especificación" más (similar a tus `/specs/`), conectando explícitamente los nuevos diseños con los 16 módulos existentes.

---

# Plan de Integración: Agente Supervisor + Inferencia INT8 en BehavioralOS

## 1. Mapa de Calor de Integración (Impacto por Módulo)

| Módulo | Impacto del Agente Supervisor | Impacto de la Cuantización INT8 |

| :--- | :--- | :--- |

| **000-Core** (Filosofía) | Se añade el principio de "Transparencia de Supervisión" (el paciente sabe cuándo habla con IA vs. humano). | Se añade la métrica "Modo de Precisión" (INT8) al perfil técnico. |

| **200-Backend** | Nuevos endpoints para orquestar workflows de Temporal y recibir señales de escalada. | Endpoints de health-check que reportan carga de modelo y TTFT. |

| **300-Frontend** | Nueva UI de "Inbox de Supervisión" para terapeutas; badge de "Confianza" para pacientes. | Barra de progreso de carga del modelo en pantalla de inicio. |

| **400-AI** | **Centro del cambio**: Se convierte en el "Orquestador de Agentes". Se añade el Confidence Scorer. | Se sustituye Gemma FP16 por Gemma INT8 (TFLite). Se añade pipeline de QAT. |

| **500-Experiencias** | El agente decide qué ejercicio poner en el *Exercise Buffer*; recibe feedback del juego en tiempo real. | La baja latencia permite feedback inmediato en juegos (ej. saltar plataformas sin delay). |

| **600-Commerce** | Sin impacto directo. | Sin impacto directo. |

| **700-PracticeOS** | **Integración crítica**: Temporal pausa workflows y crea Tickets en BPOS. Dashboard del terapeuta muestra la "Sala de Supervisión". | Sin impacto directo (el terapeuta ve la corrección, no el modelo). |

| **800-Analytics** | Se trackean: ``supervision_escalations``, ``confidence_scores``, ``human_override_ratio``. | Se trackea: ``inference_latency_p50/p99``, ``model_memory_usage``. |

| **\*\*900-Security\*\*** | El agente hereda los RLS (Row Level Security) de Supabase. La escalada humana respeta permisos. | El modelo cuantizado se distribuye firmado criptográficamente para evitar inyección. |

| **\*\*1000-Integration (BRIL)\*\*** | **\*\*Nuevos eventos\*\***: `Agent.Planning`, `Agent.Action.Executed`, `Agent.Escalated`. | Evento `Model.Quantized.Loaded` para monitoreo. |

| **\*\*1200-Deployment\*\*** | Se despliega un **\*\*Worker de Temporal\*\*** separado para el agente. | El modelo INT8 se sirve como asset descargable (CDN) o empaquetado en la app. |

| **\*\*1300-Libraries\*\*** | Nueva librería: `behavioral-agent-sdk`. | Nueva utilidad: `behavioral-quantization-tools`. |

---

## ## 2. Flujo de Datos Extremo a Extremo (E2E) Integrado

Este es el flujo unificado que conecta todos los módulos:

### 1. **\*\*Input (App Paciente)\*\***:

- El paciente escribe: **\*\*"No puedo dejar de pensar en que lo hice mal"\*\*.**
- **\*\*Frontend (React Native)\*\*** captura el texto y lo envía al **\*\*API Gateway (FastAPI)\*\*** autenticado con JWT.

### 2. **\*\*Enrutamiento y Seguridad (Backend + Security)\*\***:

- El Gateway aplica **\*\*RLS (Supabase)\*\*** para verificar que el paciente pertenece al terapeuta correcto.
- Se ejecuta el **\*\*Guardrail de Entrada\*\*** (detección de PII, crisis, jailbreak). Si pasa...

### 3. **\*\*Inicio del Agente Supervisor (Temporal.io)\*\***:

- FastAPI inicia un **Workflow de Temporal** (`SupervisedAgentWorkflow`) pasando el `user_id` y el input.

- Temporal recupera el **Behavioral Twin** (PostgreSQL + pgvector) y el **contexto de sesión** (Redis).

#### 4. **Planificación (Orquestador - 400-AI)**:

- El Supervisor (AAO potenciado) decide: **"Este es un patrón de fusión cognitiva. Necesito el Worker de Companion y el Worker de Ejercicios."**

- Genera un `Plan` en JSON.

#### 5. **Ejecución de Workers (IA + Experiencias)**:

- **Worker 1 (Companion)**: Llama a **Gemma 2B INT8** (cargada en MediaPipe) con el prompt terapéutico + contexto del Twin. Genera una respuesta empática.

- **Worker 2 (Exercise Engine)**: Consulta el **Tile Generator** (500-Experiencias) y selecciona el ejercicio **"Salta tus Pensamientos - Nivel 2"**, preparando el `Exercise Buffer`.

#### 6. **Validación y Supervisión (Guardrails + 700-PracticeOS)**:

- El **Confidence Scorer** (mini-modelo) evalúa la respuesta. Obtiene `Confidence: 0.72` (por debajo del umbral de 0.85).

- **Acción**: El Workflow de Temporal se **pausa** (human-in-the-loop).

- Se dispara un evento vía **WebSocket (BRIL)** al Dashboard del terapeuta (700-PracticeOS) con el borrador de la IA y el input del paciente.

- El terapeuta recibe la notificación, revisa, edita ligeramente la respuesta y la aprueba.

#### 7. **Reanudación y Persistencia (Backend + Analytics)**:

- Temporal reanuda el workflow con la respuesta corregida por el humano.

- Se guarda la interacción en la base de datos (historial).

- El **Behavioral Twin** se actualiza con el nuevo patrón de conducta.

- **Analytics (800)** registra una **"escalada por baja confianza"** y almacena la corrección del terapeuta como **"feedback dorado"** para futuros fine-tunings.

#### 8. **Respuesta al Paciente (Frontend)**:

- La respuesta corregida llega a la App del paciente, mostrando un pequeño ícono de "escudo" indicando que fue revisada por su terapeuta.
- Simultáneamente, el `Exercise Buffer` se precarga con el juego seleccionado, listo para jugar.

---

### ## 3. Integración Específica por Componente Técnico

#### ### 3.1. Integración con la Base de Datos (Supabase / PostgreSQL)

- **Tabla `agent_decisions`** (nueva):
  - `workflow_id` (string), `plan` (jsonb), `confidence_score` (float), `escalation_reason` (text), `therapist_override` (jsonb).
- **Tabla `model_versions`** (nueva):
  - `version` (string), `quantization` (string), `size_mb` (int), `is_active` (bool).
- **RLS (Seguridad)**: Las tablas de decisiones del agente solo son visibles para el terapeuta asignado y el propio sistema (usando `service_role`).

#### ### 3.2. Integración con Temporal.io y el Backend (FastAPI)

- **Worker Hosting**: El worker de Temporal se ejecuta como un servicio independiente en el clúster de Kubernetes (o en un contenedor Docker junto al backend en fases iniciales).
- **API Endpoints (FastAPI)**:

- ``POST /api/v1/agent/chat``: Inicia el workflow.
- ``POST /api/v1/agent/feedback/{workflow_id}``: Endpoint para que el terapeuta envíe la corrección (reanuda el workflow).
- ``GET /api/v1/agent/status/{workflow_id}``: Para que el frontend consulte si está en pausa o completado.

### ### 3.3. Integración con el Frontend (React Native / React)

- **\*\*Paciente (React Native)\*\***:
  - Componente `<ChatBubble>` ahora muestra un atributo `isSupervised` (cambia el color de la burbuja).
  - Pre-carga del modelo Gemma INT8 durante el splash screen usando `expo-file-system` para descomprimir los pesos.
- **\*\*Terapeuta (React Web)\*\***:
  - Nuevo componente `<SupervisionInbox>` en el dashboard (700-PracticeOS). Muestra la cola de workflows pausados, con opciones "Aprobar", "Editar y Enviar", "Rechazar (escalar a llamada)".
  - Conexión vía WebSocket (usando `socket.io` o el bus BRIL) para recibir las notificaciones de pausa en tiempo real.

### ### 3.4. Integración con el Motor de Juegos (Godot + jsPsych)

- **\*\*Comunicación Bidireccional\*\***:
  - El agente supervisor escribe en el **\*\*Exercise Buffer\*\*** (Redis) el ejercicio seleccionado.
  - El frontend de Godot (web/móvil) lee este buffer para iniciar el nivel correspondiente.
  - Cuando el paciente termina el juego (ej. "Salta tus Pensamientos"), Godot envía una métrica de rendimiento (ej. número de obstáculos esquivados, tiempo de reacción).
  - **\*\*El Agente Supervisor\*\*** lee esta métrica para ajustar la **\*Attribute Table\*** (dificultad, tono) en la siguiente iteración.

### ### 3.5. Integración con el Pipeline de Evaluación y Monitoreo (Observabilidad)

- **Métricas de Inferencia** (de la capa INT8): Exportadas por MediaPipe y capturadas por Prometheus.
  - ``inference_latency_seconds`` (histograma por tipo de worker).
  - ``model_memory_usage_bytes``.
- **Métricas de Supervisión** (del Agente):
  - ``agent_escalation_total`` (por motivo: `baja_confianza`, `guardrail_violacion`, `crisis_detectada`).
  - ``agent_human_override_ratio`` (qué porcentaje de respuestas son editadas por humanos).
- **Logs Estructurados**: Cada workflow de Temporal genera un ``trace_id`` que se propaga a través de todos los servicios (OpenTelemetry) para depurar flujos complejos.

---

## ## 4. Estrategia de Despliegue y Distribución del Modelo INT8

Dado que BehavioralOS es multi-tenant y altamente privado, la integración del modelo cuantizado debe manejar la distribución sin comprometer la seguridad.

1. **Almacenamiento del Modelo**: Los archivos `.tflite`` (o `.onnx``) se suben a un **bucket privado de S3/Supabase Storage** con firmas de URL de tiempo limitado.
2. **Descarga en el Dispositivo**:
  - La App (React Native) verifica la versión del modelo contra la API (``/api/v1/model/current``).
  - Si hay una nueva versión, descarga el modelo en segundo plano (usando ``expo-background-fetch``) y lo almacena en el almacenamiento local cifrado (``expo-secure-store`` para la ruta, pero el archivo en sandbox).

### 3. **Carga en MediaPipe**:

- Al iniciar el Companion, se carga el modelo desde el sistema de archivos local. Si falla (corrupción), se recurre a un modelo de respaldo empaquetado en la versión inicial de la app.

---

## ## 5. Actualización del Orden de Construcción (Build Order)

Dado que el Agente Supervisor y la Cuantización afectan a casi todas las capas, propongo ajustar el orden de implementación para minimizar bloqueos:

| Orden | Módulo | Integración específica a realizar |

| :--- | :--- | :--- |

| 1 | **000-Core / 900-Security** | Definir nuevos permisos y ontología de "supervisión". |

| 2 | **200-Backend** | Crear las tablas `agent\_decisions` y endpoints base. |

| 3 | **1300-Libraries** | Crear la librería `agent-sdk` y `quantization-utils`. |

| 4 | **400-AI (Parte 1)** | Integrar **Gemma INT8** en MediaPipe (la inferencia pura, sin orquestación). |

| 5 | **400-AI (Parte 2)** | Implementar el **Orquestador (AAO)** y los Workers (Companion, Exercise). |

| 6 | **1000-Integration** | Configurar Temporal.io y los primeros workflows. |

| 7 | **700-PracticeOS** | Integrar la "Sala de Supervisión" (Inbox del terapeuta). |

| 8 | **500-Experiencias** | Conectar el Exercise Buffer con las decisiones del agente. |

| 9 | **300-Frontend** | UI de paciente (badges) y UI de terapeuta (aprobación). |

| 10 | **800-Analytics** | Dashboards de rendimiento del agente y latencia del modelo. |

| 11 | **\*\*1200-Deployment\*\*** | Empaquetar todo en Docker/K8s con los workers de Temporal. |

---

## **## 6. Resumen de Contratos de Integración (APIs y Eventos)**

Para que los equipos (o agentes de IA) trabajen en paralelo, dejo definidos estos contratos:

**\*\*Contrato entre Backend (FastAPI) y Agente (Temporal)\*\*:**

```
```python
# Payload de inicio de workflow
{
    "session_id": "uuid",
    "patient_id": "uuid",
    "therapist_id": "uuid",
    "input_text": "No puedo dejar de pensar...",
    "context": { "behavioral_twin_snapshot": {...} }
}
```
```

**\*\*Contrato entre Agente y Módulo de Juegos (Buffer)\*\*:**

```
```python
# Escritura en Redis por el Agente
redis.set(f"exercise_buffer:{patient_id}", {
    "exercise_id": "salta_pensamientos_v2",

```



```
"attributes": { "difficulty": 0.7, "tone": "compasivo" },  
"ttl": 3600  
})  
...
```

****Contrato entre Agente y PracticeOS (Escalada)**:**

```
```python  
Evento BRIL (NATS/Redis Pub/Sub)

{
 "event_type": "agent.escalation.requested",
 "workflow_id": "temporal-123",
 "therapist_id": "uuid",
 "patient_input": "...",
 "ai_draft": "...",
 "confidence": 0.72,
 "reason": "low_confidence"
}
...

```

**## Conclusión y Siguiente Paso**

Hemos trazado el mapa completo: la cuantización INT8 da **\*\*velocidad y privacidad\*\***, el agente supervisor da **\*\*control y seguridad clínica\*\***, y esta integración los conecta con el resto del ecosistema (base de datos, frontend, juegos, analítica y consultorio).

**\*\*¿Qué sigue?\*\*** Con este plan de integración en mano, ya no hay ambigüedad sobre cómo encaja todo.

Te propongo que **\*\*empecemos a escribir el código\*\*** del primer eslabón de esta cadena: **\*\*la integración de Gemma INT8 en el Worker de Temporal (400-AI)\*\***.

Puedo generar para ti:

- El script de carga de Gemma INT8 en MediaPipe usando Python (`mediapipe_model_loader.py`).
- La Activity de Temporal que invoca a este modelo (`companion_activity.py`).

**\*\*O\*\***, si prefieres empezar por la infraestructura, puedo escribir el `docker-compose.yml` para levantar Temporal + Redis + Supabase local y simular el flujo completo de integración.

Dime por dónde quieres que empiece a codificar.